IT516 – Advanced Computer Graphics : Final Exam Cheat Sheet

Bilingual Reference (EN | VI) – HCMIU MIT – 25/04/2026 – Open-book, paper only

Last-Minute Formula Sheet / Công thức cấp tốc

1. Bezier Curves | Đường cong Bezier

Remember / Nhớ: degree $n \Rightarrow n{+}1$ control points.

- General: $B(t) = \sum_{i=0}^{n} \binom{n}{i}(1-t)^{n-i}t^i P_i$
- n=1: $B(t) = (1-t)P_0 + tP_1$
- n=2: $B(t) = (1-t)^2P_0 + 2(1-t)tP_1 + t^2P_2$
- n=3: $B(t) = (1-t)^3P_0 + 3(1-t)^2tP_1 + 3(1-t)t^2P_2 + t^3P_3$
- Tangent: $B'(0) = n(P_1 - P_0)$, $B'(1) = n(P_n - P_{n-1})$
- $B'(t) = n\sum_{i=0}^{n-1} B_i^{n-1}(t)(P_{i+1} - P_i)$
- Degree elevate: $P_i' = \frac{i}{n+1}P_{i-1} + (1 - \frac{i}{n+1})P_i$

2. De Casteljau | Thuật toán De Casteljau

Remember: pyramid of lerps; gives point + subdivision.

- $P_i^{(0)} = P_i$; $P_i^{(k)}(t) = (1-t)P_i^{(k-1)} + tP_{i+1}^{(k-1)}$
- Curve point: $B(t) = P_0^{(n)}(t)$
- Subdivide @ t: left $= \{P_0^{(0)}, P_0^{(1)}, \ldots, P_0^{(n)}\}$
- right $= \{P_0^{(n)}, P_1^{(n-1)}, \ldots, P_n^{(0)}\}$

3. Bernstein @ common t | Giá trị Bernstein

Remember: symmetric, sums to 1.

- t=0.5: $B^3 = (\frac{1}{8}, \frac{3}{8}, \frac{3}{8}, \frac{1}{8})$
- t=0.5: $B^4 = (\frac{1}{16}, \frac{4}{16}, \frac{6}{16}, \frac{4}{16}, \frac{1}{16})$
- t=0.25: $B^3 = (\frac{27}{64}, \frac{27}{64}, \frac{9}{64}, \frac{1}{64})$
- t=0.75: $B^3 = (\frac{1}{64}, \frac{9}{64}, \frac{27}{64}, \frac{27}{64})$

4. Bezier Surface | Mặt cong Bezier

Remember: bicubic = 4×4 = 16 points.

- $S(u,v) = \sum_i \sum_j B_i^m(u)B_j^n(v)P_{ij}$
- Bicubic (m=n=3): 16 control points
- @ (0.5, 0.5) bicubic: $w_{ij} = B_i^3(0.5)B_j^3(0.5)$
- Weights: corners $\frac{1}{64}$, edges $\frac{3}{64}$, inner $\frac{9}{64}$

5. Geometric Modeling | Mô hình hình học

Remember: QEM = quadric per plane; Taubin = no shrink.

- QEM: $\Delta(v) = v^TQv$, $Q = \sum_p K_p$, $K = nn^T$ (4×4)
- Laplacian: $L(v_i) = \frac{1}{|N(i)|}\sum_{j\in N(i)}(v_j - v_i)$
- Update: $v' = v + \lambda L(v)$
- Taubin: alternate λ>0 then μ<0, $|\mu|>\lambda$
- Bilateral: $v' = v + n\dfrac{\sum w_c(\|v_i - v\|)w_s(\langle n, v_i - v\rangle)\langle n, v_i - v\rangle}{\sum w_c w_s}$
- In-circle (CCW): $\det\begin{pmatrix} a-d & \|a-d\|^2 \\ b-d & \|b-d\|^2 \\ c-d & \|c-d\|^2 \end{pmatrix} > 0$
- SOR: keep if $\bar{d}_{kNN} < \mu + \alpha\sigma$
- Liepa: $W(i,j) = \min_{i<m<j} W(i,m) + W(m,j) + w(\triangle v_i v_m v_j)$

6. Medical Imaging | Ảnh y tế

Remember: HU air −1000, water 0, bone +400 to +1000.

- HU $= 1000(\mu - \mu_w)/\mu_w$
- Larmor: $\omega_0 = \gamma B_0$, $\gamma_{1H}/2\pi = 42.58\,\text{MHz/T}$
- FBP: $f(x,y) = \int_0^\pi \tilde{p}(x\cos\theta + y\sin\theta, \theta)\,d\theta$
- $\tilde{p} = \mathcal{F}^{-1}(|\omega|P)$
- MC edge: $p = p_a + \frac{T - V_a}{V_b - V_a}(p_b - p_a)$
- Otsu: $\arg\max_T w_0 w_1(\mu_0 - \mu_1)^2$
- Snake: $E = \int(\alpha|C'|^2 + \beta|C''|^2 + E_{ext})\,ds$
- Level set: $\partial\phi/\partial t + F|\nabla\phi| = 0$
- MI: $\text{MI}(X,Y) = H(X) + H(Y) - H(X,Y)$
- DVR (front-to-back): $C_{out} = C_{in}(1-\alpha) + C\alpha$

7. Segmentation Metrics | Chỉ số phân vùng

Remember: Dice = F1; IoU $= D/(2-D)$.

- Dice: $D = 2|A\cap B|/(|A| + |B|)$
- IoU/Jaccard: $J = |A\cap B|/|A\cup B| = D/(2-D)$
- HD95: 95th-percentile of surface distances
- ASSD: $\frac{1}{|S_A|+|S_B|}\left(\sum_{a\in S_A} d(a, S_B) + \sum_{b\in S_B} d(b, S_A)\right)$
- Sens $= TP/(TP{+}FN)$; Spec $= TN/(TN{+}FP)$
- Prec $= TP/(TP{+}FP)$; Rec = Sens
- $F_1 = 2PR/(P{+}R)$ = Dice

8. VR/AR Numbers | Thông số VR/AR

Remember: $<$ 20 ms latency, $\geq$ 90 Hz refresh.

- Motion-to-photon latency: $<$ 20 ms
- Refresh: $\geq$ 90 Hz (VR), 120+ Hz preferred
- FOV: 100–110° typical (Quest 3 ∼ 110°)
- IPD: 58–72 mm
- 6DoF = 3 position + 3 orientation
- AR pipeline: capture → feature → pose (PnP) → render → composite

9. BraTS Sub-regions | Vùng khối u BraTS

Remember: WT ⊃ TC ⊃ ET.

- WT (Whole Tumor) = ET ∪ NCR ∪ ED (edema)
- TC (Tumor Core) = ET ∪ NCR
- ET (Enhancing Tumor): only contrast-enhancing

10. Pascal Triangle | Tam giác Pascal

Remember: row n = Bernstein numerators for degree n.

- n=0: 1
- n=1: 1 1
- n=2: 1 2 1
- n=3: 1 3 3 1
- n=4: 1 4 6 4 1
- n=5: 1 5 10 10 5 1

Part I — Reference Cheatsheet

Theory + algorithms + tables, organised by topic

1. Virtual Reality & Augmented Reality

1.1 Definitions (VR / AR / MR / XR)

- **VR (Virtual Reality)**: *fully synthetic* immersive 3D environment that **replaces** the real world. User wears HMD; senses (sight, sound, sometimes touch) are isolated from reality. Goal = *presence* (feeling of "being there").
- **AR (Augmented Reality)**: virtual objects **overlaid** onto the real world in real time and registered in 3D. Real world remains dominant; CG content adds info. Milgram's 3 criteria: (1) combines real+virtual, (2) interactive in real time, (3) registered in 3D.
- **MR (Mixed Reality)**: virtual and real objects *co-exist and interact* (e.g., a virtual ball bounces on a real table). HoloLens-class devices.
- **XR (Extended Reality)**: umbrella term covering VR + AR + MR.
- **Milgram Reality–Virtuality Continuum**: Real → AR → AV (Augmented Virtuality) → VR.
- Key distinguishing axes: *Immersion* (low → high), *Registration* (none → 3D-locked), *Display* (screen / video see-through / optical see-through), *World model* (real / hybrid / synthetic).

1.2 History & Milestones

- **1838** Wheatstone — stereoscope (binocular depth).
- **1962** Morton Heilig — *Sensorama*: multi-sensory arcade (sight, sound, smell, vibration).
- **1968** Ivan Sutherland — *Sword of Damocles*: first HMD with head tracking; ceiling-mounted.
- **1980s** Jaron Lanier coins "Virtual Reality"; VPL Research sells DataGlove + EyePhone.
- **1990** Tom Caudell coins "Augmented Reality" (Boeing).
- **1992** CAVE (UIC) — projection-based VR room.
- **1997** Feiner et al. — Touring Machine (mobile AR).
- **1999** *ARToolKit* (Kato & Billinghurst) — fiducial-marker AR library.
- **2010s** consumer wave: Oculus Rift DK1 (2013), Google Glass (2013), HTC Vive (2016), HoloLens (2016), PSVR (2016).
- **2017+** ARKit (Apple), ARCore (Google), Magic Leap One (2018), Quest 2 (2020), Quest Pro/3 (2022/23), Vision Pro (2024).
- **2019** OpenXR 1.0 standard ratified by Khronos.

1.3 VR Components / Hardware

HMD (Head-Mounted Display):

- *Tethered*: HTC Vive, Valve Index, PSVR (PC/console).
- *Standalone*: Meta Quest 2/3/Pro (mobile SoC, no PC).
- *High-end*: Apple Vision Pro, Varjo XR-3 (eye tracking, micro-OLED, very high PPD).
- Specs to remember: **FOV** 90°–110° typical, 200°+ Pimax; **refresh** 72/90/120/144 Hz; **resolution** per-eye 2K–4K; **IPD** adjustable 58–72 mm.

Tracking:

- **DoF**: 3DoF = rotation only (yaw, pitch, roll) — phone VR; 6DoF = rotation + translation (x,y,z) — proper VR.
- **Outside-in**: external sensors track HMD (Vive Lighthouse base stations sweep IR lasers; Oculus Constellation IR LEDs + cameras). Pros: precise; Cons: setup, occlusion.
- **Inside-out**: cameras on HMD track environment via SLAM (Quest, WMR). Pros: portable; Cons: needs lighting/features.
- **Lighthouse**: 2 base stations emit horizontal+vertical IR sweeps; photodiodes on HMD/controller measure timing ⇒ pose.

Input: 6DoF controllers (Touch, Index knuckles), haptic gloves (HaptX, Manus), full-body suits (Teslasuit), **omni-treadmills** (Virtuix, KAT Walk), motion platforms / chairs.
Audio: *spatial / binaural* via HRTF (Head-Related Transfer Function); HMD has built-in speakers or 3.5mm jack; engines: Steam Audio, Oculus Spatializer, Resonance Audio.

1.4 AR Components / Hardware

Display types:

- **Optical see-through (OST)**: user sees the real world directly through transparent optics (waveguide, half-mirror); CG is added via tiny projector. Devices: *Microsoft HoloLens 1/2*, *Magic Leap 1/2*, *Google Glass*. Pros: zero camera latency for real world; Cons: limited FOV, low contrast in bright light.
- **Video see-through (VST)**: cameras capture the world → composited with CG → shown on opaque screen. Devices: phone/tablet AR, Quest 3 passthrough, Vision Pro. Pros: better occlusion, full opacity; Cons: latency, low real-world fidelity.
- **Projection / Spatial AR**: project light onto real surfaces (museum installations, projection mapping).
- **Handheld**: smartphones / tablets — most common AR platform.

1. Thực tế ảo & Thực tế tăng cường

1.1 Định nghĩa (VR / AR / MR / XR)

- **VR (Thực tế ảo)**: môi trường 3D *hoàn toàn nhân tạo*, **thay thế** thế giới thật. Người dùng đeo HMD; các giác quan bị cô lập khỏi thực tế. Mục tiêu = *presence* (cảm giác "đang ở đó").
- **AR (Thực tế tăng cường)**: vật thể ảo được **chồng lên** thế giới thật theo thời gian thực và được *đăng ký* (registered) trong không gian 3D. Thế giới thật vẫn là chính. 3 tiêu chí Milgram: (1) kết hợp thật+ảo, (2) tương tác thời gian thực, (3) đăng ký 3D.
- **MR (Thực tế hỗn hợp)**: vật ảo và vật thật *cùng tồn tại và tương tác* (ví dụ: bóng ảo nảy trên bàn thật). Thiết bị tiêu biểu: HoloLens.
- **XR**: thuật ngữ ô bao trùm VR + AR + MR.
- **Milgram continuum**: Thật → AR → AV → VR.
- Trục phân biệt: *độ đắm chìm*, *độ đăng ký*, *loại hiển thị* (screen / video see-through / optical see-through), *mô hình thế giới*.

1.2 Lịch sử & Cột mốc

- **1838** Wheatstone — kính lập thể (stereoscope).
- **1962** Morton Heilig — *Sensorama*: máy đa giác quan (hình, tiếng, mùi, rung).
- **1968** Ivan Sutherland — *Sword of Damocles*: HMD đầu tiên có tracking đầu, treo trên trần.
- **1980** Jaron Lanier đặt ra từ "Virtual Reality"; VPL Research bán DataGlove, EyePhone.
- **1990** Tom Caudell đặt từ "Augmented Reality" tại Boeing.
- **1992** CAVE (UIC) — phòng VR dùng máy chiếu.
- **1997** Feiner — Touring Machine, AR di động đầu tiên.
- **1999** *ARToolKit* — thư viện AR dùng marker.
- **2010s** làn sóng tiêu dùng: Oculus Rift DK1, Google Glass, HTC Vive, HoloLens, PSVR.
- **2017+** ARKit, ARCore, Magic Leap, Quest 2/3, Vision Pro.
- **2019** chuẩn OpenXR 1.0 (Khronos).

1.3 Phần cứng VR

HMD (Kính thực tế ảo):

- *Tethered*: HTC Vive, Valve Index, PSVR (cần PC/console).
- *Standalone*: Meta Quest 2/3/Pro (chip di động, không cần PC).
- *Cao cấp*: Vision Pro, Varjo XR-3 (eye-tracking, micro-OLED).
- Thông số quan trọng: **FOV** 90–110°; **refresh** 72/90/120/144 Hz; **độ phân giải** mỗi mắt 2K–4K; **IPD** chỉnh được 58–72 mm.

Tracking:

- **DoF**: 3DoF chỉ xoay (yaw, pitch, roll); 6DoF có thêm tịnh tiến — chuẩn VR.
- **Outside-in**: cảm biến ngoài theo dõi HMD (Vive Lighthouse quét laser IR; Oculus Constellation dùng LED IR + camera). Chính xác cao nhưng phải lắp đặt.
- **Inside-out**: camera gắn trên HMD chạy SLAM (Quest, WMR). Linh hoạt nhưng cần ánh sáng + đặc trưng môi trường.
- **Lighthouse**: 2 trạm phát quét IR ngang + dọc; cảm biến quang trên HMD đo thời gian ⇒ tính pose.

Input: tay cầm 6DoF (Touch, Index), găng haptic (HaptX, Manus), bộ đồ toàn thân (Teslasuit), **máy chạy đa hướng** (Virtuix), bệ chuyển động/ghế.
Âm thanh: *không gian / binaural* qua HRTF; SDK: Steam Audio, Oculus Spatializer, Resonance Audio.

1.4 Phần cứng AR

Loại hiển thị:

- **Optical see-through (OST)**: nhìn xuyên qua quang học trong suốt (waveguide, gương bán phản); CG được chiếu thêm. Thiết bị: HoloLens 1/2, Magic Leap, Google Glass. Ưu: không trễ ảnh thật; Nhược: FOV nhỏ, tương phản kém ngoài sáng.
- **Video see-through (VST)**: camera chụp → ghép với CG → hiển thị trên màn hình kín. Thiết bị: AR điện thoại, passthrough Quest 3, Vision Pro. Ưu: occlusion tốt; Nhược: trễ, độ trung thực thấp hơn.
- **AR chiếu / không gian**: chiếu hình lên bề mặt thật (bảo tàng, projection mapping).
- **Cầm tay**: smartphone, tablet — phổ biến nhất.

Cảm biến:

Sensors:

- RGB camera (feature tracking), IMU (gyro+accel for orientation), depth: **ToF**, **structured light**, **LiDAR** (iPhone Pro / iPad Pro), stereo cameras.
- GPS + compass (outdoor AR like Pokemon GO).

Tracking modes: **marker-based** (fiducial markers, QR-like), **markerless** (natural features), **model-based** (CAD model), **SLAM** (build map on the fly).

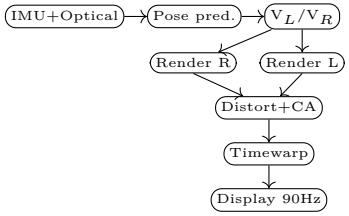
1.5 How VR Works (Pipeline)

Per-frame pipeline:

1. **Sensor sample**: HMD pose (IMU + optical) and controller poses.
2. **Prediction**: extrapolate pose to expected display time (reduce latency).
3. **View matrix update**: build V_L, V_R for left/right eye using IPD offset and head pose.
4. **Stereoscopic rendering**: render scene twice (one viewport per eye) into a stereo render target. Off-axis (asymmetric) frustum for each eye.
5. **Lens distortion correction**: apply *barrel pre-distortion* so that the lens's pincushion distortion cancels out; also chromatic aberration correction per RGB channel.
6. **Reprojection / Timewarp**: just before scanout, warp the rendered image using latest head pose (Asynchronous Timewarp = ATW; Asynchronous Spacewarp = ASW interpolates frames).
7. **Display scan-out** at 90/120 Hz; spatial audio mixed via HRTF.

Critical numbers:

- Refresh $\geq$ 90 Hz to feel smooth; 120 Hz preferred.
- **Motion-to-photon latency** $<$ 20 ms (ideal $<$ 11 ms) to avoid sickness.
- Stereo projection: $\mathbf{P}_{\text{eye}} = \mathbf{P}_{\text{proj}} \cdot \mathbf{T}(\pm\frac{IPD}{2}) \cdot \mathbf{V}_{\text{head}}$.
- **Foveated rendering**: render high-res only where the eye looks (eye-tracker), low-res in periphery $\Rightarrow$ 30–70% GPU saving (Quest Pro, PSVR2).



1.6 How AR Works (Pipeline)

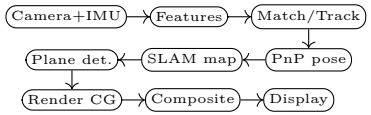
Generic AR pipeline:

1. **Capture**: camera frame + IMU sample.
2. **Pre-process**: undistort, grayscale, downsample.
3. **Feature detection**: corners (FAST/Harris), descriptors (ORB, SIFT, SURF, BRIEF).
4. **Tracking / pose estimation**:
   - Marker-based: detect square/QR-like marker, find 4 corners, compute homography $\mathbf{H}$, decompose to $[R|t]$ (PnP).
   - Markerless: match features to map, run **PnP** (Perspective-n-Point) for camera pose, refine with RANSAC.
5. **SLAM update**: insert keyframe, triangulate new map points, run **bundle adjustment** (joint optim of poses + points).
6. **Scene understanding**: plane detection (RANSAC on point cloud), depth from LiDAR/ToF, semantic segmentation, light estimation.
7. **Rendering**: project virtual model with same intrinsics K and pose $[R|t]$; handle **occlusion** via depth (real depth from sensor vs virtual z-buffer); composite over camera image (VST) or display via see-through.

Math kernels:

- Homography $\mathbf{x}' = \mathbf{H}\mathbf{x}$ (planar), 8 DoF.
- Camera projection $\mathbf{x} = K[R|t]\mathbf{X}$.
- PnP: $\geq$ 3 3D-2D correspondences $\Rightarrow$ pose.

SLAM variants: ORB-SLAM (feature-based), LSD-SLAM (direct), DSO, PTAM (parallel tracking + mapping), VINS-Mono (visual-inertial), ARKit/ARCore use **VIO** (Visual-Inertial Odometry).



- Camera RGB, IMU (gyro+accel), độ sâu: ToF, ánh sáng cấu trúc, **LiDAR** (iPhone Pro/iPad Pro), camera stereo.
- GPS + la bàn (AR ngoài trời, Pokemon GO).

Chế độ tracking: dùng marker (fiducial), không marker (đặc trưng tự nhiên), model-based (mô hình CAD), **SLAM** (tự dựng bản đồ).

1.5 Cách VR hoạt động (Pipeline)

Pipeline mỗi khung hình:

1. **Lấy mẫu cảm biến**: pose của HMD (IMU + quang học) và controller.
2. **Dự đoán**: ngoại suy pose tới thời điểm hiển thị $\Rightarrow$ giảm latency.
3. **Cập nhật ma trận view**: dựng V_L, V_R cho mắt trái/phải dùng độ lệch IPD và pose đầu.
4. **Render lập thể**: render cảnh 2 lần (mỗi mắt 1 viewport) vào stereo target. Mỗi mắt dùng frustum lệch trục (asymmetric).
5. **Hiệu chỉnh méo ống kính**: *barrel pre-distortion* ngược chiều méo pincushion của lens; cộng thêm hiệu chỉnh sắc sai cho từng kênh R,G,B.
6. **Reprojection / Timewarp**: ngay trước khi quét hình, warp ảnh với pose đầu mới nhất (ATW); ASW nội suy thêm khung.
7. **Quét ra màn hình** 90/120 Hz; trộn âm thanh không gian qua HRTF.

Con số quan trọng:

- Refresh $\geq$ 90 Hz mới mượt; 120 Hz tốt hơn.
- **Motion-to-photon latency** $<$ 20 ms (lý tưởng $<$ 11 ms) để không gây say.
- Phép chiếu lập thể: $\mathbf{P}_{\text{mắt}} = \mathbf{P}_{\text{proj}} \cdot \mathbf{T}(\pm\frac{IPD}{2}) \cdot \mathbf{V}_{\text{đầu}}$.
- **Foveated rendering**: chỉ render độ phân giải cao nơi mắt nhìn (cần eye-tracking), giảm 30–70% tải GPU.

1.6 Cách AR hoạt động (Pipeline)

Pipeline AR tổng quát:

1. **Chụp**: khung hình từ camera + mẫu IMU.
2. **Tiền xử lý**: gỡ méo, chuyển xám, giảm kích thước.
3. **Phát hiện đặc trưng**: góc (FAST/Harris), descriptor (ORB, SIFT, SURF, BRIEF).
4. **Tracking / ước lượng pose**:
   - Marker-based: tìm marker vuông/QR, lấy 4 góc, tính homography $\mathbf{H}$, phân rã thành $[R|t]$.
   - Markerless: khớp đặc trưng với bản đồ, chạy **PnP** để tìm pose camera, lọc bằng RANSAC.
5. **Cập nhật SLAM**: thêm keyframe, tam giác hóa điểm bản đồ mới, chạy **bundle adjustment**.
6. **Hiểu cảnh**: phát hiện mặt phẳng (RANSAC trên point cloud), độ sâu từ LiDAR/ToF, segment, ước lượng ánh sáng.
7. **Render**: chiếu mô hình ảo với cùng nội tham số K và pose $[R|t]$; xử lý **occlusion** bằng độ sâu thực vs z-buffer; ghép lên ảnh camera (VST) hoặc qua optical see-through.

Công thức toán:

- Homography $\mathbf{x}' = \mathbf{H}\mathbf{x}$ (mặt phẳng), 8 bậc tự do.
- Phép chiếu camera $\mathbf{x} = K[R|t]\mathbf{X}$.
- PnP: $\geq$ 3 tương ứng 3D-2D $\Rightarrow$ pose.

Biến thể SLAM: ORB-SLAM, LSD-SLAM, DSO, PTAM, VINS-Mono; ARKit/ARCore dùng **VIO** (kết hợp camera + IMU).

1.7 Tracking Techniques

- **Optical**: cameras + features/markers/LEDs. High accuracy, occlusion-prone.
- **Inertial (IMU)**: gyroscope (angular vel.), accelerometer (linear accel), magnetometer. High frequency (1 kHz), drifts over time.
- **Magnetic**: tracks coil in EM field (Polhemus). Immune to occlusion, distorted by metal.
- **Acoustic**: ultrasonic time-of-flight. Cheap, low precision.
- **Mechanical**: jointed arm encoders (very precise, restricted volume).
- **Hybrid / Sensor fusion**: complementary filter or **Extended Kalman Filter (EKF)** fusing IMU (fast, drifts) + vision (slow, absolute) ⇒ used in all modern HMDs.

CV building blocks:

- Feature detectors: Harris, FAST, DoG. Descriptors: SIFT (scale-invariant), SURF, ORB (binary, fast), BRIEF, AKAZE.
- Matching: brute-force / FLANN; Lowe's ratio test; RANSAC for outlier rejection.
- Homography (planar) vs Fundamental/Essential matrix (general epipolar geometry).
- Pose: solvePnP (OpenCV), Levenberg–Marquardt refinement.

1.8 Display Technologies

- **LCD**: cheap, full RGB sub-pixels, slower switch, backlight bleeding (Quest 2).
- **OLED**: deep blacks, fast response, but PenTile sub-pixel (loss of effective res), burn-in (PSVR1).
- **Micro-OLED (OLEDoS)**: silicon-backplane OLED, very high PPI (3000+ ppi), used in Vision Pro, Bigscreen Beyond.
- **LCoS / DLP**: used in projector-based AR.
- **Waveguides**: thin glass plates with diffraction gratings; light from micro-projector totally-internally-reflects to eye (HoloLens 2, Magic Leap).
- **Birdbath optics**: half-mirror combiner (Nreal/XReal Air).
- **Light-field displays**: multiple focal planes → correct vergence-accommodation (VAC) conflict (Magic Leap 1 had 2 planes).
- **Pancake lenses**: folded optics ⇒ thinner HMD (Quest Pro/3, Vision Pro) at cost of light efficiency.

Key metrics:

- **FOV** (field of view): horizontal × vertical degrees. Human ≈ 200° H.
- **IPD** (inter-pupillary distance) 54–74 mm; mismatch ⇒ eye strain.
- **PPD** (pixels per degree): retina-quality ≈ 60 PPD; Quest 2 ≈ 20, Vision Pro ≈ 34.
- **Screen-door effect (SDE)**: visible gaps between pixels.
- **God rays / glare**: stray light around bright objects (Fresnel lenses).
- **VAC**: eye *vergence* (where eyes converge) vs *accommodation* (focus depth) — fixed focal plane at ∼2 m causes conflict.

1.9 Software / SDKs / Tools

Game engines (course uses Unity):

- **Unity 3D**: *XR Interaction Toolkit*, *XR Plug-in Management*, OpenXR plug-in, AR Foundation (cross-platform AR over ARKit/ARCore).
- **Unreal Engine**: built-in OpenXR + ARKit/ARCore plug-ins; Niagara/MetaHuman.
- Godot 4 also has OpenXR.

AR SDKs:

- **Apple ARKit**: iOS/iPadOS; world tracking, plane detection, face/body tracking, RealityKit, LiDAR scene reconstruction.
- **Google ARCore**: Android; motion tracking, environmental understanding, light estimation, Geospatial API.
- **Vuforia**: Image targets, Model Targets, VuMarks; cross-platform, runs in Unity.
- **ARToolKit / ARToolKitX**: open-source fiducial-marker AR (oldest).
- **8th Wall, Zappar**: WebAR.
- **Microsoft MRTK**: Mixed Reality Toolkit for HoloLens/WMR.
- **Niantic Lightship**: AR cloud, persistent multiplayer.

VR / Standards:

- **OpenXR** (Khronos, 2019): cross-vendor API; replaces OpenVR/Oculus SDK lock-in.
- **OpenVR / SteamVR**: Valve's runtime.
- **Oculus Integration / Meta XR SDK**: Quest specific features.
- **WebXR Device API**: browser-based XR (Three.js, A-Frame, Babylon.js).

3D content: Blender, Maya, 3ds Max, ZBrush (sculpt), Substance (texture), Reality Capture / Meshroom (photogrammetry), Quill / Gravity Sketch (in-VR modeling).

1.10 Implementation in Unity (VR + AR)

VR project setup (XR Interaction Toolkit):

1. Create 3D project (URP recommended). *Edit* → *Project Settings* → *XR Plug-in Management* → install, tick **OpenXR** (PC) or Oculus (Android Quest).
2. Add Interaction Profiles in OpenXR settings (Oculus Touch, Valve Index, etc.).
3. Install *XR Interaction Toolkit* via Package Manager + Samples (Starter Assets).

1.7 Kỹ thuật tracking

- **Quang học**: camera + đặc trưng/marker/LED. Chính xác cao, dễ bị che.
- **Quán tính (IMU)**: con quay (vận tốc góc), gia tốc kế, từ kế. Tần số cao (1 kHz) nhưng trôi theo thời gian.
- **Từ trường**: theo dõi cuộn dây trong từ trường (Polhemus). Không bị che nhưng méo bởi kim loại.
- **Âm thanh**: siêu âm đo thời gian truyền. Rẻ, kém chính xác.
- **Cơ học**: cánh tay khớp có encoder (cực chính xác, vùng nhỏ).
- **Lai / Sensor fusion**: complementary filter hoặc **Extended Kalman Filter (EKF)** kết hợp IMU + vision ⇒ tất cả HMD hiện đại đều dùng.

Thị giác máy tính cơ bản:

- Phát hiện đặc trưng: Harris, FAST, DoG. Descriptor: SIFT (bất biến tỉ lệ), SURF, ORB (nhị phân, nhanh), BRIEF, AKAZE.
- Khớp: brute-force / FLANN; kiểm tra tỉ lệ Lowe; RANSAC loại điểm sai.
- Homography (mặt phẳng) vs ma trận Cơ bản/Bản chất (epipolar tổng quát).
- Pose: solvePnP (OpenCV), tinh chỉnh Levenberg–Marquardt.

1.8 Công nghệ hiển thị

- **LCD**: rẻ, sub-pixel RGB đầy đủ, chuyển chậm hơn, hở đèn nền (Quest 2).
- **OLED**: đen sâu, đáp ứng nhanh, nhưng PenTile làm giảm độ phân giải hiệu dụng, dễ burn-in (PSVR1).
- **Micro-OLED (OLEDoS)**: OLED trên đế silicon, PPI rất cao (3000+), dùng trong Vision Pro, Bigscreen Beyond.
- **LCoS / DLP**: dùng trong AR chiếu.
- **Waveguide**: tấm kính mỏng có grating; ánh sáng từ projector mini phản xạ toàn phần đến mắt (HoloLens 2, Magic Leap).
- **Birdbath**: gương bán phản (Nreal/XReal Air).
- **Light-field**: nhiều mặt tiêu cự → sửa xung đột vergence-accommodation.
- **Pancake lens**: thấu kính gập ⇒ HMD mỏng hơn (Quest Pro/3, Vision Pro), đánh đổi hiệu suất sáng.

Chỉ số quan trọng:

- **FOV**: độ ngang × dọc. Người ≈ 200° ngang.
- **IPD**: 54–74 mm; sai lệch gây mỏi mắt.
- **PPD**: chuẩn võng mạc ≈ 60; Quest 2 ≈ 20, Vision Pro ≈ 34.
- **Screen-door effect**: thấy khe giữa các pixel.
- **God rays**: vệt sáng quanh vật sáng (Fresnel).
- **VAC**: xung đột giữa hội tụ mắt và độ nét — gây mỏi sau lâu.

1.9 Phần mềm / SDK / Công cụ

Game engine (môn học dùng Unity):

- **Unity 3D**: *XR Interaction Toolkit*, *XR Plug-in Management*, OpenXR plug-in, AR Foundation (đa nền tảng qua ARKit/ARCore).
- **Unreal Engine**: tích hợp OpenXR + ARKit/ARCore; Niagara, MetaHuman.
- Godot 4 cũng có OpenXR.

SDK AR:

- **Apple ARKit**: iOS/iPadOS; world tracking, plane, mặt/cơ thể, RealityKit, dựng cảnh LiDAR.
- **Google ARCore**: Android; motion tracking, hiểu môi trường, ước lượng sáng, Geospatial API.
- **Vuforia**: Image Target, Model Target, VuMark; chạy trong Unity.
- **ARToolKit / ARToolKitX**: AR marker mã nguồn mở, cổ nhất.
- **8th Wall, Zappar**: WebAR.
- **Microsoft MRTK**: cho HoloLens/WMR.
- **Niantic Lightship**: AR cloud, multiplayer bền vững.

VR / Chuẩn:

- **OpenXR** (Khronos, 2019): API đa nhà sản xuất.
- **OpenVR / SteamVR**: runtime của Valve.
- **Oculus / Meta XR SDK**: tính năng riêng Quest.
- **WebXR**: XR trên trình duyệt (Three.js, A-Frame, Babylon.js).

Nội dung 3D: Blender, Maya, 3ds Max, ZBrush, Substance, Reality Capture / Meshroom (photogrammetry), Quill / Gravity Sketch (vẽ trong VR).

1.10 Triển khai trong Unity (VR + AR)

Cài VR (XR Interaction Toolkit):

1. Tạo project 3D (nên URP). *Edit* → *Project Settings* → *XR Plug-in Management* → cài, tick **OpenXR** (PC) hoặc Oculus (Quest).
2. Bật Interaction Profile trong OpenXR (Oculus Touch, Valve Index...).
3. Cài *XR Interaction Toolkit* qua Package Manager + Samples.

4. Replace Main Camera with **XR Origin (XR Rig)** prefab (camera offset + camera + LeftHand/RightHand controllers).
5. Add *XR Interaction Manager* to scene.
6. Add interactables: `XRGrabInteractable` on objects, *XR Ray Interactor* or *Direct Interactor* on hands.
7. **Locomotion**: add *Locomotion System*, *Teleportation Provider*, *Continuous Move/Turn Provider*; teleport areas/anchors on floor.
8. Build settings: Android → enable IL2CPP, ARM64, Oculus loader; deploy via adb.

AR project setup (AR Foundation):

1. Install AR Foundation + ARKit XR Plugin (iOS) and/or ARCore XR Plugin (Android).
2. Scene: **AR Session + XR Origin (AR)** (with AR Camera, AR Camera Manager, AR Camera Background).
3. Add *AR Plane Manager* for plane detection, *AR Raycast Manager* for hit-testing taps.
4. Place objects: cast ray from screen tap → `ARRaycastHit` → instantiate prefab as child of trackable, optionally add *AR Anchor*.
5. Image tracking: *AR Tracked Image Manager* + Reference Image Library.
6. Face tracking (front cam): *AR Face Manager*.
7. Build: iOS Xcode (camera usage description in Info.plist) / Android (min API 24, ARCore supported).

Tip: use *XR Device Simulator* package to test without headset.

1.11 Key Challenges

Cybersickness / Motion sickness:

- Causes: (1) sensory mismatch (eyes see motion, vestibular feels none) — **vection**; (2) **latency** > 20 ms; (3) low frame rate / dropped frames; (4) wrong IPD; (5) bad lens distortion; (6) artificial locomotion (smooth turn).
- Mitigations: high refresh (≥ 90 Hz), low motion-to-photon latency, **teleport** or snap-turn instead of smooth, vignette / tunneling during motion, stable horizon reference, comfort settings, fixed-rest frames (virtual nose, cockpit), proper IPD.

AR-specific:

- *Tracking drift* accumulating over time ⇒ loop closure, anchors, relocalization.
- *Occlusion* of virtual by real: requires depth map (LiDAR/ToF) or learned depth.
- *Lighting / shadow consistency*: estimate ambient light intensity + direction (ARKit Light Estimation, spherical harmonics).
- *Registration error*: small camera–display latency ⇒ swimming. OST-AR especially sensitive.

Hardware / UX:

- Battery, weight, heat, social acceptance ("glasshole"), privacy of always-on cameras, eye safety.
- *Vergence-Accommodation Conflict*, *God rays*, screen-door effect, FOV vs form-factor trade-off.

1.12 Applications (Domains)

- **Healthcare**: surgical training (Osso VR, FundamentalVR), *AR surgical navigation* (overlay CT/MRI on patient — Brainlab), exposure therapy (PTSD, phobias), pain distraction, stroke rehab, anatomy education (HoloAnatomy, Complete Anatomy).
- **Education / Training**: *flight simulators* (high-fidelity, FAA certified), military (squad training, JTAC), industrial (ABB welding, oil-rig safety), classroom (Labster, Google Expeditions / Arts & Culture).
- **Entertainment**: gaming (Half-Life: Alyx, Beat Saber, VRChat), immersive cinema (Notes on Blindness), 360-video, location-based VR (The VOID).
- **Architecture / CAD / AEC**: design walkthroughs (Twinmotion, Enscape), *BIM* visualization (Revit + Unity Reflect), client review.
- **Cultural heritage**: digital museums (*eMuseum*), virtual reconstruction of ruins (Pompeii VR, Angkor Wat), eTourism, AR guides on-site.
- **Retail**: virtual try-on (IKEA Place, Warby Parker, Sephora), product configurators, virtual showrooms.
- **Real estate**: Matterport 3D tours, virtual staging.
- **Remote collaboration**: Spatial, Horizon Workrooms, Microsoft Mesh, immersive telepresence.
- **Automotive**: design review (Ford), HUD AR, driver training.
- **Manufacturing**: AR maintenance instructions (PTC Vuforia, Scope AR), digital twins.

4. Thay Main Camera bằng prefab **XR Origin (XR Rig)** (camera offset + camera + 2 controller).
5. Thêm *XR Interaction Manager* vào scene.
6. Thêm interactable: `XRGrabInteractable` cho vật, *XR Ray Interactor* hoặc *Direct Interactor* cho tay.
7. **Di chuyển**: thêm *Locomotion System*, *Teleportation Provider*, *Continuous Move/Turn*; đặt teleport area trên sàn.
8. Build: Android, IL2CPP, ARM64, loader Oculus, deploy bằng adb.

Cài AR (AR Foundation):

1. Cài AR Foundation + ARKit (iOS) hoặc ARCore (Android).
2. Scene: **AR Session + XR Origin (AR)** (gồm AR Camera, Camera Manager, Camera Background).
3. Thêm *AR Plane Manager* để phát hiện mặt phẳng, *AR Raycast Manager* để hit-test.
4. Đặt vật: bắn ray từ điểm chạm → `ARRaycastHit` → instantiate prefab làm con của trackable, thêm *AR Anchor* nếu cần neo cố định.
5. Tracking ảnh: *AR Tracked Image Manager* + Reference Image Library.
6. Tracking khuôn mặt: *AR Face Manager* (camera trước).
7. Build: iOS (khai báo NSCameraUsageDescription) / Android (API ≥ 24, máy hỗ trợ ARCore).

Mẹo: dùng *XR Device Simulator* để test không cần kính.

1.11 Thách thức chính

Say VR (cybersickness):

- Nguyên nhân: (1) lệch giác quan (mắt thấy chuyển động, tai trong không) — **vection**; (2) **latency** > 20 ms; (3) FPS thấp / rớt khung; (4) sai IPD; (5) méo lens không sửa đúng; (6) di chuyển nhân tạo (smooth turn).
- Khắc phục: refresh ≥ 90 Hz, latency thấp, **teleport** hoặc snap-turn thay smooth, vignette/tunnel khi di chuyển, có chân trời ổn định, đặt IPD đúng, có khung tham chiếu (mũi ảo, cabin).

Riêng AR:

- *Trôi tracking* theo thời gian ⇒ loop closure, anchor, relocalization.
- *Occlusion*: cần depth map (LiDAR/ToF) hoặc deep learning.
- *Nhất quán ánh sáng/bóng*: ước lượng cường độ + hướng sáng (Light Estimation, spherical harmonics).
- *Lệch đăng ký*: trễ giữa camera và hiển thị ⇒ "trôi". OST-AR rất nhạy cảm.

Phần cứng / UX:

- Pin, trọng lượng, nhiệt, chấp nhận xã hội, riêng tư khi camera bật, an toàn mắt.
- *Vergence-Accommodation Conflict*, *god rays*, screen-door, đánh đổi FOV vs hình dáng.

1.12 Ứng dụng (lĩnh vực)

- **Y tế**: huấn luyện phẫu thuật (Osso VR, FundamentalVR), *điều hướng phẫu thuật AR* (chồng CT/MRI lên bệnh nhân — Brainlab), trị liệu phơi nhiễm (PTSD, ám ảnh), giảm đau, phục hồi đột quỵ, giáo dục giải phẫu (HoloAnatomy).
- **Giáo dục / Huấn luyện**: *mô phỏng bay* (FAA chứng nhận), quân sự (huấn luyện đội hình, JTAC), công nghiệp (hàn ABB, an toàn giàn khoan), lớp học (Labster, Google Arts & Culture).
- **Giải trí**: game (Half-Life: Alyx, Beat Saber, VRChat), phim immersive, 360-video, VR theo địa điểm (The VOID).
- **Kiến trúc / CAD / AEC**: dạo qua thiết kế (Twinmotion, Enscape), *BIM* (Revit + Unity Reflect), khách hàng duyệt mẫu.
- **Di sản văn hóa**: bảo tàng số (*eMuseum*), tái dựng phế tích (Pompeii VR, Angkor Wat), eTourism, hướng dẫn AR tại chỗ.
- **Bán lẻ**: thử đồ ảo (IKEA Place, Warby Parker, Sephora), cấu hình sản phẩm, showroom ảo.
- **Bất động sản**: tour 3D Matterport, staging ảo.
- **Cộng tác từ xa**: Spatial, Horizon Workrooms, Microsoft Mesh.
- **Ô tô**: review thiết kế (Ford), HUD AR, huấn luyện tài xế.
- **Sản xuất**: hướng dẫn bảo trì AR (Vuforia, Scope AR), digital twin.

1.13 Tools ↔ Techniques Mapping

Application	Recommended Tool	Key Technique
Surgical training	Unity + Oculus + haptic	6DoF tracking, force fb
AR surgery nav.	HoloLens + MRTK	Patient-CT registration, ICP
Anatomy edu.	HoloLens / Quest + MRTK / XRI	Volumetric render, hand tracking
Flight sim	Unreal + Varjo + motion seat	Stereo render, CAVE-like
Museum (eMuseum)	WebXR / Unity + ARFound.	Photogrammetry, image target
Heritage reconstr.	Blender + Unity + Quest	High-poly LOD, baked lighting
eTourism (city)	ARCore Geospatial / 8thWall	VPS, GPS+SLAM
BIM walkthrough	Unity Reflect / Twinmotion	IFC import, VR Rig
Furniture try-on	ARKit + RealityKit	Plane det., PBR shading
Industrial maint.	Vuforia Studio / Scope AR	Model target, work-order UI
Education AR book	Unity + Vuforia Image Target	Marker pose, animation
VR exposure ther.	Unity + Quest + biosensor	Procedural anxiety scenes
Remote meeting	Microsoft Mesh / Spatial	Avatar lip-sync, spatial audio

1.14 Performance & Evaluation Metrics

System metrics:

- **Latency**: motion-to-photon < 20 ms (target < 11 ms). End-to-end = sensor + processing + render + scan-out.
- **Refresh rate**: 72 / 90 / 120 / 144 Hz.
- **Jitter**: temporal variation in pose estimate.
- **FOV**: degrees H × V (90° entry-level VR, 35–50° AR-OST).
- **PPD** pixels per degree; **angular resolution**.
- **Tracking**: positional accuracy (mm), rotational (deg), drift (cm/min), volume (m^3).
- **Frame time budget**: 90 Hz ⇒ 11.1 ms / frame for both eyes.

User metrics (subjective):

- **SSQ** (Simulator Sickness Questionnaire) — Kennedy et al. — nausea, oculomotor, disorientation sub-scales.
- **IPQ** (Igroup Presence Questionnaire) — spatial presence, involvement, realism.
- **Witmer–Singer Presence Questionnaire**.
- **SUS** (Slater–Usoh–Steed) presence; or System Usability Scale.
- **NASA-TLX** for workload.
- Behavioral: time-on-task, error rate, EDA / heart rate.

1.15 Future Trends

- **Foveated rendering** + eye tracking standard (Quest Pro, PSVR2, Vision Pro).
- **Pancake lenses** ⇒ thinner, lighter HMD.
- **Varifocal / light-field displays** solve VAC.
- **Neural rendering**: NeRF, 3D Gaussian Splatting for photo-real scene capture and immersive replay; Instant-NGP for real-time NeRF.
- **AI-driven content**: text-to-3D (DreamFusion, GET3D), AI avatars, generative environments, ML-based hand/body tracking.
- **Passthrough MR** convergence (Quest 3, Vision Pro) — VR and AR on same device.
- **Metaverse / persistent worlds**: shared social VR (Horizon Worlds, VRChat, Roblox).
- **Cloud / edge XR rendering**: 5G + GeForce NOW-style streaming.
- **Brain-Computer Interfaces (BCI)**: Neuralink, Galea (EEG/EMG); thought-driven UI.
- **Haptics evolution**: ultrasound mid-air haptics (Ultraleap), electro-tactile.
- **Digital twins** of cities and factories (NVIDIA Omniverse, USD).
- **Standardization**: OpenXR ecosystem; **glTF** + **USD** for asset interop.
- **Privacy / ethics**: gaze data, biometric identification, mixed-reality bystanders.

1.13 Bảng mapping Công cụ ↔ Kỹ thuật

Ứng dụng	Công cụ đề xuất	Kỹ thuật chính
Huấn luyện PT	Unity + Oculus + haptic	Tracking 6DoF, lực phản hồi
PT AR điều hướng	HoloLens + MRTK	Đăng ký CT-bệnh nhân, ICP
Giải phẫu học	HoloLens / Quest + MRTK / XRI	Render thể tích, hand tracking
Mô phỏng bay	Unreal + Varjo + ghế động	Stereo render, kiểu CAVE
Bảo tàng (eMuseum)	WebXR / Unity + ARFound.	Photogrammetry, image target
Tái dựng di sản	Blender + Unity + Quest	LOD đa cấp, bake ánh sáng
eTourism thành phố	ARCore Geospatial / 8thWall	VPS, GPS+SLAM
BIM walkthrough	Unity Reflect / Twinmotion	Import IFC, VR Rig
Thử đồ nội thất	ARKit + RealityKit	Plane det., shader PBR
Bảo trì CN	Vuforia / Scope AR	Model target, UI lệnh việc
Sách AR giáo dục	Unity + Vuforia Image Target	Pose marker, animation
VR phơi nhiễm	Unity + Quest + biosensor	Sinh cảnh lo âu thủ tục
Họp từ xa	Microsoft Mesh / Spatial	Avatar lip-sync, âm thanh KG

1.14 Hiệu năng & Đánh giá

Đo hệ thống:

- **Latency**: motion-to-photon < 20 ms (mục tiêu < 11 ms). Tổng = cảm biến + xử lý + render + scan-out.
- **Refresh**: 72 / 90 / 120 / 144 Hz.
- **Jitter**: dao động pose theo thời gian.
- **FOV**: H × V độ (VR cơ bản 90°, AR-OST 35–50°).
- **PPD**; độ phân giải góc.
- **Tracking**: chính xác vị trí (mm), góc (deg), drift (cm/phút), thể tích (m^3).
- **Ngân sách thời gian khung**: 90 Hz ⇒ 11.1 ms/khung cho cả 2 mắt.

Đo người dùng (chủ quan):

- **SSQ** — say mô phỏng (buồn nôn, thị giác, mất phương hướng).
- **IPQ** — presence (không gian, nhập vai, hiện thực).
- **Witmer–Singer PQ**.
- **SUS** (Slater–Usoh–Steed) presence; hoặc System Usability Scale.
- **NASA-TLX** đo tải nhận thức.
- Hành vi: thời gian, tỉ lệ lỗi, EDA / nhịp tim.

1.15 Xu hướng tương lai

- **Foveated rendering** + eye tracking thành tiêu chuẩn (Quest Pro, PSVR2, Vision Pro).
- **Thấu kính pancake** ⇒ HMD mỏng nhẹ.
- **Varifocal / light-field** giải quyết VAC.
- **Neural rendering**: NeRF, 3D Gaussian Splatting cho dựng cảnh siêu thực; Instant-NGP cho NeRF thời gian thực.
- **Nội dung AI**: text-to-3D (DreamFusion, GET3D), avatar AI, môi trường sinh, tracking tay/cơ thể bằng ML.
- **MR passthrough hội tụ**: Quest 3, Vision Pro — VR và AR trên cùng thiết bị.
- **Metaverse / thế giới bền**: VR xã hội (Horizon Worlds, VRChat, Roblox).
- **Render đám mây/biên**: 5G + streaming kiểu GeForce NOW.
- **BCI**: Neuralink, Galea (EEG/EMG); UI điều khiển bằng suy nghĩ.
- **Haptic mới**: siêu âm chạm không khí (Ultraleap), điện-xúc giác.
- **Digital twin** của thành phố, nhà máy (NVIDIA Omniverse, USD).
- **Chuẩn hóa**: OpenXR; **glTF** + **USD** cho liên thông tài sản.
- **Riêng tư / đạo đức**: dữ liệu gaze, định danh sinh trắc, người ngoài cuộc trong MR.

1.16 Quick Reference Numbers

Quantity	Min OK	Good
Refresh rate	72 Hz	90–120 Hz
M2P latency	< 20 ms	< 11 ms
FOV (VR)	90°	$110^\circ+$
FOV (AR-OST)	30°	$50^\circ+$
PPD	15	30–60
IPD range	60–66 mm	55–72 mm
Tracking accuracy	5 mm / 1°	< 1 mm / 0.1°
Frame time @ 90 Hz	11.1 ms	—

Common abbreviations: HMD, FOV, IPD, PPD, DoF, IMU, SLAM, VIO, VST, OST, MR, XR, ATW, ASW, VAC, SDE, SSQ, IPQ, NeRF, LiDAR, ToF, HRTF, BCI.

1.16 Tham chiếu nhanh

Đại lượng	Tối thiểu	Tốt
Tần số quét	72 Hz	90–120 Hz
Trễ M2P	< 20 ms	< 11 ms
FOV (VR)	90°	$110^\circ+$
FOV (AR-OST)	30°	$50^\circ+$
PPD	15	30–60
Khoảng IPD	60–66 mm	55–72 mm
Độ chính xác tracking	5 mm / 1°	< 1 mm / 0.1°
Thời gian khung @ 90 Hz	11.1 ms	—

Viết tắt thường gặp: HMD, FOV, IPD, PPD, DoF, IMU, SLAM, VIO, VST, OST, MR, XR, ATW, ASW, VAC, SDE, SSQ, IPQ, NeRF, LiDAR, ToF, HRTF, BCI.

2. Geometric Modeling

2.1 Overview

Geometric modeling = mathematical & algorithmic representation, construction, and manipulation of 2D/3D shapes for graphics, CAD/CAM, simulation, 3D printing, VR/AR, medical imaging.

Representations:

- **Point cloud**: unordered set $P = \{p_i \in \mathbb{R}^3\}$. From LiDAR, photogrammetry, RGB-D. No topology.
- **Polygonal mesh**: vertices V, edges E, faces F. Triangle (most common, planar, GPU friendly), quad (subdivision/animation), n-gon. Euler: $V - E + F = 2 - 2g$ (g=genus). *Manifold*: every edge incident to ≤ 2 faces; vertex link is a disk/half-disk. *Watertight*=closed manifold (no boundary).
- **Parametric surfaces**: $S(u, v) = \sum B_i(u)B_j(v)P_{ij}$. Bezier, B-spline, NURBS. Smooth, exact, but harder to render directly.
- **Implicit / level sets**: zero-set $\{x : f(x) = 0\}$. SDF (signed distance field), metaballs. Easy boolean ops, free topology change.
- **Volumetric / voxels**: 3D grid V_{ijk}. CT/MRI, sparse octree, OpenVDB. High memory.
- **CSG (Constructive Solid Geometry)**: tree of $\cup, \cap, \setminus$ on primitives.
- **Subdivision surfaces**: Catmull-Clark, Loop, Doo-Sabin — refine coarse mesh to smooth limit.

Pipeline: acquisition → registration/alignment → denoising/outlier removal → surface reconstruction (triangulation) → hole filling/repair → simplification/remeshing → parameterization/UV → analysis (curvature, geodesics) → rendering / export.

File formats:

- **OBJ** (Wavefront): ASCII, `v/vn/vt/f`, no topology guarantee.
- **PLY** (Stanford): ASCII or binary, header-driven, supports per-vertex color/normal/custom attrs.
- **STL**: triangles only, no shared vertices, used for 3D printing.
- **OFF**: simple ASCII (vertex count, face count, then data).
- **FBX** (Autodesk): rigging, animation, materials.
- **DAE** (COLLADA): XML, exchange format.
- **glTF/GLB**: web/runtime, PBR materials.

2.2 Mesh Simplification

Goal: reduce triangle count $|F|$ from $N \to N' \ll N$ while preserving shape, attributes (UV, color, normal), and topology where possible.

Algorithms:

1. **Vertex clustering** (Rossignac & Borrel 1993). Overlay 3D grid; collapse all vertices in each cell to one representative (centroid or quadric-optimal). $O(N)$, very fast, low quality, topology may change. Used as out-of-core preprocess.
2. **Vertex decimation** (Schroeder 1992). Iteratively remove "removable" vertices (classified by curvature/feature angle), retriangulate hole. Keeps subset of original vertices.
3. **Edge collapse + Quadric Error Metric (QEM)** (Garland & Heckbert 1997). Most popular, high quality. *Math:*
   - Each face has plane $\pi = (a, b, c, d)^T$ with $a^2 + b^2 + c^2 = 1$. Squared distance from $v = (x, y, z, 1)^T$ to plane: $(\pi^T v)^2 = v^T(\pi\pi^T)v$.
   - Per-vertex *quadric* $Q_v = \sum_{\pi \in \text{planes}(v)} \pi\pi^T$ (a 4×4 symmetric PSD).
   - Cost of collapsing edge $(v_i, v_j) \to \bar{v}$: $E(\bar{v}) = \bar{v}^T(Q_i + Q_j)\bar{v}$.
   - Optimal $\bar{v}$: solve $\partial E/\partial \bar{v} = 0$, i.e., set top 3×4 block of $Q_i + Q_j$ to identity-target; if invertible, $\bar{v} = Q'^{-1}(0, 0, 0, 1)^T$. Else fall back to midpoint or endpoint with minimum E.
   - Maintain min-heap keyed on E; pop, collapse, update neighbors' quadrics ($Q_{\bar{v}} = Q_i + Q_j$) and edge costs.
4. **Progressive meshes** (Hoppe 1996). Store base mesh M^0 + sequence of vertex splits $\{vs_i\}$ enabling continuous LOD, streaming, geomorph.
5. **Edge collapse with attribute quadrics** (extended QEM): augment Q to handle color, UV, normals as extra dims.

2. Mô hình hình học

2.1 Tổng quan

Geometric modeling = biểu diễn và xử lý toán học/thuật toán cho hình dạng 2D/3D, dùng trong đồ họa, CAD/CAM, mô phỏng, in 3D, VR/AR, ảnh y khoa.

Các dạng biểu diễn:

- **Đám mây điểm (point cloud)**: tập điểm rời rạc $P = \{p_i\}$. Từ LiDAR, photogrammetry, RGB-D. Không có topology.
- **Lưới đa giác (mesh)**: đỉnh V, cạnh E, mặt F. Tam giác phổ biến nhất; quad cho subdivision/animation. Euler: $V - E + F = 2 - 2g$. *Manifold*: mỗi cạnh ≤ 2 mặt kề; link đỉnh là đĩa. *Watertight*=manifold kín, không biên.
- **Mặt tham số**: Bezier, B-spline, NURBS — mượt, chính xác, khó render trực tiếp.
- **Implicit/level set**: $\{x : f(x) = 0\}$. SDF, metaballs. Dễ phép boolean, đổi topology tự do.
- **Voxel/khối**: lưới 3D, dùng CT/MRI, octree, OpenVDB. Tốn bộ nhớ.
- **CSG**: cây boolean $\cup, \cap, \setminus$ trên các khối nguyên thủy.
- **Subdivision**: Catmull-Clark, Loop, Doo-Sabin — tinh chỉnh lưới thô thành mặt mượt.

Pipeline: thu nhận → căn chỉnh (registration) → khử nhiễu/loại outlier → tái tạo bề mặt (triangulation) → lấp lỗ/sửa lưới → đơn giản hóa/remesh → tham số hóa UV → phân tích (độ cong, geodesic) → render/xuất.

Định dạng file:

- **OBJ**: ASCII, dòng `v/vn/vt/f`.
- **PLY**: ASCII hoặc nhị phân, header linh hoạt, hỗ trợ thuộc tính tùy biến.
- **STL**: chỉ tam giác, không chia sẻ đỉnh, in 3D.
- **OFF**: ASCII đơn giản.
- **FBX**: rigging, animation.
- **DAE**: XML, format trao đổi.
- **glTF/GLB**: chuẩn web/runtime, vật liệu PBR.

2.2 Đơn giản hóa lưới

Mục tiêu: giảm số tam giác $|F|$ trong khi giữ hình dạng, thuộc tính (UV, màu, pháp tuyến), và topology.

Thuật toán:

1. **Vertex clustering** (Rossignac & Borrel 1993). Phủ lưới 3D grid; mọi đỉnh trong cùng cell gộp làm một (centroid hoặc QEM-optimal). $O(N)$, rất nhanh, chất lượng thấp, có thể đổi topology. Hữu dụng tiền xử lý dữ liệu lớn.
2. **Vertex decimation** (Schroeder 1992). Lặp xóa các đỉnh "có thể xóa" (đánh giá theo độ cong, góc đặc trưng), tam giác hóa lại lỗ. Giữ tập con đỉnh gốc.
3. **Edge collapse + QEM** (Garland & Heckbert 1997). Phổ biến nhất, chất lượng cao.
   - Mỗi mặt có mặt phẳng $\pi = (a, b, c, d)^T$, $a^2 + b^2 + c^2 = 1$. Khoảng cách bình phương từ v đến π: $(\pi^T v)^2 = v^T(\pi\pi^T)v$.
   - Quadric mỗi đỉnh: $Q_v = \sum_\pi \pi\pi^T$ (4×4 đối xứng PSD).
   - Chi phí gộp cạnh $(v_i, v_j) \to \bar{v}$: $E(\bar{v}) = \bar{v}^T(Q_i + Q_j)\bar{v}$.
   - Tối ưu $\bar{v}$ bằng giải $\partial E/\partial \bar{v} = 0$; nếu suy biến dùng trung điểm.
   - Heap nhỏ nhất theo E; pop, collapse, cập nhật Q và chi phí lân cận.
4. **Progressive meshes** (Hoppe 1996). Lưu mesh gốc M^0 và chuỗi vertex split, hỗ trợ LOD liên tục, streaming, geomorph.
5. **QEM mở rộng**: Q có thêm dimension cho màu/UV/normal.

Đo chất lượng:

- **Hausdorff**: $d_H(A, B) = \max(\sup \inf, \sup \inf)$.
- **RMS**: căn quân phương khoảng cách (Metro tool).
- Lệch normal, đánh giá thị giác.

Ứng dụng: LOD real-time, streaming mạng, mobile/VR, collision proxy, mip-map hình học.

Quality metrics:

- **Hausdorff distance**: $d_H(A,B) = \max(\sup_{a\in A}\inf_{b\in B}\|a-b\|, \sup_b \inf_a \|a-b\|)$.
- **RMS error**: $\sqrt{\frac{1}{|A|}\int_A d(a,B)^2 dA}$ — Metro tool (Cignoni 1998).
- Normal angle deviation, visual fidelity (perceptual).

Applications: real-time rendering LOD, network streaming, mobile/VR, collision proxy, mip-map of geometry.

2.3 Triangulation (Point Cloud → Mesh)

Delaunay triangulation (DT):

- *Definition (empty-circle property)*: triangulation T of $P \subset \mathbb{R}^2$ is Delaunay iff for every triangle, its circumcircle contains no other point of P in its interior. 3D: empty circumsphere.
- *Optimality*: maximizes the minimum angle ⇒ avoids slivers (best in lex-order of angle vectors). Unique if no 4 cocircular points.
- *Dual* of **Voronoi diagram**: $V_i = \{x : \|x-p_i\| \le \|x-p_j\| \forall j\}$. Connect p_i, p_j iff their Voronoi cells share an edge.
- *Complexity (2D)*: $O(N \log N)$.
- *Algorithms*:
  - **Incremental insertion + edge flip** (Lawson). Insert point, locate triangle, split into 3, flip non-Delaunay edges (use in-circle test).
  - **Bowyer–Watson** (incremental, dual view, see pseudocode).
  - **Divide & Conquer** (Guibas & Stolfi) $O(N \log N)$.
  - **Sweep-line** (Fortune, primarily Voronoi).
- *In-circle predicate* for triangle abc and query d (CCW): d is inside circumcircle iff

$$\det\begin{pmatrix} a_x-d_x & a_y-d_y & \|a-d\|^2 \\ b_x-d_x & b_y-d_y & \|b-d\|^2 \\ c_x-d_x & c_y-d_y & \|c-d\|^2 \end{pmatrix} > 0.$$

Bowyer–Watson algorithm:

Input: Points P
Output: Delaunay triangulation T

```
T ← {super-triangle containing all P}  foreach point p ∈ P do
    B ← {t ∈ T : p ∈ circumcircle(t)} ;                    // bad triangles
    P ← boundary edges of B that are not shared by two bad triangles ;   // polygon hole
    T ← T \ B  foreach edge e ∈ P do
        add triangle formed by e and p to T
    end
end
remove all triangles touching the super-triangle  return T
```

Algorithm: Bowyer–Watson Delaunay

Constrained Delaunay (CDT): enforce specified edges; relaxation: edge (u,v) is locally Delaunay if no constrained edge separates u, v from a fourth point. Used in GIS, mesh generation.

3D point-cloud surface reconstruction:

- **Ball Pivoting Algorithm (BPA)** (Bernardini 1999). Pivot a ball of radius ρ around boundary edges; whenever it touches three points without containing any others, form triangle. Sensitive to ρ vs sampling density. $O(N \log N)$ with KD-tree.
- **Poisson Surface Reconstruction** (Kazhdan, Bolitho, Hoppe 2006). Treat oriented points as samples of a vector field $\vec{V}$ approximating gradient of indicator function χ; solve $\nabla\cdot\nabla\chi = \nabla\cdot\vec{V}$ on octree, extract iso-surface via Marching Cubes. Smooth, watertight, robust to noise. *Screened Poisson* (2013) adds point-fitting term.
- **α-shapes** (Edelsbrunner). Generalization of convex hull: from DT keep simplices with circumradius $\le \alpha$. $\alpha \to \infty$: convex hull; $\alpha \to 0$: just points.
- **Marching Cubes** (Lorensen & Cline 1987). For implicit/scalar volume: per cube of 8 corners, sign pattern indexes into 256-entry lookup table (15 base cases by symmetry); produce 0–4 triangles. Linearly interpolate iso-vertex on edges. *Marching Tetrahedra* avoids ambiguities.
- **Greedy Projection Triangulation** (Marton, PCL). Project local neighborhood to tangent plane (from PCA normal), incrementally grow front by adding triangles that meet angle/edge-length thresholds.
- **Convex hull / Gift wrap / QuickHull**: only for convex point sets. 3D QuickHull is $O(N \log N)$ avg.
- **Crust / Cocone** (Amenta, Bern, Dey): theoretically guaranteed reconstruction from ϵ-sampled smooth surfaces, uses DT and Voronoi poles.

2.4 Hole Filling

Boundary detection (manifold mesh):

- Half-edge with no twin / edge with exactly 1 incident face ⇒ *boundary edge*.
- Walk boundary edges to extract *boundary loops* (each hole = one loop).
- Algorithm: pick unmarked boundary edge; from its end vertex, follow next boundary edge around the umbrella until back to start.

Hole filling families:

2.3 Tam giác hóa (Point Cloud → Lưới)

Delaunay triangulation (DT):

- *Định nghĩa (vòng tròn rỗng)*: tam giác hóa T của $P \subset \mathbb{R}^2$ là Delaunay khi mỗi tam giác có vòng ngoại tiếp không chứa điểm khác bên trong. 3D: hình cầu ngoại tiếp rỗng.
- *Tính tối ưu*: cực đại góc nhỏ nhất ⇒ tránh tam giác "lưỡi dao". Duy nhất nếu không có 4 điểm đồng viên.
- *Đối ngẫu của **Voronoi***: cell $V_i = \{x : \|x-p_i\| \le \|x-p_j\|\}$; nối p_i, p_j nếu cell chia cạnh.
- Độ phức tạp 2D: $O(N \log N)$.
- Thuật toán:
  - **Incremental + edge flip** (Lawson): chèn điểm, định vị, chia 3 tam giác, flip cạnh không Delaunay (in-circle test).
  - **Bowyer–Watson** (incremental).
  - **Chia để trị** $O(N \log N)$.
  - **Sweep-line** (Fortune, chủ yếu cho Voronoi).
- *Phép thử in-circle* cho abc, d (CCW): định thức 3×3 với cột là lệch tọa độ và bình phương khoảng cách; $> 0 \Leftrightarrow d$ trong vòng ngoại tiếp.

Bowyer–Watson: (xem pseudocode bên cột tiếng Anh)

1. Khởi tạo T với super-triangle bao toàn bộ P.
2. Với mỗi p: tìm tam giác "xấu" có p trong vòng ngoại tiếp; lấy biên đa giác từ chúng; xóa và nối lại với p.
3. Cuối cùng xóa tam giác kề super-triangle.

Constrained Delaunay (CDT): ép giữ các cạnh chỉ định; locally Delaunay khi không có cạnh constraint chia cách. Dùng trong GIS, sinh lưới.

Tái tạo bề mặt từ point cloud 3D:

- **Ball Pivoting (BPA)** (Bernardini 1999): lăn quả bóng bán kính ρ quanh cạnh biên; khi chạm 3 điểm mà không chứa điểm khác → tạo tam giác. Phải chọn ρ phù hợp mật độ.
- **Poisson Reconstruction** (Kazhdan 2006): coi điểm có normal là gradient của hàm chỉ thị χ; giải $\nabla\cdot\nabla\chi = \nabla\cdot\vec{V}$ trên octree, trích iso-surface bằng Marching Cubes. Mượt, watertight, kháng nhiễu. *Screened Poisson* (2013) thêm constraint khớp điểm.
- **α-shapes**: tổng quát hóa bao lồi; từ DT giữ simplex có bán kính ngoại tiếp $\le \alpha$.
- **Marching Cubes** (1987): mỗi cube 8 góc, dấu các góc index vào bảng 256 (15 case cơ bản); sinh 0–4 tam giác, nội suy tuyến tính trên cạnh. *Marching Tetrahedra* tránh ambiguity.
- **Greedy Projection** (PCL/Marton): chiếu vùng lân cận lên mặt phẳng tiếp tuyến, mở rộng front theo ngưỡng góc/độ dài.
- **Convex hull** (QuickHull 3D): chỉ với tập lồi.
- **Crust / Cocone** (Amenta): có đảm bảo lý thuyết với ϵ-sampling từ mặt mượt; dùng Voronoi poles.

2.4 Lấp lỗ (Hole Filling)

Phát hiện biên (mesh manifold):

- Half-edge không có twin / cạnh có đúng 1 mặt ⇒ cạnh biên.
- Đi vòng các cạnh biên để trích boundary loop (mỗi lỗ = 1 loop).
- Thuật toán: chọn cạnh biên chưa đánh dấu; từ đỉnh cuối, đi cạnh biên tiếp theo trong umbrella đến khi quay lại.

Các họ thuật toán:

- **Liepa (2003)**: classic 3-stage triangulation–refinement–fairing.
  1. *Min-weight triangulation* of boundary loop. Weight $w(t)$=(area, dihedral) lex. DP: $W(i,j)=\min_{i<m<j} W(i,m)+W(m,j)+w(\triangle v_i v_m v_j)$, $O(n^3)$.
  2. *Refinement*: subdivide patch triangles whose edge length far from surrounding density (relax via centroid-split + edge swap).
  3. *Fairing*: minimize membrane/thin-plate energy via Laplacian/biharmonic smoothing on patch interior, boundary fixed.
- **Advancing front**: start at boundary, repeatedly add best triangle (smallest angle / closest pair) until front closes.
- **RBF-based** (Carr, Branch & Olague): fit radial basis function $\phi(\|x-x_i\|)$ on boundary points + normals, evaluate iso-surface to plug hole. Smooth, C^∞, expensive globally.
- **Poisson hole filling** (Zhao, Gao, Lin 2007): solve Poisson equation for vertex positions over patch.
- **Volumetric** (Curless & Levoy 1996): rasterize mesh to SDF in voxel grid, diffuse/extend SDF into hole, extract surface via Marching Cubes. Robust but loses detail.
- **Context-based / template-based** (Sharf, Pauly, Bischoff): search the mesh for self-similar patches and copy.
- **MLS-based** (Jun 2005): moving least squares fit on boundary normals/positions.

NFD — Normal Field Diffusion (user's project, MeshLab plugin):

1. **Step 1 — Hole detection**: find boundary loops by walking border half-edges; each loop L_h stored as ordered vertex indices, validated by size $3 \le |L_h| \le$ `MaxHoleSize`.
2. **Step 2 — Boundary analysis**: for each boundary vertex, compute *normal* (area-weighted from one-ring) and *discrete mean curvature*.
3. **Step 3 — Initial triangulation**: build local frame from average boundary normal $\bar{\mathbf{n}} = \sum \mathbf{n}_i / \|\cdot\|$, project boundary loop to 2D plane (u,v), triangulate (2D Delaunay/ear-clipping), insert interior Steiner vertices on a regular grid; lift back to 3D as flat patch.
4. **Step 4 — Normal field diffusion (CORE)**: solve heat equation on patch graph for each component $k \in \{x,y,z\}$ of normal, $\partial n_k/\partial t = \Delta n_k$, Dirichlet BC $n_k|_{\partial\Omega} = n_{k,\text{fix}}$. Discretize with cotangent Laplacian $L_{ij} = \frac{1}{2}(\cot\alpha_{ij} + \cot\beta_{ij})$ and *implicit Euler*: $(I-\lambda L)\mathbf{n}_k^{t+1} = \mathbf{n}_k^t$. Cholesky/$LDL^T$ via Eigen `SimplicialLDLT`; unconditionally stable. Re-normalize $\|\mathbf{n}\| = 1$ per vertex after iterations.
5. **Step 5 — Laplacian smoothing of positions** (uniform, boundary fixed) — 3 iters to even spacing.
6. **Step 6 — Curvature-guided displacement (spherical-cap)**:
   - Estimate cap height h from boundary geometry: $h = R(1-\cos\bar{\theta})$ with R = mean boundary distance to centroid, $\bar{\theta} = \text{avg}_i \arccos(\mathbf{n}_i \cdot \bar{\mathbf{n}})$ (average is more robust than max).
   - Geodesic distance d_i from boundary via Dijkstra (multi-source); normalize $t_i = d_i / \max d_j \in [0,1]$.
   - Weight profile $w(t) = \sqrt{2t - t^2}$ (spherical cap, $w(0)=0, w(1)=1, w'(0)=\infty \Rightarrow C^1$ at boundary).
   - Displace: $v_i' = v_i + h\, w(t_i)\, \mathbf{n}_i^{\text{diffused}}$ (per-vertex direction $\Rightarrow$ patch follows local curvature).
7. **Step 7 — Patch merging**: re-index boundary to original mesh vertices (no duplicates), add interior vertices/faces, average colors from neighborhood for visual continuity. Optional Taubin $(\lambda,\mu) = (0.40, -0.44)$ to remove spikes.

Quality criteria: continuity (C^0 position match, C^1 tangent match, C^2 curvature match), smoothness (low Dirichlet/bending energy), no self-intersection, feature preservation.

Challenges: complex hole topology (islands, multiply-connected), sharp features, large holes, non-manifold input, mesh with handles.

2.5 Noise Removal / Mesh & Point Cloud Denoising

Sources of noise: scanner sensor noise (Kinect $\sigma \sim$ mm), registration misalignment, photogrammetry artifacts, quantization, reconstruction artifacts.

Mesh smoothing:

- **Uniform Laplacian**: $\mathbf{v}_i^{(k+1)} = \mathbf{v}_i^{(k)} + \lambda L(\mathbf{v}_i)$, $L(\mathbf{v}_i) = \frac{1}{|N(i)|}\sum_{j\in N(i)}(\mathbf{v}_j - \mathbf{v}_i)$. Causes *shrinkage* (volume loss) and over-smoothing of features.
- **Cotangent Laplacian** (more accurate, conformal): $L(\mathbf{v}_i) = \frac{1}{2A_i}\sum_j(\cot\alpha_{ij} + \cot\beta_{ij})(\mathbf{v}_j - \mathbf{v}_i)$, A_i = Voronoi area. Discretizes mean curvature: $\Delta\mathbf{v} = -2H\mathbf{n}$.
- **Taubin $\lambda|\mu$ smoothing** (1995): alternate $\lambda > 0$ shrink and $\mu < -\lambda < 0$ inflate, $|\mu| > \lambda$. Ideally $\frac{1}{\lambda} + \frac{1}{\mu} = k_{pb}$ (pass-band) keeps low-freq, kills high-freq. Eliminates shrinkage.
- **Bilateral mesh denoising** (Fleishman 2003 / Jones 2003): per vertex v with normal $\mathbf{n}$, neighbors $\{q_i\}$:
$$\hat{v} = v + \mathbf{n}\cdot\frac{\sum_i w_c(\|q_i - v\|)\, w_s(\langle\mathbf{n}, q_i - v\rangle)\,\langle\mathbf{n}, q_i - v\rangle}{\sum_i w_c\, w_s}$$
  where $w_c(d) = e^{-d^2/2\sigma_c^2}$ (spatial), $w_s(h) = e^{-h^2/2\sigma_s^2}$ (signal). Edge-preserving (anisotropic).
- **Mean curvature flow**: $\partial\mathbf{x}/\partial t = -H\,\mathbf{n}$. Implicit (Desbrun 1999): $(I - \lambda L_{cot})\mathbf{x}^{t+1} = \mathbf{x}^t$ — unconditionally stable.
- **Anisotropic diffusion** (Desbrun, Hildebrandt): diffusion coefficient depends on local feature ($\to 0$ near sharp edges).
- **Bilateral normal filter** (Zheng 2011): smooth normals first, then update positions to match.
- **L_0 mesh smoothing** (He & Schaefer 2013): minimize $\sum_e \|D(e)\|_0$ — preserves planar regions, sharp edges.
- **Robust statistics**: replace mean by median in Laplacian for outlier resilience.

Point cloud denoising:

- **Statistical Outlier Removal (SOR)** (PCL): for each point compute mean distance μ_i to its k nearest neighbors; global $\mu = \text{avg}(\mu_i)$, $\sigma = \text{std}(\mu_i)$; remove p_i if $\mu_i > \mu + \alpha\sigma$ (typical $k = 20, \alpha = 1.0$).

- **Liepa (2003)**: ba giai đoạn.
  1. *Min-weight triangulation* loop biên: trọng số (diện tích, góc dihedral) thứ tự lex; DP $O(n^3)$ với $W(i,j) = \min_m W(i,m) + W(m,j) + w(\triangle)$.
  2. *Refinement*: chia nhỏ tam giác có cạnh quá lớn so với mật độ xung quanh.
  3. *Fairing*: tối thiểu năng lượng màng/uốn bằng Laplacian/biharmonic, biên cố định.
- **Advancing front**: từ biên, lặp thêm tam giác tốt nhất (góc nhỏ nhất / cặp gần nhất) đến khi front đóng.
- **RBF**: khớp $\phi(\|x - x_i\|)$ trên điểm/normal biên, lấy iso-surface; mượt C^∞.
- **Poisson**: giải Poisson cho vị trí đỉnh trên patch.
- **Volumetric** (Curless–Levoy 1996): rasterize SDF trên voxel grid, diffuse SDF vào lỗ, Marching Cubes. Chắc chắn, mất chi tiết.
- **Template/context-based**: tìm patch tự tương tự trên mesh và sao chép.
- **MLS** (Jun 2005): moving least squares trên normal/vị trí biên.

NFD — Normal Field Diffusion (project của bạn, plugin MeshLab):

1. **Bước 1 — Phát hiện lỗ**: đi vòng border half-edge; mỗi loop là dãy chỉ số đỉnh, kiểm tra $3 \le |L_h| \le$ `MaxHoleSize`.
2. **Bước 2 — Phân tích biên**: tính normal (area-weighted một-vòng) và mean curvature rời rạc cho từng đỉnh biên.
3. **Bước 3 — Tam giác hóa khởi tạo**: dựng frame từ $\bar{\mathbf{n}} = \sum \mathbf{n}_i / \|\cdot\|$, chiếu loop xuống 2D, tam giác hóa (Delaunay/ear-clip), thêm Steiner trong lưới đều, nâng lên 3D thành patch phẳng.
4. **Bước 4 — Normal Field Diffusion (LÕI)**: giải heat equation $\partial n_k/\partial t = \Delta n_k$ (mỗi component k), Dirichlet $n_k|_{\partial\Omega}$ cố định. Rời rạc bằng cotangent Laplacian $L_{ij} = \frac{1}{2}(\cot\alpha + \cot\beta)$, *implicit Euler*: $(I - \lambda L)\mathbf{n}^{t+1} = \mathbf{n}^t$. Cholesky LDL^T qua Eigen `SimplicialLDLT`; ổn định vô điều kiện. Chuẩn hóa $\|\mathbf{n}\| = 1$.
5. **Bước 5 — Laplacian smoothing vị trí** (uniform, biên cố định, 3 iter) để đều khoảng cách.
6. **Bước 6 — Dịch chuyển theo độ cong (chỏm cầu)**:
   - Cao độ vòm $h = R(1 - \cos\bar{\theta})$, R = bán kính trung bình từ centroid, $\bar{\theta}$ = trung bình $\arccos(\mathbf{n}_i \cdot \bar{\mathbf{n}})$ (trung bình bền hơn max).
   - Khoảng cách geodesic d_i từ biên (Dijkstra multi-source), chuẩn hóa $t_i = d_i / \max d_j$.
   - Trọng số $w(t) = \sqrt{2t - t^2}$ (chỏm cầu, $w(0) = 0$, $w'(0) = \infty \Rightarrow C^1$ ở biên).
   - Dịch chuyển: $v_i' = v_i + h\, w(t_i)\, \mathbf{n}_i^{\text{diffused}}$.
7. **Bước 7 — Ghép patch**: tái-index biên về đỉnh gốc của mesh (không tạo trùng), thêm đỉnh/mặt nội bộ, lấy màu trung bình lân cận. Tùy chọn Taubin $(\lambda,\mu) = (0.40, -0.44)$ để khử gai.

Tiêu chí chất lượng: liên tục $C^0/C^1/C^2$, mượt (Dirichlet/bending energy thấp), không tự cắt, giữ chi tiết.

Thách thức: lỗ topology phức tạp (đảo, đa liên thông), chi tiết sắc, lỗ lớn, mesh không manifold, mesh có quai.

2.5 Khử nhiễu lưới và point cloud

Nguồn nhiễu: cảm biến scanner (Kinect $\sigma \sim$ mm), lệch registration, photogrammetry, lượng tử hóa, artifact tái tạo.

Smoothing trên mesh:

- **Laplacian đều**: $\mathbf{v}_i^{(k+1)} = \mathbf{v}_i^{(k)} + \lambda L(\mathbf{v}_i)$, $L(\mathbf{v}_i) = \frac{1}{|N(i)|}\sum(\mathbf{v}_j - \mathbf{v}_i)$. Gây *co rút* thể tích, làm mòn chi tiết.
- **Cotangent Laplacian**: $L(\mathbf{v}_i) = \frac{1}{2A_i}\sum(\cot\alpha + \cot\beta)(\mathbf{v}_j - \mathbf{v}_i)$, A_i = diện tích Voronoi. Rời rạc mean curvature: $\Delta\mathbf{v} = -2H\mathbf{n}$.
- **Taubin $\lambda|\mu$**: xen kẽ $\lambda > 0$ (co) và $\mu < -\lambda$ (giãn). Điều kiện $|\mu| > \lambda$. Pass-band $\frac{1}{\lambda} + \frac{1}{\mu} = k_{pb}$. Tránh co rút.
- **Bilateral mesh denoising** (Fleishman/Jones 2003): với v có normal $\mathbf{n}$, $\hat{v} = v + \mathbf{n}\cdot\frac{\sum w_c\, w_s\, \langle\mathbf{n}, q_i - v\rangle}{\sum w_c\, w_s}$, w_c = Gaussian không gian, w_s = Gaussian tín hiệu — bảo toàn cạnh.
- **Mean curvature flow**: $\partial\mathbf{x}/\partial t = -H\mathbf{n}$. Implicit (Desbrun 1999): $(I - \lambda L_{cot})\mathbf{x}^{t+1} = \mathbf{x}^t$, ổn định vô điều kiện.
- **Khuếch tán bất đẳng hướng**: hệ số khuếch tán nhỏ ở cạnh sắc.
- **Bilateral normal filter** (Zheng 2011): mượt normal trước, cập nhật vị trí khớp.
- **L_0 smoothing** (He–Schaefer 2013): tối thiểu $\|D(e)\|_0$, giữ vùng phẳng và cạnh.
- **Thống kê bền**: dùng median thay mean để chống outlier.

Khử nhiễu point cloud:

- **SOR**: tính μ_i = trung bình khoảng cách đến k-NN; loại p_i nếu $\mu_i > \mu + \alpha\sigma$ (thường $k = 20, \alpha = 1.0$).
- **Radius outlier**: loại điểm có $< n_{\min}$ neighbor trong bán kính r.
- **Voxel downsampling**: thay điểm trong mỗi voxel bằng centroid.
- **MLS** (Levin): khớp đa thức cục bộ trên mặt phẳng tiếp tuyến (PCA), chiếu điểm.
- **WLOP** (Huang 2009): chiếu điểm đến tập sạch và phân bố đều, kết hợp lực hút và đẩy.
- **PCA normal**: hiệp phương sai k-NN, eigenvector của giá trị riêng nhỏ nhất là normal; định hướng nhất quán bằng MST.
- **Bilateral** cho point cloud.

- **Radius outlier removal**: discard point with $< n_{\min}$ neighbors within radius r.
- **Voxel-grid downsampling**: replace points in each voxel by centroid (uniform density).
- **Moving Least Squares (MLS)** (Levin, Alexa): fit local polynomial $g(u, v)$ on tangent plane (PCA), project point onto g. Smooths and resamples.
- **WLOP (Weighted Locally Optimal Projection)** (Huang 2009): iteratively project points to a clean, evenly distributed set; minimizes attraction $\sum \|x_i - p_j\|\theta(\cdot)$ + repulsion.
- **PCA normal estimation**: covariance of k-NN, smallest eigenvalue's eigenvector = normal; sign disambiguated by Riemannian propagation (MST).
- **Bilateral point cloud filter**: similar to mesh bilateral but on neighborhoods.
- **RIMLS** (Robust Implicit MLS, Öztireli 2009): robust kernel (Geman-McClure) preserves features.
- **Learning-based**: PointCleanNet, Total Denoising, score-based diffusion (Luo & Hu 2021).

Edge-preserving = bilateral, anisotropic, L_0, RIMLS.

Quality measures: Hausdorff distance, RMS-to-ground-truth, normal angle error $\arccos(\hat{\mathbf{n}} \cdot \mathbf{n}_{gt})$, feature preservation (chamfer on edge sets).

2.6 Worked Example

(A) QEM edge collapse on 4-vertex tetra-fan.
Vertices $v_1 = (0, 0, 0), v_2 = (1, 0, 0), v_3 = (0, 1, 0), v_4 = (1, 1, 0), v_5 = (0.5, 0.5, 1)$. Faces all share v_5 apex on a square base.
Step 1. Plane of base face $v_1v_2v_3$: normal $(0, 0, 1)$, $d = 0$, so $\pi = (0, 0, 1, 0)^T$, $\pi\pi^T = \mathrm{diag}(0, 0, 1, 0)$.
Step 2. Compute $Q_i = \sum_{\pi \in F(i)} \pi\pi^T$ for each vertex. For base vertices, Q_i accumulates plane $z = 0$ once + side planes.
Step 3. Candidate edge (v_1, v_2). $Q_{12} = Q_1 + Q_2$. Solve $\partial(\bar{v}^T Q_{12} \bar{v})/\partial\bar{v} = 0$ by setting top 3×4 rows of Q_{12} replaced as $[I \,|\, 0]$ targeted to $(0, 0, 0, 1)^T$. If singular fall back: try midpoint $(0.5, 0, 0)$ and endpoints, pick min E. Suppose $E_{mid} = 0.012$ is smallest. Push $(v_1, v_2, 0.012, (0.5, 0, 0))$ to heap. Repeat for all edges; pop min, collapse, update incident Q's.
(B) Bowyer–Watson on 4 points $A = (0, 0), B = (2, 0), C = (2, 2), D = (1, 1)$.
Init super-triangle S enclosing all (e.g., huge $(-100, -100)$, $(100, -100)$, $(0, 100)$). $T = \{S\}$.
Insert A: S contains A in circumcircle (yes); bad=$\{S\}$, polygon = 3 edges of S; new $T = \{SA_1, SA_2, SA_3\}$ (three triangles around A).
Insert B: Find triangles whose circumcircle contains B (some subset). Remove, retriangulate hole boundary fan around B.
Insert C: similar.
Insert D: $D = (1, 1)$ may lie inside circumcircle of $\triangle ABC$ (yes, since ABC is right-isoceles with circumcircle through them; check: circumcenter of $A, B, C = (1, 1)$, radius $= \sqrt{2}$; D at center is inside) so $\triangle ABC$ is bad; remove, retriangulate cavity into $\triangle ABD, \triangle BCD, \triangle CAD$.
Cleanup: delete every triangle touching a super-triangle vertex. Final DT $= \{ABD, BCD, CAD\}$ (the unique DT of these 4 points if no 4 cocircular, else flip-equivalent).
(C) Laplacian smoothing — 5-vertex umbrella. Center $v_0 = (0, 0, 1)$, ring $v_1 = (1, 0, 0), v_2 = (0, 1, 0), v_3 = (-1, 0, 0), v_4 = (0, -1, 0)$. Uniform Laplacian on v_0: $L(v_0) = \frac{1}{4}\sum(v_i - v_0) = \frac{1}{4}(0, 0, -4) = (0, 0, -1)$. With $\lambda = 0.5$: $v_0' = (0, 0, 1) + 0.5 \cdot (0, 0, -1) = (0, 0, 0.5)$ — moved halfway toward ring centroid. Repeat $\rightarrow$ shrinkage.
To prevent shrinkage, follow with Taubin $\mu = -0.53$: $v_0'' = v_0' + (-0.53) \cdot L(v_0')$. Recompute $L(v_0') = \frac{1}{4}\sum(v_i - v_0') = \frac{1}{4}(0, 0, -2) = (0, 0, -0.5)$, so $v_0'' = (0, 0, 0.5) + (-0.53)(0, 0, -0.5) = (0, 0, 0.765)$ — partially restored.

- **RIMLS** (Öztireli 2009): kernel bền (Geman-McClure) giữ chi tiết.
- **Học máy**: PointCleanNet, score-based (Luo & Hu 2021).

Bảo toàn cạnh = bilateral, anisotropic, L_0, RIMLS.
Đo chất lượng: Hausdorff, RMS so với ground-truth, sai số góc normal, chamfer trên tập cạnh.

2.6 Ví dụ tính toán

(A) QEM edge collapse trên fan 5 đỉnh.
Đỉnh: $v_1 = (0, 0, 0)$, $v_2 = (1, 0, 0)$, $v_3 = (0, 1, 0)$, $v_4 = (1, 1, 0)$, đỉnh chóp $v_5 = (0.5, 0.5, 1)$.
B1. Mặt phẳng đáy có normal $(0, 0, 1)$, $\pi = (0, 0, 1, 0)$, $\pi\pi^T = \mathrm{diag}(0, 0, 1, 0)$.
B2. $Q_i = \sum_{\pi \in F(i)} \pi\pi^T$ cộng dồn các mặt kề.
B3. Cạnh (v_1, v_2): $Q_{12} = Q_1 + Q_2$. Giải $\nabla(\bar{v}^T Q_{12} \bar{v}) = 0$. Nếu suy biến, thử trung điểm $(0.5, 0, 0)$ và hai endpoint, chọn E nhỏ nhất. Đẩy vào heap; pop nhỏ nhất; collapse; cập nhật Q các đỉnh kề.
(B) Bowyer–Watson 4 điểm $A = (0, 0), B = (2, 0), C = (2, 2), D = (1, 1)$.
Init: super-triangle S rất lớn bao toàn bộ. $T = \{S\}$.
Chèn A: S có A trong vòng ngoại tiếp; bad=$\{S\}$; xóa S, tạo 3 tam giác từ 3 cạnh S với A.
Chèn B, C: tương tự.
Chèn D $= (1, 1)$*:* $\triangle ABC$ có tâm $(1, 1)$, bán kính $\sqrt{2}$, D ở chính tâm $\Rightarrow$ trong vòng. $\triangle ABC$ bad; xóa, lấp khoang bằng $\triangle ABD, \triangle BCD, \triangle CAD$.
Cleanup: xóa tam giác kề super vertex. DT cuối cùng: $\{ABD, BCD, CAD\}$.
(C) Laplacian smoothing — umbrella 5 đỉnh.
Tâm v_0=$(0, 0, 1)$; vòng $(1, 0, 0)$, $(0, 1, 0)$, $(-1, 0, 0)$, $(0, -1, 0)$.
$L(v_0) = \frac{1}{4}\sum(v_i - v_0) = (0, 0, -1)$. Với $\lambda = 0.5$: $v_0' = (0, 0, 0.5)$ — di chuyển nửa đường về centroid vòng (co rút).
Bù bằng Taubin $\mu = -0.53$: $L(v_0') = (0, 0, -0.5)$; $v_0'' = (0, 0, 0.5) + (-0.53)(0, 0, -0.5) = (0, 0, 0.765)$ — phục hồi một phần.

2.7 Quick Reference Table

Algorithm	Input	Output	Cmplx.	Key param.	When to use
Vertex clustering	mesh	coarse mesh	$O(N)$	grid cell size	out-of-core, fast LOD
Vertex decimation	mesh	coarse mesh	$O(N \log N)$	feature angle	moderate quality
QEM edge collapse	mesh	coarse mesh	$O(N \log N)$ heap	target #faces	best quality LOD
Progressive mesh	mesh	PM stream	$O(N \log N)$	—	streaming, geomorph
Delaunay (B-W)	2D points	DT	$O(N \log N)$ avg	—	quality 2D mesh
Constrained DT	pts + edges	CDT	$O((N + k) \log N)$	constraint set	GIS, FEM domain
Ball Pivoting	oriented pts	mesh	$O(N \log N)$	ball radius ρ	uniform-density scan
Poisson Recon	oriented pts	watertight mesh	$O(N \log N)$ octree	depth d, samp/node	noisy scans
α-shapes	3D pts	mesh	$O(N^2)$ 3D	α	coarse hull family
Marching Cubes	scalar volume	mesh	$O(K^3)$	iso-value	SDF/volume data
Greedy proj. tri.	oriented pts	mesh	$O(N \log N)$	search radius, ang.	PCL pipeline
Liepa hole fill	hole loop	patch	$O(n^3)$ DP	weight (A, θ)	generic small holes
Advancing front	loop	patch	$O(n^2)$	angle threshold	moderate holes
RBF fill	loop + nrm	patch	$O(n^3)$ solve	kernel ϕ	smooth C^∞
Volumetric fill	mesh	repaired mesh	$O(K^3)$ voxels	voxel size	topology fix
NFD hole fill	mesh+hole	patch	$O(n)$ Cholesky	λ, iters	curvature-aware
Uniform Laplacian	mesh	smoothed	$O(N)$/iter	λ, iter	quick smooth
Cotangent Lap.	mesh	smoothed	$O(N)$/iter	λ, iter	accurate, conformal
Taubin $\lambda\|\mu$	mesh	smoothed	$O(N)$/iter	λ, μ	no shrink
Bilateral mesh	mesh	smoothed	$O(Nk)$	σ_c, σ_s	feature-preserving
Mean curv. flow	mesh	smoothed	solve/iter	step Δt	PDE smoothing
L_0 smoothing	mesh	smoothed	ADMM iters	β	sharp-edge cleanup
SOR	point cloud	cleaner	$O(Nk)$ kNN	k, α	remove outliers
Radius outlier	pts	cleaner	$O(N)$ kdtree	$r, n_{\min}$	sparse outliers
Voxel grid	pts	down-sampled	$O(N)$	voxel size	uniform resamp.
MLS proj.	pts	smooth pts	$O(Nk)$	poly. deg	smooth+resamp
WLOP	pts	even pts	iterative	h, μ	even distribution
PCA normal est.	pts	normals	$O(Nk)$	k	normal init for recon
RIMLS	pts	smooth pts	iter	robust σ_r	feature preserving

2.8 Memory Aids / Pitfalls

- *Cotangent weights* can be *negative* on obtuse triangles → matrix loses M-property, possible flips. Mitigate via intrinsic Delaunay.
- *Implicit Euler* for diffusion: stable for any Δt but each step needs sparse solve; precompute Cholesky factor once if topology fixed.
- *QEM optimal* $\bar{v}$: always check matrix invertibility; degenerate ⇒ midpoint fallback.
- *Hole detection on non-manifold* edges: an edge with ≥ 3 incident faces is non-manifold, not a hole; pre-clean.
- *Marching Cubes ambiguity*: face/internal ambiguity ⇒ cracks; use asymptotic decider or Marching Tetrahedra.

2.7 Bảng tham chiếu nhanh

Thuật toán	Đầu vào	Đầu ra	Phức tạp	T.số chính	Khi nào dùng
Vertex clustering	mesh	mesh thô	$O(N)$	cỡ ô grid	LOD nhanh, dữ liệu lớn
Vertex decimation	mesh	mesh thô	$O(N \log N)$	góc đặc trưng	chất lượng vừa
QEM collapse	mesh	mesh thô	$O(N \log N)$ heap	số face đích	LOD chất lượng cao
Progressive mesh	mesh	PM	$O(N \log N)$	—	streaming, geomorph
Delaunay (B-W)	điểm 2D	DT	$O(N \log N)$	—	lưới 2D chất lượng
CDT	điểm + cạnh	CDT	$O((N + k) \log N)$	tập constraint	GIS, FEM
Ball Pivoting	điểm + normal	mesh	$O(N \log N)$	bán kính ρ	scan đều mật độ
Poisson Recon	điểm + normal	mesh kín	$O(N \log N)$	depth, samp/node	scan nhiễu
α-shapes	điểm 3D	mesh	$O(N^2)$	α	họ bao thô
Marching Cubes	volume	mesh	$O(K^3)$	iso-value	SDF/volume
Greedy proj.	điểm + normal	mesh	$O(N \log N)$	bán kính, góc	pipeline PCL
Liepa fill	loop biên	patch	$O(n^3)$ DP	(A, θ)	lỗ nhỏ tổng quát
Advancing front	loop	patch	$O(n^2)$	ngưỡng góc	lỗ vừa
RBF fill	loop + nrm	patch	$O(n^3)$	kernel ϕ	lỗ mượt C^∞
Volumetric fill	mesh	mesh sửa	$O(K^3)$ voxel	cỡ voxel	sửa topology
NFD fill	mesh+lỗ	patch	$O(n)$ Cholesky	λ, số iter	lỗ theo độ cong
Laplacian đều	mesh	mượt	$O(N)$/iter	λ, iter	smooth nhanh
Cotangent Lap.	mesh	mượt	$O(N)$/iter	λ, iter	chính xác, conformal
Taubin $\lambda\|\mu$	mesh	mượt	$O(N)$/iter	λ, μ	không co rút
Bilateral mesh	mesh	mượt	$O(Nk)$	σ_c, σ_s	giữ cạnh
Mean curv. flow	mesh	mượt	solve/iter	Δt	smooth PDE
L_0 smoothing	mesh	mượt	ADMM	β	dọn cạnh sắc
SOR	point cloud	sạch hơn	$O(Nk)$	k, α	loại outlier
Radius outlier	điểm	sạch hơn	$O(N)$ kdtree	$r, n_{\min}$	outlier rời rạc
Voxel grid	điểm	downsample	$O(N)$	cỡ voxel	lấy mẫu đều
MLS proj.	điểm	mượt	$O(Nk)$	bậc đa thức	smooth+resamp
WLOP	điểm	đều	lặp	h, μ	phân bố đều
PCA normal	điểm	normal	$O(Nk)$	k	init normal
RIMLS	điểm	mượt	lặp	σ_r bền	giữ chi tiết

2.8 Mẹo nhớ / Bẫy thường gặp

- Trọng số *cotangent* có thể âm trên tam giác tù ⇒ ma trận mất M-property, có thể flip. Khắc phục bằng intrinsic Delaunay.
- *Implicit Euler* ổn định mọi Δt nhưng mỗi bước phải solve sparse; precompute Cholesky nếu topology cố định.
- *QEM* $\bar{v}$ tối ưu: luôn kiểm tra suy biến; nếu suy biến dùng trung điểm.
- Phát hiện lỗ trên mesh không manifold: cạnh có ≥ 3 mặt là non-manifold, không phải lỗ; cần dọn trước.
- *Marching Cubes ambiguity*: face/internal ⇒ lỗ rạn; dùng asymptotic decider hoặc MT.
- *Taubin* cần $|\mu| > \lambda$; nếu $\mu = -\lambda$ thì pass-band $k_{pb} = 0$, không smooth.

- *Taubin*: requires $|\mu| > \lambda$; if $\mu = -\lambda$ pass-band $k_{pb} = 0$ no smoothing.
- *Laplacian* on irregular mesh = uniform $\neq$ cotangent; uniform behaves like graph diffusion (curvature flow only on regular meshes).
- *Hausdorff* is asymmetric in general; use 2-sided max.
- *Poisson recon* blurs sharp features (low-pass); try *screened* variant or feature-aware methods.
- *Boundary loops $\neq$ holes 1-1* when surface has multiple holes sharing topology (e.g., torus with bites).

- Laplacian đều $\neq$ cotangent trên mesh không đều; uniform = graph diffusion.
- *Hausdorff* bất đối xứng; nên dùng max hai chiều.
- *Poisson recon* làm mờ chi tiết sắc; dùng *screened* hoặc feature-aware.
- Boundary loop $\neq$ lỗ 1-1 khi topology phức (ví dụ torus bị cắn).

3. Medical Image Processing

3.1 Imaging Modalities

MRI (Magnetic Resonance Imaging).

- *Physics:* hydrogen (^{1}H) protons in water/fat align with strong static field B_0 (1.5T or 3T). RF pulse at Larmor freq. $\omega_0 = \gamma B_0$ tips magnetization; gradient coils (G_x, G_y, G_z) encode space; receive coil samples FID.
- *Relaxation:* T_1 longitudinal recovery, T_2 transverse decay, T_2^* (with field inhomogeneity), PD (proton density).
- *Sequences:* T1w, T2w, FLAIR (fluid-attenuated, suppresses CSF), DWI (diffusion), MRA (angiography), fMRI BOLD (blood oxygen level dependent), MRS (spectroscopy), SWI.
- *Strengths:* excellent soft-tissue contrast; no ionizing radiation; multiplanar.
- *Weaknesses:* slow ($\sim$30 min), expensive, claustrophobia, contraindication with metal/pacemakers.
- *Output:* k-space (Fourier domain) $\to$ DICOM volume.

CT (Computed Tomography).

- *Physics:* rotating X-ray source + arc detector; multi-angle line integrals of attenuation $\mu(x, y)$ form sinogram $p(t, \theta)$.
- *Reconstruction:* Filtered Back Projection (FBP) using Radon inverse; or iterative (ART, SIRT, MLEM, MBIR), now deep-learning recon.
- *Hounsfield Units:* HU $= 1000\frac{\mu-\mu_w}{\mu_w}$. Air -1000, fat -100, water 0, soft tissue $-100..+100$, bone $+400..+1000$, metal $> +2000$.
- Fast (<1 s/slice), high spatial resolution, but ionizing.
- CT angiography (CTA), perfusion CT, dual-energy CT, cone-beam CT (CBCT).

Ultrasound (US).

- Piezoelectric transducer emits 2–15 MHz pulses; echo time $\to$ depth $d = ct/2$ ($c \approx 1540$ m/s in soft tissue).
- Modes: A-mode (1D amplitude), B-mode (2D brightness), M-mode (motion), Doppler (flow velocity via freq shift).
- Real-time, cheap, no radiation, portable; operator-dependent, poor through bone/air. Use: obstetrics, cardiac (echo), abdominal, vascular.

PET (Positron Emission Tomography).

- Radiotracer (^{18}F-FDG, ^{11}C, ^{68}Ga) emits β^+; positron annihilates with electron $\to$ pair of 511 keV γ photons at $\sim 180°$; ring detectors record coincidences.
- Functional/metabolic imaging (glucose uptake $\Rightarrow$ tumor activity).
- Hybrid PET-CT and PET-MRI co-register anatomy + function.

X-ray (planar radiography, mammography). 2D projection of attenuation; cheap; bone/lung screening; mammography uses low-energy X-ray for breast soft tissue.

Other modalities (brief).

- **SPECT:** γ-emitting tracer (Tc-99m); cheaper than PET, lower res.
- **OCT:** optical coherence tomography, low-coherence interferometry, micrometer resolution; ophthalmology, dermatology.
- **Fluoroscopy:** continuous X-ray for real-time guidance (catheter, GI).
- **Endoscopy, microscopy (H&E, IHC) — pathology.**

DICOM (Digital Imaging and Communications in Medicine). File = header (tag-VR-length-value triples: PatientID, Modality, PixelSpacing, SliceThickness, RescaleIntercept/Slope, StudyUID, SeriesUID, SOPInstanceUID) + pixel data. Network protocol DIMSE. PACS (Picture Archive and Communication System) stores; HL7/FHIR for textual records.

3. Xử lý ảnh y tế

3.1 Phương thức tạo ảnh

MRI (Cộng hưởng từ).

- *Vật lý:* proton ^{1}H trong nước/mỡ xếp theo từ trường tĩnh B_0 (1.5T hoặc 3T). Xung RF tại tần số Larmor $\omega_0 = \gamma B_0$ làm lệch từ hóa; cuộn gradient G_x, G_y, G_z mã hóa không gian; cuộn thu nhận tín hiệu FID.
- *Hồi phục:* T_1 dọc, T_2 ngang, T_2^* (kèm bất đồng nhất từ trường), PD (mật độ proton).
- *Chuỗi xung:* T1w, T2w, FLAIR (xóa dịch não tủy), DWI (khuếch tán), MRA (mạch máu), fMRI BOLD (oxy hóa máu), MRS (phổ), SWI.
- *Ưu:* tương phản mô mềm xuất sắc; không bức xạ ion hóa; đa mặt phẳng.
- *Nhược:* chậm ($\sim$30 phút), đắt, sợ kín, chống chỉ định với kim loại/máy tạo nhịp.
- *Đầu ra:* k-space (miền Fourier) $\to$ thể tích DICOM.

CT (Cắt lớp vi tính).

- *Vật lý:* ống X-quang quay + dãy đầu dò; tích phân đường suy giảm $\mu(x, y)$ ở nhiều góc $\to$ sinogram $p(t, \theta)$.
- *Tái tạo:* Filtered Back Projection (FBP) qua biến đổi Radon ngược; hoặc lặp (ART, SIRT, MLEM, MBIR), nay có deep-learning recon.
- *Hounsfield Unit:* HU $= 1000\frac{\mu-\mu_w}{\mu_w}$. Khí -1000, mỡ -100, nước 0, mô mềm $-100..+100$, xương $+400..+1000$, kim loại $> +2000$.
- Nhanh (<1 s/lát), độ phân giải cao, nhưng có bức xạ.
- CT mạch máu (CTA), CT tưới máu, CT đa năng lượng, CBCT (cone-beam).

Siêu âm (US).

- Đầu dò áp điện phát xung 2–15 MHz; thời gian dội $\to$ độ sâu $d = ct/2$ ($c \approx 1540$ m/s trong mô mềm).
- Chế độ: A-mode (biên độ 1D), B-mode (sáng 2D), M-mode (chuyển động), Doppler (vận tốc dòng theo dịch chuyển tần số).
- Thời gian thực, rẻ, không bức xạ, di động; phụ thuộc kỹ thuật viên, kém qua xương/khí. Dùng: sản phụ khoa, tim mạch, ổ bụng, mạch máu.

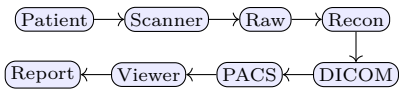
PET (Cắt lớp phát xạ positron).

- Chất đánh dấu phóng xạ (^{18}F-FDG, ^{11}C, ^{68}Ga) phát β^+; positron hủy với electron $\to$ cặp photon γ 511 keV cách nhau $\sim 180°$; vòng đầu dò ghi nhận trùng hợp.

Mod.	Physics	Res.	Rad.	Speed	Best for
MRI	H+RF+B_0	~1 mm	no	slow	soft tissue, brain
CT	X-ray	<0.5 mm	yes	fast	bone, lung, trauma
US	sound echo	~0.5 mm	no	real-time	OB, cardiac
PET	β^+ tracer	~4 mm	yes	medium	metabolism, oncology
X-ray	X-ray 2D	~0.1 mm	yes	instant	chest, bone, mammo
SPECT	γ tracer	~8 mm	yes	medium	cardiac, bone scan
OCT	near-IR light	μm	no	fast	retina, skin

3.2 Acquisition → Reconstruction Pipeline

Patient prep (fasting, contrast injection, positioning) → scan (raw signal: *k-space* for MRI, *sinogram* for CT, RF echo for US, list-mode coincidences for PET) → reconstruction → post-processing (filter, MPR, MIP) → DICOM tag-write → PACS server → workstation/viewer (radiologist) → structured report (HL7/FHIR) → EMR.



3.3 Reconstruction Methods

CT — Filtered Back Projection (FBP). Sinogram $p(t,\theta)$ where $t = x\cos\theta + y\sin\theta$.

1. 1D Fourier transform of each projection: $P(\omega,\theta) = \mathcal{F}_t\{p\}$.
2. Apply ramp filter $|\omega|$ (often Shepp-Logan / Hann window): $\tilde{P} = |\omega|P$.
3. Inverse 1D FT → $\tilde{p}(t,\theta)$.
4. Back-project:

$$f(x,y) = \int_0^\pi \tilde{p}(x\cos\theta + y\sin\theta, \theta)\, d\theta$$

Central Slice Theorem: 1D FT of projection at angle θ = central slice of 2D FT of f.

- Hình ảnh chức năng/chuyển hóa (hấp thu glucose ⇒ hoạt động khối u).
- PET-CT và PET-MRI lai ghép giải phẫu + chức năng.

X-quang (chụp phẳng, chụp nhũ ảnh). Hình chiếu 2D của hệ số suy giảm; rẻ; tầm soát xương/phổi; nhũ ảnh dùng X-quang năng lượng thấp cho mô mềm vú.

Các phương thức khác (tóm tắt).

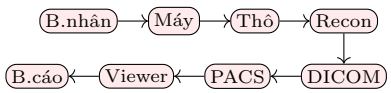
- **SPECT:** chất đánh dấu phát γ (Tc-99m); rẻ hơn PET, độ phân giải kém hơn.
- **OCT:** chụp cắt lớp quang học giao thoa độ kết hợp thấp, độ phân giải micromet; nhãn khoa, da liễu.
- **Soi huỳnh quang (Fluoroscopy):** X-quang liên tục thời gian thực (đặt catheter, tiêu hóa).
- **Nội soi, kính hiển vi (H&E, IHC) — giải phẫu bệnh.**

DICOM. File = header (bộ ba tag-VR-length-value: PatientID, Modality, PixelSpacing, SliceThickness, RescaleIntercept/Slope, StudyUID, SeriesUID, SOPInstanceUID) + dữ liệu pixel. Giao thức mạng DIMSE. PACS lưu trữ; HL7/FHIR cho dữ liệu văn bản.

P.thức	Vật lý	Đ.giải	B.xạ	T.độ	Tốt cho
MRI	H+RF+B_0	~1 mm	không	chậm	mô mềm, não
CT	X-quang	<0.5 mm	có	nhanh	xương, phổi, c.thương
US	sóng âm	~0.5 mm	không	realtime	sản, tim
PET	tracer β^+	~4 mm	có	vừa	ch.hóa, ung thư
X-q	X-quang 2D	~0.1 mm	có	tức thì	ngực, xương, vú
SPECT	tracer γ	~8 mm	có	vừa	tim, xương
OCT	ánh sáng IR	μm	không	nhanh	võng mạc, da

3.2 Pipeline thu nhận → tái tạo

Chuẩn bị bệnh nhân (nhịn ăn, tiêm thuốc cản quang, tư thế) → quét (tín hiệu thô: *k-space* cho MRI, *sinogram* cho CT, dội RF cho US, list-mode trùng hợp cho PET) → tái tạo → hậu xử lý (lọc, MPR, MIP) → ghi tag DICOM → máy chủ PACS → trạm đọc (BS chẩn đoán hình ảnh) → báo cáo có cấu trúc (HL7/FHIR) → EMR (hồ sơ bệnh án).



3.3 Phương pháp tái tạo

CT — Filtered Back Projection (FBP). Sinogram $p(t,\theta)$ với $t = x\cos\theta + y\sin\theta$.

1. Biến đổi Fourier 1D mỗi hình chiếu: $P(\omega,\theta) = \mathcal{F}_t\{p\}$.
2. Áp bộ lọc *ramp* $|\omega|$ (thường kèm cửa sổ Shepp-Logan/Hann): $\tilde{P} = |\omega|P$.
3. FT ngược 1D → $\tilde{p}(t,\theta)$.
4. Chiếu ngược:

$$f(x,y) = \int_0^\pi \tilde{p}(x\cos\theta + y\sin\theta, \theta)\, d\theta$$

Định lý lát cắt trung tâm: FT 1D của hình chiếu góc θ chính là lát cắt qua tâm của FT 2D

Iterative CT.

- ART (Algebraic Reconstruction): Kaczmarz row-action $f^{k+1} = f^k + \lambda \frac{p_i - A_i f^k}{\|A_i\|^2} A_i^T$.
- SIRT (Simultaneous), MLEM (max-likelihood expectation maximization, used in PET).
- Model-Based Iterative Recon (MBIR): noise model + prior (TV, Markov).
- Deep-learning: AUTOMAP, DnCNN-prior, unrolled networks (LEARN, Primal-Dual).

MRI reconstruction.

- Fully sampled: $f(x, y) = \mathcal{F}^{-1}\{S(k_x, k_y)\}$ (2D inverse FFT of k-space).
- Parallel imaging: SENSE (image domain unfolding via coil sensitivities), GRAPPA (k-space interpolation).
- Compressed sensing: $\min_x \|F_u x - y\|_2^2 + \lambda \|\Psi x\|_1$ (sparsity in wavelet/TV).
- DL recon: VarNet, MoDL, AUTOMAP.

3D Reconstruction from 2D Slices (graphics-relevant).

- Stack DICOM slices → regular voxel grid $V(x, y, z)$ with anisotropic spacing.
- **Direct Volume Rendering (DVR):**
  - Ray casting front-to-back; emission-absorption integral

    $$I = \int_0^L C(s)\, \mu(s)\, e^{-\int_0^s \mu(t)dt}\, ds$$

    Discrete compositing: $C_{out} = C_{in}(1 - \alpha) + C\alpha$.
  - Transfer function TF : intensity → (R, G, B, α) — tunable for tissue selection.
  - Variants: splatting (footprint of voxel), 3D texture mapping (GPU slabs), shear-warp (Lacroute & Levoy), GPU ray-marching.
- **Surface Extraction — Marching Cubes (Lorensen & Cline 1987):**
  1. For each voxel cube, classify each of 8 corners as inside/outside iso-surface ($\geq T$).
  2. 8-bit index → one of $2^8 = 256$ cases; using rotational+reflection symmetry → 15 base cases.
  3. Lookup table gives edge list; linear interp on each edge: $\mathbf{p} = \mathbf{p}_a + \frac{T - V_a}{V_b - V_a}(\mathbf{p}_b - \mathbf{p}_a)$.
  4. Connect to triangles; estimate normal from ∇V for shading.

  Ambiguity: same edge index can have two topologies; resolved by *asymptotic decider* (sign of bilinear interp at saddle).
- Variants:
  - Marching Tetrahedra (split cube into 5 or 6 tets — no ambiguity, more triangles).
  - Dual Contouring (Ju et al.) — uses Hermite data on edges, places one vertex per cube via QEF, preserves sharp features.
  - Cuberille (blocky), Extended/Topological MC.
- **Pipeline:** segment ROI → MC iso-surface → smoothing (Taubin $\lambda|\mu$ — keeps volume) → simplification (QEM — Garland-Heckbert) → texturing/lighting/render.

VTK pipeline.

`Reader (vtkDICOMImageReader) → Filter (vtkImageGaussianSmooth, vtkContourFilter, vtkMarchingCubes) → Mapper (vtkPolyDataMapper) → Actor → Renderer → RenderWindow+Interactor.`

của f.

CT lặp.

- ART (Đại số): Kaczmarz $f^{k+1} = f^k + \lambda \frac{p_i - A_i f^k}{\|A_i\|^2} A_i^T$.
- SIRT (đồng thời), MLEM (cực đại hóa kỳ vọng — dùng trong PET).
- MBIR (dựa mô hình): mô hình nhiễu + tiên nghiệm (TV, Markov).
- Deep-learning: AUTOMAP, DnCNN-prior, mạng "unrolled" (LEARN, Primal-Dual).

Tái tạo MRI.

- Lấy mẫu đầy đủ: $f(x, y) = \mathcal{F}^{-1}\{S(k_x, k_y)\}$ (FFT ngược 2D từ k-space).
- Parallel imaging: SENSE (gỡ rối miền ảnh qua bản đồ độ nhạy cuộn), GRAPPA (nội suy k-space).
- Compressed sensing: $\min_x \|F_u x - y\|_2^2 + \lambda \|\Psi x\|_1$ (thưa trên wavelet/TV).
- DL recon: VarNet, MoDL, AUTOMAP.

Tái tạo 3D từ các lát 2D (liên quan đồ họa).

- Xếp chồng các lát DICOM → lưới voxel đều $V(x, y, z)$, khoảng cách bất đẳng hướng.
- **Direct Volume Rendering (DVR):**
  - Ray casting front-to-back; tích phân phát xạ-hấp thụ

    $$I = \int_0^L C(s)\, \mu(s)\, e^{-\int_0^s \mu(t)dt}\, ds$$

    Hợp thành rời rạc: $C_{out} = C_{in}(1 - \alpha) + C\alpha$.
  - Transfer function TF : cườngđộ → (R, G, B, α) — chỉnh để chọn loại mô.
  - Biến thể: splatting (dấu chân voxel), 3D texture (GPU slabs), shear-warp (Lacroute–Levoy), ray-marching trên GPU.
- **Trích bề mặt — Marching Cubes (Lorensen & Cline 1987):**
  1. Với mỗi khối voxel, phân loại 8 góc bên trong/ngoài bề mặt đẳng trị ($\geq T$).
  2. Chỉ số 8-bit → một trong $2^8 = 256$ trường hợp; dùng đối xứng quay+phản xạ → 15 trường hợp gốc.
  3. Bảng tra tra ra danh sách cạnh; nội suy tuyến tính trên mỗi cạnh: $\mathbf{p} = \mathbf{p}_a + \frac{T - V_a}{V_b - V_a}(\mathbf{p}_b - \mathbf{p}_a)$.
  4. Nối thành tam giác; tính pháp tuyến từ ∇V để tô bóng.

  Mơ hồ: cùng một chỉ số cạnh có thể có hai topology; xử lý bằng *asymptotic decider* (dấu của nội suy song tuyến tại điểm yên ngựa).
- Biến thể:
  - Marching Tetrahedra (chia khối thành 5 hoặc 6 tứ diện — không mơ hồ, nhiều tam giác hơn).
  - Dual Contouring (Ju et al.) — dùng Hermite trên cạnh, đặt một đỉnh/khối qua QEF, giữ được cạnh sắc.
  - Cuberille (lập phương), Extended/Topological MC.
- **Quy trình:** phân vùng ROI → MC iso-surface → làm mịn (Taubin $\lambda|\mu$ — bảo toàn thể tích) → giảm tam giác (QEM — Garland-Heckbert) → gán texture/chiếu sáng/render.

3.4 Image Processing Operations

Preprocessing.

- Denoising: Gaussian (LP linear), median (impulse), bilateral (range+space, edge-preserving), non-local means (patch similarity), BM3D, Total Variation, deep denoisers (DnCNN, Noise2Noise).
- Contrast: histogram equalization; CLAHE (contrast-limited adaptive HE) per tile + clip.
- N4ITK bias-field correction (MRI inhomogeneity, multiplicative low-frequency field).
- Intensity normalization: z-score, min-max, percentile clip, white-stripe, Nyul piecewise-linear.
- Resampling to isotropic spacing; skull stripping; cropping; padding.

Registration.

- Goal: align moving image to fixed image; $T : \mathbb{R}^3 \to \mathbb{R}^3$.
- Rigid (6 DOF — 3 translation + 3 rotation); affine (12 DOF + scale + shear); non-rigid: B-spline FFD (free-form deformation, control grid), Demons (optical flow), diffeomorphic LDDMM (large-deformation), SyN (ANTs).
- Similarity metrics: SSD $= \sum (I_f - T(I_m))^2$ (mono-modal), NCC (illumination-invariant), Mutual Information MI $= H(A) + H(B) - H(A, B)$ (multi-modal MR-CT).
- Optimizer: gradient descent, LBFGS, Powell.
- DL: VoxelMorph, TransMorph.

Segmentation.

- *Thresholding:* Otsu maximizes inter-class variance $\sigma_B^2 = w_0 w_1 (\mu_0 - \mu_1)^2$.
- *Region growing:* seed + similarity criterion.
- *Watershed:* gradient = topographic surface; flood from minima; over-segments ⇒ marker-controlled.
- *Active contours / Snakes (Kass et al.):* minimize

$$E = \int (\alpha |C'|^2 + \beta |C''|^2 + E_{\text{ext}})\, ds$$

  α elasticity, β rigidity; $E_{\text{ext}} = -|\nabla (G_\sigma * I)|^2$.
- *Level Sets (Osher-Sethian):* embed contour as zero-set of ϕ; evolve

$$\frac{\partial \phi}{\partial t} + F|\nabla \phi| = 0$$

  Chan-Vese: piecewise-constant Mumford-Shah → no edge needed.
- *Graph cuts:* energy $E = \sum D_p(l_p) + \sum V_{pq}(l_p, l_q)$, min-cut/max-flow $O(V^2 E)$ (Boykov-Kolmogorov); GrabCut iterates GMM + graph cut.
- *Atlas-based:* register atlas → propagate labels; multi-atlas + label fusion (majority vote, STAPLE).

Pipeline VTK.

`Reader (vtkDICOMImageReader) → Filter (vtkImageGaussianSmooth, vtkContourFilter, vtkMarchingCubes) → Mapper (vtkPolyDataMapper) → Actor → Renderer → RenderWindow+Interactor.`

3.4 Các phép xử lý ảnh

Tiền xử lý.

- Khử nhiễu: Gauss (LP tuyến tính), median (xung), bilateral (giá trị+không gian, giữ cạnh), non-local means (tương đồng patch), BM3D, Total Variation, deep denoiser (DnCNN, Noise2Noise).
- Tương phản: cân bằng histogram; CLAHE (cân bằng thích nghi có giới hạn) theo ô + cắt đỉnh.
- Hiệu chỉnh từ trường lệch N4ITK (MRI bias field, trường nhân tần số thấp).
- Chuẩn hóa cường độ: z-score, min-max, cắt percentile, white-stripe, Nyul phân đoạn tuyến tính.
- Resample về spacing đẳng hướng; lột sọ; cắt; pad.

Đăng ký ảnh (Registration).

- Mục tiêu: căn ảnh moving với ảnh fixed; $T : \mathbb{R}^3 \to \mathbb{R}^3$.
- Cứng (6 DOF — 3 tịnh tiến + 3 quay); affine (12 DOF + tỷ lệ + cắt); phi cứng: B-spline FFD (biến dạng tự do, lưới điều khiển), Demons (optical flow), diffeomorphic LDDMM (biến dạng lớn), SyN (ANTs).
- Hàm tương đồng: SSD $= \sum (I_f - T(I_m))^2$ (đơn modal), NCC (bất biến chiếu sáng), Mutual Information MI $= H(A) + H(B) - H(A, B)$ (đa modal MR-CT).
- Tối ưu hóa: gradient descent, LBFGS, Powell.
- DL: VoxelMorph, TransMorph.

Phân vùng (Segmentation).

- *Ngưỡng:* Otsu cực đại hóa phương sai liên lớp $\sigma_B^2 = w_0 w_1 (\mu_0 - \mu_1)^2$.
- *Region growing:* hạt mầm + tiêu chí tương đồng.
- *Watershed:* gradient = bề mặt địa hình; tràn từ điểm thấp; dễ over-seg ⇒ điều khiển bằng marker.
- *Active contour / Snake (Kass):* cực tiểu

$$E = \int (\alpha |C'|^2 + \beta |C''|^2 + E_{\text{ext}})\, ds$$

  α co giãn, β độ cứng; $E_{\text{ext}} = -|\nabla (G_\sigma * I)|^2$.
- *Level Set (Osher-Sethian):* nhúng đường viền là tập-không của ϕ; tiến hóa

$$\frac{\partial \phi}{\partial t} + F|\nabla \phi| = 0$$

  Chan-Vese: Mumford-Shah từng đoạn-hằng → không cần cạnh.
- *Graph cuts:* năng lượng $E = \sum D_p(l_p) + \sum V_{pq}(l_p, l_q)$, min-cut/max-flow $O(V^2 E)$ (Boykov-Kolmogorov); GrabCut lặp GMM + graph cut.
- *Dựa atlas:* đăng ký atlas → truyền nhãn; multi-atlas + hợp nhãn (đa số, STAPLE).

- *Deep learning:*
  - **U-Net** (Ronneberger 2015): symmetric encoder-decoder with skip-connections.
  - **V-Net** (Milletari 2016): 3D, residual blocks, Dice loss $1 - \frac{2|A \cap B|}{|A|+|B|}$.
  - **nnU-Net** (Isensee 2021): self-configuring (preprocessing, topology, training); state-of-the-art baseline.
  - SegResNet, Attention U-Net, TransUNet, Swin-UNet, SAM-Med2D, MedSAM.

Feature extraction.

- Gradients: Sobel, Prewitt, Scharr; Canny edge (smooth → gradient → NMS → hysteresis).
- Laplacian of Gaussian (LoG), DoG, Hessian (vesselness — Frangi filter).
- Texture: GLCM (Haralick: contrast, correlation, energy, homogeneity), LBP, Gabor banks.
- **Radiomics:** shape (volume, sphericity, elongation), first-order intensity (mean, kurtosis), texture (GLCM, GLRLM, GLSZM, NGTDM, GLDM), wavelet/Laplacian filtered features. **IBSI** standard for reproducibility.

Classification / Detection.

- Traditional: SVM (kernel RBF), Random Forest, kNN, AdaBoost — on hand-crafted features.
- Deep CNN: LeNet-5, AlexNet, VGG-16/19, GoogLeNet (Inception), ResNet (skip), DenseNet, EfficientNet (compound scaling).
- Detection: R-CNN, Fast/Faster R-CNN, YOLO, RetinaNet, DETR.
- Vision Transformers: ViT, Swin Transformer, SegFormer.
- Foundation models: SAM (Segment Anything), MedSAM, BiomedCLIP, Med-PaLM (text), LLaVA-Med (multi-modal).

3.5 Brain Tumor Detection (BraTS context)

BraTS challenge (Brain Tumor Segmentation, MICCAI annual since 2012). Provides 4 co-registered MRI modalities per patient:

- **T1** — anatomy, gray vs white matter.
- **T1ce** (post-Gadolinium) — enhancing tumor highlighted (BBB breakdown).
- **T2** — edema bright.
- **FLAIR** — edema bright, CSF dark (best for whole tumor).

Tumor sub-regions (labels).

- **ET** — Enhancing Tumor (hyper-intense T1ce relative to T1).
- **NCR/NET** — Necrotic / non-enhancing tumor core.
- **ED** — peritumoral Edema.
- Hierarchical:
  - **WT** (Whole Tumor) = ET ∪ NCR ∪ ED.
  - **TC** (Tumor Core) = ET ∪ NCR.
  - **ET** alone.

Standard pipeline.

- *Học sâu:*
  - **U-Net** (Ronneberger 2015): encoder-decoder đối xứng, skip-connection.
  - **V-Net** (Milletari 2016): 3D, khối residual, Dice loss $1 - \frac{2|A \cap B|}{|A|+|B|}$.
  - **nnU-Net** (Isensee 2021): tự cấu hình (tiền xử lý, kiến trúc, training); baseline SOTA.
  - SegResNet, Attention U-Net, TransUNet, Swin-UNet, SAM-Med2D, MedSAM.

Trích đặc trưng.

- Gradient: Sobel, Prewitt, Scharr; cạnh Canny (mịn → gradient → NMS → hysteresis).
- Laplacian of Gaussian (LoG), DoG, Hessian (mạch máu — Frangi filter).
- Texture: GLCM (Haralick: contrast, correlation, energy, homogeneity), LBP, Gabor.
- **Radiomics:** hình dạng (thể tích, độ cầu, độ kéo dài), bậc một (mean, kurtosis), texture (GLCM, GLRLM, GLSZM, NGTDM, GLDM), đặc trưng wavelet/Laplacian. Chuẩn **IBSI** để tái lập được.

Phân loại / Phát hiện.

- Truyền thống: SVM (kernel RBF), Random Forest, kNN, AdaBoost — trên đặc trưng thủ công.
- Deep CNN: LeNet-5, AlexNet, VGG-16/19, GoogLeNet (Inception), ResNet (skip), DenseNet, EfficientNet (scale tổng hợp).
- Phát hiện: R-CNN, Fast/Faster R-CNN, YOLO, RetinaNet, DETR.
- Vision Transformer: ViT, Swin Transformer, SegFormer.
- Foundation model: SAM (Segment Anything), MedSAM, BiomedCLIP, Med-PaLM (văn bản), LLaVA-Med (đa modal).

3.5 Phát hiện u não (BraTS)

Thử thách BraTS (MICCAI hàng năm từ 2012). Mỗi bệnh nhân có 4 modality MRI đã đăng ký:

- **T1** — giải phẫu, chất xám/trắng.
- **T1ce** (sau Gadolinium) — vùng u tăng quang (hàng rào máu não bị phá).
- **T2** — phù não sáng.
- **FLAIR** — phù sáng, dịch não tủy tối (tốt nhất cho whole tumor).

Vùng u con (nhãn).

- **ET** — Enhancing Tumor (tăng quang T1ce so với T1).
- **NCR/NET** — hoại tử / không tăng quang.
- **ED** — phù quanh u.
- Phân cấp:
  - **WT** (Toàn bộ u) = ET ∪ NCR ∪ ED.
  - **TC** (Lõi u) = ET ∪ NCR.
  - **ET** riêng.

Pipeline chuẩn.

1. Skull stripping (BET, ROBEX, HD-BET).
2. Co-registration of 4 modalities (rigid to T1 atlas, e.g., SRI24).
3. Bias field correction (N4ITK).
4. Resample to $1\,\mathrm{mm}^3$ isotropic.
5. Intensity normalization (z-score per modality, brain mask).
6. Patch / full-volume training of 3D U-Net / nnU-Net.
7. Inference + post-processing: connected components, fill holes, conditional random field (CRF), morphology, ensemble (TTA, k-fold).

Evaluation metrics.

- **Dice** $= \dfrac{2|A \cap B|}{|A| + |B|} \in [0, 1]$, harmonic mean of precision/recall.
- **Jaccard / IoU** $= \dfrac{|A \cap B|}{|A \cup B|} = \dfrac{D}{2 - D}$.
- **Hausdorff95** = 95-th percentile of $\{d(a, B)\} \cup \{d(b, A)\}$ — robust to outliers.
- **ASSD** $= \dfrac{1}{|S_A| + |S_B|}\big(\sum_{a \in S_A} d(a, S_B) + \sum_{b \in S_B} d(b, S_A)\big)$.
- Sensitivity $= \mathrm{TP}/(\mathrm{TP} + \mathrm{FN})$, Specificity $= \mathrm{TN}/(\mathrm{TN} + \mathrm{FP})$, Precision, Recall, F1.
- Volumetric similarity, Cohen's κ, AUC for classification heads.

3.6 Paper 1 — GAN Brain Tumor Segmentation

Title: *A Research for Segmentation of Brain Tumors Based on GAN Model.* Multi-scale 3D-GAN evaluated on BraTS 2021.

Architecture.

- **Generator G:** 3D Residual U-Net (encoder-decoder with residual blocks, skip connections, 3D convs).
- **Bridge module:** sits between G and Discriminator; aggregates multi-scale features from several G decoder stages, fuses them, and feeds into D so that D sees both high-level context and low-level boundary details simultaneously.
- **Discriminator D:** judges (input MRI $\oplus$ predicted seg) vs (input $\oplus$ ground truth).

Three contributions.

1. Bridge module — fuses multi-scale features into D.
2. **Algorithm 1** — dynamically adjusts the number of D training epochs using the linear-regression *slope* of the adversarial loss; if loss is decreasing too fast, train D more (or less) to balance G-vs-D.
3. Combined loss: $\mathcal{L} = \mathcal{L}_{\text{Dice}} + \lambda \mathcal{L}_{\text{MSE-adv}}$.

Results (Dice, BraTS 2021 validation split).

- ET = 82.3%, WT = 88.6%, TC = 83.1% — beats U-Net, V-Net, VoxelGAN, Vox2Vox.
- Also reports Jaccard, Hausdorff95, ASSD vs same baselines.

Strengths.

- Bridge well-motivated: gives D both context + boundary detail.
- Algorithm 1 is novel and pragmatic — addresses classic GAN G-D imbalance.
- 4 metrics (Dice, Jaccard, HD95, ASSD) and 4 baselines (CNN + GAN families).

1. Lột sọ (BET, ROBEX, HD-BET).
2. Đăng ký đồng vị 4 modality (cứng về atlas T1, vd SRI24).
3. Hiệu chỉnh bias field (N4ITK).
4. Resample về $1\,\mathrm{mm}^3$ đẳng hướng.
5. Chuẩn hóa cường độ (z-score mỗi modality, mask não).
6. Train 3D U-Net / nnU-Net theo patch hoặc full-volume.
7. Suy luận + hậu xử lý: thành phần liên thông, lấp lỗ, CRF, hình thái, ensemble (TTA, k-fold).

Độ đo đánh giá.

- **Dice** $= \dfrac{2|A \cap B|}{|A| + |B|} \in [0, 1]$, trung bình điều hòa precision/recall.
- **Jaccard / IoU** $= \dfrac{|A \cap B|}{|A \cup B|} = \dfrac{D}{2 - D}$.
- **Hausdorff95** = percentile thứ 95 của $\{d(a, B)\} \cup \{d(b, A)\}$ — kháng outlier.
- **ASSD** $= \dfrac{1}{|S_A| + |S_B|}\big(\sum_{a \in S_A} d(a, S_B) + \sum_{b \in S_B} d(b, S_A)\big)$.
- Sensitivity $= \mathrm{TP}/(\mathrm{TP} + \mathrm{FN})$, Specificity $= \mathrm{TN}/(\mathrm{TN} + \mathrm{FP})$, Precision, Recall, F1.
- Tương đồng thể tích, Cohen's κ, AUC cho head phân loại.

3.6 Paper 1 — GAN phân vùng u não

Tiêu đề: *A Research for Segmentation of Brain Tumors Based on GAN Model.* Multi-scale 3D-GAN, đánh giá trên BraTS 2021.

Kiến trúc.

- **Generator G:** 3D Residual U-Net (encoder-decoder với khối residual, skip connection, conv 3D).
- **Bridge module:** nằm giữa G và Discriminator; tổng hợp đặc trưng đa tỷ lệ từ vài tầng decoder của G, hợp nhất rồi đưa vào D để D nhìn được cả ngữ cảnh cấp cao lẫn chi tiết biên cấp thấp.
- **Discriminator D:** phân biệt (MRI $\oplus$ seg dự đoán) vs (MRI $\oplus$ ground truth).

Ba đóng góp.

1. Bridge module — hợp đặc trưng đa tỷ lệ vào D.
2. **Algorithm 1** — điều chỉnh động số epoch huấn luyện D theo *độ dốc* hồi quy tuyến tính của adversarial loss; nếu loss giảm quá nhanh thì train D thêm (hoặc bớt) để cân bằng G-D.
3. Hàm loss kết hợp: $\mathcal{L} = \mathcal{L}_{\text{Dice}} + \lambda \mathcal{L}_{\text{MSE-adv}}$.

Kết quả (Dice, validation split BraTS 2021).

- ET = 82.3%, WT = 88.6%, TC = 83.1% — vượt U-Net, V-Net, VoxelGAN, Vox2Vox.
- Báo cáo thêm Jaccard, Hausdorff95, ASSD so cùng baselines.

Điểm mạnh.

- Bridge có động cơ rõ: cho D cả ngữ cảnh + chi tiết biên.
- Algorithm 1 mới và thực dụng — xử lý mất cân bằng kinh điển G-D trong GAN.
- 4 độ đo (Dice, Jaccard, HD95, ASSD) và 4 baseline (CNN + GAN).

Weaknesses.

- Only validation split from training — no cross-validation, no submission to BraTS server ⇒ hard to compare with literature.
- No ablation isolating Bridge vs Algorithm 1 — individual contribution unknown.
- *Paradox:* on ET, GAN gives higher Dice than ResUnet-no-GAN but *worse* HD95 (9.6 mm vs 6.4 mm) and ASSD (3.37 vs 2.35) ⇒ GAN improved overlap but *degraded boundary.*
- Hyperparams of Algorithm 1 (k, l, threshold β) untuned; no theoretical justification for linear regression on adversarial loss.
- No parameter count, training/inference time, no significance tests, no code release.
- Bridge module deserves its own dedicated figure (currently merged in Fig. 1).

Suggested improvements.

- Submit to BraTS online server, or 5-fold CV with mean±std.
- Ablations: Baseline / +Bridge / +Algo1 / +both.
- Analyze ET HD95/ASSD paradox (boundary smoothing artifacts of MSE-adv?).
- Sensitivity analysis on k, l, β.
- Release source code; report params/FLOPs/runtime; statistical significance (paired Wilcoxon).

3.7 Paper 2 — ANN → DCNN Paradigm Shift Review

Title: *Paradigm shift from Artificial Neural Networks (ANNs) to deep Convolutional Neural Networks (DCNNs) in the field of medical image processing.* 2024 review.

Thesis. Past pipeline: hand-crafted feature extraction (SIFT, GLCM, Hu moments) → shallow ANN classifier. Modern pipeline: end-to-end DCNN learns features directly from raw images.

Strengths.

- Concrete numerical convolution example with a Sobel kernel applied to a hand X-ray ⇒ very accessible for newcomers.
- Two summary tables: ANN-based vs DCNN-based studies — *the "feature extraction" column literally vanishes* in DCNN row.
- Honest about DCNN limitations: data-hungry, prone to overfitting, black-box.

Weaknesses.

- No mention of **Vision Transformers**, **SAM**, or medical foundation models (Med-PaLM, LLaVA-Med) — yet the paper claims a "paradigm shift" in 2024. The actual ongoing shift (DCNN → Transformer/Foundation) is ignored. ANN → DCNN was already widely accepted by 2017.
- Focuses on **classification**; ignores **segmentation** (U-Net, nnU-Net) and standard datasets (**BraTS**, LIDC, CheXpert).

Điểm yếu.

- Chỉ chia validation từ training — không cross-validation, không nộp BraTS server ⇒ khó so với cộng đồng.
- Thiếu ablation tách bạch Bridge vs Algorithm 1 — không biết phần nào đóng góp bao nhiêu.
- *Nghịch lý:* với ET, GAN có Dice cao hơn ResUnet không-GAN nhưng HD95 (9.6 mm vs 6.4 mm) và ASSD (3.37 vs 2.35) lại *tệ hơn* ⇒ GAN cải thiện overlap thể tích nhưng *làm xấu biên.*
- Siêu tham số Algorithm 1 (k, l, ngưỡng β) không khảo sát; không có lý thuyết nào biện minh cho việc hồi quy tuyến tính trên loss.
- Không có số tham số, thời gian train/infer, không có kiểm định ý nghĩa thống kê, không công khai code.
- Module Bridge đáng có hình riêng (đang gộp vào Fig. 1).

Đề xuất cải tiến.

- Nộp BraTS online server, hoặc 5-fold CV với mean±std.
- Ablation: Baseline / +Bridge / +Algo1 / +cả hai.
- Phân tích nghịch lý HD95/ASSD ở ET (nghi do MSE-adv làm trơn biên?).
- Khảo sát độ nhạy k, l, β.
- Công khai mã nguồn; báo cáo params/FLOPs/runtime; kiểm định ý nghĩa (paired Wilcoxon).

3.7 Paper 2 — Tổng quan chuyển dịch ANN → DCNN

Tiêu đề: *Paradigm shift from Artificial Neural Networks (ANNs) to deep Convolutional Neural Networks (DCNNs) in the field of medical image processing.* Bài tổng quan 2024.

Luận điểm. Quy trình cũ: trích đặc trưng thủ công (SIFT, GLCM, mô-men Hu) → ANN nông để phân loại. Quy trình hiện đại: DCNN end-to-end học đặc trưng trực tiếp từ ảnh thô.

Điểm mạnh.

- Ví dụ tích chập số cụ thể với kernel Sobel áp lên ảnh X-quang bàn tay ⇒ rất dễ hiểu cho người mới.
- Hai bảng tổng kết: nghiên cứu dùng ANN vs nghiên cứu dùng DCNN — *cột "feature extraction" biến mất hoàn toàn* ở dòng DCNN.
- Trung thực về hạn chế của DCNN: cần nhiều dữ liệu, dễ overfit, hộp đen.

Điểm yếu.

- Không nhắc đến **Vision Transformer**, **SAM**, hay các foundation model y tế (Med-PaLM, LLaVA-Med) — mà bài lại tuyên bố "paradigm shift" năm 2024. Cuộc chuyển dịch đang xảy ra (DCNN → Transformer/Foundation) bị bỏ qua. ANN → DCNN đã được công nhận từ 2017.
- Chỉ tập trung **phân loại**; bỏ qua **phân vùng** (U-Net, nnU-Net) và các bộ dữ liệu chuẩn (**BraTS**, LIDC, CheXpert).

- English-language context only; no **low-resource language** coverage. Critical for Vietnamese CDSS — labeled corpora missing, mixed VN-EN-Latin terminology, no local medical knowledge graph.
- No **XAI** (Grad-CAM, conformal prediction, Dempster-Shafer) and no **uncertainty quantification** — yet the EU AI Act classifies CDSS as *high-risk*, so explainability is mandatory.
- No PRISMA / inclusion-exclusion criteria for a review paper; no positioning vs Litjens 2017 / Esteva 2019 (already authoritative surveys).
- Long, rambling history (1980s–90s) padding; the actually-important CNN mechanism section is too short. AI/ML/DL definition boxes are too basic for a Q1 journal.

Takeaway. Useful intro for newcomers, but for proper publication needs: (i) Transformers + foundation models, (ii) segmentation track + standard datasets, (iii) low-resource language section, (iv) XAI + uncertainty for CDSS, (v) PRISMA methodology and survey-positioning.

3.8 Diagnosis-Support Applications

Use cases.

- Triage / worklist prioritization (e.g., critical findings in head CT).
- Second-reader / double-reading (mammography).
- Population screening (chest X-ray TB, retinal DR, breast cancer).
- Surgical planning (3D mesh from CT/MRI for tumor resection).
- Image-guided intervention (real-time fusion US+MRI for biopsy/ablation).
- Quantification: lesion volume, perfusion, ejection fraction.
- Treatment response monitoring (RECIST, longitudinal radiomics).

End-to-end CDSS pipeline. DICOM input → preprocessing (denoise, normalize, register) → segmentation (U-Net family) → classification / detection (CNN/ViT) → uncertainty quantification (MC-Dropout, deep ensembles, conformal prediction) → explanation (Grad-CAM, Integrated Gradients, attention maps, SHAP) → structured report (HL7 FHIR DiagnosticReport) → clinician review.

Regulatory.

- **FDA 510(k)** / De Novo / PMA in US; AI/ML SaMD action plan; predetermined change control.
- **CE marking**, **MDR** (EU 2017/745).
- **EU AI Act** — CDSS is high-risk: requires risk-management, data governance, technical docs, transparency, human oversight, accuracy/robustness/cybersecurity logs.
- Privacy: HIPAA (US), GDPR (EU), Vietnam Decree 13/2023/ND-CP on personal data.

Vietnamese context.

- Lacking large-scale Vietnamese-labeled medical corpora (radiology reports, clinical notes).
- Mixed VN-EN-Latin terminology (e.g., "viêm phổi / pneumonia / bronchopneumonia").
- Missing local medical knowledge graph / SNOMED-CT Vietnamese mapping.
- Hospital PACS/HIS heterogeneous; limited GPU; class imbalance for rare endemic dis-

- Bối cảnh chỉ tiếng Anh; không đề cập **ngôn ngữ low-resource**. Rất quan trọng với CDSS tiếng Việt — thiếu corpus có nhãn, thuật ngữ trộn VN-EN-Latin, chưa có knowledge graph y tế Việt.
- Không bàn **XAI** (Grad-CAM, conformal prediction, Dempster-Shafer) lẫn **đo bất định** — nhưng EU AI Act xếp CDSS vào *high-risk* nên giải thích được là bắt buộc.
- Bài tổng quan mà không có PRISMA / tiêu chí lựa chọn-loại trừ; không định vị so với Litjens 2017 / Esteva 2019 (đã là survey kinh điển).
- Phần lịch sử (1980s–90s) dài lan man; phần cơ chế CNN — phần quan trọng — lại quá ngắn. Hộp định nghĩa AI/ML/DL quá cơ bản cho một tạp chí Q1.

Tóm lại. Hữu ích như bài đọc nhập môn, nhưng để xuất bản đàng hoàng cần: (i) Transformer + foundation models, (ii) nhánh segmentation + dataset chuẩn, (iii) phần ngôn ngữ low-resource, (iv) XAI + uncertainty cho CDSS, (v) phương pháp PRISMA và định vị so với survey trước.

3.8 Ứng dụng hỗ trợ chẩn đoán

Trường hợp sử dụng.

- Phân loại ưu tiên / triage (vd phát hiện CT sọ não nguy kịch).
- Người đọc thứ hai / đọc kép (nhũ ảnh).
- Tầm soát quần thể (X-quang phổi lao, võng mạc tiểu đường, ung thư vú).
- Lập kế hoạch phẫu thuật (mesh 3D từ CT/MRI để cắt u).
- Can thiệp dưới hình ảnh (hợp nhất US+MRI thời gian thực để sinh thiết/đốt).
- Định lượng: thể tích tổn thương, tưới máu, phân suất tống máu.
- Theo dõi đáp ứng điều trị (RECIST, radiomics dọc).

Pipeline CDSS đầu cuối. DICOM đầu vào → tiền xử lý (khử nhiễu, chuẩn hóa, đăng ký) → phân vùng (họ U-Net) → phân loại / phát hiện (CNN/ViT) → đo bất định (MC-Dropout, deep ensembles, conformal prediction) → giải thích (Grad-CAM, Integrated Gradients, bản đồ chú ý, SHAP) → báo cáo có cấu trúc (HL7 FHIR DiagnosticReport) → bác sĩ duyệt.

Pháp lý.

- **FDA 510(k)** / De Novo / PMA tại Mỹ; kế hoạch SaMD AI/ML; kiểm soát thay đổi đã định trước.
- **CE marking**, **MDR** (EU 2017/745).
- **EU AI Act** — CDSS thuộc nhóm rủi ro cao: yêu cầu quản lý rủi ro, quản trị dữ liệu, tài liệu kỹ thuật, minh bạch, giám sát của người, log độ chính xác/độ bền/an ninh mạng.
- Bảo mật: HIPAA (Mỹ), GDPR (EU), Nghị định 13/2023/NĐ-CP về dữ liệu cá nhân ở Việt Nam.

Bối cảnh Việt Nam.

- Thiếu corpus y tế tiếng Việt có nhãn quy mô lớn (báo cáo chẩn đoán hình ảnh, ghi chú lâm sàng).
- Thuật ngữ trộn VN-EN-Latin (vd "viêm phổi / pneumonia / bronchopneumonia").
- Chưa có knowledge graph y tế tiếng Việt / ánh xạ SNOMED-CT.

eases.

3.9 Quick Reference Summary

Mod.	Best for	Reconstruction	Mesh extraction
MRI	soft tissue, brain, MS	2D iFFT k-space; SENSE/GRAPPA; CS	MC on T1/FLAIR after seg.
CT	bone, lung, trauma	FBP (ramp); iter MBIR; DL recon	MC on HU threshold (bone >300)
US 3D	OB, cardiac, vascular	beamforming; free-hand 3D	MC on B-mode envelope (noisy)
PET	oncology, brain metab.	MLEM, OSEM, TOF	threshold SUV → MC, smooth
X-ray	chest, mammo	none (2D projection)	N/A (unless tomosynth.)
SPECT	cardiac, bone, thyroid	FBP / OSEM	rare
OCT	retina, anterior segment	Fourier-domain iFFT	MC on layer prob. map

Key formulas to remember.

- HU: $\mathrm{HU} = 1000(\mu - \mu_w)/\mu_w$.
- Larmor: $\omega_0 = \gamma B_0$ ($\gamma_{^1H}/2\pi = 42.58$ MHz/T).
- FBP: $f(x,y) = \int_0^\pi \tilde{p}(x\cos\theta + y\sin\theta, \theta)d\theta$.
- MC edge interp: $\mathbf{p} = \mathbf{p}_a + \frac{T-V_a}{V_b-V_a}(\mathbf{p}_b - \mathbf{p}_a)$.
- DVR composite: $C_{out} = C_{in}(1-\alpha) + C\alpha$.
- Dice: $D = 2|A\cap B|/(|A|+|B|)$; $\mathrm{IoU} = D/(2-D)$.
- Otsu: $\arg\max_T w_0 w_1(\mu_0 - \mu_1)^2$.
- Snake energy: $E = \int(\alpha|C'|^2 + \beta|C''|^2 + E_{\text{ext}})ds$.
- Level set: $\partial\phi/\partial t + F|\nabla\phi| = 0$.
- MI: $\mathrm{MI}(A,B) = H(A) + H(B) - H(A,B)$.

- PACS/HIS bệnh viện không đồng nhất; GPU hạn chế; mất cân bằng lớp với bệnh đặc hữu hiếm.

3.9 Bảng tổng kết tham khảo nhanh

P.thức	Tốt cho	Tái tạo	Trích mesh
MRI	mô mềm, não, MS	iFFT 2D k-space; SENSE/GRAPPA; CS	MC trên T1/FLAIR sau seg
CT	xương, phổi, ch.thương	FBP (ramp); MBIR lặp; DL recon	MC ngưỡng HU (xương >300)
US 3D	sản, tim, mạch	beamforming; 3D tự do	MC trên envelope B-mode, nhiễu
PET	ung thư, ch.hóa não	MLEM, OSEM, TOF	ngưỡng SUV → MC, làm mịn
X-q	ngực, vú	không (chiếu 2D)	N/A (trừ tomosynthesis)
SPECT	tim, xương, giáp	FBP / OSEM	hiếm
OCT	võng mạc, đoạn trước	FT ngược miền Fourier	MC trên bản đồ xác suất lớp

Công thức cần nhớ.

- HU: $\mathrm{HU} = 1000(\mu - \mu_w)/\mu_w$.
- Larmor: $\omega_0 = \gamma B_0$ ($\gamma_{^1H}/2\pi = 42.58$ MHz/T).
- FBP: $f(x,y) = \int_0^\pi \tilde{p}(x\cos\theta + y\sin\theta, \theta)d\theta$.
- Nội suy cạnh MC: $\mathbf{p} = \mathbf{p}_a + \frac{T-V_a}{V_b-V_a}(\mathbf{p}_b - \mathbf{p}_a)$.
- Hợp DVR: $C_{out} = C_{in}(1-\alpha) + C\alpha$.
- Dice: $D = 2|A\cap B|/(|A|+|B|)$; $\mathrm{IoU} = D/(2-D)$.
- Otsu: $\arg\max_T w_0 w_1(\mu_0 - \mu_1)^2$.
- Năng lượng Snake: $E = \int(\alpha|C'|^2 + \beta|C''|^2 + E_{\text{ext}})ds$.
- Level set: $\partial\phi/\partial t + F|\nabla\phi| = 0$.
- MI: $\mathrm{MI}(A,B) = H(A) + H(B) - H(A,B)$.

4. Bezier Curves, Bezier Surfaces, De Casteljau | *Đường cong, Mặt Bezier, Thuật toán De Casteljau*

4.1 Bezier Curve — Bernstein Form

A **Bezier curve** of degree n with $n+1$ control points $P_0, \ldots, P_n$ is the parametric curve

$$B(t) = \sum_{i=0}^{n} \binom{n}{i}(1-t)^{n-i}t^i P_i, \quad t \in [0,1].$$

The polynomials

$$B_i^n(t) = \binom{n}{i}(1-t)^{n-i}t^i, \quad i = 0, \ldots, n$$

are called **Bernstein basis polynomials**.

Key properties.

- *Endpoint interpolation:* $B(0) = P_0,\ B(1) = P_n$.
- *Partition of unity:* $\sum_{i=0}^n B_i^n(t) = 1$ for all $t \Rightarrow$ *affine invariance.*
- *Convex hull:* $B(t) \in \mathrm{conv}\{P_0, \ldots, P_n\}$ since $B_i^n(t) \geq 0$.
- *Variation diminishing:* the curve crosses any line/plane no more times than its control polygon does $\Rightarrow$ no extra wiggles.
- *Symmetry:* $B(t)$ with control points reversed equals $B(1-t)$, i.e. $B_i^n(t) = B_{n-i}^n(1-t)$.
- *Endpoint tangents:* $B'(0) = n(P_1 - P_0),\ B'(1) = n(P_n - P_{n-1})$.
- *Pseudo-local control:* moving P_i shifts $B(t)$ proportionally to $B_i^n(t)$ (max at $t = i/n$).
- *Degree:* a Bezier curve with $n+1$ points is a polynomial of degree $\leq n$.

4.1 Đường cong Bezier — Dạng Bernstein

Đường cong Bezier bậc n với $n+1$ control point $P_0, \ldots, P_n$ là đường cong tham số

$$B(t) = \sum_{i=0}^{n} \binom{n}{i}(1-t)^{n-i}t^i P_i, \quad t \in [0,1].$$

Các đa thức $B_i^n(t) = \binom{n}{i}(1-t)^{n-i}t^i$ gọi là **đa thức cơ sở Bernstein**.

Tính chất quan trọng.

- *Nội suy đầu mút:* $B(0) = P_0,\ B(1) = P_n$.
- *Tổng bằng 1:* $\sum B_i^n(t) = 1 \Rightarrow$ *bất biến affine* (xoay/tịnh tiến control point thì curve cũng biến đổi tương ứng).
- *Bao lồi:* đường cong luôn nằm trong bao lồi của các control point.
- *Giảm dao động (variation diminishing):* đường cong cắt một đường thẳng không nhiều hơn số lần control polygon cắt đường đó.
- *Đối xứng:* đảo thứ tự control point thì $B(t)$ trở thành $B(1-t)$.
- *Tiếp tuyến tại đầu mút:* $B'(0) = n(P_1 - P_0)$, $B'(1) = n(P_n - P_{n-1})$.
- *Điều khiển gần địa phương:* khi dời P_i, curve dịch chuyển nhiều nhất tại $t = i/n$.
- *Bậc:* curve là đa thức bậc $\leq n$.

4.2 Một số bậc thường gặp

Tuyến tính ($n = 1$): $B(t) = (1-t)P_0 + tP_1$ – chính là phép nội suy tuyến tính giữa hai điểm.

4.2 Specific Low Degrees

Linear ($n = 1,\ P_0, P_1$):

$$B(t) = (1-t)P_0 + tP_1.$$

Quadratic ($n = 2,\ P_0, P_1, P_2$):

$$B(t) = (1-t)^2 P_0 + 2(1-t)tP_1 + t^2 P_2.$$

Cubic ($n = 3,\ P_0, P_1, P_2, P_3$) – the most-used in graphics:

$$B(t) = (1-t)^3 P_0 + 3(1-t)^2 tP_1 + 3(1-t)t^2 P_2 + t^3 P_3.$$

Quartic ($n = 4$):

$$B(t) = \sum_{i=0}^{4} \binom{4}{i} (1-t)^{4-i} t^i P_i.$$

Pascal's triangle (binomial coefficients):

n	coefficients					
0				1		
1			1		1	
2			1	2	1	
3		1	3	3	1	
4	1	4	6	4	1	
5	1	5	10	10	5	1

4.3 De Casteljau Algorithm

A geometric, recursive way to evaluate $B(t)$ without the Bernstein expansion.

Recurrence. Given control points $P_0, \ldots, P_n$ and parameter $t \in [0,1]$, define

$$P_i^{(0)} = P_i, \quad P_i^{(k)}(t) = (1-t)P_i^{(k-1)}(t) + t\,P_{i+1}^{(k-1)}(t),$$

for $k = 1, \ldots, n$ and $i = 0, \ldots, n-k$. Then

$$B(t) = P_0^{(n)}(t).$$

Geometric meaning. On every edge of the control polygon, divide the segment in ratio $t : (1-t)$. Connect the resulting points to obtain a new (shorter) polygon. Repeat. After n steps a single point remains – this is $B(t)$.

Why use it.

- Numerically stable (only convex combinations of input points).
- Cheaper than Bernstein for one t (no powers, only linear interpolations: $O(n^2)$ adds/mults but tiny constants).
- It *also* yields the **subdivision** of the curve at t for free (see §4.9).

Input: control points $P_0, \ldots, P_n$; parameter t
Output: $B(t)$

```
for i ← 0 to n do
    Q_i ← P_i
for k ← 1 to n do
    for i ← 0 to n − k do
        Q_i ← (1 − t) Q_i + t Q_{i+1}
return Q_0
```

Algorithm: De Casteljau (curve)

4.4 Worked Examples (Cubic)

Setup. $P_0 = (0,0),\ P_1 = (1,3),\ P_2 = (3,3),\ P_3 = (4,0)$.

Example A: $t = 0.75$ **via De Casteljau.** $1-t = 0.25$.
Level 1 (3 points):

$$P_0^{(1)} = 0.25(0,0) + 0.75(1,3) = (0.75,\ 2.25),$$
$$P_1^{(1)} = 0.25(1,3) + 0.75(3,3) = (2.50,\ 3.00),$$
$$P_2^{(1)} = 0.25(3,3) + 0.75(4,0) = (3.75,\ 0.75).$$

Bậc hai ($n = 2$): $B(t) = (1-t)^2 P_0 + 2(1-t)tP_1 + t^2 P_2$.
Bậc ba ($n = 3$) – thông dụng nhất trong đồ hoạ và font chữ: $B(t) = (1-t)^3 P_0 + 3(1-t)^2 tP_1 + 3(1-t)t^2 P_2 + t^3 P_3$.
Bậc bốn ($n = 4$): tương tự, hệ số $\binom{4}{i} = 1, 4, 6, 4, 1$.
Tam giác Pascal dùng để tra hệ số nhị thức $\binom{n}{i}$:

n	hệ số					
0				1		
1			1		1	
2			1	2	1	
3		1	3	3	1	
4	1	4	6	4	1	
5	1	5	10	10	5	1

4.3 Thuật toán De Casteljau

Cách tính $B(t)$ bằng *nội suy lặp* – không cần khai triển Bernstein.

Công thức truy hồi. Cho $P_0, \ldots, P_n$ và $t \in [0,1]$, đặt

$$P_i^{(0)} = P_i, \quad P_i^{(k)}(t) = (1-t)P_i^{(k-1)} + t\,P_{i+1}^{(k-1)},$$

với $k = 1, \ldots, n$ và $i = 0, \ldots, n-k$. Khi đó $B(t) = P_0^{(n)}(t)$.

Ý nghĩa hình học. Trên mỗi cạnh của control polygon, lấy điểm chia theo tỉ lệ $t : (1-t)$. Nối các điểm vừa tìm được thành đa giác mới (ngắn hơn). Lặp lại n lần – điểm cuối cùng chính là $B(t)$.

Ưu điểm.

- Ổn định số học (chỉ dùng tổ hợp lồi của các điểm gốc).
- Rẻ hơn Bernstein khi cần một t (không có luỹ thừa); $O(n^2)$ phép cộng-nhân nhưng hệ số rất nhỏ.
- *Tự động cho phép phân chia* (subdivision) đường cong tại t – xem §4.9.

Input: control point $P_0, \ldots, P_n$; tham số t
Output: $B(t)$

```
for i ← 0 to n do
    Q_i ← P_i
for k ← 1 to n do
    for i ← 0 to n − k do
        Q_i ← (1 − t) Q_i + t Q_{i+1}
return Q_0
```

Algorithm: De Casteljau (đường cong)

4.4 Ví dụ chi tiết (bậc 3)

Dữ liệu. $P_0 = (0,0),\ P_1 = (1,3),\ P_2 = (3,3),\ P_3 = (4,0)$.

Ví dụ A: $t = 0.75$ – **thuật toán De Casteljau.** Trọng số 0.25 và 0.75.
Mức 1:

$$P_0^{(1)} = 0.25P_0 + 0.75P_1 = (0.75, 2.25),$$
$$P_1^{(1)} = 0.25P_1 + 0.75P_2 = (2.5, 3),$$
$$P_2^{(1)} = 0.25P_2 + 0.75P_3 = (3.75, 0.75).$$

Level 2 (2 points):

$$\begin{aligned} P_0^{(2)} &= 0.25(0.75, 2.25) + 0.75(2.5, 3) \\ &= (0.1875 + 1.875,\ 0.5625 + 2.25) = (2.0625,\ 2.8125), \\ P_1^{(2)} &= 0.25(2.5, 3) + 0.75(3.75, 0.75) \\ &= (0.625 + 2.8125,\ 0.75 + 0.5625) = (3.4375,\ 1.3125). \end{aligned}$$

Level 3 (final):

$$\begin{aligned} B(0.75) &= 0.25(2.0625, 2.8125) + 0.75(3.4375, 1.3125) \\ &= (0.515625 + 2.578125,\ 0.703125 + 0.984375) \\ &= \boxed{(3.09375,\ 1.6875)}. \end{aligned}$$

Verify with Bernstein. At $t = 0.75$: $B_0^3 = (0.25)^3 = 0.015625$; $B_1^3 = 3(0.25)^2(0.75) = 0.140625$; $B_2^3 = 3(0.25)(0.75)^2 = 0.421875$; $B_3^3 = (0.75)^3 = 0.421875$ (sum $= 1$ ✓).

$$B(0.75) = 0.015625(0, 0) + 0.140625(1, 3) + 0.421875(3, 3) + 0.421875(4, 0).$$

x: $0 + 0.140625 + 1.265625 + 1.6875 = 3.09375$. y: $0 + 0.421875 + 1.265625 + 0 = 1.6875$. ✓

Example B: $t = 0.5$. ($1-t = 0.5$, all weights $\frac{1}{2}$.)

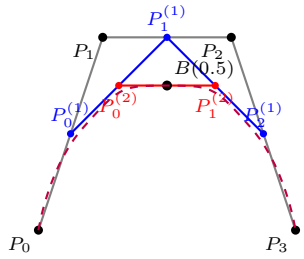
$$\begin{aligned} P_0^{(1)} &= \tfrac{1}{2}((0, 0) + (1, 3)) = (0.5, 1.5), \\ P_1^{(1)} &= \tfrac{1}{2}((1, 3) + (3, 3)) = (2.0, 3.0), \\ P_2^{(1)} &= \tfrac{1}{2}((3, 3) + (4, 0)) = (3.5, 1.5). \\ P_0^{(2)} &= \tfrac{1}{2}((0.5, 1.5) + (2, 3)) = (1.25, 2.25), \\ P_1^{(2)} &= \tfrac{1}{2}((2, 3) + (3.5, 1.5)) = (2.75, 2.25). \\ B(0.5) &= \tfrac{1}{2}((1.25, 2.25) + (2.75, 2.25)) = \boxed{(2.0,\ 2.25)}. \end{aligned}$$

Bernstein check ($B^3 = \frac{1}{8}, \frac{3}{8}, \frac{3}{8}, \frac{1}{8}$): $x = \frac{1}{8} \cdot 0 + \frac{3}{8} \cdot 1 + \frac{3}{8} \cdot 3 + \frac{1}{8} \cdot 4 = \frac{0+3+9+4}{8} = \frac{16}{8} = 2$. $y = \frac{0+9+9+0}{8} = 2.25$. ✓

Example C: $t = 1/3$. ($1-t = 2/3$.)

$$\begin{aligned} P_0^{(1)} &= \tfrac{2}{3}(0, 0) + \tfrac{1}{3}(1, 3) = (\tfrac{1}{3}, 1), \\ P_1^{(1)} &= \tfrac{2}{3}(1, 3) + \tfrac{1}{3}(3, 3) = (\tfrac{5}{3}, 3), \\ P_2^{(1)} &= \tfrac{2}{3}(3, 3) + \tfrac{1}{3}(4, 0) = (\tfrac{10}{3}, 2). \\ P_0^{(2)} &= \tfrac{2}{3}(\tfrac{1}{3}, 1) + \tfrac{1}{3}(\tfrac{5}{3}, 3) = (\tfrac{2}{9} + \tfrac{5}{9},\ \tfrac{2}{3} + 1) = (\tfrac{7}{9}, \tfrac{5}{3}), \\ P_1^{(2)} &= \tfrac{2}{3}(\tfrac{5}{3}, 3) + \tfrac{1}{3}(\tfrac{10}{3}, 2) = (\tfrac{10}{9} + \tfrac{10}{9},\ 2 + \tfrac{2}{3}) = (\tfrac{20}{9}, \tfrac{8}{3}). \\ B(\tfrac{1}{3}) &= \tfrac{2}{3}(\tfrac{7}{9}, \tfrac{5}{3}) + \tfrac{1}{3}(\tfrac{20}{9}, \tfrac{8}{3}) \\ &= (\tfrac{14}{27} + \tfrac{20}{27},\ \tfrac{10}{9} + \tfrac{8}{9}) = \boxed{(\tfrac{34}{27},\ 2)} \approx (1.259,\ 2). \end{aligned}$$

4.4* TikZ Figure: De Casteljau Construction



EN: grey = control polygon; blue = level-1 polygon; red = level-2 segment; black = $B(t)$; dashed purple = the actual Bezier curve.

Mức 2:

$$\begin{aligned} P_0^{(2)} &= 0.25P_0^{(1)} + 0.75P_1^{(1)} = (2.0625, 2.8125), \\ P_1^{(2)} &= 0.25P_1^{(1)} + 0.75P_2^{(1)} = (3.4375, 1.3125). \end{aligned}$$

Mức 3:

$$B(0.75) = 0.25P_0^{(2)} + 0.75P_1^{(2)} = \boxed{(3.09375, 1.6875)}.$$

Kiểm tra bằng Bernstein. Hệ số tại t=0.75: $\frac{1}{64}, \frac{9}{64}, \frac{27}{64}, \frac{27}{64}$ (= 0.015625; 0.140625; 0.421875; 0.421875).
x: 0+0.140625+1.265625+1.6875=3.09375 ✓ y: 0+0.421875+1.265625+0=1.6875 ✓.

Ví dụ B: $t = 0.5$. Mọi trọng số là 0.5.

$$\begin{aligned} P_0^{(1)} &= (0.5, 1.5),\ P_1^{(1)} = (2, 3),\ P_2^{(1)} = (3.5, 1.5), \\ P_0^{(2)} &= (1.25, 2.25),\ P_1^{(2)} = (2.75, 2.25), \\ B(0.5) &= (2.0, 2.25). \end{aligned}$$

Kiểm tra Bernstein $\frac{1}{8}, \frac{3}{8}, \frac{3}{8}, \frac{1}{8}$: $x = \frac{0+3+9+4}{8} = 2$, $y = \frac{0+9+9+0}{8} = 2.25$ ✓.

Ví dụ C: $t = 1/3$. Trọng số 2/3 và 1/3.

$$\begin{aligned} P_0^{(1)} &= (\tfrac{1}{3}, 1),\ P_1^{(1)} = (\tfrac{5}{3}, 3),\ P_2^{(1)} = (\tfrac{10}{3}, 2), \\ P_0^{(2)} &= (\tfrac{7}{9}, \tfrac{5}{3}),\ P_1^{(2)} = (\tfrac{20}{9}, \tfrac{8}{3}), \\ B(\tfrac{1}{3}) &= (\tfrac{34}{27}, 2) \approx (1.259, 2). \end{aligned}$$

Mẹo: khi t là phân số đơn giản, giữ phép tính dạng phân số để tránh sai số.

4.4* Hình minh hoạ De Casteljau

(Hình bên cột trái)

VI: xám = control polygon gốc; xanh = đa giác mức 1 (lấy điểm chia t trên mỗi cạnh); đỏ = đoạn mức 2; đen = điểm $B(t)$; tím nét đứt = đường cong Bezier thực sự.
Quan sát: các đoạn nối các điểm trung gian (đa giác xanh, đỏ) ngày càng tiệm cận đường cong; đa giác mức 1 thậm chí *tiếp xúc* với curve tại $B(t)$ (đó là tiếp tuyến của curve tại t).

Tổng kết các mức:

- Mức 0: $n+1$ điểm gốc $P_0, \ldots, P_n$.
- Mức 1: n điểm $P_i^{(1)}$ – mỗi điểm chia một cạnh.
- Mức k: $n+1-k$ điểm.
- Mức n: *một* điểm duy nhất $= B(t)$.

Mẹo thi: với $n = 3$, kẻ *tam giác De Casteljau* 4 cột → 3 cột → 2 cột → 1 cột. Khi $t = 0.5$, các trọng số đối xứng nên tính nhanh hơn.

4.5 Bezier Surface (Tensor Product)

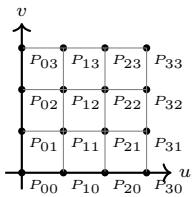
A **Bezier surface (patch)** of bi-degree (m, n) is defined by an $(m{+}1) \times (n{+}1)$ grid of control points P_{ij} (the *control net*):

$$S(u, v) = \sum_{i=0}^{m} \sum_{j=0}^{n} B_i^m(u)\, B_j^n(v)\, P_{ij}, \quad (u, v) \in [0, 1]^2.$$

Properties.

- Corner interpolation: $S(0, 0) = P_{00}$, $S(1, 0) = P_{m0}$, $S(0, 1) = P_{0n}$, $S(1, 1) = P_{mn}$.
- Convex hull of the control net contains the patch.
- Affine invariance.
- *Isoparametric curves* $u = u_0$ or $v = v_0$ are themselves Bezier curves of degree n (resp. m).
- *Boundary curves* are the four Bezier curves through the four edges of the control net.
- Tangent at corner $(0, 0)$ along u: $S_u(0, 0) = m(P_{10} - P_{00})$; along v: $S_v(0, 0) = n(P_{01} - P_{00})$.

Bicubic patch $(m = n = 3)$: $4 \times 4 = 16$ control points – the workhorse of CAD/animation.



4.6 De Casteljau on Surfaces

To compute $S(u, v)$ for a $(m{+}1) \times (n{+}1)$ control net:

Method 1 (rows first).

1. For each row $i = 0, \ldots, m$, run de Casteljau on $(P_{i0}, \ldots, P_{in})$ at parameter $v \Rightarrow$ point Q_i.
2. Run de Casteljau on $(Q_0, \ldots, Q_m)$ at parameter $u \Rightarrow S(u, v)$.

Method 2 (columns first). Swap the order: first reduce columns at u, then the resulting $(n{+}1)$ points at v. Same answer (tensor-product structure commutes).

Cost. $O(m^2 n + mn^2)$ multiplications – dominated by the smaller of the two passes. For bicubic $(m = n = 3)$: $4 \times 6 + 4 \times 6 = 48$ scalar lerps per coordinate $\approx$ trivial.

Input: control net P_{ij}, $i{=}0..m$, $j{=}0..n$; (u, v)
for $i \leftarrow 0$ **to** m **do**
  $Q_i \leftarrow$ **DeCasteljau**$(P_{i0}, \ldots, P_{in};\, v)$
return DeCasteljau$(Q_0, \ldots, Q_m;\, u)$

Algorithm: De Casteljau (surface, row-first)

4.7 Worked Example — Bezier Surface at $(u, v) = (0.5, 0.5)$

Setup (biquadratic, $m = n = 2$). Use a 3×3 control net (only the z-coordinate varies, $x = i$, $y = j$):

$$P_{ij} = (i,\ j,\ h_{ij}), \qquad h = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 0 \end{pmatrix},$$

i.e. a small "tent" peaking at the centre. Compute $S(0.5, 0.5)$.

Step 1 – reduce each row at $v = 0.5$ (quadratic De Casteljau).
For row i, control points are h_{i0}, h_{i1}, h_{i2}; for $v = 0.5$:

$$Q_i = \tfrac{1}{4} h_{i0} + \tfrac{1}{2} h_{i1} + \tfrac{1}{4} h_{i2}.$$

- Row 0: $Q_0^h = \frac{1}{4} \cdot 0 + \frac{1}{2} \cdot 1 + \frac{1}{4} \cdot 0 = \frac{1}{2}$. So $Q_0 = (0,\ 1,\ 0.5)$.
- Row 1: $Q_1^h = \frac{1}{4} \cdot 1 + \frac{1}{2} \cdot 3 + \frac{1}{4} \cdot 1 = \frac{1}{4} + \frac{3}{2} + \frac{1}{4} = 2$. So $Q_1 = (1,\ 1,\ 2)$.
- Row 2: $Q_2^h = \frac{1}{4} \cdot 0 + \frac{1}{2} \cdot 1 + \frac{1}{4} \cdot 0 = \frac{1}{2}$. So $Q_2 = (2,\ 1,\ 0.5)$.

(Note the y-coords come out as 1 because $\frac{1}{4} \cdot 0 + \frac{1}{2} \cdot 1 + \frac{1}{4} \cdot 2 = 1$.)

Step 2 – reduce (Q_0, Q_1, Q_2) at $u = 0.5$.

$$S = \tfrac{1}{4} Q_0 + \tfrac{1}{2} Q_1 + \tfrac{1}{4} Q_2.$$

4.5 Mặt Bezier (tích tensor)

Mặt (patch) Bezier bậc (m, n) được xác định bởi *lưới control net* kích thước $(m{+}1){\times}(n{+}1)$:

$$S(u, v) = \sum_{i=0}^{m} \sum_{j=0}^{n} B_i^m(u) B_j^n(v) P_{ij}.$$

Tính chất.

- Nội suy 4 góc: $S(0, 0) = P_{00}$, $S(1, 0) = P_{m0}$, $S(0, 1) = P_{0n}$, $S(1, 1) = P_{mn}$.
- Nằm trong bao lồi của control net.
- Bất biến affine.
- *Đường isoparametric* $u = u_0$ hoặc $v = v_0$ là đường cong Bezier bậc n (hoặc m).
- *4 đường biên* là 4 curve Bezier nằm trên 4 cạnh của control net.
- Tiếp tuyến tại góc $(0, 0)$: $S_u = m(P_{10} - P_{00})$, $S_v = n(P_{01} - P_{00})$.

Bicubic patch $(m = n = 3)$: 16 control point – dạng phổ biến nhất trong CAD/animation, mô phỏng vải, hoạt cảnh nhân vật...

Đường iso có nghĩa là cố định một biến (vd $u = 0.3$) rồi cho biến kia chạy. Đó vẫn là Bezier curve nên có thể áp dụng lại De Casteljau.

4.6 De Casteljau cho mặt

Để tính $S(u, v)$ trên control net $(m{+}1) \times (n{+}1)$:

Cách 1 (theo hàng trước).

1. Với mỗi hàng $i = 0, \ldots, m$: chạy De Casteljau trên $(P_{i0}, \ldots, P_{in})$ tại v, được điểm Q_i.
2. Chạy De Casteljau trên $(Q_0, \ldots, Q_m)$ tại u, được $S(u, v)$.

Cách 2 (theo cột trước). Đảo thứ tự: rút cột trước (theo u), được $n{+}1$ điểm; rồi rút theo v. Kết quả *giống nhau* do cấu trúc tensor.

Lựa chọn: dùng *cách hàng trước* nếu $m < n$ (ít vòng lặp ngoài), ngược lại dùng *cách cột trước*. Với bicubic thì gần như tương đương.

Quan trọng cho thi: vì cách nào cũng đúng, hãy chọn cách *tận dụng được* các giá trị tròn (vd nếu $v = 0.5$ thì giảm hàng trước cho dễ tính nhẩm).

4.7 Ví dụ mặt Bezier tại $(u, v) = (0.5, 0.5)$

Dữ liệu (bậc $2{\times}2$). Lưới 3×3, $x = i, y = j$, độ cao h_{ij} như bảng:

$$h = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 0 \end{pmatrix}.$$

Tính $S(0.5, 0.5)$.

Bước 1 – giảm hàng tại $v = 0.5$: trọng số bậc 2 là $(\frac{1}{4}, \frac{1}{2}, \frac{1}{4})$.

- Hàng 0: $z = \frac{1}{4}(0) + \frac{1}{2}(1) + \frac{1}{4}(0) = 0.5 \Rightarrow Q_0 = (0, 1, 0.5)$.
- Hàng 1: $z = \frac{1}{4}(1) + \frac{1}{2}(3) + \frac{1}{4}(1) = 2 \Rightarrow Q_1 = (1, 1, 2)$.
- Hàng 2: $z = 0.5 \Rightarrow Q_2 = (2, 1, 0.5)$.

Bước 2 – giảm (Q_0, Q_1, Q_2) tại $u = 0.5$:

$$S = \tfrac{1}{4} Q_0 + \tfrac{1}{2} Q_1 + \tfrac{1}{4} Q_2 = (1, 1, 1.25).$$

$$\boxed{S(0.5, 0.5) = (1, 1, 1.25)}.$$

Kiểm tra. Theo Bernstein 2D, các trọng số là tích Kronecker $(\frac{1}{4}, \frac{1}{2}, \frac{1}{4}) \otimes (\frac{1}{4}, \frac{1}{2}, \frac{1}{4})$, ma trận trọng số $\frac{1}{16}\begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix}$. Nhân với ma trận h rồi cộng tổng: $20/16 = 1.25$ ✓.

Mở rộng cho bicubic: cũng tương tự nhưng có 4 hàng, mỗi hàng giảm theo De Casteljau bậc 3. Sau đó còn 4

$$x : \tfrac{1}{4}\cdot 0 + \tfrac{1}{2}\cdot 1 + \tfrac{1}{4}\cdot 2 = 1,$$
$$y : \tfrac{1}{4}\cdot 1 + \tfrac{1}{2}\cdot 1 + \tfrac{1}{4}\cdot 1 = 1,$$
$$z : \tfrac{1}{4}\cdot 0.5 + \tfrac{1}{2}\cdot 2 + \tfrac{1}{4}\cdot 0.5 = 0.125 + 1 + 0.125 = 1.25.$$
$$\boxed{S(0.5, 0.5) = (1,\ 1,\ 1.25)}.$$

Sanity check (Bernstein direct). For $n = m = 2$ the Bernstein weights at $(0.5, 0.5)$ are $(\frac{1}{4}, \frac{1}{2}, \frac{1}{4}) \otimes (\frac{1}{4}, \frac{1}{2}, \frac{1}{4})$, the z-sum equals

$$\tfrac{1}{16}\sum_{ij} w_{ij}h_{ij} \quad \text{with} \quad w = \begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix}.$$

$\sum w_{ij}h_{ij} = 1\cdot 0 + 2\cdot 1 + 1\cdot 0 + 2\cdot 1 + 4\cdot 3 + 2\cdot 1 + 1\cdot 0 + 2\cdot 1 + 1\cdot 0 = 0 + 2 + 0 + 2 + 12 + 2 + 0 + 2 + 0 = 20$. So $z = 20/16 = 1.25$. ✓

4.8 Other Parametric Curves (Context)

- **Hermite**: defined by 2 endpoints P_0, P_1 and 2 tangents T_0, T_1. Cubic: $H(t) = h_{00}P_0 + h_{10}T_0 + h_{01}P_1 + h_{11}T_1$ with the four Hermite basis polynomials $h_{00} = 2t^3 - 3t^2 + 1$ etc.
- **Catmull–Rom**: an *interpolating* spline through control points; tangent at P_i is $\frac{1}{2}(P_{i+1} - P_{i-1})$. Local control, C^1.
- **B-splines**: piecewise polynomials with *knot vector*; only $k+1$ basis functions are non-zero on a span $\Rightarrow$ *local control* (move one CP and only nearby curve changes). Bezier is the special case of a single non-uniform knot vector $\{0, \ldots, 0, 1, \ldots, 1\}$.
- **NURBS** = Non-Uniform Rational B-Splines. Adds positive weights w_i: $C(t) = \dfrac{\sum w_i N_i^p(t) P_i}{\sum w_i N_i^p(t)}$. Can represent conics exactly (circle, ellipse).

Comparison.

	Interp.?	Local ctrl?	Tangent ctl?	Conics?
Bezier	endpoints only	no	via P_1, P_{n-1}	no
Hermite	endpoints	no	yes (explicit)	no
Catmull–Rom	yes (all CP)	yes	no (auto)	no
B-spline	no	yes	no	no
NURBS	no (ctrl + w)	yes	no	yes

4.9 Subdivision, Derivatives, Degree Elevation

Subdivision (free from De Casteljau). Running de Casteljau at t *produces*, as a side product, the control polygons of the two halves of the curve:

$$\text{Left half: } P_0^{(0)},\ P_0^{(1)},\ P_0^{(2)},\ \ldots,\ P_0^{(n)},$$
$$\text{Right half: } P_0^{(n)},\ P_1^{(n-1)},\ P_2^{(n-2)},\ \ldots,\ P_n^{(0)}.$$

The two halves together reproduce the original curve; this is the basis of *adaptive rendering* (recursively subdivide until the polygon is "flat enough").

Derivative. The derivative of a degree-n Bezier curve is itself a degree-$(n-1)$ Bezier curve on the *forward differences*:

$$B'(t) = n\sum_{i=0}^{n-1} B_i^{n-1}(t)\,(P_{i+1} - P_i).$$

At endpoints: $B'(0) = n(P_1 - P_0)$, $B'(1) = n(P_n - P_{n-1})$ (already noted).

Degree elevation. A degree-n Bezier with points $P_0, \ldots, P_n$ is also a degree-$(n+1)$ Bezier with $n+2$ points P_i':

$$P_i' = \tfrac{i}{n+1}P_{i-1} + \left(1 - \tfrac{i}{n+1}\right)P_i, \quad i = 0, \ldots, n+1,$$

(with P_{-1}, P_{n+1} interpreted via the boundary cases $P_0' = P_0$, $P_{n+1}' = P_n$). Useful to make two Bezier patches/curves have matching degrees before joining.

C^1 joining of two cubic Bezier curves with control points $\{P_0, P_1, P_2, P_3\}$ then $\{Q_0, Q_1, Q_2, Q_3\}$: need $Q_0 = P_3$ (C^0) and $Q_1 - Q_0 = P_3 - P_2$ (C^1), i.e. $Q_1 = 2P_3 - P_2$.

4.10 Exam-Style Problems & Strategies

1. *"Compute $B(t)$ at given t"* → De Casteljau (faster, fewer arithmetic mistakes than Bernstein for one t). Always lay out the triangle.
2. *"Compute $S(u, v)$"* → row-wise De Casteljau then column-wise. Decide order based on which u/v is "nicer".
3. *"Subdivide at t"* → run De Casteljau, then read off the left/right control polygons (§4.9).
4. *"Tangent at endpoint"* → $B'(0) = n(P_1 - P_0)$ or $B'(1) = n(P_n - P_{n-1})$.
5. *"Match C^0/C^1 between curves"* → shared endpoint + collinear adjacent control points with correct length ratio.
6. *"Verify a property"* (convex hull, partition of unity, etc.) → cite Bernstein property; show $\sum B_i^n(t) = 1$ via điểm $Q_0..Q_3$, giảm tiếp 1 lần De Casteljau bậc 3 nữa → ra $S(u, v)$.

4.8 Các loại đường cong tham số khác

- **Hermite**: xác định bởi 2 điểm đầu mút + 2 tiếp tuyến. Bậc 3, hữu ích khi đã có thông tin tiếp tuyến.
- **Catmull–Rom**: spline *nội suy* đi qua các control point; tiếp tuyến tại P_i tự động tính $= \frac{1}{2}(P_{i+1} - P_{i-1})$. Có local control, liên tục C^1.
- **B-spline**: đa thức piecewise, có *knot vector*; mỗi hàm cơ sở chỉ ảnh hưởng cục bộ $\Rightarrow$ *điều khiển địa phương*. Bezier là trường hợp đặc biệt với knot $\{0, \ldots, 0, 1, \ldots, 1\}$.
- **NURBS** = B-spline *hữu tỉ + không đều*; có trọng số w_i, biểu diễn được đường tròn / ellipse *chính xác*, là chuẩn trong CAD.

So sánh.

	Nội suy?	Local?	Tiếp tuyến?	Conic?
Bezier	2 đầu mút	không	qua P_1, P_{n-1}	không
Hermite	2 đầu mút	không	có (tường minh)	không
Catmull–Rom	qua mọi CP	có	tự động	không
B-spline	không	có	không	không
NURBS	không (CP $+w$)	có	không	có

4.9 Phân chia, Đạo hàm, Nâng bậc

Phân chia (subdivision) miễn phí từ De Casteljau. Khi chạy thuật toán tại t, các điểm trung gian *tự sinh ra* 2 control polygon mới:

$$\text{Trái: } P_0^{(0)}, P_0^{(1)}, \ldots, P_0^{(n)}; \quad \text{Phải: } P_0^{(n)}, P_1^{(n-1)}, \ldots, P_n^{(0)}.$$

Hợp lại tái tạo đúng curve gốc – là cơ sở để *vẽ thích nghi* (đệ quy cho tới khi polygon đủ "phẳng" thì vẽ luôn các đoạn thẳng).

Đạo hàm. Đạo hàm của Bezier bậc n là Bezier bậc $n-1$ trên *sai phân*:

$$B'(t) = n\sum_{i=0}^{n-1} B_i^{n-1}(t)(P_{i+1} - P_i).$$

Ở đầu mút: $B'(0) = n(P_1 - P_0)$, $B'(1) = n(P_n - P_{n-1})$.

Nâng bậc (degree elevation). Curve bậc n có thể viết lại thành bậc $n+1$ với $n+2$ điểm

$$P_i' = \tfrac{i}{n+1}P_{i-1} + \left(1 - \tfrac{i}{n+1}\right)P_i.$$

Hữu ích khi cần *khớp bậc* 2 đoạn curve trước khi nối.

Nối C^1 giữa 2 curve bậc 3 $\{P_i\}$ và $\{Q_i\}$: $Q_0 = P_3$ (cùng điểm), $Q_1 = 2P_3 - P_2$ (cùng tiếp tuyến).

4.10 Dạng bài thi & Chiến thuật

1. *"Tính $B(t)$ tại t cho trước"* → dùng De Casteljau (ít lỗi số học hơn Bernstein cho một giá trị t). Vẽ bảng tam giác rõ ràng.
2. *"Tính $S(u, v)$"* → giảm hàng theo v trước, rồi giảm theo u (hoặc ngược lại). Chọn thứ tự sao cho t nào "đẹp" thì làm cuối.
3. *"Phân chia tại t"* → chạy De Casteljau, đọc 2 polygon ở 2 cạnh tam giác (§4.9).
4. *"Tiếp tuyến tại đầu mút"* → $B'(0) = n(P_1 - P_0)$ hoặc $B'(1) = n(P_n - P_{n-1})$.
5. *"Nối C^0/C^1"* → trùng đầu mút + 3 điểm liền kề thẳng hàng, tỉ lệ độ dài hợp lý.

$((1-t)+t)^n = 1$.

7. *"Convert Bezier → monomial"* (rare): multiply out $\sum \binom{n}{i}(1-t)^{n-i}t^i P_i$ symbolically.
8. *"Pick t to minimise computation"*: $t = 0, 1, \frac{1}{2}$ are easiest. $t = \frac{1}{3}, \frac{2}{3}$ keep things in thirds.

Common pitfalls.

- Forgetting that weights are $(1-t)$ for the *left* point and t for the *right* – not the other way around.
- Using $\binom{n}{i}$ wrong (e.g. $\binom{3}{1} = 3$ not 1).
- In a surface problem, mixing up u/v – always label rows vs columns first.
- Forgetting that $B(0) = P_0$, $B(1) = P_n$ (not $P_1, \ldots, P_{n-1}$).

4.11 Bernstein Coefficient Tables

Quadratic ($n = 2$): weights $((1-t)^2,\ 2(1-t)t,\ t^2)$.

t	B_0^2	B_1^2	B_2^2	sum
0.25	$9/16 = 0.5625$	$6/16 = 0.3750$	$1/16 = 0.0625$	1
0.50	$1/4 = 0.25$	$1/2 = 0.50$	$1/4 = 0.25$	1
0.75	$1/16 = 0.0625$	$6/16 = 0.3750$	$9/16 = 0.5625$	1

Cubic ($n = 3$): weights $((1-t)^3,\ 3(1-t)^2t,\ 3(1-t)t^2,\ t^3)$.

t	B_0^3	B_1^3	B_2^3	B_3^3	sum
0.25	$27/64 = 0.421875$	$27/64 = 0.421875$	$9/64 = 0.140625$	$1/64 = 0.015625$	1
0.50	$1/8 = 0.125$	$3/8 = 0.375$	$3/8 = 0.375$	$1/8 = 0.125$	1
0.75	$1/64 = 0.015625$	$9/64 = 0.140625$	$27/64 = 0.421875$	$27/64 = 0.421875$	1
1/3	$8/27 \approx 0.296$	$12/27 \approx 0.444$	$6/27 \approx 0.222$	$1/27 \approx 0.037$	1
2/3	1/27	6/27	12/27	8/27	1

Quartic ($n = 4$) at $t = 0.5$:
$B^4 = (\frac{1}{16}, \frac{4}{16}, \frac{6}{16}, \frac{4}{16}, \frac{1}{16}) = (.0625, .25, .375, .25, .0625)$.
Quintic ($n = 5$) at $t = 0.5$: $B^5 = (1/32,\ 5/32,\ 10/32,\ 10/32,\ 5/32,\ 1/32)$.
Symmetry: $B_i^n(t) = B_{n-i}^n(1-t)$ – so the row at $t = 0.75$ is the row at $t = 0.25$ reversed (visible above).

4.12 One-Glance Formula Box

Bezier curve: $B(t) = \sum_{i=0}^n \binom{n}{i}(1-t)^{n-i}t^i P_i$.
Bernstein: $B_i^n(t) = \binom{n}{i}(1-t)^{n-i}t^i$, $\sum_i B_i^n(t) = 1$.
De Casteljau: $P_i^{(k)} = (1-t)P_i^{(k-1)} + tP_{i+1}^{(k-1)}$; $B(t) = P_0^{(n)}$.
Tangent: $B'(0) = n(P_1 - P_0)$, $B'(1) = n(P_n - P_{n-1})$.
Derivative curve: $B'(t) = n\sum B_i^{n-1}(t)(P_{i+1} - P_i)$.
Degree elevation: $P_i' = \frac{i}{n+1}P_{i-1} + (1 - \frac{i}{n+1})P_i$.
Subdivision halves:
left $= \{P_0^{(0)}, P_0^{(1)}, \ldots, P_0^{(n)}\}$,
right $= \{P_0^{(n)}, P_1^{(n-1)}, \ldots, P_n^{(0)}\}$.
Surface: $S(u,v) = \sum_{i,j} B_i^m(u)B_j^n(v)P_{ij}$.
Surface eval: rows-at-v then column-at-u (or vice-versa).
Corner: $S(0,0) = P_{00}$, etc.
Surface tangents at $(0,0)$**:** $S_u = m(P_{10} - P_{00})$, $S_v = n(P_{01} - P_{00})$.

6. *"Chứng minh tính chất"* → trích dẫn Bernstein; tổng đa thức Bernstein $= ((1-t)+t)^n = 1$.

Lỗi hay gặp.

- Lẫn trọng số: $(1-t)$ đi với điểm *trái*, t đi với điểm *phải*.
- Sai $\binom{n}{i}$ – nên chép luôn dòng Pascal trước khi tính.
- Trong bài surface, lẫn vai trò u/v – ghi rõ i là chỉ số hàng, j là cột.
- Nhớ $B(0) = P_0, B(1) = P_n$ – KHÔNG đi qua các CP ở giữa.

4.11 Bảng tra hệ số Bernstein

Bậc 2: trọng số $((1-t)^2, 2(1-t)t, t^2)$.

t	B_0^2	B_1^2	B_2^2	tổng
0.25	0.5625	0.3750	0.0625	1
0.50	0.25	0.50	0.25	1
0.75	0.0625	0.3750	0.5625	1

Bậc 3:

t	B_0^3	B_1^3	B_2^3	B_3^3	tổng
0.25	0.421875	0.421875	0.140625	0.015625	1
0.50	0.125	0.375	0.375	0.125	1
0.75	0.015625	0.140625	0.421875	0.421875	1
1/3	8/27	12/27	6/27	1/27	1
2/3	1/27	6/27	12/27	8/27	1

Bậc 4 tại $t = 0.5$: $(1, 4, 6, 4, 1)/16 = (0.0625, 0.25, 0.375, 0.25, 0.0625)$.
Bậc 5 tại $t = 0.5$: $(1, 5, 10, 10, 5, 1)/32$.
Đối xứng: $B_i^n(t) = B_{n-i}^n(1-t)$. Vì vậy hàng $t = 0.75$ là hàng $t = 0.25$ đảo chiều – tiết kiệm thời gian khi tra bảng.
Mẹo nhớ: hệ số bậc n là hàng n của tam giác Pascal; phần $(1-t)^{n-i}t^i$ chính là "phép phối $n-i$ lần $1-t$ và i lần t".

4.12 Khung công thức tóm tắt

Curve Bezier: $B(t) = \sum \binom{n}{i}(1-t)^{n-i}t^i P_i$.
Bernstein: $B_i^n = \binom{n}{i}(1-t)^{n-i}t^i$, tổng $= 1$.
De Casteljau: $P_i^{(k)} = (1-t)P_i^{(k-1)} + tP_{i+1}^{(k-1)}$.
Tiếp tuyến đầu mút: $n(P_1 - P_0)$ và $n(P_n - P_{n-1})$.
Đạo hàm: $B'(t) = n\sum B_i^{n-1}(t)(P_{i+1} - P_i)$.
Nâng bậc: $P_i' = \frac{i}{n+1}P_{i-1} + (1 - \frac{i}{n+1})P_i$.
Phân chia: polygon trái $\{P_0^{(0)}, \ldots, P_0^{(n)}\}$, phải $\{P_0^{(n)}, \ldots, P_n^{(0)}\}$.
Mặt: $S(u,v) = \sum B_i^m(u)B_j^n(v)P_{ij}$.
Tính S: giảm hàng tại v rồi giảm cột tại u (hoặc ngược lại).
Góc: $S(0,0) = P_{00}$, $S(1,0) = P_{m0}$, $S(0,1) = P_{0n}$, $S(1,1) = P_{mn}$.
Tiếp tuyến tại $(0,0)$**:** $m(P_{10} - P_{00})$ và $n(P_{01} - P_{00})$.
Mẹo phòng thi: luôn vẽ *tam giác De Casteljau*, đánh dấu trọng số $1-t$ (trái) và t (phải).

6. Worked Example: Bicubic Bezier Surface (4×4) at $(u, v) = (0.5, 0.5)$

6.1 Setup — the 4×4 Control Net

A **bicubic Bezier surface** (degree 3 in both u and v) needs a 4×4 control net $\{P_{ij}\}_{i,j=0}^{3}$. We use the following "hill / tent" net (rows i run along u, columns j run along v):

	$j=0$	$j=1$	$j=2$	$j=3$
i=0	$(0,0,0)$	$(1,0,1)$	$(2,0,1)$	$(3,0,0)$
i=1	$(0,1,1)$	$(1,1,3)$	$(2,1,3)$	$(3,1,1)$
i=2	$(0,2,1)$	$(1,2,3)$	$(2,2,3)$	$(3,2,1)$
i=3	$(0,3,0)$	$(1,3,1)$	$(2,3,1)$	$(3,3,0)$

The (x, y) are on the integer grid; only the z-component varies (saddle/tent peak in the centre). We want $S(0.5,\ 0.5)$. The surface formula:

$$S(u, v) = \sum_{i=0}^{3} \sum_{j=0}^{3} B_i^3(u)\, B_j^3(v)\, P_{ij}.$$

We will compute $S(0.5, 0.5)$ **three ways** and check they match.

6.2 Method 1 — Row-first De Casteljau

Step A. For each row i, run De Casteljau in v at $v = 0.5$ on the four points $P_{i0}, P_{i1}, P_{i2}, P_{i3}$ to obtain Q_i.

Row 0: $(0, 0, 0), (1, 0, 1), (2, 0, 1), (3, 0, 0)$

$$\begin{aligned}
\text{L1:}\ & \tfrac{1}{2}(P_{00}{+}P_{01}) = (0.5, 0, 0.5)\\
& \tfrac{1}{2}(P_{01}{+}P_{02}) = (1.5, 0, 1)\\
& \tfrac{1}{2}(P_{02}{+}P_{03}) = (2.5, 0, 0.5)\\
\text{L2:}\ & \tfrac{1}{2}((0.5, 0, 0.5){+}(1.5, 0, 1)) = (1.0, 0, 0.75)\\
& \tfrac{1}{2}((1.5, 0, 1){+}(2.5, 0, 0.5)) = (2.0, 0, 0.75)\\
\text{L3:}\ & Q_0 = \tfrac{1}{2}((1.0, 0, 0.75){+}(2.0, 0, 0.75))\\
& \quad = (1.5,\ 0,\ 0.75)
\end{aligned}$$

Row 1: $(0, 1, 1), (1, 1, 3), (2, 1, 3), (3, 1, 1)$

$$\begin{aligned}
\text{L1:}\ & (0.5, 1, 2),\ (1.5, 1, 3),\ (2.5, 1, 2)\\
\text{L2:}\ & (1.0, 1, 2.5),\ (2.0, 1, 2.5)\\
\text{L3:}\ & Q_1 = (1.5,\ 1,\ 2.5)
\end{aligned}$$

Row 2: same z-pattern as Row 1 (only y changes from 1 to 2), so by the affine-invariance of De Casteljau the result is

$$Q_2 = (1.5,\ 2,\ 2.5).$$

Row 3: same z-pattern as Row 0, so

$$Q_3 = (1.5,\ 3,\ 0.75).$$

Step B. Now run De Casteljau in u at $u = 0.5$ on Q_0, Q_1, Q_2, Q_3:

$$\begin{aligned}
\text{L1:}\ & \tfrac{1}{2}(Q_0{+}Q_1) = (1.5,\ 0.5,\ 1.625)\\
& \tfrac{1}{2}(Q_1{+}Q_2) = (1.5,\ 1.5,\ 2.5)\\
& \tfrac{1}{2}(Q_2{+}Q_3) = (1.5,\ 2.5,\ 1.625)\\
\text{L2:}\ & \tfrac{1}{2}((1.5, 0.5, 1.625){+}(1.5, 1.5, 2.5))\\
& \quad = (1.5,\ 1.0,\ 2.0625)\\
& \tfrac{1}{2}((1.5, 1.5, 2.5){+}(1.5, 2.5, 1.625))\\
& \quad = (1.5,\ 2.0,\ 2.0625)\\
\text{L3:}\ & S(0.5, 0.5) = \tfrac{1}{2}((1.5, 1.0, 2.0625){+}(1.5, 2.0, 2.0625))
\end{aligned}$$

$$\boxed{S(0.5,\ 0.5) = (1.5,\ 1.5,\ 2.0625)}$$

6.3 Method 2 — Column-first verification

Swap the order: first De Casteljau in u on each column j at $u = 0.5$ to obtain R_j, then in v on R_0, R_1, R_2, R_3 at $v = 0.5$.

Column 0: $(0, 0, 0), (0, 1, 1), (0, 2, 1), (0, 3, 0)$
$\Rightarrow R_0 = (0,\ 1.5,\ 0.75)$ (same z-arithmetic as Row 0).
Column 1: $(1, 0, 1), (1, 1, 3), (1, 2, 3), (1, 3, 1)$
$\Rightarrow R_1 = (1,\ 1.5,\ 2.5)$.
Column 2: by symmetry $R_2 = (2,\ 1.5,\ 2.5)$.
Column 3: by symmetry $R_3 = (3,\ 1.5,\ 0.75)$.

6. Ví dụ tính tay: Mặt Bezier bicubic (4×4) tại $(u, v) = (0.5, 0.5)$

6.1 Thiết lập — lưới control net 4×4

Một **mặt Bezier bicubic** (bậc 3 theo cả u và v) cần lưới 4×4 control point $\{P_{ij}\}_{i,j=0}^{3}$. Ta dùng lưới "đồi/lều" sau (hàng i chạy theo hướng u, cột j chạy theo hướng v):

	$j=0$	$j=1$	$j=2$	$j=3$
i=0	$(0,0,0)$	$(1,0,1)$	$(2,0,1)$	$(3,0,0)$
i=1	$(0,1,1)$	$(1,1,3)$	$(2,1,3)$	$(3,1,1)$
i=2	$(0,2,1)$	$(1,2,3)$	$(2,2,3)$	$(3,2,1)$
i=3	$(0,3,0)$	$(1,3,1)$	$(2,3,1)$	$(3,3,0)$

Toạ độ (x, y) nằm trên lưới số nguyên; chỉ thành phần z thay đổi (tạo dạng yên ngựa/đỉnh đồi ở giữa). Ta cần tính $S(0.5,\ 0.5)$.
Công thức mặt:

$$S(u, v) = \sum_{i=0}^{3} \sum_{j=0}^{3} B_i^3(u)\, B_j^3(v)\, P_{ij}.$$

Ta sẽ tính $S(0.5, 0.5)$ **theo ba cách** và kiểm tra trùng kết quả.

6.2 Cách 1 — De Casteljau theo hàng trước

Bước A. Với mỗi hàng i, chạy De Casteljau theo v tại $v = 0.5$ trên 4 điểm $P_{i0}, P_{i1}, P_{i2}, P_{i3}$ để được Q_i.

Hàng 0: $(0, 0, 0), (1, 0, 1), (2, 0, 1), (3, 0, 0)$

$$\begin{aligned}
\text{L1:}\ & \tfrac{1}{2}(P_{00}{+}P_{01}) = (0.5, 0, 0.5)\\
& \tfrac{1}{2}(P_{01}{+}P_{02}) = (1.5, 0, 1)\\
& \tfrac{1}{2}(P_{02}{+}P_{03}) = (2.5, 0, 0.5)\\
\text{L2:}\ & \tfrac{1}{2}((0.5, 0, 0.5){+}(1.5, 0, 1)) = (1.0, 0, 0.75)\\
& \tfrac{1}{2}((1.5, 0, 1){+}(2.5, 0, 0.5)) = (2.0, 0, 0.75)\\
\text{L3:}\ & Q_0 = \tfrac{1}{2}((1.0, 0, 0.75){+}(2.0, 0, 0.75))\\
& \quad = (1.5,\ 0,\ 0.75)
\end{aligned}$$

Hàng 1: $(0, 1, 1), (1, 1, 3), (2, 1, 3), (3, 1, 1)$

$$\begin{aligned}
\text{L1:}\ & (0.5, 1, 2),\ (1.5, 1, 3),\ (2.5, 1, 2)\\
\text{L2:}\ & (1.0, 1, 2.5),\ (2.0, 1, 2.5)\\
\text{L3:}\ & Q_1 = (1.5,\ 1,\ 2.5)
\end{aligned}$$

Hàng 2: có cùng pattern z với Hàng 1 (chỉ y đổi từ 1 sang 2), nhờ tính bất biến affine của De Casteljau ta có

$$Q_2 = (1.5,\ 2,\ 2.5).$$

Hàng 3: có cùng pattern z với Hàng 0, nên

$$Q_3 = (1.5,\ 3,\ 0.75).$$

Bước B. Bây giờ chạy De Casteljau theo u tại $u = 0.5$ trên Q_0, Q_1, Q_2, Q_3:

$$\begin{aligned}
\text{L1:}\ & \tfrac{1}{2}(Q_0{+}Q_1) = (1.5,\ 0.5,\ 1.625)\\
& \tfrac{1}{2}(Q_1{+}Q_2) = (1.5,\ 1.5,\ 2.5)\\
& \tfrac{1}{2}(Q_2{+}Q_3) = (1.5,\ 2.5,\ 1.625)\\
\text{L2:}\ & \tfrac{1}{2}((1.5, 0.5, 1.625){+}(1.5, 1.5, 2.5))\\
& \quad = (1.5,\ 1.0,\ 2.0625)\\
& \tfrac{1}{2}((1.5, 1.5, 2.5){+}(1.5, 2.5, 1.625))\\
& \quad = (1.5,\ 2.0,\ 2.0625)\\
\text{L3:}\ & S(0.5, 0.5) = \tfrac{1}{2}((1.5, 1.0, 2.0625){+}(1.5, 2.0, 2.0625))
\end{aligned}$$

$$\boxed{S(0.5,\ 0.5) = (1.5,\ 1.5,\ 2.0625)}$$

6.3 Cách 2 — Kiểm tra bằng cột trước

Đảo thứ tự: De Casteljau theo u trên mỗi cột j tại $u = 0.5$ để được R_j, rồi De Casteljau theo v trên R_0, R_1, R_2, R_3 tại $v = 0.5$.

Cột 0: $(0, 0, 0), (0, 1, 1), (0, 2, 1), (0, 3, 0)$
$\Rightarrow R_0 = (0,\ 1.5,\ 0.75)$ (z tính giống Hàng 0).
Cột 1: $(1, 0, 1), (1, 1, 3), (1, 2, 3), (1, 3, 1)$
$\Rightarrow R_1 = (1,\ 1.5,\ 2.5)$.
Cột 2: đối xứng $R_2 = (2,\ 1.5,\ 2.5)$.
Cột 3: đối xứng $R_3 = (3,\ 1.5,\ 0.75)$.

Now De Casteljau on $R_0, \ldots, R_3$ at $v = 0.5$:

$$\begin{aligned}
&\text{L1: } (0.5, 1.5, 1.625),\ (1.5, 1.5, 2.5),\ (2.5, 1.5, 1.625)\\
&\text{L2: } (1.0, 1.5, 2.0625),\ (2.0, 1.5, 2.0625)\\
&\text{L3: } (1.5,\ 1.5,\ 2.0625)\quad \checkmark
\end{aligned}$$

Same answer — the tensor-product structure makes the two orders commute.

6.4 Method 3 — Direct Bernstein verification

At $t = 0.5$ the cubic Bernstein basis is

$$B_i^3(0.5) = \tfrac{1}{8}\big(1,\ 3,\ 3,\ 1\big), \quad i = 0, 1, 2, 3.$$

The bicubic weights $w_{ij} = B_i^3(0.5)\, B_j^3(0.5)$ form (in units of 1/64):

$$[w_{ij}] \cdot 64 = \begin{pmatrix} 1 & 3 & 3 & 1 \\ 3 & 9 & 9 & 3 \\ 3 & 9 & 9 & 3 \\ 1 & 3 & 3 & 1 \end{pmatrix}.$$

Sum check: $4(1) + 8(3) + 4(9) = 4{+}24{+}36 = 64$. ✓

Compute z-component first (the only non-trivial one). Z-matrix is

$$[z_{ij}] = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 3 & 3 & 1 \\ 1 & 3 & 3 & 1 \\ 0 & 1 & 1 & 0 \end{pmatrix}.$$

$$\begin{aligned}
64\,z &= \underbrace{1{\cdot}0{\cdot}4}_{\text{corners}} + \underbrace{3{\cdot}1{\cdot}8}_{\text{edge mids}} + \underbrace{9{\cdot}3{\cdot}4}_{\text{interior}}\\
&= 0 + 24 + 108 = 132,\\
z &= 132/64 = 2.0625. \checkmark
\end{aligned}$$

x-component: by row sums, $\sum_j w_{ij} x_{ij}$ uses $x_{ij} = j$, weighted along each row by $(1, 3, 3, 1)/8$:

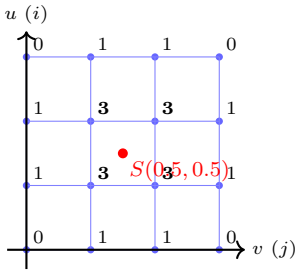
$$\bar{x}_{\text{row}} = \tfrac{1{\cdot}0+3{\cdot}1+3{\cdot}2+1{\cdot}3}{8} = \tfrac{12}{8} = 1.5,$$

and this is the same for every row, so $x = 1.5$.

y-component: symmetric argument with $y_{ij} = i$ gives $y = 1.5$.

Final result $(1.5, 1.5, 2.0625)$ *matches the De Casteljau answer.* ✓

6.5 Control Net Diagram



Top-down view: numbers are z-heights of P_{ij}. Red dot = projected evaluation point.

6.6 Exam Tips

- **Direction discipline:** write down which axis is u (rows i) and which is v (columns j) before the first arithmetic step.
- **Row-first is usually faster** on paper: you can reuse symmetry between rows (e.g. Row 0 vs Row 3, Row 1 vs Row 2) to skip duplicate work.
- **Always sanity-check with Bernstein.** At $(0.5, 0.5)$ the cubic weights are $(1, 3, 3, 1)/8$, so the 4×4 weights are $\{1, 3, 9\}/64$ — *memorise these.*
- Check the corner/edge/interior tally: $4{\cdot}1 + 8{\cdot}3 + 4{\cdot}9 = 64$.
- For other (u, v), compute $B_i^3(u)$ and $B_j^3(v)$ separately, then form the outer product.
- $S(u, v)$ at the corners equals the corner control points: $S(0, 0) = P_{00}$ etc. — handy for instant validation.

De Casteljau trên $R_0, \ldots, R_3$ tại $v = 0.5$:

$$\begin{aligned}
&\text{L1: } (0.5, 1.5, 1.625),\ (1.5, 1.5, 2.5),\ (2.5, 1.5, 1.625)\\
&\text{L2: } (1.0, 1.5, 2.0625),\ (2.0, 1.5, 2.0625)\\
&\text{L3: } (1.5,\ 1.5,\ 2.0625)\quad \checkmark
\end{aligned}$$

Cùng kết quả — cấu trúc tích tensor (tensor product) khiến hai thứ tự giao hoán.

6.4 Cách 3 — Kiểm tra trực tiếp bằng Bernstein

Tại $t = 0.5$ cơ sở Bernstein bậc 3 là

$$B_i^3(0.5) = \tfrac{1}{8}\big(1,\ 3,\ 3,\ 1\big), \quad i = 0, 1, 2, 3.$$

Trọng số bicubic $w_{ij} = B_i^3(0.5)\, B_j^3(0.5)$ (đơn vị 1/64):

$$[w_{ij}] \cdot 64 = \begin{pmatrix} 1 & 3 & 3 & 1 \\ 3 & 9 & 9 & 3 \\ 3 & 9 & 9 & 3 \\ 1 & 3 & 3 & 1 \end{pmatrix}.$$

Kiểm tra tổng: $4(1) + 8(3) + 4(9) = 4{+}24{+}36 = 64$. ✓

Tính thành phần z (chỗ duy nhất không tầm thường). Ma trận z:

$$[z_{ij}] = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 3 & 3 & 1 \\ 1 & 3 & 3 & 1 \\ 0 & 1 & 1 & 0 \end{pmatrix}.$$

$$\begin{aligned}
64\,z &= \underbrace{1{\cdot}0{\cdot}4}_{\text{4 góc}} + \underbrace{3{\cdot}1{\cdot}8}_{\text{8 ô cạnh}} + \underbrace{9{\cdot}3{\cdot}4}_{\text{4 ô trong}}\\
&= 0 + 24 + 108 = 132,\\
z &= 132/64 = 2.0625. \checkmark
\end{aligned}$$

Thành phần x: với $x_{ij} = j$, mỗi hàng cho cùng giá trị

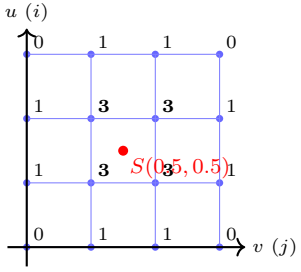
$$\bar{x}_{\text{hàng}} = \tfrac{1{\cdot}0+3{\cdot}1+3{\cdot}2+1{\cdot}3}{8} = \tfrac{12}{8} = 1.5,$$

nên $x = 1.5$.

Thành phần y: đối xứng với $y_{ij} = i$ cho $y = 1.5$.

Kết quả $(1.5, 1.5, 2.0625)$ *trùng với De Casteljau.* ✓

6.5 Sơ đồ control net



Nhìn từ trên xuống: số ghi là độ cao z của P_{ij}. Chấm đỏ = hình chiếu của điểm cần tính.

6.6 Mẹo làm bài thi

- **Ghi rõ hướng tham số:** viết ngay từ đầu trục nào là u (hàng i), trục nào là v (cột j) trước khi làm phép tính đầu tiên.
- **Cách hàng-trước (row-first) thường nhanh hơn** khi tính tay: tận dụng đối xứng giữa các hàng (vd. Hàng 0 đối xứng Hàng 3, Hàng 1 đối xứng Hàng 2) để bỏ qua phép tính trùng.
- **Luôn check chéo bằng Bernstein.** Tại $(0.5, 0.5)$ trọng số bậc 3 là $(1, 3, 3, 1)/8$, nên ma trận $4{\times}4$ là $\{1, 3, 9\}/64$ — *thuộc lòng bộ số này.*
- Kiểm tra tổng góc/cạnh/trong: $4{\cdot}1 + 8{\cdot}3 + 4{\cdot}9 = 64$.
- Với (u, v) khác, tính $B_i^3(u)$ và $B_j^3(v)$ riêng, rồi nhân ngoài (outer product) thành ma trận 4×4.
- $S(u, v)$ ở 4 góc bằng đúng 4 control point góc: $S(0, 0) = P_{00}$,...— dùng để kiểm tra nhanh.

Part II — Theoretical Q&A Bank

Essay-style answers organised by topic for quick recall during exam

QA1. Theoretical Q&A — VR / AR

Q1. What is Virtual Reality (VR)?
Virtual Reality is a computer-generated, fully **immersive** 3D environment that completely replaces the user's real-world view. The user wears a **Head-Mounted Display (HMD)** such as the **Meta Quest 3**, **HTC Vive Pro**, **Valve Index**, or **PlayStation VR2**, plus tracked controllers and sometimes haptic gloves. The system tracks the user's head and hand pose in real time and renders a **stereoscopic** pair of images (one per eye) at **90–120 Hz** so the brain perceives true depth. The goals of VR are **presence** (the feeling of "being there") and **immersion** (sensory replacement). Typical applications include training simulators, surgical rehearsal, exposure therapy, architectural walkthroughs, and games such as *Half-Life: Alyx* and *Beat Saber*.

Q2. What is Augmented Reality (AR)?
Augmented Reality *augments* the real world by overlaying digital content (3D models, text, animations) onto the user's view of the physical environment, instead of replacing it. AR can run on **smartphones/tablets** (video see-through, e.g. Pokemon Go, IKEA Place) or on **AR glasses** (optical see-through, e.g. Microsoft HoloLens 2, Magic Leap 2, Apple Vision Pro in passthrough). Core capabilities are **6DoF tracking**, **plane detection**, **anchoring**, **occlusion**, and **light estimation**. Common SDKs are **ARKit** (iOS), **ARCore** (Android), **Vuforia** (cross-platform marker-based), and **Unity AR Foundation** (unifies them). Applications include navigation arrows on streets, surgical guidance overlay, AR try-on for retail, and industrial maintenance instructions.

Q3. What is Mixed Reality (MR)?
Mixed Reality sits between AR and VR on the **Milgram Reality–Virtuality Continuum**. In MR, virtual objects are not just overlaid but actually **interact with** the real environment: they understand the geometry of walls and furniture, support **occlusion** (a real hand can hide a virtual ball), respond to real lighting, and accept physical interaction. MR therefore requires **spatial mapping** (a real-time mesh of the room produced by depth sensors) plus **hand and eye tracking**. Devices designed for MR include the **Microsoft HoloLens 2**, **Magic Leap 2**, **Meta Quest 3** (color passthrough), and **Apple Vision Pro**. Use cases: collaborative engineering review, holographic medical training, factory digital twins.

Q4. What is XR (Extended Reality)?
XR is the **umbrella term** that encompasses VR, AR, and MR — "X" stands for any reality. It is used in industry standards (e.g., **OpenXR**) and in research because the boundaries between the three are increasingly blurry: a Quest 3 in passthrough mode is AR/MR, the same device with passthrough off is VR. Talking about "XR" lets us describe **cross-cutting concerns** such as tracking, rendering, input, ergonomics, and content authoring without committing to one mode. The **OpenXR** standard (Khronos Group) is the de-facto cross-vendor API, supported by Meta, Microsoft, Valve, HTC, Pico, and Apple. Modern game engines like **Unity** and **Unreal** target OpenXR so a single codebase runs on all major XR headsets.

Q5. Compare VR vs AR vs MR (in one paragraph and a table).
The three differ mainly in **how much real world remains visible** and **how virtual objects interact with reality**.

	VR	AR	MR
Real world	hidden	visible	visible
Virtual obj.	full env.	overlay	integrated
Interaction	with virt.	passive	with real
Display	opaque HMD	phone/glass	MR HMD
Example	Quest VR	Pokemon Go	HoloLens 2

In short: **VR replaces**, **AR adds on top**, **MR blends with awareness**. All three sit on the Milgram continuum and are unified under **XR**.

Q6. What is an HMD and what are its main components?
A **Head-Mounted Display (HMD)** is the wearable display that puts visuals close to the eyes. Its main components are: (1) two **micro-displays** (LCD, OLED, or micro-OLED), one per eye, with resolutions like 2064×2208 per eye on Quest 3; (2) **Fresnel** or **pancake lenses** that magnify the image so the eye can focus at near distance; (3) an **IPD adjustment** for inter-pupillary distance; (4) an **IMU** (gyroscope + accelerometer + magnetometer) for fast head rotation tracking; (5) outward-facing **cameras** for inside-out 6DoF tracking and passthrough; (6) sometimes **eye-tracking cameras** for foveated rendering. Standalone HMDs (Quest 3) include an SoC like Snapdragon XR2 Gen 2; PC-VR HMDs (Vive Pro 2, Index) stream from a GPU through DisplayPort.

Q7. What is 6DoF tracking? Compare with 3DoF.
Degrees of Freedom (DoF) = number of independent motions the system can sense. **3DoF** senses only *rotations* around three axes (yaw, pitch, roll) — the user can look around but cannot walk; typical of cheap mobile VR like the original Cardboard or Oculus Go. **6DoF** adds the three *translations* (forward/back, left/right, up/down), so the user can physically walk around and lean. 6DoF requires either **outside-in** tracking (e.g., HTC Vive Lighthouse base stations emitting IR sweeps) or **inside-out** tracking (cameras on the HMD doing visual SLAM, e.g., Quest 3, HoloLens 2). 6DoF is essential for **presence**, room-scale interaction, and is the standard for any modern VR/AR experience.

Q8. What is SLAM and how does it work?
SLAM = Simultaneous Localization And Mapping. The headset (or robot, drone, phone) builds a map of an unknown environment *while* tracking its own pose inside that map, in real time. Pipeline: (1) **Feature**

QA1. Câu hỏi lý thuyết — VR / AR

Câu 1. Thực tại ảo (VR) là gì?
Virtual Reality (VR) là một môi trường 3D do máy tính tạo ra, **hoàn toàn thay thế** thế giới thực mà người dùng nhìn thấy. Người dùng đeo một **Head-Mounted Display (HMD)** như **Meta Quest 3**, **HTC Vive Pro**, **Valve Index** hoặc **PSVR2**, kèm theo controller và đôi khi haptic gloves. Hệ thống **tracking** đầu và tay người dùng theo thời gian thực, rồi **render** một cặp ảnh **stereoscopic** (mỗi mắt một ảnh) ở **90–120 Hz** để não cảm nhận chiều sâu. Mục tiêu của VR là tạo **presence** (cảm giác "đang ở trong đó") và **immersion** (thay thế cảm giác). Ứng dụng phổ biến: huấn luyện, mô phỏng phẫu thuật, trị liệu phơi nhiễm, đi dạo kiến trúc, game như *Half-Life: Alyx*, *Beat Saber*.

Câu 2. Thực tại tăng cường (AR) là gì?
Augmented Reality (AR) *bổ sung* nội dung số (mô hình 3D, text, animation) lên thế giới thực thay vì thay thế nó. AR chạy trên **smartphone/tablet** (video see-through, ví dụ Pokemon Go, IKEA Place) hoặc trên **AR glasses** (optical see-through như HoloLens 2, Magic Leap 2, Apple Vision Pro). Năng lực cốt lõi gồm: **6DoF tracking**, **plane detection**, **anchoring**, **occlusion**, **light estimation**. Các SDK phổ biến: **ARKit** (iOS), **ARCore** (Android), **Vuforia** (cross-platform, marker-based), **Unity AR Foundation** (gói chung). Ứng dụng: chỉ đường ngoài phố, hướng dẫn phẫu thuật, thử đồ ảo, hướng dẫn bảo trì công nghiệp.

Câu 3. Mixed Reality (MR) là gì?
Mixed Reality (MR) nằm giữa AR và VR trên **Milgram Reality–Virtuality Continuum**. Vật thể ảo không chỉ "đè lên" mà thực sự **tương tác** với môi trường thật: hiểu được hình học tường, bàn ghế, hỗ trợ **occlusion** (tay thật che được quả bóng ảo), phản ứng với ánh sáng thật, và cho phép tương tác vật lý. MR vì vậy cần **spatial mapping** (lưới 3D của phòng từ depth sensor) cộng với **hand tracking** và **eye tracking**. Thiết bị MR: **HoloLens 2**, **Magic Leap 2**, **Meta Quest 3** (color passthrough), **Apple Vision Pro**. Use case: review kỹ thuật phối hợp, huấn luyện y khoa hologram, digital twin nhà máy.

Câu 4. XR (Extended Reality) là gì?
XR là **thuật ngữ ô** bao gồm VR, AR và MR — "X" nghĩa là bất kỳ loại reality nào. Nó được dùng trong chuẩn công nghiệp (ví dụ **OpenXR**) và trong nghiên cứu, vì ranh giới giữa ba loại ngày càng mờ: Quest 3 bật passthrough là AR/MR, tắt passthrough thì là VR. Dùng từ "XR" giúp ta nói về các vấn đề **xuyên suốt** như tracking, render, input, ergonomic mà không cần chọn loại cụ thể. Chuẩn **OpenXR** (Khronos Group) là API cross-vendor de-facto, hỗ trợ bởi Meta, Microsoft, Valve, HTC, Pico, Apple. Các engine **Unity**, **Unreal** đều target OpenXR nên một codebase chạy được trên đa số headset.

Câu 5. So sánh VR, AR, MR (1 đoạn + bảng).
Khác biệt chính nằm ở **lượng thế giới thực còn nhìn thấy** và **mức độ tương tác** của vật thể ảo với thực tại.

	VR	AR	MR
TG thực	ẩn	thấy	thấy
Vật ảo	toàn bộ	overlay	tích hợp
Tương tác	với ảo	thụ động	với thực
Display	HMD đen	phone/kính	MR HMD
Ví dụ	Quest VR	Pokemon Go	HoloLens 2

Tóm lại: **VR thay thế**, **AR thêm vào**, **MR hòa quyện**. Cả ba nằm trên Milgram continuum và gộp chung thành **XR**.

Câu 6. HMD là gì? Các thành phần chính.
Head-Mounted Display (HMD) là thiết bị đeo đầu đưa hình ảnh sát mắt. Thành phần chính: (1) hai **micro-display** (LCD, OLED hoặc micro-OLED), mỗi mắt một, ví dụ Quest 3 có 2064×2208 mỗi mắt; (2) **Fresnel** hoặc **pancake lens** phóng đại để mắt focus được ở khoảng gần; (3) **IPD adjustment** để chỉnh khoảng cách hai mắt; (4) **IMU** (gyro + accel + magnetometer) đo xoay đầu siêu nhanh; (5) **camera** hướng ngoài để inside-out 6DoF tracking và passthrough; (6) đôi khi có **eye-tracking camera** cho foveated rendering. HMD standalone (Quest 3) có SoC kiểu Snapdragon XR2 Gen 2; PC-VR HMD (Vive Pro 2, Index) nhận stream từ GPU qua DisplayPort.

Câu 7. 6DoF tracking là gì? So với 3DoF?
Degrees of Freedom (DoF) = số chuyển động độc lập hệ thống cảm nhận được. **3DoF** chỉ cảm nhận *xoay* quanh 3 trục (yaw/pitch/roll) — người dùng nhìn quanh được nhưng không đi được; đặc trưng của mobile VR rẻ như Cardboard hoặc Oculus Go. **6DoF** thêm 3 chiều *tịnh tiến* (trước/sau, trái/phải, lên/xuống), người dùng đi và nghiêng được. 6DoF cần **outside-in** (HTC Vive Lighthouse phát IR sweep) hoặc **inside-out** (camera trên HMD chạy visual SLAM, ví dụ Quest 3, HoloLens 2). 6DoF là điều kiện cần để có **presence**, room-scale interaction, và là chuẩn của VR/AR hiện đại.

Câu 8. SLAM là gì? Hoạt động ra sao?
SLAM = Simultaneous Localization And Mapping. Headset (hoặc robot, drone, phone) vừa *xây bản đồ* môi trường lạ, vừa *định vị bản thân* trong bản đồ đó, theo thời gian thực. Pipeline: (1) **Feature extraction** từ mỗi

extraction from each frame (corners/keypoints via FAST, ORB, or learned features); (2) **Feature matching** between consecutive frames; (3) **Pose estimation** via PnP / essential matrix; (4) **IMU fusion** with an Extended Kalman Filter or factor graph for smoothness during fast motion; (5) **Mapping**: triangulate matched features into 3D landmarks; (6) **Loop closure** corrects accumulated **drift** when a previously seen place is recognized; (7) **Bundle adjustment** optimizes all poses + landmarks. Visual–inertial SLAM (VI-SLAM) underlies ARKit, ARCore, HoloLens, and Quest tracking.

Q9. What is presence? What is immersion?

Immersion is an **objective property of the system**: how well the hardware replaces sensory input (high resolution, wide **FOV**, low latency, spatial audio, haptics). **Presence** is the **subjective feeling** of "being there" that the user experiences *because* immersion is high. Immersion enables presence but does not guarantee it: bad content, low frame rate, or cybersickness can break presence even on great hardware. Presence is usually measured with **questionnaires** like the Slater–Usoh–Steed (SUS) or Witmer–Singer Presence Questionnaire, or with physiological proxies (heart rate, EDA). Designers maximize presence with consistent **6DoF tracking**, $\geq$ 90 Hz refresh, $<$ 20 ms motion-to-photon latency, spatial audio, and believable interaction.

Q10. How does VR work step-by-step (the rendering loop)?

The VR loop runs at 90–120 Hz and must finish each frame in $\sim$11 ms or less:
(1) Sense: read **IMU** + cameras to update head pose; **(2) Predict:** extrapolate the pose to the photon-display time (**motion prediction**); **(3) Update:** run game logic, physics, animation; **(4) Cull:** frustum + occlusion culling; **(5) Render twice:** produce the *left-eye* and *right-eye* images using two slightly offset projection matrices (one per camera, separated by IPD); **(6) Distortion correction:** pre-warp each image to cancel the lens's pincushion distortion and chromatic aberration; **(7) Asynchronous Time Warp / Space Warp:** just before scan-out, re-project the finished image with the latest pose to compensate for any remaining latency; **(8) Scan out** to the panel, scanline-synced. Spatial audio is computed in parallel with HRTFs.

Q11. How does the AR pipeline work step-by-step?

(1) Camera capture: grab the next frame from the rear camera (or passthrough cameras); **(2) Tracking:** run **VI-SLAM** to estimate the device's 6DoF pose; **(3) Scene understanding:** detect **planes** (floor, table), feature points, meshes, faces, hands, image targets; **(4) Light estimation:** infer ambient intensity, color, and a directional source (or spherical harmonics) from the camera image; **(5) Anchor management:** attach virtual objects to anchors that are stable in the real world; **(6) Rendering:** render virtual objects with the camera's intrinsics and the estimated pose; apply real-world lighting; perform **occlusion** using the depth map or scene mesh; **(7) Composite:** blend the rendered overlay with the camera image (video see-through) or display on the optical combiner (optical see-through); **(8) Display** at $\geq$ 60 Hz.

Q12. How does inside-out tracking work?

In **inside-out tracking**, the cameras are *on the headset* and look outward; there are no external base stations. Each frame, the system extracts **visual features** (corners), matches them across multiple cameras (stereo or wider arrays) and across time, and combines this with the **IMU** via **visual–inertial odometry**. The result is a 6DoF pose at very high rate (1000 Hz from IMU, corrected by 30–60 Hz visual updates). For controllers, the same outward cameras observe **IR LEDs** on the controller (Quest's constellation) or perform fully optical hand tracking (HoloLens, Quest hand tracking, Vision Pro). Pros: no setup, portable, fewer cables. Cons: needs textured environment, can lose tracking in the dark or in front of a blank wall, controller can be lost behind the back.

Q13. Compare outside-in (Lighthouse) vs inside-out tracking.

Outside-in uses external sensors (Lighthouse base stations sweeping IR planes; or external cameras like the original Oculus Rift CV1) that observe the headset/controllers; **inside-out** reverses this: cameras are on the headset. Outside-in offers **sub-millimeter precision**, near-zero jitter, large play areas (Vive Lighthouse covers $\sim$10 m), and reliable controller tracking even behind the back. Its drawbacks: cumbersome setup, fixed play space, needs power outlets and line-of-sight. Inside-out is **plug-and-play**, portable, cheaper, supports any room, but has slightly more jitter, can drift, struggles in low light or feature-poor rooms, and may lose controllers occluded from the headset cameras. Modern consumer HMDs (Quest, HoloLens, Vision Pro, PSVR2) all use inside-out; pro/research setups still favor Lighthouse.

Q14. How does marker-based AR work?

Marker-based AR relies on a known image (**fiducial marker** like ARTag/ArUco, or a textured **image target** in Vuforia) printed in the real world. Steps: (1) The camera grabs a frame; (2) The image is binarized/thresholded and contours are found; (3) Quadrilateral contours are matched against the marker's known pattern (or feature descriptors are matched against the image database); (4) The four corners give a 2D–3D correspondence; (5) Solving **PnP (Perspective-n-Point)** with the camera intrinsics yields the 6DoF pose of the marker relative to the camera; (6) A virtual object is rendered at that pose. Marker-based AR is **robust, fast, and free of drift**, used in print-to-AR books, museum exhibits, packaging promotions, and education — but the marker must remain visible.

Q15. Compare marker-based vs markerless AR.

Marker-based requires a printed marker/image target in the scene; **markerless** relies on natural features and SLAM. Marker-based is **simple, robust, drift-free, low-cost** but constrained: the experience dies if the marker is occluded, lit poorly, or out of frame. Markerless (ARKit, ARCore, HoloLens) discovers planes and feature points anywhere, supports walking around the scene, and enables persistent **world anchors** via cloud anchoring (ARCore Cloud Anchors, ARKit ARWorldMap). Its trade-offs are higher compute, dependence on texture and lighting, and **drift** that needs loop closure. Practical rule: use marker-based for short, controlled experiences (museum cards, packaging, education); use markerless for room-scale, navigation, and large-area AR.

Q16. Compare optical see-through vs video see-through.

Optical see-through (OST): light from the real world reaches the eye through a transparent **combiner** (waveguide, beam-splitter); virtual content is projected onto it. Used by HoloLens 2, Magic Leap 2, smart glasses. Pros: **zero camera latency** for the real view, natural depth, safer (you actually see). Cons: limited **FOV** ($\sim$50° on HL2),

frame (góc/keypoint qua FAST, ORB hoặc deep features); (2) **Feature matching** giữa các frame liên tiếp; (3) **Pose estimation** qua PnP / essential matrix; (4) **IMU fusion** bằng Extended Kalman Filter hoặc factor graph cho mượt khi chuyển động nhanh; (5) **Mapping**: tam giác hóa các feature thành landmark 3D; (6) **Loop closure** sửa **drift** khi nhận ra nơi cũ; (7) **Bundle adjustment** tối ưu toàn bộ pose + landmark. Visual–inertial SLAM (VI-SLAM) là nền của ARKit, ARCore, HoloLens, Quest.

Câu 9. Presence là gì? Immersion là gì?

Immersion là **thuộc tính khách quan của hệ thống**: phần cứng thay thế giác quan tốt đến đâu (độ phân giải cao, **FOV** rộng, latency thấp, âm thanh không gian, haptic). **Presence** là **cảm giác chủ quan** "đang ở đó" mà người dùng cảm thấy *nhờ* immersion cao. Immersion là điều kiện cần nhưng chưa đủ: nội dung dở, FPS thấp, cybersickness đều có thể phá vỡ presence. Presence thường đo bằng **questionnaire** như Slater–Usoh–Steed (SUS), Witmer–Singer, hoặc đo sinh lý (nhịp tim, EDA). Thiết kế tốt để tối đa presence: **6DoF tracking** ổn định, $\geq$ 90 Hz, motion-to-photon $<$ 20 ms, spatial audio, tương tác thuyết phục.

Câu 10. VR hoạt động từng bước thế nào (rendering loop)?

Vòng VR chạy 90–120 Hz, mỗi frame phải xong trong $\sim$11 ms:
(1) Sense: đọc **IMU** + camera, cập nhật head pose; **(2) Predict:** ngoại suy pose tới thời điểm photon ra màn (**motion prediction**); **(3) Update:** game logic, physics, animation; **(4) Cull:** frustum + occlusion culling; **(5) Render hai lần:** ảnh *mắt trái* và *mắt phải* với hai ma trận chiếu lệch nhau (cách nhau IPD); **(6) Distortion correction:** pre-warp để khử pincushion và chromatic aberration của thấu kính; **(7) Asynchronous Time Warp / Space Warp:** ngay trước scan-out, re-project ảnh đã render bằng pose mới nhất để bù latency; **(8) Scan out** ra panel theo scanline. Spatial audio tính song song bằng HRTF.

Câu 11. AR pipeline hoạt động từng bước?

(1) Camera capture: lấy frame mới từ camera sau (hoặc camera passthrough); **(2) Tracking:** chạy **VI-SLAM** để ước lượng 6DoF pose của thiết bị; **(3) Scene understanding:** dò **plane** (sàn, bàn), feature point, mesh, mặt người, tay, image target; **(4) Light estimation:** suy ra cường độ, màu sắc môi trường + nguồn sáng định hướng (hoặc spherical harmonics) từ ảnh camera; **(5) Anchor management:** gắn vật ảo vào anchor cố định trong không gian thật; **(6) Rendering:** render vật ảo theo intrinsic của camera và pose ước lượng; áp ánh sáng thực; thực hiện **occlusion** dùng depth map hoặc scene mesh; **(7) Composite:** blend overlay với ảnh camera (video see-through) hoặc hiển thị qua combiner quang (optical see-through); **(8) Hiển thị** ở $\geq$ 60 Hz.

Câu 12. Inside-out tracking hoạt động ra sao?

Trong **inside-out tracking**, camera nằm *trên headset*, hướng ra ngoài; không cần base station. Mỗi frame, hệ thống trích **feature** (góc), match qua nhiều camera (stereo hoặc nhiều hơn) và qua thời gian, kết hợp với **IMU** bằng **visual–inertial odometry**. Kết quả là 6DoF pose tần số rất cao (IMU 1000 Hz, hiệu chỉnh bằng visual 30–60 Hz). Với controller: cùng camera đó quan sát **IR LED** trên tay cầm (Quest constellation) hoặc làm hand tracking thuần optical (HoloLens, Quest hand tracking, Vision Pro). Ưu: không cần setup, di động, ít cáp. Nhược: cần môi trường có texture, mất tracking trong tối hoặc trước tường trắng, controller có thể mất khi giấu sau lưng.

Câu 13. So sánh outside-in (Lighthouse) vs inside-out.

Outside-in dùng cảm biến ngoài (Lighthouse phát mặt phẳng IR; hoặc camera ngoài như Rift CV1) quan sát headset/controller; **inside-out** ngược lại, camera nằm trên headset. Outside-in cho **độ chính xác sub-mm**, gần như không jitter, vùng chơi rộng (Vive Lighthouse phủ $\sim$10 m), tracking controller ổn định kể cả sau lưng. Nhược: setup phức tạp, vùng chơi cố định, cần ổ cắm và line-of-sight. Inside-out là **plug-and-play**, di động, rẻ, hỗ trợ mọi phòng, nhưng jitter hơn, có drift, kém trong tối hoặc phòng ít texture, mất controller khi bị che. HMD tiêu dùng hiện nay (Quest, HoloLens, Vision Pro, PSVR2) dùng inside-out; setup chuyên nghiệp/nghiên cứu vẫn ưa Lighthouse.

Câu 14. Marker-based AR hoạt động thế nào?

Marker-based AR dựa vào ảnh đã biết (**fiducial marker** kiểu ARTag/ArUco, hoặc **image target** có texture trong Vuforia) đặt trong thế giới thực. Bước: (1) Camera lấy frame; (2) Ảnh được binarize/threshold, tìm contour; (3) Contour tứ giác được match với pattern marker (hoặc feature descriptor match với database); (4) Bốn góc cho tương ứng 2D–3D; (5) Giải **PnP (Perspective-n-Point)** với intrinsic camera $\Rightarrow$ 6DoF pose của marker so với camera; (6) Render vật ảo tại pose đó. Marker-based AR **ổn định, nhanh, không drift**, dùng trong sách AR, bảo tàng, bao bì khuyến mại, giáo dục — nhưng marker phải luôn nhìn thấy.

Câu 15. So sánh marker-based vs markerless AR.

Marker-based cần marker/image target in sẵn; **markerless** dựa vào feature tự nhiên + SLAM. Marker-based **đơn giản, ổn, không drift, rẻ** nhưng bị ràng buộc: marker bị che, thiếu sáng, ngoài khung là "chết". Markerless (ARKit, ARCore, HoloLens) phát hiện plane và feature mọi nơi, đi lại được, hỗ trợ **world anchor** bền qua cloud (ARCore Cloud Anchors, ARKit ARWorldMap). Nhược điểm: tốn compute, phụ thuộc texture và ánh sáng, có **drift** cần loop closure. Quy tắc thực tế: marker-based cho trải nghiệm ngắn, có kiểm soát (bảo tàng, bao bì, giáo dục); markerless cho room-scale, dẫn đường, AR diện rộng.

Câu 16. So sánh optical vs video see-through.

Optical see-through (OST): ánh sáng thực đi qua **combiner** trong suốt (waveguide, beam-splitter); nội dung ảo được chiếu lên đó. HoloLens 2, Magic Leap 2, smart glasses dùng cách này. Ưu: **0 ms camera latency** cho cảnh thực, depth tự nhiên, an toàn hơn. Nhược: **FOV** hẹp ($\sim$50° ở HL2), vật ảo trong mờ, khó render màu đen thật,

virtual objects look semi-transparent, hard to render true black, expensive optics. **Video see-through (VST)**: outward cameras capture the world, the system composites virtuals on top, and the result is shown on opaque displays (Quest 3, Vision Pro, Varjo XR-4). Pros: **wide FOV**, perfect occlusion, dynamic range, full opacity for virtuals. Cons: camera latency, distortion, lower fidelity than direct vision, total visual blackout if the system crashes.

Q17. How does stereoscopic rendering produce depth?
The brain extracts depth from many cues; the strongest near-field cue is **binocular disparity** — the slight horizontal offset between the images received by the left and right eye. To replicate this, a VR engine maintains **two virtual cameras** separated by the user's **IPD** ($\sim$63 mm) and renders each frame twice with slightly different view matrices but the same model and projection setup (with appropriate eye offsets). Each rendered image is sent to its eye's panel through the lens. The brain fuses the two images and infers depth. Additional cues are **motion parallax** (provided by 6DoF head tracking) and **vergence**. Mismatch between vergence and the fixed lens focus causes the **vergence–accommodation conflict (VAC)**, a known cause of fatigue and a target of varifocal R&D.

Q18. How does foveated rendering work and why use it?
The human retina has high resolution only in the **fovea** ($\sim 5^\circ$ at the center of gaze); peripheral vision is much lower resolution. **Foveated rendering** exploits this by rendering the **gaze-centered region at full resolution** and the periphery at progressively lower resolution, then blending. **Eye-tracked** variant (PSVR2, Vision Pro, Quest Pro) uses real-time gaze; **fixed** variant (Quest 2/3 default) just lowers resolution at panel edges. Benefits: **30–70% GPU savings**, enabling higher-fidelity scenes or lower power on standalone HMDs. Implementation in graphics APIs: **Variable Rate Shading (VRS)**, Vulkan VK_EXT_fragment_density_map, NVIDIA VRSS. The technique is essential for high-resolution headsets ($>$ 3K per eye) where full-res rendering would not fit a 90 Hz budget.

Q19. Why is low latency critical in VR? What is motion-to-photon?
Motion-to-photon (M2P) latency is the total time from a real head movement until the corresponding photon hits the eye. If M2P is too high, the image lags the head, the brain detects a mismatch between vestibular and visual systems, and **cybersickness** results. Research and industry consensus: **M2P should be under 20 ms**; great systems achieve $<$ **15 ms**. Latency is reduced by: (1) high-frequency IMU + prediction; (2) pipelined GPU rendering; (3) **Asynchronous Time Warp (ATW)** re-projects the last rendered frame using fresh head pose right before scan-out; (4) **Asynchronous Space Warp (ASW)** synthesizes intermediate frames; (5) racing-the-beam scan-out. High refresh rate (90/120/144 Hz) further reduces effective latency and judder.

Q20. Why does motion sickness (cybersickness) occur in VR? How to prevent it?
Cybersickness arises mainly from a **sensory conflict** between the visual system (which says "I'm moving") and the **vestibular system** (which says "I'm sitting still"). Triggers: high latency, low frame rate, artificial **smooth locomotion**, vection, narrow FOV vignetting absent, miscalibrated IPD, lens distortion errors. Mitigation: **(1)** keep M2P $<$ 20 ms and frame rate $\geq$ 90 Hz; **(2)** use **teleport locomotion** or short **snap turns** (30°); **(3)** apply **tunnel-vision FOV reducer** during artificial movement; **(4)** provide a **static reference** like a cockpit or nose anchor; **(5)** keep the floor calibrated and the horizon stable; **(6)** let users sit and start with short sessions; **(7)** avoid acceleration — use constant velocity. Always include comfort settings in shipping VR apps.

Q21. Why is high refresh rate (90+ Hz) needed?
At lower refresh rates (60 Hz or below), persistent flicker and judder are visible because each frame is held in front of the eye while the head moves; this causes **smearing** and contributes strongly to nausea. **90 Hz is the empirical comfort threshold**; **120 Hz** is preferred for fast titles (Beat Saber, racing) and **144 Hz** on Index. Higher rates also lower effective M2P latency by reducing the wait until the next vsync. To hit these rates, engines use **stereo instancing / multiview** (rendering both eyes in one pass), **single-pass instanced rendering**, fixed/eye-tracked foveation, lower shader complexity, and **ATW/ASW** as a safety net when rendering misses the deadline. Standalone HMDs additionally use dynamic resolution and aggressive LOD to maintain rate.

Q22. What is FOV and why does it matter? What is IPD?
FOV (Field of View) is the angular size of what the user can see. Human binocular FOV is $\sim 210^\circ$ horizontally; consumer VR HMDs offer 90°–110° (Quest 3 $\sim 110^\circ$, Index $\sim 130^\circ$, Varjo XR-4 $\sim 120^\circ$ at higher resolution; HoloLens 2 only $\sim 52^\circ$). **Wider FOV** $\Rightarrow$ **stronger immersion** but more pixels and lens complexity. **IPD (Inter-Pupillary Distance)** is the distance between the user's pupils, typically 58–72 mm; HMDs need to match the user's IPD by mechanical or software adjustment for a sharp, comfortable image and correct stereo. Wrong IPD causes blur, eye strain, and incorrect depth scale. Eye-tracking can perform **automatic IPD compensation** in software.

Q23. What is the screen-door effect? How is it solved?
The **screen-door effect (SDE)** is the appearance of dark gaps between subpixels — the user sees through a fine "mesh" over the image. It happens because lenses magnify the panel close to the eye, exaggerating the gaps in the **PenTile / RGB stripe** subpixel layout. SDE is reduced by: (1) higher **pixel density** (PPD — pixels per degree); (2) **full-RGB stripe** instead of PenTile; (3) **micro-OLED** panels that pack subpixels tightly (Vision Pro, Bigscreen Beyond); (4) **pancake lenses** that allow finer optical performance; (5) optical anti-aliasing diffusers. Modern HMDs (Quest 3 with $\sim$25 PPD, Vision Pro with $\sim$34 PPD) have nearly eliminated SDE; older HMDs (Vive, Rift CV1) had it badly.

Q24. Compare HMD display technologies: LCD, OLED, micro-OLED, waveguide.
LCD: cheap, high pixel density, full-RGB stripe, but limited contrast, gray blacks, slower response, backlight bleed. Used in Quest 2/3, Pico 4, Varjo. **OLED**: per-pixel emission, deep blacks, fast response, but typically PenTile (worse SDE), risk of burn-in. Used in Vive Pro, PSVR2 (HDR OLED). **Micro-OLED** (silicon-backed OLED): tiny panel, very high pixel density ($>$ 3000 PPI), excellent blacks, low power; used in Vision Pro (4K per eye), Bigscreen Beyond, Varjo XR-4. **Waveguides** (used in HoloLens 2, Magic Leap 2): not a panel but a light-guide that channels micro-display output into the eye through diffraction gratings; enables thin AR glasses but limits FOV and adds rainbow artifacts.

optics đắt. **Video see-through (VST)**: camera ngoài quay cảnh thực, ghép ảo lên, hiển thị qua màn hình kín (Quest 3, Vision Pro, Varjo XR-4). Ưu: **FOV rộng**, occlusion hoàn hảo, dải động cao, ảo phủ kín được. Nhược: latency camera, distortion, độ trung thực kém mắt thường, mất hình toàn bộ nếu hệ thống crash.

Câu 17. Stereoscopic render tạo depth thế nào?
Não suy ra depth từ nhiều cue; cue gần mạnh nhất là **binocular disparity** — chênh lệch ngang giữa ảnh hai mắt. Để mô phỏng, engine VR đặt **hai camera ảo** cách nhau bằng **IPD** ($\sim$63 mm), mỗi frame render hai lần với view matrix lệch (chung model + projection có offset mắt). Mỗi ảnh đi qua thấu kính tới panel của mắt tương ứng. Não fuse hai ảnh và tính ra depth. Cue phụ: **motion parallax** (do 6DoF head tracking cung cấp) và **vergence**. Khi vergence lệch tiêu cự cố định của thấu kính sẽ gây **vergence–accommodation conflict (VAC)** — nguyên nhân mỏi mắt, đang là chủ đề R&D varifocal.

Câu 18. Foveated rendering hoạt động thế nào, vì sao dùng?
Võng mạc người chỉ có độ phân giải cao ở **fovea** ($\sim 5^\circ$ trung tâm); vùng ngoại vi thấp hơn nhiều. **Foveated rendering** tận dụng điều đó: render **vùng nhìn** ở full resolution, vùng ngoại vi giảm dần resolution rồi blend. Bản **eye-tracked** (PSVR2, Vision Pro, Quest Pro) dùng gaze thực; bản **fixed** (Quest 2/3 mặc định) chỉ giảm resolution ở rìa panel. Lợi ích: **tiết kiệm GPU 30–70%**, cho phép scene đẹp hơn hoặc tiết kiệm pin trên HMD standalone. Triển khai bằng **Variable Rate Shading (VRS)**, Vulkan VK_EXT_fragment_density_map, NVIDIA VRSS. Kỹ thuật này gần như bắt buộc với headset $>$ 3K mỗi mắt vì render full-res không kịp 90 Hz.

Câu 19. Vì sao latency thấp quan trọng trong VR? Motion-to-photon là gì?
Motion-to-photon (M2P) latency là tổng thời gian từ khi đầu chuyển động thực tới khi photon tới mắt. Nếu M2P cao, ảnh trễ so với đầu, não phát hiện lệch giữa tiền đình và thị giác $\Rightarrow$ **cybersickness**. Đồng thuận: **M2P dưới 20 ms**; hệ tốt đạt $<$ **15 ms**. Cách giảm latency: (1) IMU tần số cao + prediction; (2) GPU render pipelined; (3) **Asynchronous Time Warp (ATW)** re-project frame cũ bằng pose mới trước khi scan-out; (4) **Asynchronous Space Warp (ASW)** sinh frame trung gian; (5) racing-the-beam scan-out. Refresh rate cao (90/120/144 Hz) giảm thêm latency hữu hiệu và judder.

Câu 20. Vì sao bị say (cybersickness) trong VR? Cách phòng?
Cybersickness chủ yếu do **xung đột giác quan** giữa thị giác ("đang di chuyển") và **tiền đình** ("đang ngồi yên"). Nguyên nhân: latency cao, FPS thấp, **smooth locomotion** nhân tạo, vection, không có vignette, IPD sai, distortion lệch. Cách giảm: **(1)** giữ M2P $<$ 20 ms và $\geq$ 90 Hz; **(2)** dùng **teleport** hoặc **snap turn** 30°; **(3) tunnel-vision** thu hẹp FOV khi di chuyển nhân tạo; **(4)** thêm **tham chiếu tĩnh** (cockpit, nose anchor); **(5)** giữ sàn calibrated, đường chân trời ổn định; **(6)** ngồi và bắt đầu phiên ngắn; **(7)** tránh acceleration, dùng vận tốc đều. Ứng dụng VR thương mại bắt buộc có comfort settings.

Câu 21. Vì sao cần refresh rate cao (90+ Hz)?
Ở refresh thấp (60 Hz trở xuống), flicker và judder rất rõ vì mỗi frame "đứng yên" trước mắt trong khi đầu di chuyển, gây **smearing** và buồn nôn. **90 Hz là ngưỡng thoải mái thực nghiệm**; **120 Hz** cho game nhanh (Beat Saber, racing); **144 Hz** trên Index. Refresh cao cũng giảm M2P do thời gian chờ vsync ngắn. Để đạt được, engine dùng **stereo instancing/multiview** (render hai mắt trong một pass), **single-pass instanced rendering**, foveation cố định/eye-tracked, shader nhẹ, **ATW/ASW** làm lưới an toàn khi miss deadline. HMD standalone còn dùng dynamic resolution và LOD để giữ FPS.

Câu 22. FOV là gì, vì sao quan trọng? IPD là gì?
FOV (Field of View) là góc thị trường người dùng thấy. Mắt người: $\sim 210^\circ$ ngang; VR tiêu dùng: 90–110° (Quest 3 $\sim 110^\circ$, Index $\sim 130^\circ$, Varjo XR-4 $\sim 120^\circ$ độ phân giải cao; HoloLens 2 chỉ $\sim 52^\circ$). **FOV rộng** $\Rightarrow$ **immersion mạnh** nhưng tốn pixel và optics phức tạp. **IPD (Inter-Pupillary Distance)** là khoảng cách hai đồng tử, 58–72 mm; HMD phải khớp IPD bằng cơ học hoặc phần mềm để ảnh nét và stereo đúng. IPD sai $\Rightarrow$ mờ, mỏi mắt, sai tỉ lệ depth. Eye-tracking có thể **IPD compensation** tự động bằng phần mềm.

Câu 23. Screen-door effect là gì? Giải quyết ra sao?
Screen-door effect (SDE) là hiện tượng nhìn thấy khoảng tối giữa các subpixel — ảnh như có "lưới". Lý do: thấu kính phóng đại panel sát mắt, làm rõ khe của subpixel layout **PenTile / RGB stripe**. Cách giảm: (1) tăng **pixel density** (PPD); (2) dùng **full-RGB stripe** thay PenTile; (3) panel **micro-OLED** đóng subpixel rất khít (Vision Pro, Bigscreen Beyond); (4) **pancake lens** cho optics mỏng hơn; (5) diffuser quang chống aliasing. HMD mới (Quest 3 $\sim$25 PPD, Vision Pro $\sim$34 PPD) gần như hết SDE; HMD cũ (Vive, Rift CV1) bị rõ.

Câu 24. So sánh display HMD: LCD, OLED, micro-OLED, waveguide.
LCD: rẻ, mật độ pixel cao, full-RGB stripe, nhưng tương phản kém, đen ngả xám, chậm, lọt sáng. Dùng ở Quest 2/3, Pico 4, Varjo. **OLED**: phát sáng từng pixel, đen sâu, đáp ứng nhanh, nhưng thường PenTile (SDE rõ hơn), nguy cơ burn-in. Dùng ở Vive Pro, PSVR2 (HDR OLED). **Micro-OLED** (OLED trên silicon): panel rất nhỏ, mật độ rất cao ($>$ 3000 PPI), đen tốt, ít điện; dùng ở Vision Pro (4K mỗi mắt), Bigscreen Beyond, Varjo XR-4. **Waveguide** (HoloLens 2, Magic Leap 2): không phải panel mà là ống dẫn sáng đưa output micro-display vào mắt qua cách tử nhiễu xạ; cho kính AR mỏng nhưng FOV hẹp và có rainbow artifact.

Q25. What are the main components of a VR/AR system?
A complete XR system has: **(1) Display**: the HMD (panels + lenses); **(2) Tracking**: 6DoF head tracking (IMU + cameras), controller tracking, hand and eye tracking; **(3) Input devices**: motion controllers (Quest Touch, Index Knuckles), hand tracking, voice (microphone), bare-hand pinch (Vision Pro); **(4) Audio**: spatial audio with HRTF (binaural), often via on-strap speakers or earphones; **(5) Compute**: a tethered PC with GPU (PC-VR) or an on-board SoC (standalone); **(6) Software stack**: OS runtime (Quest OS, visionOS, SteamVR), **OpenXR runtime**, engine (Unity/Unreal/native), application logic; **(7) Optional**: haptics (vibration, gloves), treadmills, smell, biometric sensors. AR adds outward RGB+depth cameras and scene-understanding modules.

Q26. What is OpenXR and why is it important?
OpenXR is an **open, royalty-free standard API** from the Khronos Group that lets a single application target many XR devices. Before OpenXR, developers wrote separate code for Oculus SDK, OpenVR, Windows Mixed Reality, Pico, etc.; now they call one API and the runtime (Meta, SteamVR, Pico, Snapdragon Spaces, visionOS via emulation) does the device-specific work. OpenXR exposes **spaces**, **actions**, **swapchains**, and **composition layers** in a vendor-neutral way. Importance: **(1) portability** (one codebase, many headsets); **(2) longevity** (apps survive HMD generations); **(3) ecosystem unity**; **(4) industry standard for enterprise procurement**. Unity (XR Plugin Management with OpenXR plugin) and Unreal natively target OpenXR; native C++/Vulkan engines link to it directly.

Q27. Compare ARKit, ARCore, and Vuforia.
ARKit (Apple, iOS/visionOS): top-tier VI-SLAM, plane and mesh detection, body and face tracking, people occlusion, motion capture; iOS-only but rock-solid quality. **ARCore** (Google, Android + iOS): cross-platform mobile AR with motion tracking, plane finding, light estimation, **Cloud Anchors** for shared/persistent AR, Geospatial API (anchors at GPS+VPS coordinates worldwide). **Vuforia** (PTC): the veteran cross-platform SDK, strongest in **marker-based** (Image, Multi-Image, Cylinder, 3D Object Targets), Model Targets, Area Targets; works on iOS, Android, HoloLens, Magic Leap, Unity Editor. Choice: **Vuforia** for industrial / education / image-target apps; **ARKit** for premium iOS experiences; **ARCore** for Android-first, geo-located, or shared-anchor scenarios. Unity **AR Foundation** wraps ARKit + ARCore behind one API.

Q28. Describe the steps to build a VR app in Unity.
(1) Install Unity LTS (e.g. 2022.3) with Android Build Support (for Quest); **(2)** Open Package Manager, install **XR Plugin Management**, then enable the **OpenXR** or **Oculus** provider; **(3)** Add the **XR Interaction Toolkit (XRI)** package for ray/grab/teleport interactors; **(4)** In the scene, replace the Main Camera with an **XR Origin (VR)** prefab (rig with tracked camera + two controllers); **(5)** Set up **tracked input actions** (Input System) for triggers, grips, thumbsticks; **(6)** Add **XR Grab Interactable** to objects, **Teleportation Areas** to floor; **(7)** Configure quality (URP, MSAA $\geq$ 4x, 90 Hz); enable **single-pass instanced** rendering; **(8)** Build to Quest (Android) or PC (Windows + SteamVR/OpenXR); **(9)** Profile with the Oculus Performance HUD; iterate on draw calls, fill rate, and frame timing.

Q29. Describe the steps to build an AR app in Unity with AR Foundation.
(1) Unity LTS + Build Support for iOS and Android; **(2)** Install **AR Foundation** + **ARKit XR Plugin** + **ARCore XR Plugin** via Package Manager; **(3)** Replace Main Camera with **AR Session Origin** (camera + tracked pose driver) and add an **AR Session** object; **(4)** Add managers for the features you need: **ARPlaneManager**, **ARRaycastManager**, **ARAnchorManager**, **ARFaceManager**, **ARMeshManager**, **AROcclusionManager**; **(5)** Write a script: tap $\rightarrow$ raycast against detected planes $\rightarrow$ instantiate a prefab anchored at the hit; **(6)** Configure player settings: ARM64, Vulkan/Metal, ARCore/ARKit required, camera permission strings; **(7)** Build and deploy to device (USB or TestFlight); **(8)** Iterate using **AR Foundation Editor Remote** or **XR Simulation** to test in editor without a phone.

Q30. Application: How is VR used in healthcare and surgical training?
VR provides **safe, repeatable, high-fidelity** training environments for clinicians. (1) **Surgical simulators**: trainees rehearse laparoscopic, orthopedic, cataract, or endovascular procedures with haptic instruments (e.g. **Osso VR**, **FundamentalVR**, **Surgical Theater**, **ImmersiveTouch**); the system records metrics (path length, errors, time) for objective assessment. (2) **Anatomy education**: students explore segmented patient-specific 3D models reconstructed from CT/MRI (e.g. **3D Organon**, **Complete Anatomy**). (3) **Patient-specific surgical planning**: surgeon walks through the patient's CT in VR before operating. (4) **Pain distraction and anxiety reduction**: pediatric burns, dentistry, chemotherapy. **Tools**: Unity + XRI + Oculus/OpenXR, Vuforia for AR overlays, custom haptics (HaptX, Phantom Omni). The clinical evidence shows VR-trained residents achieve proficiency in significantly less OR time.

Q31. Application: How is AR used in surgical navigation?
AR surgical navigation overlays preoperative imaging (CT, MRI, vessels, tumor boundaries) directly onto the patient anatomy in the surgeon's view. Pipeline: **(1)** segment the CT/MRI to extract organs and target structures (3D Slicer, MITK); **(2)** register the preoperative model to the patient's body via **fiducial markers**, surface scanning, or feature-based registration; **(3)** track the patient and the instruments in real time (optical trackers, IR LEDs, or AR HMD inside-out); **(4)** render the segmented model at the registered pose on **HoloLens 2**, **Magic Leap 2**, or a tablet; **(5)** update with respiration / deformation. Real systems: **Brainlab Mixed Reality Viewer**, **Medivis SurgicalAR**, **Augmedics xvision** (FDA-approved for spine). Reported benefits: less radiation (fewer fluoroscopy shots), better screw placement accuracy, shorter OR time.

Q32. Application: VR/AR in education and training.
VR/AR is transforming education by turning abstract concepts into **embodied, interactive 3D experiences**. **VR examples**: **Google Expeditions / Wonderscope** (virtual field trips); **Labster** (virtual labs — run a chemistry experiment safely); **Engage**, **Mozilla Hubs**, **ENGAGE XR** for VR classrooms; **Tilt Brush** for art. **AR examples**: **JigSpace** for product visualization, **Merge Cube** for a holdable 3D model, **Quiver** for AR coloring books, **Civilisations AR** (BBC) for museum artifacts at home. **Why it works**: spatial presence improves recall;

Câu 25. Các thành phần chính của hệ thống VR/AR.
Một hệ XR đầy đủ gồm: **(1) Display**: HMD (panel + lens); **(2) Tracking**: 6DoF head (IMU + camera), controller, hand, eye tracking; **(3) Input**: motion controller (Quest Touch, Index Knuckles), hand tracking, voice (mic), pinch (Vision Pro); **(4) Audio**: spatial audio HRTF (binaural) qua loa trên dây đeo hoặc tai nghe; **(5) Compute**: PC tethered có GPU (PC-VR) hoặc SoC on-board (standalone); **(6) Software stack**: OS runtime (Quest OS, visionOS, SteamVR), **OpenXR runtime**, engine (Unity/Unreal/native), logic ứng dụng; **(7) Tùy chọn**: haptic (rung, glove), treadmill, mùi, biometric. AR thêm camera RGB + depth hướng ngoài và module scene-understanding.

Câu 26. OpenXR là gì? Vì sao quan trọng?
OpenXR là **chuẩn API mở, miễn phí bản quyền** của Khronos Group, cho phép một app chạy trên nhiều thiết bị XR. Trước OpenXR phải viết riêng cho Oculus SDK, OpenVR, Windows MR, Pico... Giờ chỉ gọi một API, runtime (Meta, SteamVR, Pico, Snapdragon Spaces, visionOS qua emulation) lo phần thiết bị. OpenXR phơi ra **space**, **action**, **swapchain**, **composition layer** theo cách trung lập. Tầm quan trọng: **(1) portable** (một codebase, nhiều HMD); **(2) bền theo thời gian** (app sống qua nhiều thế hệ); **(3) thống nhất ecosystem**; **(4) chuẩn cho mua sắm doanh nghiệp**. Unity (XR Plugin Management + OpenXR plugin) và Unreal hỗ trợ OpenXR sẵn; engine C++/Vulkan link trực tiếp.

Câu 27. So sánh ARKit, ARCore, Vuforia.
ARKit (Apple, iOS/visionOS): VI-SLAM hàng đầu, plane + mesh, body + face tracking, people occlusion, mocap; chỉ iOS nhưng cực ổn. **ARCore** (Google, Android + iOS): mobile AR đa nền, motion tracking, plane, light estimation, **Cloud Anchors** cho AR chia sẻ/bền, Geospatial API (anchor theo GPS+VPS toàn cầu). **Vuforia** (PTC): SDK cross-platform lâu đời, mạnh nhất ở **marker-based** (Image, Multi-Image, Cylinder, 3D Object Target), Model Target, Area Target; chạy iOS, Android, HoloLens, Magic Leap, Unity Editor. Chọn: **Vuforia** cho công nghiệp/giáo dục/image-target; **ARKit** cho iOS cao cấp; **ARCore** cho Android-first, geo-located, shared anchor. Unity **AR Foundation** bọc ARKit + ARCore sau một API chung.

Câu 28. Các bước build app VR trong Unity.
(1) Cài Unity LTS (vd 2022.3) kèm Android Build Support (cho Quest); **(2)** Mở Package Manager, cài **XR Plugin Management**, bật provider **OpenXR** hoặc **Oculus**; **(3)** Cài **XR Interaction Toolkit (XRI)** cho ray/grab/teleport interactor; **(4)** Trong scene, thay Main Camera bằng prefab **XR Origin (VR)** (rig camera + 2 controller); **(5)** Setup **tracked input action** (Input System) cho trigger, grip, thumbstick; **(6)** Thêm **XR Grab Interactable** cho object, **Teleportation Area** cho sàn; **(7)** Cấu hình quality (URP, MSAA $\geq$ 4x, 90 Hz), bật **single-pass instanced**; **(8)** Build cho Quest (Android) hoặc PC (Windows + SteamVR/OpenXR); **(9)** Profile bằng Oculus Performance HUD; tối ưu draw call, fill rate, frame time.

Câu 29. Các bước build app AR trong Unity với AR Foundation.
(1) Unity LTS + Build Support cho iOS và Android; **(2)** Cài **AR Foundation** + **ARKit XR Plugin** + **ARCore XR Plugin** qua Package Manager; **(3)** Thay Main Camera bằng **AR Session Origin** (camera + tracked pose driver) và thêm object **AR Session**; **(4)** Thêm manager cho tính năng cần: **ARPlaneManager**, **ARRaycastManager**, **ARAnchorManager**, **ARFaceManager**, **ARMeshManager**, **AROcclusionManager**; **(5)** Viết script: chạm $\rightarrow$ raycast vào plane $\rightarrow$ instantiate prefab với anchor tại hit; **(6)** Player settings: ARM64, Vulkan/Metal, bắt buộc ARCore/ARKit, chuỗi camera permission; **(7)** Build và deploy lên thiết bị (USB hoặc TestFlight); **(8)** Lặp bằng **AR Foundation Editor Remote** hoặc **XR Simulation** để test trong editor.

Câu 30. Ứng dụng: VR trong y tế và đào tạo phẫu thuật.
VR cung cấp môi trường huấn luyện **an toàn, lặp lại được, fidelity cao** cho bác sĩ. (1) **Simulator phẫu thuật**: học nội soi, chỉnh hình, đục thủy tinh thể, can thiệp mạch với dụng cụ haptic (**Osso VR**, **FundamentalVR**, **Surgical Theater**, **ImmersiveTouch**); hệ thống ghi metric (đường đi, lỗi, thời gian) để đánh giá khách quan. (2) **Giáo dục giải phẫu**: sinh viên khám phá mô hình 3D bệnh nhân từ CT/MRI (**3D Organon**, **Complete Anatomy**). (3) **Lập kế hoạch phẫu thuật cá nhân hóa**: bác sĩ "đi qua" CT bệnh nhân trong VR trước khi mổ. (4) **Giảm đau, giảm lo âu**: bỏng nhi, nha khoa, hóa trị. **Tool**: Unity + XRI + Oculus/OpenXR, Vuforia overlay, haptic riêng (HaptX, Phantom Omni). Bằng chứng lâm sàng: bác sĩ trẻ huấn luyện VR đạt thành thạo nhanh hơn nhiều.

Câu 31. Ứng dụng: AR trong dẫn đường phẫu thuật.
AR surgical navigation chiếu hình ảnh tiền phẫu (CT, MRI, mạch máu, biên u) trực tiếp lên cơ thể bệnh nhân trong tầm nhìn bác sĩ. Pipeline: **(1)** segment CT/MRI để lấy cơ quan và cấu trúc đích (3D Slicer, MITK); **(2)** register mô hình tiền phẫu với cơ thể bằng **fiducial marker**, quét bề mặt, hoặc feature-based; **(3)** track bệnh nhân và dụng cụ theo thời gian thực (optical tracker, IR LED, hoặc AR HMD inside-out); **(4)** render mô hình ở pose đã register lên **HoloLens 2**, **Magic Leap 2**, hoặc tablet; **(5)** cập nhật theo hô hấp / biến dạng. Hệ thực: **Brainlab Mixed Reality Viewer**, **Medivis SurgicalAR**, **Augmedics xvision** (FDA cho cột sống). Lợi: giảm tia X, đặt vít chính xác hơn, rút ngắn thời gian mổ.

Câu 32. Ứng dụng: VR/AR trong giáo dục và đào tạo.
VR/AR biến khái niệm trừu tượng thành **trải nghiệm 3D nhập vai, tương tác**. **VR**: **Google Expeditions/Wonderscope** (đi tham quan ảo); **Labster** (lab ảo — thí nghiệm hóa an toàn); **Engage**, **Mozilla Hubs**, **ENGAGE XR** làm lớp học VR; **Tilt Brush** cho mỹ thuật. **AR**: **JigSpace** cho product viz, **Merge Cube** cầm mô hình 3D, **Quiver** sách tô màu AR, **Civilisations AR** (BBC) đưa hiện vật bảo tàng về nhà. **Vì sao hiệu quả**: presence giúp nhớ lâu; học trải nghiệm theo Kolb; setup nguy hiểm/đắt (hóa, sinh, máy móc) trở nên an toàn và tái

experiential learning matches Kolb's cycle; expensive/dangerous setups (chemistry, biology dissection, machinery) become safe and reusable. **Tools to build**: Unity + AR Foundation + Vuforia for image-target textbook AR; Unity + XRI for VR labs; **CoSpaces Edu** or **Spatial** for non-coders.

Q33. Application: VR for cultural heritage, e-Museum, and e-Tourism.
VR/AR enables **remote access** to fragile or distant heritage and increases **engagement** with cultural assets. Pipeline: scan the site/object using **photogrammetry** (Reality Capture, Meshroom), **LiDAR / terrestrial laser scanning**, or structured light; clean the mesh in Blender/MeshLab; texture it; import into Unity/Unreal; build a **VR walkthrough** (Quest) or **web AR / WebXR** viewer. Examples: **Mona Lisa: Beyond the Glass** (Louvre, HTC Vive), **Ancient Olympia in VR**, the digital reconstruction of **Notre-Dame de Paris**, **Smithsonian Voices VR**. For Vietnam: Hòang Thành Thăng Long, Mỵ Sơn, Huê Imperial City have all been targets of digital twin / VR projects (using NFD-based hole filling for damaged statues, Bezier surface modeling for restoration). Tools: Unity, Unreal, **8th Wall WebXR**, AR Foundation, Vuforia (image targets on museum cards).

Q34. Application: AR in industrial maintenance and Industry 4.0.
AR maintenance overlays step-by-step procedures, IoT sensor data, and 3D animations onto the actual equipment, reducing errors and the need for senior experts on-site. Pipeline: **(1)** import the CAD model (CATIA/SolidWorks/Inventor) into Unity via **PiXYZ** or **CAD Exchanger**; **(2)** author procedures in **Vuforia Studio** or **Microsoft Dynamics 365 Guides**; **(3)** register the model to the real machine via **Vuforia Model Targets** or **Area Targets** (no marker needed — the CAD itself is the recognition target); **(4)** run on **HoloLens 2**, **Magic Leap 2**, or rugged tablets; **(5)** stream live IoT data via **ThingWorx** / Azure Digital Twins; **(6) Remote Assist**: a senior expert sees the technician's view and annotates in real time. Documented benefits at Boeing, Airbus, Lockheed: 30–90% time savings, fewer wiring errors.

Q35. Application: VR in psychotherapy — exposure therapy and PTSD.
VR Exposure Therapy (VRET) treats anxiety disorders, phobias, and PTSD by gradually exposing patients to controlled, virtual versions of feared stimuli (heights, flying, public speaking, combat). The therapist controls intensity, can pause, can repeat, and the patient never has to leave the clinic. Validated programs: **Bravemind** (USC ICT, for combat PTSD — Iraq/Afghanistan/Vietnam scenarios), **Limbix VR**, **XRHealth** (FDA-cleared for several indications), **Psious / Amelia**. **Why it works**: emotional engagement is high (presence!); therapist can grade exposure precisely; cost is lower than real-world exposure (you cannot easily "schedule a flight"). **Tools**: Unity + XRI + Oculus + biometric sensors (heart-rate variability, EDA, breathing) for biofeedback. Evidence base: dozens of RCTs show VRET non-inferior or superior to in-vivo exposure for several phobias.

Q36. Application: VR/AR in architecture, engineering, and construction (AEC).
AEC firms use VR/AR throughout the design lifecycle. (1) **Design review in VR**: walk a 1:1 scale model of the building before construction (**Enscape**, **Twinmotion**, **IrisVR Prospect**, **Autodesk VRED**); detect ergonomic issues (low ceilings, cramped corridors) early. (2) **Client presentations**: photorealistic VR walkthroughs sell projects. (3) **On-site AR overlay**: surveyors view the BIM model registered to the construction site via HoloLens (**Trimble XR10**, **Trimble Connect**); they verify pipe placement, rebar layout. (4) **Clash detection** and coordination meetings in VR, multi-user. (5) **Safety training** (working at heights, hot work). **Pipeline**: Revit/ArchiCAD/Rhino → FBX/IFC → Unity Reflect or Unreal Datasmith → HMD. Real impact: fewer RFIs, fewer change orders, faster client buy-in.

Q37. Application: AR in retail and e-commerce.
AR boosts conversion by letting customers **visualize products in their own space** before buying. (1) **Furniture try-out**: **IKEA Place**, **Wayfair View in Room**, **Amazon AR View** place a 3D sofa on the actual living-room floor with correct scale and shadows. (2) **AR try-on for fashion/cosmetics**: **Sephora Virtual Artist**, **L'Oréal ModiFace**, **Snapchat AR Lenses for Gucci/Dior**, eyeglass try-on (Warby Parker), shoe try-on (Nike). (3) **AR product packaging**: scan a soda can, see a campaign animation. **How it works**: ARKit/ARCore for plane detection and tracking; **ARFaceManager** for face mesh; **8th Wall** for browser AR (no install); **glTF** or **USDZ** as 3D format; **WebXR / Quick Look** delivery. **Documented impact**: AR-equipped product pages convert 2–3× better and reduce returns.

Q38. Application: VR/AR in entertainment, social, and the metaverse.
The largest commercial market for VR is **games and social experiences**. Hits: **Beat Saber**, **Half-Life: Alyx**, **Resident Evil 4 VR**, **Asgard's Wrath 2**, **Gorilla Tag**. Social platforms: **VRChat**, **Rec Room**, **Horizon Worlds**, **Spatial** support hundreds of users in shared virtual spaces with full-body avatars, lip-sync, and **eye contact** via eye tracking. AR side: **Pokemon GO**, **Snapchat Lenses**, **TikTok AR effects** reach billions. Production tools: Unity + Photon Fusion / **Mirror** / **Normcore** for networking; **Ready Player Me** avatars; **Lens Studio** (Snap), **Spark AR** (Meta), **Effect House** (TikTok) for AR filters. Concert/live: **Travis Scott in Fortnite**, **ABBA Voyage** mixed-reality stage. The "metaverse" vision unifies these: persistent, multi-user, cross-device 3D internet.

Q39. What sensors are needed for AR tracking? (and what each one provides)
A modern AR device combines complementary sensors: **(1) Wide-angle RGB camera(s)** for visual features, image targets, hand and face tracking; **(2) IMU** (3-axis gyroscope + 3-axis accelerometer + sometimes magnetometer) at 800–1000 Hz for fast rotational tracking and motion prediction; **(3) Depth sensor** (**ToF** on Pixel/iPad LiDAR, or **structured light** on iPhone Pro front for FaceID, or **stereo** pairs) for plane detection, occlusion, mesh reconstruction; **(4) GPS + barometer** for outdoor coarse localization; **(5) Magnetometer** for heading; **(6) Microphones** for voice + spatial audio. **Sensor fusion** (Extended Kalman Filter / factor graphs) combines these into a single, smooth 6DoF pose. The IMU provides high frequency and short-term accuracy; the camera corrects long-term drift; depth gives metric scale and occlusion.

Q40. What are the main challenges in VR? How are they mitigated?
(1) Cybersickness: caused by latency or artificial locomotion — mitigate with $\geq$ 90 Hz, M2P $<$ 20 ms, teleport, snap turn, vignette. **(2) Hardware ergonomics**: weight on the face, heat, sweat, cable — mitigated by lighter

sử dụng. **Tool**: Unity + AR Foundation + Vuforia cho AR sách giáo khoa; Unity + XRI cho lab VR; **CoSpaces Edu** hoặc **Spatial** cho người không code.

Câu 33. Ứng dụng: VR cho di sản văn hóa, e-Museum, e-Tourism.
VR/AR cho phép **tiếp cận từ xa** các di sản dễ vỡ/xa xôi và tăng **tương tác** với tài sản văn hóa. Pipeline: quét địa điểm/hiện vật bằng **photogrammetry** (Reality Capture, Meshroom), **LiDAR/terrestrial laser scanning**, hoặc structured light; làm sạch mesh trong Blender/MeshLab; texture; nhập Unity/Unreal; làm **VR walkthrough** (Quest) hoặc **web AR/WebXR**. Ví dụ: **Mona Lisa: Beyond the Glass** (Louvre, Vive), **Olympia cổ đại VR**, dựng số **Notre-Dame Paris**, **Smithsonian Voices VR**. Việt Nam: Hoàng Thành Thăng Long, Mỹ Sơn, Cố đô Huế đều có dự án digital twin/VR (dùng NFD hole filling cho tượng vỡ, Bezier để phục dựng). Tool: Unity, Unreal, **8th Wall WebXR**, AR Foundation, Vuforia (image target trên thẻ bảo tàng).

Câu 34. Ứng dụng: AR trong bảo trì công nghiệp và Industry 4.0.
AR maintenance chiếu hướng dẫn từng bước, dữ liệu IoT, animation 3D lên máy thật ⇒ giảm lỗi, ít cần chuyên gia tại chỗ. Pipeline: **(1)** đưa CAD (CATIA/SolidWorks/Inventor) vào Unity qua **PiXYZ** hoặc **CAD Exchanger**; **(2)** soạn quy trình trong **Vuforia Studio** hoặc **Dynamics 365 Guides**; **(3)** register mô hình với máy thật bằng **Vuforia Model Targets** hoặc **Area Targets** (không cần marker — chính CAD là target); **(4)** chạy trên **HoloLens 2**, **Magic Leap 2**, hoặc tablet công nghiệp; **(5)** stream IoT qua **ThingWorx**/Azure Digital Twins; **(6) Remote Assist**: chuyên gia thấy view của thợ và chú thích thời gian thực. Boeing, Airbus, Lockheed: tiết kiệm 30–90% thời gian, giảm lỗi.

Câu 35. Ứng dụng: VR trong tâm lý trị liệu — exposure therapy và PTSD.
VR Exposure Therapy (VRET) điều trị rối loạn lo âu, ám ảnh sợ, PTSD bằng cách phơi nhiễm dần với phiên bản ảo của tác nhân sợ (cao, máy bay, nói trước đám đông, chiến đấu). Nhà trị liệu điều khiển cường độ, dừng, lặp; bệnh nhân không cần ra khỏi phòng khám. Chương trình đã kiểm chứng: **Bravemind** (USC ICT, PTSD chiến tranh — Iraq/Afghanistan/Vietnam), **Limbix VR**, **XRHealth** (FDA-cleared), **Psious/Amelia**. **Vì sao hiệu quả**: cảm xúc cao (presence!); phân cấp phơi nhiễm chính xác; chi phí thấp hơn ngoài đời. **Tool**: Unity + XRI + Oculus + sensor sinh lý (HRV, EDA, hô hấp) cho biofeedback. Bằng chứng: hàng chục RCT cho thấy VRET không kém hoặc hơn phơi nhiễm thực với nhiều loại phobia.

Câu 36. Ứng dụng: VR/AR trong kiến trúc và xây dựng (AEC).
AEC dùng VR/AR xuyên suốt vòng đời thiết kế. (1) **Design review VR**: đi trong mô hình tỉ lệ 1:1 trước khi xây (**Enscape**, **Twinmotion**, **IrisVR Prospect**, **Autodesk VRED**); phát hiện trần thấp, hành lang chật từ sớm. (2) **Trình bày khách hàng**: walkthrough VR photorealistic. (3) **AR tại công trường**: kỹ sư xem BIM đè lên công trường qua HoloLens (**Trimble XR10**, **Trimble Connect**); kiểm tra ống, cốt thép. (4) **Clash detection** và họp phối hợp đa người trong VR. (5) **Đào tạo an toàn** (làm việc trên cao, hàn). **Pipeline**: Revit/ArchiCAD/Rhino → FBX/IFC → Unity Reflect hoặc Unreal Datasmith → HMD. Lợi ích thực: ít RFI, ít change order, khách duyệt nhanh.

Câu 37. Ứng dụng: AR trong bán lẻ và thương mại điện tử.
AR tăng tỉ lệ chuyển đổi bằng cách cho khách **hình dung sản phẩm trong không gian riêng** trước khi mua. (1) **Thử nội thất**: **IKEA Place**, **Wayfair View in Room**, **Amazon AR View** đặt sofa 3D lên sàn phòng khách thật với tỉ lệ và bóng đúng. (2) **Thử thời trang/mỹ phẩm**: **Sephora Virtual Artist**, **L'Oréal ModiFace**, **Snapchat AR Lens cho Gucci/Dior**, thử kính (Warby Parker), thử giày (Nike). (3) **Bao bì AR**: quét lon nước ngọt → hiện animation chiến dịch. **Cách làm**: ARKit/ARCore cho plane + tracking; **ARFaceManager** cho face mesh; **8th Wall** cho AR trên browser (không cần cài); **glTF** hoặc **USDZ** làm format 3D; **WebXR/Quick Look** để phát. **Tác động**: trang có AR chuyển đổi cao gấp 2–3 lần và giảm tỉ lệ trả hàng.

Câu 38. Ứng dụng: VR/AR trong giải trí, mạng xã hội và metaverse.
Thị trường thương mại lớn nhất của VR là **game và mạng xã hội**. Hit: **Beat Saber**, **Half-Life: Alyx**, **Resident Evil 4 VR**, **Asgard's Wrath 2**, **Gorilla Tag**. Nền tảng xã hội: **VRChat**, **Rec Room**, **Horizon Worlds**, **Spatial** hỗ trợ hàng trăm người trong không gian ảo, full-body avatar, lip-sync, **giao tiếp mắt** qua eye tracking. AR: **Pokemon GO**, **Snapchat Lens**, **TikTok AR effect** tiếp cận hàng tỉ người. Tool: Unity + Photon Fusion/**Mirror**/**Normcore** cho networking; **Ready Player Me** cho avatar; **Lens Studio** (Snap), **Spark AR** (Meta), **Effect House** (TikTok) cho AR filter. Sự kiện trực tiếp: **Travis Scott trong Fortnite**, **ABBA Voyage**. Tầm nhìn "metaverse": internet 3D bền, đa người dùng, đa thiết bị.

Câu 39. AR tracking cần sensor nào? (mỗi loại làm gì)
Thiết bị AR hiện đại phối hợp các sensor bù trừ: **(1) Camera RGB góc rộng** cho feature, image target, hand + face; **(2) IMU** (gyro 3 trục + accel 3 trục + đôi khi magnetometer) 800–1000 Hz cho xoay nhanh và motion prediction; **(3) Depth sensor** (**ToF** trên Pixel/iPad LiDAR, hoặc **structured light** trên iPhone Pro mặt trước cho FaceID, hoặc **stereo**) cho plane, occlusion, dựng mesh; **(4) GPS + barometer** cho định vị ngoài trời; **(5) Magnetometer** cho hướng; **(6) Mic** cho voice + spatial audio. **Sensor fusion** (EKF / factor graph) gộp lại thành 6DoF pose mượt. IMU cho tần số cao + chính xác ngắn hạn; camera sửa drift dài hạn; depth cho scale chuẩn và occlusion.

Câu 40. Thách thức chính của VR và cách giảm thiểu.
(1) Cybersickness: do latency hoặc locomotion nhân tạo — giải bằng $\geq$ 90 Hz, M2P $<$ 20 ms, teleport, snap turn, vignette. **(2) Ergonomic**: nặng mặt, nóng, mồ hôi, dây — pancake optics nhẹ, dây cân bằng (Bigscreen Beyond,

pancake optics, balanced strap (Bigscreen Beyond, Quest Elite), wireless. **(3) Eye fatigue / VAC**: mismatch between vergence and fixed lens accommodation — research on **varifocal** (Half Dome, Vision Pro hints), **light-field** displays. **(4) Content cost**: 3D assets, animation, optimization — partly addressed by asset stores, photogrammetry, AI-generated content. **(5) Onboarding** for non-gamers: tutorials, hand tracking instead of controllers. **(6) Compute budget** on standalone — foveated rendering, fixed-foveation, level-of-detail, baked lighting. **(7) Social acceptance / privacy** — guardian boundaries, cameras off in private spaces.

Q41. What are the main challenges in AR (drift, occlusion, lighting, registration)?
(1) Tracking drift: visual–inertial SLAM accumulates error; mitigated by **loop closure**, **cloud anchors**, persistent maps, fiducial markers as anchors. **(2) Occlusion**: virtual objects must hide behind real ones — solved with **depth maps** (ARKit People Occlusion, ARCore Depth API), **scene mesh**, or instance segmentation. **(3) Lighting consistency**: virtuals must match real light — **Light Estimation** (ambient intensity, color temperature, spherical harmonics, primary directional source) provided by ARKit/ARCore; HDR environment probes. **(4) Registration accuracy**: alignment between virtual model and real object — improved with marker-based or model-target tracking and recalibration. **(5) Display limits**: small FOV (HoloLens 52°), low brightness in sunlight, semi-transparent virtuals on OST. **(6) Power and thermals** on phones / glasses — duty cycling, low-power tracking modes.

Q42. Why does AR need plane detection, and how does it work?
Without **plane detection**, an AR app cannot place virtual objects on real surfaces with correct scale and orientation; objects would float arbitrarily. Pipeline (ARCore/ARKit): **(1)** The SLAM module continuously produces a **point cloud** of feature points; **(2)** Points are clustered by spatial proximity; **(3) RANSAC** fits planes through clusters with low residual error; **(4)** Each plane is classified as **horizontal** (floor, table) or **vertical** (wall) using gravity from the IMU; **(5)** Plane boundaries are estimated using a convex hull or alpha shape; **(6)** Planes grow over frames as more points are seen; nearby planes can merge. The app uses **ARRaycastManager** to cast a ray from a screen tap to the planes and instantiate a prefab at the hit. On LiDAR-equipped iPads, the depth map is used directly for instant plane detection and meshing.

Q43. Application: VR/AR in cultural and tourism — e-Tourism Vietnam case.
e-Tourism brings physical destinations to remote audiences and enhances on-site visits. (1) **360° VR videos / VR walkthroughs** of Ha Long Bay, Phong Nha caves, Tràng An, Cu Chi tunnels — captured with **Insta360 Pro 2**, served via YouTube VR or a Unity native app on Quest. (2) **AR guides at heritage sites**: scan a QR code or use VPS (ARCore Geospatial API) to overlay reconstructions of original temple roofs, painted murals, or Cham towers onto today's ruins. (3) **Photogrammetric digital twins**: scan statues with a mobile phone (Polycam, RealityScan), reconstruct online, and let visitors examine details impossible to see from a distance. (4) **Hole filling & restoration**: damaged statues at My Son Sanctuary can be "virtually restored" using normal-field-driven hole filling (the topic of NFD-HoleFill) and Bezier surface modeling. **Tools**: Unity, Reality Composer, AR Foundation, 8th Wall WebXR for in-browser AR (no install needed for tourists).

Q44. What are the key performance metrics for VR/AR (latency, FOV, refresh rate, presence)?
(1) Motion-to-photon latency (< 20 ms target); **(2) Frame rate / refresh rate** (90 Hz minimum, 120/144 Hz preferred); **(3) Resolution** (per eye, e.g. Quest 3 2064×2208) and **pixels per degree (PPD)** (> 25 reduces SDE; Vision Pro ∼34); **(4) FOV** (110° for VR, 50° for HoloLens AR); **(5) Tracking accuracy and jitter** (sub-millimeter for Lighthouse, ∼mm for inside-out); **(6) Tracking volume** (room-scale 3×3 m typical); **(7) Battery / weight / thermals** on standalone; **(8) Subjective: presence questionnaires** (SUS, Witmer–Singer); **(9) Cybersickness scores** (Simulator Sickness Questionnaire SSQ); **(10) Task performance** (time, errors) for training and clinical apps. **(11) For AR**: registration accuracy in mm, drift over time, light estimation error, FPS under thermal load.

Q45. What are future trends in VR/AR?
(1) Light, all-day AR glasses (Meta Orion prototype, Apple AR glasses, Snap Spectacles 5) replacing the smartphone over the next decade. **(2) Pancake / metalens optics** for thin, high-clarity HMDs. **(3) Eye- and face-tracking everywhere** for foveated rendering, social presence, and biometric input. **(4) Higher resolution** (> 4K per eye, retina-class); micro-OLED mainstream. **(5) AI-generated 3D content**: text-to-3D (Luma AI Genie, Meshy, Stable Dreamer), one-shot avatars, neural radiance fields for capture. **(6) Generative agents and LLM-driven NPCs** in social VR. **(7) WebXR and 8th Wall growing browser-based AR**. **(8) Standardization** (OpenXR, glTF, USD/USDZ). **(9) Healthcare and enterprise verticals** maturing (FDA-cleared therapeutics). **(10) Brain-computer interfaces** and neural wristbands for input (Meta CTRL-Labs). **(11) Spatial computing** (Apple's term) merges VR/AR into general-purpose desktop replacement.

Q46. Exam-style: "Application of VR in [field]" — a template answer you can adapt.
Field: [healthcare / education / culture / industry / retail / psychology / AEC / military].
Structure your essay in 5 parts:
(1) Problem statement: what real-world pain (training cost, danger, accessibility, scale, knowledge transfer) does VR solve here? **(2) How VR helps**: which specific affordance (presence, safe failure, visualization, repetition, remote presence) addresses the problem? **(3) How it works** (3–5 steps): capture data → build 3D scene → track user → render stereo → interact + assess. **(4) Tools & techniques**: HMD (Quest 3 / HoloLens 2 / Vision Pro), engine (Unity + XR Interaction Toolkit), SDKs (OpenXR, ARKit/ARCore, Vuforia for marker), supporting tech (photogrammetry, MRI segmentation, BIM, IoT, biometrics). **(5) Concrete example** (a real product or paper): name a system, the institution, and a measured outcome. End with one sentence on remaining limitations. This 5-part structure handles any "application" question on the exam.

Quest Elite), wireless. **(3) Mỏi mắt/VAC**: lệch vergence vs accommodation — nghiên cứu **varifocal** (Half Dome, gợi ý Vision Pro), **light-field**. **(4) Chi phí nội dung**: asset 3D, animation, tối ưu — asset store, photogrammetry, AI-generated. **(5) Onboarding** cho người không phải gamer: tutorial, hand tracking thay controller. **(6) Compute** trên standalone — foveated rendering, fixed foveation, LOD, baked lighting. **(7) Xã hội/quyền riêng tư** — guardian, camera tắt trong không gian riêng.

Câu 41. Thách thức chính của AR (drift, occlusion, ánh sáng, registration).
(1) Drift của tracking: VI-SLAM cộng dồn sai số — giải bằng **loop closure**, **cloud anchor**, bản đồ bền, dùng marker làm anchor. **(2) Occlusion**: vật ảo phải lẫn sau vật thật — dùng **depth map** (ARKit People Occlusion, ARCore Depth API), **scene mesh**, hoặc instance segmentation. **(3) Ánh sáng nhất quán**: vật ảo phải hợp ánh sáng thật — **Light Estimation** (cường độ, color temperature, spherical harmonics, hướng chính) của ARKit/ARCore; HDR env probe. **(4) Độ chính xác registration**: căn chỉnh model ảo với vật thật — dùng marker hoặc model target và recalibrate. **(5) Hạn chế display**: FOV nhỏ (HoloLens 52°), độ sáng thấp ngoài nắng, vật ảo trong mờ trên OST. **(6) Pin và nhiệt** trên phone/kính — duty cycle, low-power tracking.

Câu 42. Vì sao AR cần plane detection và hoạt động ra sao?
Không có **plane detection**, app AR không đặt được vật ảo lên bề mặt thật đúng tỉ lệ + hướng; vật sẽ trôi lung tung. Pipeline (ARCore/ARKit): **(1)** SLAM sinh **point cloud** feature liên tục; **(2)** Cluster point theo khoảng cách; **(3) RANSAC** fit plane qua cluster với sai số nhỏ; **(4)** Plane được phân loại **ngang** (sàn, bàn) hay **dọc** (tường) dựa vào gravity từ IMU; **(5)** Biên plane ước lượng bằng convex hull hoặc alpha shape; **(6)** Plane lớn dần qua các frame; plane gần nhau có thể merge. App dùng **ARRaycastManager** bắn tia từ điểm chạm tới plane và instantiate prefab tại hit. Trên iPad có LiDAR, depth map dùng trực tiếp để dò plane và mesh tức thì.

Câu 43. Ứng dụng: VR/AR trong văn hóa và du lịch — e-Tourism Việt Nam.
e-Tourism đưa địa điểm thực tới khán giả từ xa và làm phong phú trải nghiệm tại chỗ. (1) **Video VR 360°/VR walkthrough** Hạ Long, Phong Nha, Tràng An, địa đạo Củ Chi — quay bằng **Insta360 Pro 2**, phát qua YouTube VR hoặc app Unity trên Quest. (2) **AR guide tại di tích**: quét QR hoặc dùng VPS (ARCore Geospatial API) để overlay phục dựng mái đền, tranh tường, tháp Chăm lên di tích hiện tại. (3) **Digital twin photogrammetry**: quét tượng bằng phone (Polycam, RealityScan), dựng online, cho khách xem chi tiết không thấy từ xa. (4) **Hole filling và phục dựng**: tượng vỡ ở Mỹ Sơn có thể "phục chế ảo" bằng normal-field-driven hole filling (đề tài NFD-HoleFill) và Bezier surface. **Tool**: Unity, Reality Composer, AR Foundation, 8th Wall WebXR cho AR trong browser (du khách không cần cài app).

Câu 44. Các metric hiệu năng quan trọng của VR/AR.
(1) Motion-to-photon latency (mục tiêu < 20 ms); **(2) FPS/refresh** (tối thiểu 90 Hz, tốt 120/144 Hz); **(3) Độ phân giải** mỗi mắt (Quest 3 2064×2208) và **pixel per degree (PPD)** (> 25 giảm SDE; Vision Pro ∼34); **(4) FOV** (110° VR, 50° HoloLens); **(5) Độ chính xác và jitter của tracking** (sub-mm Lighthouse, ∼mm inside-out); **(6) Vùng tracking** (room-scale 3×3 m); **(7) Pin/nặng/nhiệt** trên standalone; **(8) Chủ quan: questionnaire presence** (SUS, Witmer–Singer); **(9) Cybersickness** (SSQ); **(10) Hiệu suất task** (thời gian, lỗi) cho training và y tế. **(11) AR**: độ chính xác registration mm, drift theo thời gian, sai số light estimation, FPS dưới tải nhiệt.

Câu 45. Xu hướng tương lai của VR/AR.
(1) Kính AR nhẹ, đeo cả ngày (Meta Orion, Apple AR glasses, Snap Spectacles 5) thay smartphone trong thập kỷ tới. **(2) Pancake/metalens optics** cho HMD mỏng, sắc nét. **(3) Eye + face tracking khắp nơi** cho foveated rendering, social presence, biometric input. **(4) Resolution cao hơn** (> 4K mỗi mắt, retina-class); micro-OLED phổ thông. **(5) AI sinh nội dung 3D**: text-to-3D (Luma AI Genie, Meshy, Stable Dreamer), avatar one-shot, NeRF cho capture. **(6) Generative agent + LLM-driven NPC** trong social VR. **(7) WebXR và 8th Wall** đẩy AR trên browser. **(8) Chuẩn hóa** (OpenXR, glTF, USD/USDZ). **(9) Y tế và doanh nghiệp** trưởng thành (therapeutic được FDA duyệt). **(10) BCI / dây đeo cổ tay thần kinh** làm input (Meta CTRL-Labs). **(11) Spatial computing** (Apple) hợp nhất VR/AR thành máy tính cá nhân thế hệ mới.

Câu 46. Mẫu trả lời "Ứng dụng VR trong [lĩnh vực]" có thể tái sử dụng.
Lĩnh vực: [y tế / giáo dục / văn hóa / công nghiệp / bán lẻ / tâm lý / AEC / quân sự].
Cấu trúc 5 phần:
(1) Vấn đề: nỗi đau thực tế nào (chi phí huấn luyện, nguy hiểm, khó tiếp cận, quy mô, truyền tri thức) mà VR giải được? **(2) VR giúp thế nào**: affordance cụ thể (presence, sai mà không nguy hiểm, trực quan, lặp, hiện diện từ xa) trị vấn đề ra sao? **(3) Cách hoạt động** (3–5 bước): thu dữ liệu → dựng scene 3D → track người dùng → render stereo → tương tác + đánh giá. **(4) Tool & kỹ thuật**: HMD (Quest 3/HoloLens 2/Vision Pro), engine (Unity + XRI), SDK (OpenXR, ARKit/ARCore, Vuforia cho marker), công nghệ phụ (photogrammetry, segmentation MRI, BIM, IoT, biometric). **(5) Ví dụ cụ thể**: tên hệ thống, đơn vị, kết quả đo được. Kết bằng một câu về giới hạn còn lại. Cấu trúc 5 phần này dùng được cho mọi câu "ứng dụng" trong đề thi.

QA2a. Theoretical Q&A — Triangulation & Hole Filling

Q1. What is triangulation? Why do we need it in 3D modeling?
Triangulation is the process of decomposing a geometric domain (a point cloud, polygon, or surface) into a set of non-overlapping triangles whose union covers the domain. In 3D modeling we triangulate because the **triangle** is the

QA2a. Câu hỏi lý thuyết — Tam giác hóa & Vá lỗ

Câu 1. Tam giác hóa là gì? Tại sao cần trong mô hình hóa 3D?
Tam giác hóa (triangulation) là quá trình phân hoạch một miền hình học (point cloud, đa giác, bề mặt) thành tập các tam giác không chồng lấn phủ kín toàn miền. Trong mô hình hóa 3D ta cần tam giác hóa vì **triangle**

simplest planar primitive: it is always convex, always planar, and uniquely defined by three vertices, which makes it ideal for GPU rasterization, ray-tracing, and finite-element analysis. Most rendering APIs (OpenGL, Vulkan, DirectX) only consume triangle meshes natively, so any higher-order surface (NURBS, subdivision, implicit) must be tessellated into triangles before display. Triangulations also support **barycentric interpolation** of attributes (color, normal, UV), which is the foundation of Gouraud and Phong shading. In geometry processing, triangle meshes form a discrete **2-manifold** on which we can define discrete differential operators (Laplacian, divergence, curvature). Without triangulation, downstream tasks like physics simulation, 3D printing slicing, and collision detection are impossible. Hence triangulation is the universal bridge between raw geometric data and computable structure.

Q2. Given a 3D point cloud, how do you triangulate it? List the main algorithms.

Triangulating a 3D point cloud means recovering a surface mesh that interpolates or approximates the points. The main algorithm families are: (i) **Ball Pivoting Algorithm** (BPA) — rolls a virtual ball of radius ρ over the cloud, creating a triangle whenever the ball touches three points without containing any other; (ii) **Poisson Surface Reconstruction** — solves an indicator-function PDE driven by oriented normals, then extracts an isosurface via Marching Cubes; (iii) **Delaunay-based** methods (3D Delaunay tetrahedralization $\to$ alpha-shape filtering) which are mathematically optimal but expensive, $O(N^2)$ in the worst case; (iv) **Greedy projection triangulation** (PCL) — projects local neighborhoods onto a tangent plane and triangulates incrementally; (v) **Marching Cubes** on a signed-distance field for implicit surfaces; (vi) **Power Crust** and **Cocone** — provable reconstruction guarantees from samples. The choice depends on noise level, density uniformity, and whether normals are available. Typical pipeline: **denoise** $\to$ **normal estimation** $\to$ **reconstruction** $\to$ **post-processing** (hole filling, decimation).

Q3. Explain Delaunay triangulation and its empty-circle property.

A **Delaunay triangulation** $\mathrm{DT}(P)$ of a point set $P \subset \mathbb{R}^2$ is a triangulation such that for every triangle, its **circumscribed circle** contains no other point of P in its interior. This is the **empty circle property** (or empty circumcircle criterion). Equivalently, a triangulation is Delaunay iff every internal edge is **locally Delaunay**: the sum of the two opposite angles is at most 180°. The Delaunay triangulation exists and is unique whenever points are in **general position** (no four cocircular). It can be computed in $O(N \log N)$ using divide-and-conquer, sweep-line (Fortune), or randomized incremental insertion (Bowyer-Watson). The empty-circle property guarantees that triangles are as "fat" as possible — Delaunay maximizes the **minimum angle** over all triangulations of P, avoiding thin slivers. In 3D, the analogue is the Delaunay tetrahedralization with the empty-sphere property. This makes Delaunay the de-facto standard for high-quality meshing in CAD, GIS, and FEM.

Q4. Why does Delaunay maximize the minimum angle? Why is this desirable?

The **max-min angle** property follows from the **flip lemma**: if an internal edge e shared by two triangles fails the local Delaunay condition (the opposite vertex lies inside the circumcircle), then **flipping** e to the other diagonal of the convex quadrilateral strictly increases the minimum of the six angles involved. Iterating flips converges (the angle vector strictly improves in lexicographic order), and the fixed point is the Delaunay triangulation. Thus among all triangulations of the same point set, Delaunay has the **lexicographically largest sorted angle vector**, in particular the largest minimum angle. This is desirable because thin **sliver triangles** (minimum angle near 0°) cause: (i) numerical instability in linear systems (large condition numbers in FEM stiffness matrices); (ii) interpolation error blow-up since gradient estimates depend on $1/\sin\theta_{\min}$; (iii) shading artifacts due to ill-conditioned barycentric coordinates; (iv) collision-detection robustness issues. Maximizing the minimum angle therefore improves both numerical accuracy and visual quality.

Q5. Explain the Bowyer-Watson algorithm step-by-step.

Bowyer-Watson is a randomized incremental algorithm for constructing the Delaunay triangulation in expected $O(N \log N)$ time.

1. Initialize with a **super-triangle** (or super-tetrahedron in 3D) large enough to contain all points P.
2. For each point $p \in P$ inserted in random order:
   (a) Find all triangles whose **circumcircle contains** p; call this the **bad set** B.
   (b) Compute the **boundary** (cavity) of B — the polygon of edges shared by exactly one triangle in B.
   (c) Delete all triangles in B.
   (d) Re-triangulate the cavity by connecting p to every boundary edge, creating a fan of new triangles.
3. After all insertions, remove all triangles incident to the super-triangle vertices.

The key invariant is that the empty-circle property is restored after each insertion. Robustness requires **exact predicates** (in-circle, orientation) to avoid topological inconsistency from floating-point errors.

Q6. What is the in-circle predicate? Write the determinant formula.

The **in-circle predicate** tests whether a point D lies inside, on, or outside the circumcircle of triangle ABC (with A, B, C in counter-clockwise order). It is the geometric heart of Delaunay algorithms. The classical determinant formula is:

$$\mathrm{InCircle}(A,B,C,D) = \det \begin{pmatrix} A_x - D_x & A_y - D_y & (A_x - D_x)^2 + (A_y - D_y)^2 \\ B_x - D_x & B_y - D_y & (B_x - D_x)^2 + (B_y - D_y)^2 \\ C_x - D_x & C_y - D_y & (C_x - D_x)^2 + (C_y - D_y)^2 \end{pmatrix}$$

A positive value means D is **inside** the circumcircle, zero means **on** it, and negative means **outside**. This is a degree-4 polynomial in the coordinates, so floating-point evaluation can flip sign near zero and produce inconsistent triangulations. Robust implementations (Shewchuk's predicates) use **adaptive exact arithmetic** — start with cheap approximate evaluation, fall back to exact only when the magnitude is too small to trust. The 3D analogue is the in-sphere predicate, a 4×4 determinant.

Q7. Dual relationship between Delaunay triangulation and Voronoi diagram?

The **Voronoi diagram** $\mathrm{Vor}(P)$ partitions the plane into cells, where cell V_p contains all points closer to $p \in P$ than to any other site. The Delaunay triangulation is the **straight-line dual graph** of the Voronoi diagram: there is an edge pq in $\mathrm{DT}(P)$ iff Voronoi cells V_p and V_q share an edge, and a triangle pqr in $\mathrm{DT}(P)$ iff V_p, V_q, V_r meet at a

là primitive phẳng đơn giản nhất: luôn lồi, luôn phẳng, xác định duy nhất bởi ba vertex, lý tưởng cho GPU rasterization, ray-tracing và FEM. Hầu hết rendering API (OpenGL, Vulkan, DirectX) chỉ nhận triangle mesh làm input, nên các bề mặt bậc cao (NURBS, subdivision, implicit) đều phải được tessellate thành tam giác trước khi hiển thị. Triangulation cho phép **nội suy barycentric** các thuộc tính (color, normal, UV) — nền tảng của Gouraud và Phong shading. Trong geometry processing, triangle mesh tạo thành **2-manifold** rời rạc trên đó ta định nghĩa được các toán tử vi phân rời rạc (Laplacian, divergence, curvature). Không có tam giác hóa, các tác vụ downstream như mô phỏng vật lý, slicing in 3D, và collision detection đều bất khả thi. Vì vậy tam giác hóa là cầu nối phổ quát giữa dữ liệu hình học thô và cấu trúc tính toán được.

Câu 2. Cho point cloud 3D, làm sao tam giác hóa? Liệt kê các thuật toán chính.

Tam giác hóa point cloud 3D nghĩa là khôi phục mesh bề mặt nội suy hoặc xấp xỉ các điểm. Các họ thuật toán chính: (i) **Ball Pivoting Algorithm** (BPA) — lăn một quả banh ảo bán kính ρ qua point cloud, sinh tam giác khi banh chạm ba điểm mà không chứa điểm nào khác; (ii) **Poisson Surface Reconstruction** — giải PDE indicator function dẫn dắt bởi normal có hướng, rồi trích isosurface bằng Marching Cubes; (iii) phương pháp dựa trên **Delaunay** (3D Delaunay tetrahedralization $\to$ lọc alpha-shape) — tối ưu về toán học nhưng đắt, $O(N^2)$ worst case; (iv) **Greedy projection triangulation** (PCL) — chiếu lân cận cục bộ lên tangent plane rồi tam giác hóa từng bước; (v) **Marching Cubes** trên trường khoảng cách có dấu cho implicit surface; (vi) **Power Crust** và **Cocone** — đảm bảo tái tạo có chứng minh từ mẫu. Lựa chọn tùy mức độ noise, độ đồng đều mật độ, và có normal hay không. Pipeline điển hình: **denoise** $\to$ **ước lượng normal** $\to$ **reconstruction** $\to$ **post-processing** (vá lỗ, decimation).

Câu 3. Giải thích Delaunay triangulation và tính chất hình tròn rỗng.

Delaunay triangulation $\mathrm{DT}(P)$ của tập điểm $P \subset \mathbb{R}^2$ là tam giác hóa sao cho mọi tam giác có **đường tròn ngoại tiếp** không chứa điểm nào khác của P ở trong. Đây là **tính chất hình tròn rỗng** (empty circumcircle). Tương đương, một triangulation là Delaunay khi và chỉ khi mọi cạnh nội bộ là **locally Delaunay**: tổng hai góc đối diện $\leq 180^\circ$. Delaunay tồn tại và duy nhất khi các điểm ở **general position** (không có 4 điểm đồng viên). Có thể tính trong $O(N \log N)$ bằng divide-and-conquer, sweep-line (Fortune), hoặc randomized incremental insertion (Bowyer-Watson). Tính chất hình tròn rỗng đảm bảo tam giác "béo" nhất có thể — Delaunay **cực đại hóa góc nhỏ nhất** trên mọi triangulation của P, tránh sliver. Trong 3D, tương ứng là Delaunay tetrahedralization với tính chất hình cầu rỗng. Do đó Delaunay là chuẩn de-facto cho meshing chất lượng cao trong CAD, GIS, FEM.

Câu 4. Tại sao Delaunay cực đại hóa góc nhỏ nhất? Tại sao điều này mong muốn?

Tính chất **max-min angle** suy ra từ **flip lemma**: nếu cạnh nội bộ e chung bởi hai tam giác không thỏa local Delaunay (đỉnh đối diện nằm trong đường tròn ngoại tiếp), thì **flip** e sang đường chéo kia của tứ giác lồi sẽ nghiêm ngặt tăng góc nhỏ nhất trong sáu góc liên quan. Lặp flip hội tụ (vector góc cải thiện theo thứ tự từ điển), và điểm cố định chính là Delaunay. Vì vậy trong mọi triangulation của cùng tập điểm, Delaunay có **vector góc sắp xếp lớn nhất theo thứ tự từ điển**, đặc biệt là góc nhỏ nhất lớn nhất. Điều này mong muốn vì **sliver** (góc nhỏ gần 0°) gây: (i) bất ổn định số trong hệ tuyến tính (condition number lớn trong ma trận stiffness FEM); (ii) sai số nội suy bùng nổ vì gradient phụ thuộc $1/\sin\theta_{\min}$; (iii) artifact shading do barycentric không ổn định; (iv) vấn đề robustness trong collision detection. Cực đại hóa góc nhỏ nhất cải thiện cả độ chính xác số học lẫn chất lượng hiển thị.

Câu 5. Giải thích Bowyer-Watson từng bước.

Bowyer-Watson là thuật toán incremental ngẫu nhiên xây Delaunay trong kỳ vọng $O(N \log N)$.

1. Khởi tạo với **super-triangle** (super-tetrahedron trong 3D) đủ lớn chứa toàn bộ P.
2. Với mỗi điểm $p \in P$ chèn theo thứ tự ngẫu nhiên:
   (a) Tìm mọi tam giác có **circumcircle chứa** p; gọi là **bad set** B.
   (b) Tính **boundary** (cavity) của B — đa giác các cạnh thuộc đúng một tam giác trong B.
   (c) Xóa toàn bộ tam giác trong B.
   (d) Tam giác hóa lại cavity bằng cách nối p với từng cạnh biên, tạo fan tam giác mới.
3. Sau khi chèn xong, xóa mọi tam giác kề super-triangle vertex.

Bất biến chính: tính chất hình tròn rỗng được khôi phục sau mỗi lần chèn. Robustness cần **exact predicate** (in-circle, orientation) để tránh bất nhất topology do lỗi floating-point.

Câu 6. In-circle predicate là gì? Viết công thức định thức.

In-circle predicate kiểm tra điểm D nằm trong, trên, hay ngoài đường tròn ngoại tiếp tam giác ABC (với A, B, C counter-clockwise). Đây là trái tim hình học của thuật toán Delaunay. Công thức định thức cổ điển:

$$\mathrm{InCircle}(A,B,C,D) = \det \begin{pmatrix} A_x - D_x & A_y - D_y & (A_x - D_x)^2 + (A_y - D_y)^2 \\ B_x - D_x & B_y - D_y & (B_x - D_x)^2 + (B_y - D_y)^2 \\ C_x - D_x & C_y - D_y & (C_x - D_x)^2 + (C_y - D_y)^2 \end{pmatrix}$$

Giá trị dương: D **trong** circumcircle, 0: **trên**, âm: **ngoài**. Đây là đa thức bậc 4 theo tọa độ, nên đánh giá floating-point có thể đảo dấu gần 0 sinh triangulation bất nhất. Implementation robust (predicate của Shewchuk) dùng **adaptive exact arithmetic** — bắt đầu bằng đánh giá xấp xỉ rẻ, chỉ rơi về exact khi magnitude quá nhỏ. Tương ứng 3D là in-sphere predicate, định thức 4×4.

Câu 7. Quan hệ đối ngẫu giữa Delaunay và Voronoi?

Voronoi diagram $\mathrm{Vor}(P)$ chia mặt phẳng thành các cell, cell V_p chứa mọi điểm gần $p \in P$ hơn các site khác. Delaunay là **đồ thị đối ngẫu đường thẳng** của Voronoi: có cạnh pq trong $\mathrm{DT}(P)$ khi và chỉ khi cell V_p và V_q chia cạnh chung, và có tam giác pqr khi và chỉ khi V_p, V_q, V_r gặp tại một Voronoi vertex. Voronoi vertex tại điểm gặp

Voronoi vertex. The Voronoi vertex at the meeting of three cells is exactly the **circumcenter** of the corresponding Delaunay triangle, which is why Delaunay's empty-circle property is the dual of Voronoi's empty-cell property. Computationally one structure yields the other in linear time. This duality is exploited in **centroidal Voronoi tessellations** (CVT) for high-quality remeshing, in nearest-neighbor queries, and in mesh generation algorithms like Lloyd's relaxation. Together they form the foundational duo of computational geometry.

Q8. Compare Delaunay vs Constrained Delaunay (CDT). When is CDT needed?

A **Constrained Delaunay Triangulation** (CDT) is a triangulation that contains a prescribed set of edges E (constraints) and is "as Delaunay as possible" subject to those constraints. Standard Delaunay only respects the point set P and may insert edges that cross required boundaries, which is unacceptable when modeling polygons with holes, GIS shorelines, or CAD profiles. Formally, an edge e in CDT is **constrained Delaunay** if either $e \in E$, or its circumcircle contains no point of P *visible* from the interior of e (visibility blocked by constraints). CDT is needed for: (i) polygon triangulation respecting holes and concavities; (ii) terrain modeling with rivers and ridges as breaklines; (iii) FEM meshing where material boundaries must be preserved; (iv) cartographic generalization where road centerlines act as constraints. Algorithms like Shewchuk's `Triangle` compute CDT by inserting constraint edges into a Delaunay triangulation and using **constrained edge flips**. CDT may sacrifice the global max-min angle property locally near constraints.

Q9. Explain Ball Pivoting Algorithm (BPA). Role of ball radius?

Ball Pivoting Algorithm (BPA, Bernardini et al. 1999) reconstructs a triangle mesh from an oriented point cloud by simulating a virtual ball of radius ρ rolling over the points.

1. Find a **seed triangle**: any three points such that the ball touches all three without containing any other point.
2. For each boundary edge of the current mesh, **pivot** the ball around the edge until it hits a new point, creating a new triangle.
3. Repeat until no more pivots are possible; restart with a new seed if the cloud has multiple components.

The **ball radius** ρ is the critical parameter. If ρ is too small, the ball falls through the cloud creating holes wherever sample spacing exceeds 2ρ. If ρ is too large, the ball bridges across thin features, fusing geometry that should remain separate. A common heuristic sets ρ to slightly larger than the median nearest-neighbor distance. **Multi-radius BPA** runs the algorithm with increasing radii to capture both fine details and large gaps. BPA is fast, memory-efficient, and preserves input points exactly — but is sensitive to noise and density variation.

Q10. Explain Poisson Surface Reconstruction. What equation?

Poisson Surface Reconstruction (Kazhdan, Bolitho, Hoppe 2006) reconstructs a watertight surface from oriented point samples by formulating reconstruction as a **Poisson problem**. Treat the surface as the boundary of a solid, and let $\chi : \mathbb{R}^3 \to \{0, 1\}$ be the indicator function (1 inside, 0 outside). The gradient $\nabla\chi$ is zero everywhere except at the surface, where it equals the inward normal as a distribution. Approximating with a smoothed indicator $\tilde{\chi}$, the input oriented points $\{p_i, \mathbf{n}_i\}$ define a vector field $\vec{V}$, and we seek $\tilde{\chi}$ such that $\nabla\tilde{\chi} \approx \vec{V}$. Taking divergence yields the **Poisson equation**:

$$\Delta\tilde{\chi} = \nabla \cdot \vec{V}$$

This is solved on an adaptive octree, then the surface is extracted as the isosurface $\tilde{\chi} = c$ via Marching Cubes. Poisson is robust to noise, produces smooth watertight meshes, and naturally fills small holes — but tends to oversmooth sharp features and depends on accurate normals. Screened Poisson (Kazhdan-Hoppe 2013) adds a positional constraint to recover sharp detail.

Q11. Compare BPA vs Poisson — when to use each?

BPA (Ball Pivoting) is an **interpolating** reconstruction: every input point becomes a mesh vertex, preserving sample positions exactly. **Poisson** is **approximating**: it fits a smooth implicit field through the points and resamples a new mesh. Use BPA when: input is clean (LiDAR, structured-light scan), density is uniform, you need to preserve original samples (e.g., for measurement traceability in metrology), and you want fast results — BPA is roughly $O(N)$ with spatial hashing. Use Poisson when: input is noisy (photogrammetry, RGB-D), density varies, you need a guaranteed watertight mesh for 3D printing or simulation, and you can afford an $O(N)$ octree solve. Poisson naturally fills small gaps — a side-effect that is desirable for repair but bad if the gaps were intentional. BPA produces topologically faithful but possibly non-watertight meshes; Poisson produces watertight but oversmoothed meshes. In practice, modern pipelines (Meshlab, Open3D) provide both and let the user choose based on data characteristics.

Q12. What is alpha-shape? How does α affect the result?

The **alpha-shape** (Edelsbrunner-Mücke 1994) of a point set $P \subset \mathbb{R}^d$ generalizes the convex hull by parameterizing "how tightly" the boundary wraps the points. Geometrically, an edge/triangle/tetrahedron belongs to the alpha-complex if there exists an empty α-ball (radius α) that touches its vertices and contains no other point. As $\alpha \to \infty$, the alpha-shape becomes the convex hull. As $\alpha \to 0$, it degenerates to the input points themselves. Intermediate α produces shapes with concavities, holes, and multiple components — capturing the "true" shape suggested by the samples. **Small** α: more detailed, more holes, sensitive to noise. **Large** α: smoother, fewer holes, may bridge real cavities. Alpha-shapes are computed efficiently as a filtration of the Delaunay triangulation: every Delaunay simplex has a critical α at which it enters the complex. Applications: shape reconstruction from samples, molecular surface modeling, density-based clustering, and topological data analysis (persistent homology).

Q13. Explain Marching Cubes — why 256 cases reduce to 15?

Marching Cubes (Lorensen-Cline 1987) extracts a triangle mesh isosurface $f(x, y, z) = c$ from a 3D scalar field sampled on a regular grid. For each grid cube (8 corners), classify each corner as **inside** ($f \le c$) or **outside** ($f > c$), giving $2^8 = 256$ possible configurations. For each, a lookup table tells which edges are intersected by the isosurface; the intersection points (computed by linear interpolation) are connected into 0–4 triangles. Naively, 256 distinct cases — but exploiting symmetries reduces them: **complementary symmetry** (swap inside/outside) halves to 128, and **rotational symmetry** of the cube (24 rotations) plus reflections collapse equivalent topologies, yielding

của ba cell chính là **circumcenter** của tam giác Delaunay tương ứng, nên tính chất hình tròn rỗng của Delaunay là đối ngẫu của tính chất cell rỗng của Voronoi. Về tính toán, có cấu trúc này thì lấy được cấu trúc kia trong thời gian tuyến tính. Đối ngẫu được khai thác trong **centroidal Voronoi tessellations** (CVT) cho remeshing chất lượng cao, trong nearest-neighbor query, và trong các thuật toán mesh generation như Lloyd relaxation. Cùng nhau chúng tạo thành cặp nền tảng của hình học tính toán.

Câu 8. So sánh Delaunay vs Constrained Delaunay (CDT). Khi nào cần CDT?

Constrained Delaunay Triangulation (CDT) là triangulation chứa tập cạnh chỉ định E (constraint) và "Delaunay nhất có thể" với ràng buộc đó. Delaunay chuẩn chỉ tôn trọng tập điểm P, có thể chèn cạnh cắt qua biên bắt buộc — không chấp nhận được khi mô hình hóa đa giác có lỗ, đường bờ biển GIS, hay profile CAD. Hình thức, cạnh e trong CDT là **constrained Delaunay** nếu $e \in E$, hoặc circumcircle không chứa điểm P nào *nhìn thấy* từ phần trong e (visibility bị chặn bởi constraint). CDT cần cho: (i) tam giác hóa đa giác có lỗ và lõm; (ii) mô hình địa hình với sông, sống núi làm breakline; (iii) meshing FEM nơi biên vật liệu phải giữ; (iv) tổng quát hóa bản đồ với centerline đường làm constraint. Các thuật toán như `Triangle` của Shewchuk tính CDT bằng cách chèn constraint edge vào Delaunay rồi dùng **constrained edge flip**. CDT có thể hy sinh tính max-min angle toàn cục cục bộ gần constraint.

Câu 9. Giải thích Ball Pivoting (BPA). Vai trò bán kính banh?

Ball Pivoting Algorithm (BPA, Bernardini và cộng sự 1999) tái tạo triangle mesh từ point cloud có normal bằng cách mô phỏng banh ảo bán kính ρ lăn qua các điểm.

1. Tìm **seed triangle**: ba điểm sao cho banh chạm cả ba mà không chứa điểm nào khác.
2. Với mỗi cạnh biên của mesh hiện tại, **pivot** banh quanh cạnh đó đến khi chạm điểm mới, sinh tam giác mới.
3. Lặp đến khi không pivot được nữa; restart với seed mới nếu cloud có nhiều thành phần.

Bán kính banh ρ là tham số then chốt. ρ quá nhỏ: banh lọt qua cloud sinh lỗ ở mọi chỗ khoảng cách mẫu vượt 2ρ. ρ quá lớn: banh bắc cầu qua chi tiết mỏng, hợp nhất hình học phải tách. Heuristic thường: ρ hơi lớn hơn median nearest-neighbor distance. **Multi-radius BPA** chạy với bán kính tăng dần để bắt cả chi tiết nhỏ lẫn khoảng trống lớn. BPA nhanh, tiết kiệm memory, giữ input point chính xác — nhưng nhạy với noise và biến thiên mật độ.

Câu 10. Giải thích Poisson Surface Reconstruction. Phương trình gì?

Poisson Surface Reconstruction (Kazhdan, Bolitho, Hoppe 2006) tái tạo bề mặt watertight từ point sample có normal bằng cách công thức hóa thành **bài toán Poisson**. Coi bề mặt là biên của một khối, và $\chi : \mathbb{R}^3 \to \{0, 1\}$ là indicator function (1 trong, 0 ngoài). Gradient $\nabla\chi$ bằng 0 mọi nơi trừ trên bề mặt, nơi nó bằng inward normal dạng phân phối. Xấp xỉ bằng indicator được làm trơn $\tilde{\chi}$, các điểm có normal $\{p_i, \mathbf{n}_i\}$ định nghĩa vector field $\vec{V}$, và ta tìm $\tilde{\chi}$ sao cho $\nabla\tilde{\chi} \approx \vec{V}$. Lấy divergence cho **phương trình Poisson**:

$$\Delta\tilde{\chi} = \nabla \cdot \vec{V}$$

Giải trên adaptive octree, rồi trích bề mặt như isosurface $\tilde{\chi} = c$ qua Marching Cubes. Poisson robust với noise, sinh mesh watertight mượt, tự nhiên vá lỗ nhỏ — nhưng hay oversmooth feature sắc và phụ thuộc normal chính xác. Screened Poisson (2013) thêm positional constraint để khôi phục chi tiết sắc.

Câu 11. So sánh BPA vs Poisson — khi nào dùng?

BPA là tái tạo **interpolating**: mọi input point thành vertex mesh, giữ vị trí mẫu chính xác. **Poisson** là **approximating**: fit trường implicit mượt qua các điểm rồi resample mesh mới. Dùng BPA khi: input sạch (LiDAR, structured-light), mật độ đồng đều, cần giữ sample gốc (vd: traceability đo lường trong metrology), và muốn nhanh — BPA $O(N)$ với spatial hashing. Dùng Poisson khi: input noisy (photogrammetry, RGB-D), mật độ biến thiên, cần watertight mesh cho in 3D hay simulation, chấp nhận $O(N)$ octree solve. Poisson tự nhiên vá khoảng trống nhỏ — tốt cho repair nhưng xấu nếu khoảng trống là cố ý. BPA cho mesh trung thành topology nhưng có thể không watertight; Poisson watertight nhưng oversmooth. Thực tế pipeline hiện đại (Meshlab, Open3D) cung cấp cả hai cho user chọn theo đặc tính dữ liệu.

Câu 12. Alpha-shape là gì? α ảnh hưởng kết quả thế nào?

Alpha-shape (Edelsbrunner-Mücke 1994) của tập điểm $P \subset \mathbb{R}^d$ tổng quát hóa convex hull bằng cách tham số hóa biên "quấn chặt" điểm thế nào. Hình học, một edge/triangle/tetrahedron thuộc alpha-complex nếu tồn tại α-ball rỗng (bán kính α) chạm các đỉnh mà không chứa điểm khác. Khi $\alpha \to \infty$, alpha-shape trở thành convex hull. Khi $\alpha \to 0$, suy biến thành chính các điểm. α trung gian sinh hình có lõm, lỗ, nhiều thành phần — bắt được hình "thật" mà sample gợi ý. α **nhỏ**: chi tiết hơn, nhiều lỗ, nhạy noise. α **lớn**: mượt hơn, ít lỗ, có thể nối qua khoang thật. Alpha-shape tính hiệu quả như filtration của Delaunay: mọi Delaunay simplex có α tới hạn để vào complex. Ứng dụng: tái tạo shape từ sample, surface phân tử, clustering theo mật độ, topological data analysis (persistent homology).

Câu 13. Marching Cubes — tại sao 256 case rút về 15?

Marching Cubes (Lorensen-Cline 1987) trích triangle mesh isosurface $f(x, y, z) = c$ từ trường scalar 3D sample trên grid đều. Với mỗi cube grid (8 góc), phân loại mỗi góc là **trong** ($f \le c$) hay **ngoài** ($f > c$), cho $2^8 = 256$ cấu hình. Với mỗi cấu hình, lookup table chỉ ra cạnh nào bị isosurface cắt; các giao điểm (tính bằng nội suy tuyến tính) được nối thành 0–4 tam giác. Naive: 256 case riêng — nhưng khai thác symmetry rút gọn: **complementary symmetry** (đổi trong/ngoài) giảm còn 128, và **rotational symmetry** của cube (24 phép quay) cộng phản chiếu hợp nhất topology tương đương, cho **15 case duy nhất** cổ điển (một số tài liệu đếm 14 hay 16). Lúc runtime, vẫn

the classical **15 unique cases** (some references count 14 or 16). At runtime, the original 256-entry table is still used (no rotation needed per cube), but designers only had to manually enumerate 15 patterns. A subtlety: the original table has **ambiguous cases** where the chosen triangulation can produce holes between adjacent cubes; later variants (Asymptotic Decider, Marching Cubes 33) resolve this with extra topology checks.

Q14. Describe greedy projection triangulation (PCL).

Greedy Projection Triangulation (Marton-Rusu-Beetz 2009, in PCL) reconstructs a mesh from an oriented point cloud by incrementally growing triangles in local tangent planes.

1. **Setup:** build a kd-tree on P; for each point p_i have a normal $\mathbf{n}_i$.
2. Pick a seed point and project its k-nearest neighbors onto the tangent plane.
3. Sort neighbors by polar angle around the seed; greedily form triangles from the seed to consecutive admissible neighbors.
4. A neighbor is **admissible** if: distance $\leq r_{\max}$, normal angle $\leq \theta_{\max}$, the triangle does not violate visibility, and all candidate triangles obey a maximum allowed angle.
5. Maintain a **front** of fringe edges; iteratively extend the front by adding new admissible triangles until no extension is possible.

Parameters: search radius $r_{\max}$, max neighbors k, μ (relative density), maximum surface angle, and minimum/maximum triangle angles. The algorithm is fast ($O(N \log N)$), local, and parallelizable, but cannot recover topologically complex regions and requires consistent oriented normals.

Q15. What are sliver triangles and why avoid them?

A **sliver triangle** is a triangle whose minimum angle is very small (typically $< 5^\circ$) — i.e., nearly degenerate, long and thin. In 3D meshing, slivers also include tetrahedra with small dihedral angles. Slivers are problematic because: (i) the **condition number** of FEM stiffness matrices grows like $1/\sin^2\theta_{\min}$, so slivers cause iterative solvers to converge slowly or diverge; (ii) gradient and Laplacian estimates on the mesh have error $O(1/\sin\theta_{\min})$, degrading downstream physics; (iii) numerical robustness of geometric predicates fails near degeneracy; (iv) **shading** interpolation produces visible bands and Mach-band artifacts; (v) **normal estimation** via cross product becomes unreliable, since the cross product magnitude vanishes; (vi) **collision detection** and ray-casting suffer from self-intersection false positives. Mitigation strategies: enforce Delaunay during construction; **edge flipping**, **edge collapsing**, and **vertex repositioning** during cleanup; use **centroidal Voronoi tessellation** (CVT) for isotropic remeshing; in 3D, use **sliver exudation** (Cheng-Dey) which perturbs vertex weights to remove slivers from weighted Delaunay tetrahedralizations.

Q16. Triangular mesh with many holes — how to reconstruct? General approach.

The general pipeline for repairing a holed mesh has four phases. (1) **Hole detection:** scan all edges, find boundary edges (those incident to exactly one face), and group them into closed loops via half-edge traversal. (2) **Hole classification:** compute hole size (perimeter, planarity, genus), distinguish small holes (handle with simple methods) from large holes (need geometry-aware methods); detect islands and multi-component holes. (3) **Patch generation:** for each hole loop, generate an initial triangulation — common methods are minimum-weight triangulation (Liepa), advancing front, or projection-based ear clipping. (4) **Quality refinement:** apply edge flips toward Delaunay, subdivide oversized triangles to match local mesh density, then **fair** the patch by minimizing curvature energy (membrane or thin-plate spline) so the patch blends smoothly. Optionally, normal-aware methods like **Normal Field Diffusion** replace plain fairing with PDE-based normal interpolation followed by displacement. The output should be **watertight**, 2-manifold, and visually continuous with the surrounding mesh.

Q17. How to detect a hole? What is a boundary loop and how to extract?

Hole detection on a triangle mesh exploits the **half-edge** (or directed edge) data structure. An edge is a **boundary edge** (border edge) if it is incident to exactly one face — equivalently, its half-edge has no twin/opposite. A **hole** is a maximal closed cycle of boundary edges, called a **boundary loop**. Extraction algorithm: (1) iterate all half-edges and collect those whose twin is null into a set B; (2) pick any edge in B, follow next-boundary pointers (the next half-edge around the same vertex with no twin) to walk along the loop until returning to the start; (3) mark the loop as visited and remove its edges from B; (4) repeat until B is empty. The number of disjoint loops equals the number of holes. For a closed 2-manifold $|B| = 0$. Edge cases: **non-manifold edges** (incident to 3+ faces) require special handling; mesh with reversed face orientations may give incorrect loops, so one should first verify consistent orientation. Complexity is $O(E)$ — linear in edge count.

Q18. Liepa's hole filling algorithm step-by-step (3 stages).

Liepa 2003 is the canonical three-stage hole filling algorithm.

1. **Stage 1 — Minimum-weight triangulation:** given the boundary loop $v_0, \ldots, v_{n-1}$, fill it with $n-2$ triangles minimizing a weight functional (typically area + dihedral angle). Uses dynamic programming: let $W_{i,j}$ be the optimal weight for sub-polygon $v_i, \ldots, v_j$; recurrence $W_{i,j} = \min_{i<k<j}\left(W_{i,k} + W_{k,j} + w(\Delta v_i v_k v_j)\right)$ in $O(n^3)$ time.
2. **Stage 2 — Refinement:** subdivide patch triangles whose edge length exceeds the average length of their boundary neighbors; repeatedly split and edge-flip toward Delaunay until patch density matches surrounding mesh.
3. **Stage 3 — Fairing:** solve for new interior vertex positions minimizing a smoothness energy (membrane $\Delta x = 0$ or thin-plate $\Delta^2 x = 0$) with Dirichlet boundary conditions fixed at the original loop. This is a sparse linear system using the cotangent Laplacian.

Liepa is robust, parameter-free, and produces C^0/C^1-smooth patches, making it the default in tools like MeshLab.

Q19. Minimum-weight triangulation in Liepa? What weights?

The **minimum-weight triangulation** (MWT) finds the triangulation of the polygon (boundary loop) that minimizes a sum of per-triangle weights. Liepa uses a 2-component weight: $w(\Delta) = (\mathrm{Area}(\Delta), \theta_{\max}(\Delta))$, compared lexicographically — minimize maximum dihedral angle first, ties broken by minimum total area. Why dihedral

dùng table 256-entry gốc (không cần xoay từng cube), nhưng designer chỉ phải liệt kê 15 mẫu. Tinh tế: table gốc có **ambiguous case** mà tam giác hóa có thể sinh lỗ giữa cube kề; biến thể sau (Asymptotic Decider, Marching Cubes 33) giải quyết bằng kiểm tra topology bổ sung.

Câu 14. Mô tả greedy projection triangulation (PCL).

Greedy Projection Triangulation (Marton-Rusu-Beetz 2009, trong PCL) tái tạo mesh từ point cloud có normal bằng cách mọc tam giác từng bước trong tangent plane cục bộ.

1. **Setup:** dựng kd-tree trên P; mỗi p_i có normal $\mathbf{n}_i$.
2. Chọn seed point và chiếu k-nearest neighbor lên tangent plane.
3. Sắp neighbor theo polar angle quanh seed; greedy tạo tam giác từ seed đến các neighbor liên tiếp hợp lệ.
4. Neighbor **hợp lệ** nếu: khoảng cách $\leq r_{\max}$, góc normal $\leq \theta_{\max}$, tam giác không vi phạm visibility, và mọi tam giác candidate tuân thủ góc tối đa.
5. Duy trì **front** các fringe edge; mở rộng front bằng cách thêm tam giác hợp lệ đến khi không mở rộng được nữa.

Tham số: search radius $r_{\max}$, max neighbor k, μ (mật độ tương đối), max surface angle, min/max triangle angle. Thuật toán nhanh ($O(N \log N)$), cục bộ, parallel được, nhưng không khôi phục được vùng topology phức tạp và cần normal có hướng nhất quán.

Câu 15. Sliver triangle là gì? Tại sao tránh?

Sliver triangle là tam giác có góc nhỏ nhất rất nhỏ (thường $< 5^\circ$) — gần suy biến, dài và mảnh. Trong meshing 3D, sliver còn gồm tetrahedron có dihedral angle nhỏ. Sliver có vấn đề vì: (i) **condition number** ma trận stiffness FEM tăng như $1/\sin^2\theta_{\min}$, nên sliver làm iterative solver hội tụ chậm hoặc phân kỳ; (ii) ước lượng gradient và Laplacian trên mesh có sai số $O(1/\sin\theta_{\min})$, suy giảm physics downstream; (iii) robustness số của geometric predicate hỏng gần suy biến; (iv) nội suy **shading** sinh dải và Mach-band artifact; (v) **ước lượng normal** bằng cross product không tin cậy vì magnitude triệt tiêu; (vi) **collision detection** và ray-casting bị self-intersection false positive. Biện pháp: enforce Delaunay khi xây; **edge flip**, **edge collapse**, **vertex reposition** khi cleanup; dùng **CVT** cho isotropic remeshing; trong 3D dùng **sliver exudation** (Cheng-Dey) — perturb vertex weight để loại sliver khỏi weighted Delaunay tetrahedralization.

Câu 16. Mesh nhiều lỗ — làm sao tái tạo? Hướng tiếp cận chung.

Pipeline chung sửa mesh có lỗ gồm bốn pha. (1) **Phát hiện lỗ:** duyệt mọi cạnh, tìm boundary edge (kề đúng một face), gom thành chu trình đóng bằng half-edge traversal. (2) **Phân loại lỗ:** tính kích thước (perimeter, độ phẳng, genus), phân biệt lỗ nhỏ (xử lý đơn giản) vs lỗ lớn (cần phương pháp geometry-aware); phát hiện island và lỗ nhiều thành phần. (3) **Sinh patch:** cho mỗi loop, sinh tam giác hóa ban đầu — phổ biến là minimum-weight triangulation (Liepa), advancing front, hay ear clipping projection-based. (4) **Refine chất lượng:** flip cạnh hướng tới Delaunay, chia nhỏ tam giác quá lớn để khớp mật độ cục bộ, rồi **fair** patch bằng cách cực tiểu hóa năng lượng curvature (membrane hay thin-plate spline) để patch hòa mượt. Tùy chọn: phương pháp normal-aware như **Normal Field Diffusion** thay fairing thường bằng nội suy normal qua PDE rồi displacement. Output phải **watertight**, 2-manifold, và liên tục thị giác với mesh xung quanh.

Câu 17. Phát hiện lỗ thế nào? Boundary loop là gì và trích ra sao?

Phát hiện lỗ trên triangle mesh khai thác cấu trúc **half-edge** (hoặc directed edge). Cạnh là **boundary edge** (border edge) nếu kề đúng một face — tương đương half-edge không có twin/opposite. **Lỗ** là chu trình đóng cực đại của boundary edge, gọi là **boundary loop**. Thuật toán trích: (1) duyệt mọi half-edge, gom những cái twin null vào tập B; (2) chọn edge bất kỳ trong B, đi theo next-boundary pointer (half-edge kế tiếp quanh cùng vertex không có twin) men loop đến khi quay lại điểm xuất phát; (3) đánh dấu loop đã thăm và xóa edge khỏi B; (4) lặp đến khi B rỗng. Số loop rời nhau = số lỗ. Mesh đóng 2-manifold: $|B| = 0$. Trường hợp đặc biệt: **non-manifold edge** (kề ≥ 3 face) cần xử lý riêng; mesh có face hướng ngược có thể cho loop sai, nên xác minh hướng nhất quán trước. Độ phức tạp $O(E)$ — tuyến tính theo số cạnh.

Câu 18. Thuật toán Liepa từng bước (3 giai đoạn).

Liepa 2003 là thuật toán vá lỗ ba giai đoạn kinh điển.

1. **GĐ 1 — Minimum-weight triangulation:** cho boundary loop $v_0, \ldots, v_{n-1}$, lấp bằng $n-2$ tam giác tối thiểu hóa weight functional (thường là area + dihedral angle). Dùng DP: $W_{i,j}$ là weight tối ưu cho sub-polygon $v_i, \ldots, v_j$; truy hồi $W_{i,j} = \min_{i<k<j}\left(W_{i,k} + W_{k,j} + w(\Delta v_i v_k v_j)\right)$ trong $O(n^3)$.
2. **GĐ 2 — Refinement:** chia nhỏ tam giác patch có cạnh vượt trung bình cạnh boundary kế cận; lặp split và edge-flip hướng tới Delaunay đến khi mật độ patch khớp mesh xung quanh.
3. **GĐ 3 — Fairing:** giải vị trí vertex nội bộ mới tối thiểu hóa năng lượng mượt (membrane $\Delta x = 0$ hoặc thin-plate $\Delta^2 x = 0$) với điều kiện biên Dirichlet cố định tại loop gốc. Hệ tuyến tính sparse dùng cotangent Laplacian.

Liepa robust, không tham số, sinh patch C^0/C^1-mượt, là default trong MeshLab.

Câu 19. Minimum-weight triangulation trong Liepa? Weight gì?

Minimum-weight triangulation (MWT) tìm triangulation của đa giác (boundary loop) tối thiểu hóa tổng weight per-triangle. Liepa dùng weight 2-thành-phần: $w(\Delta) = (\mathrm{Area}(\Delta), \theta_{\max}(\Delta))$, so sánh lexicographic — tối thiểu max dihedral angle trước, hòa thì lấy area nhỏ nhất. Tại sao dihedral? Trong 3D các tam giác candidate trải trên

angle? In 3D the candidate triangles span a non-planar boundary, so penalizing high dihedral angles ensures the patch bends smoothly across triangle boundaries rather than creasing. Why area? Among smooth triangulations, smaller total area approximates the minimal surface (Plateau problem). The DP recurrence is:

$$W_{i,j} = \min_{i<k<j} \Big(W_{i,k} + W_{k,j} + w(\triangle v_i v_k v_j) \Big)$$

with base case $W_{i,i+1} = 0$ and $W_{i,i+2} = w(\triangle v_i v_{i+1} v_{i+2})$. Time complexity $O(n^3)$, space $O(n^2)$. Backtracking the optimal split indices recovers the triangulation. The dihedral term requires knowing neighboring triangles, so the actual algorithm is slightly more involved (caching adjacent face normals), but the structure remains DP over polygon intervals.

Q20. What is mesh fairing? Which energies are minimized?

Mesh fairing is the process of smoothing a region of a mesh by minimizing a geometric energy functional, typically applied to hole patches or noisy regions. The goal is to produce a surface that is as "visually pleasing" as possible while respecting boundary constraints. Two main energies:

- **Membrane energy** (Dirichlet): $E_m = \int \|\nabla x\|^2 \, dA$, minimized by solutions of Laplace's equation $\Delta x = 0$. Produces C^0 surfaces — minimal-area patches with sharp curvature transitions at the boundary.
- **Thin-plate energy** (bending): $E_b = \int \|\Delta x\|^2 \, dA$, minimized by bi-harmonic solutions $\Delta^2 x = 0$. Produces C^1 surfaces with smooth curvature blending — visually preferred but requires twice the boundary information (positions and normals).

Discretization uses the **cotangent Laplacian** $L_{ij} = \frac{1}{2}(\cot \alpha_{ij} + \cot \beta_{ij})$, leading to a sparse symmetric linear system $Lx = b$ (membrane) or $L^2 x = b$ (thin-plate). Solve with Cholesky factorization. Drawback: fairing is purely **intrinsic** and does not propagate **external curvature trends**, which is why methods like Normal Field Diffusion are preferred for highly curved regions.

Q21. Compare Liepa vs RBF vs Poisson hole filling.

Three competing paradigms for hole filling: **Liepa** (combinatorial), **RBF** (radial basis function interpolation), and **Poisson** (PDE).

- **Liepa:** purely intrinsic, uses only boundary loop. Fast, robust, parameter-free. Limitation: produces flat/membrane-like patches, ignores curvature outside the loop. Best for small holes in flat regions.
- **RBF (Carr et al. 2001):** fit a global implicit function $f(\mathbf{x}) = \sum_i w_i \phi(\|\mathbf{x} - c_i\|)$ to off-surface points (boundary $\pm$ normal), then extract surface via Marching Cubes. Smooth C^∞ patch, naturally extends curvature. Limitation: dense linear system $O(N^3)$ solve unless using fast multipole; sensitive to noise; non-local — modifying one point affects entire mesh.
- **Poisson:** reformulates global reconstruction; if applied locally on a hole region with normals, fills smoothly. Handles large holes well. Limitation: oversmooths sharp features; needs careful boundary integration; modifies mesh outside the hole.

Rule of thumb: small flat hole $\rightarrow$ Liepa; medium curved hole, accurate normals $\rightarrow$ RBF; large hole on noisy scan $\rightarrow$ Poisson. Hybrid methods (Liepa skeleton + curvature-aware fairing like NFD) combine strengths.

Q22. Describe the Normal Field Diffusion (NFD) hole filling method.

Normal Field Diffusion (the user's MeshLab plugin project) is a curvature-aware hole filling method that replaces plain position fairing with PDE-based normal interpolation. The seven-stage pipeline:

1. **Boundary extraction:** detect hole loops via half-edge boundary traversal.
2. **Initial triangulation:** fill each loop with ear clipping using a **best-ear** (max-min interior angle) criterion to avoid the fan-from-pivot artifact.
3. **Delaunay edge flipping:** improve patch quality by Lawson flips toward the projected Delaunay condition.
4. **Refinement:** subdivide oversized triangles to match local mesh density (target edge length from boundary average).
5. **Normal field diffusion:** treat the three components of the normal as scalar fields on the patch; solve the heat equation $\partial \mathbf{n}/\partial t = \Delta \mathbf{n}$ with Dirichlet boundary (boundary normals fixed) using **implicit Euler** with cotangent Laplacian and SimplicialLDLT factorization.
6. **Spherical-cap displacement:** push interior vertices along the diffused normal field with amplitude derived from intrinsic geometry (mean radius r, mean angle θ); weighting profile $w(t) = \sqrt{2t - t^2}$ for C^1 continuity at the boundary.
7. **Taubin smoothing:** non-shrinking low-pass filter to remove high-frequency noise from interpolation.

Quantitative validation via symmetric Hausdorff distance achieved 0.12% of bounding-box diagonal on Stanford Bunny.

Q23. Difference between hole filling and mesh repair?

Hole filling is a specific subtask within the broader category of **mesh repair**. Mesh repair encompasses all operations to convert a defective mesh into a clean, watertight, 2-manifold representation. Defects include: **holes** (open boundaries), **non-manifold edges** (incident to 3+ faces), **non-manifold vertices** (umbrella structure violated), **self-intersections** (faces that geometrically cross), **inverted normals** (inconsistent orientation), **duplicate vertices** (cracks), **degenerate triangles** (zero area), and **floating components** (disconnected debris). A typical mesh repair pipeline first **merges duplicate vertices**, **removes degenerate faces**, **splits non-manifold edges**, **resolves self-intersections**, **fixes orientation**, and **fills holes** as a final step. So hole filling assumes the mesh is otherwise topologically clean — applying it on a non-manifold mesh produces undefined behavior. Tools like MeshLab, Meshmixer, and Netfabb provide separate filters for each repair step. Confusion arises because some commercial tools market "mesh repair" but actually perform only hole filling.

Q24. How to handle complex hole topology (multiple components, islands)?

Complex hole topologies arise when scanning concave or perforated objects. Three sub-cases: (i) **Multiple disjoint holes:** simple — extract each boundary loop independently and fill them separately. The half-edge traversal

boundary không phẳng, phạt dihedral cao đảm bảo patch uốn mượt qua biên tam giác chứ không gấp khúc. Tại sao area? Trong các triangulation mượt, tổng area nhỏ xấp xỉ minimal surface (bài toán Plateau). Truy hồi DP:

$$W_{i,j} = \min_{i<k<j} \Big(W_{i,k} + W_{k,j} + w(\triangle v_i v_k v_j) \Big)$$

với base $W_{i,i+1} = 0$ và $W_{i,i+2} = w(\triangle v_i v_{i+1} v_{i+2})$. Độ phức tạp $O(n^3)$, không gian $O(n^2)$. Backtrack split index tối ưu cho triangulation. Term dihedral cần biết tam giác kề, nên thuật toán thực hơi phức tạp hơn (cache normal face kề), nhưng cấu trúc vẫn là DP trên interval đa giác.

Câu 20. Mesh fairing là gì? Năng lượng nào được tối thiểu hóa?

Mesh fairing là làm mượt một vùng mesh bằng cách cực tiểu hóa năng lượng hình học, thường áp dụng cho patch lỗ hoặc vùng noise. Mục tiêu: bề mặt "đẹp mắt" nhất có thể trong khi tôn trọng ràng buộc biên. Hai năng lượng chính:

- **Membrane energy** (Dirichlet): $E_m = \int \|\nabla x\|^2 \, dA$, cực tiểu hóa bởi nghiệm Laplace $\Delta x = 0$. Sinh bề mặt C^0 — patch minimal-area với chuyển curvature sắc tại biên.
- **Thin-plate energy** (bending): $E_b = \int \|\Delta x\|^2 \, dA$, cực tiểu hóa bởi nghiệm bi-harmonic $\Delta^2 x = 0$. Sinh bề mặt C^1 với pha trộn curvature mượt — đẹp hơn nhưng cần gấp đôi thông tin biên (vị trí và normal).

Rời rạc hóa dùng **cotangent Laplacian** $L_{ij} = \frac{1}{2}(\cot \alpha_{ij} + \cot \beta_{ij})$, dẫn tới hệ tuyến tính sparse đối xứng $Lx = b$ (membrane) hoặc $L^2 x = b$ (thin-plate). Giải bằng Cholesky factorization. Hạn chế: fairing thuần **intrinsic** và không lan tỏa **xu hướng curvature ngoài**, nên các phương pháp như Normal Field Diffusion ưu tiên cho vùng cong mạnh.

Câu 21. So sánh Liepa vs RBF vs Poisson vá lỗ.

Ba mô hình vá lỗ cạnh tranh: **Liepa** (tổ hợp), **RBF** (nội suy radial basis function), **Poisson** (PDE).

- **Liepa:** thuần intrinsic, chỉ dùng boundary loop. Nhanh, robust, không tham số. Hạn chế: patch phẳng/membrane, bỏ qua curvature ngoài loop. Tốt cho lỗ nhỏ vùng phẳng.
- **RBF (Carr 2001):** fit hàm implicit toàn cục $f(\mathbf{x}) = \sum_i w_i \phi(\|\mathbf{x} - c_i\|)$ qua off-surface point (boundary $\pm$ normal), rồi trích bề mặt qua Marching Cubes. Patch C^∞ mượt, tự nhiên mở rộng curvature. Hạn chế: hệ dense $O(N^3)$ trừ khi dùng fast multipole; nhạy noise; non-local — sửa 1 điểm ảnh hưởng toàn mesh.
- **Poisson:** công thức hóa reconstruction toàn cục; áp dụng cục bộ trên vùng lỗ có normal, lấp mượt. Xử lý lỗ lớn tốt. Hạn chế: oversmooth feature sắc; cần xử lý biên cẩn thận; sửa cả mesh ngoài lỗ.

Quy tắc: lỗ phẳng nhỏ $\rightarrow$ Liepa; lỗ cong vừa, normal chính xác $\rightarrow$ RBF; lỗ lớn scan noisy $\rightarrow$ Poisson. Phương pháp lai (Liepa skeleton + curvature-aware fairing như NFD) kết hợp ưu điểm.

Câu 22. Mô tả phương pháp Normal Field Diffusion (NFD).

Normal Field Diffusion (project MeshLab plugin của user) là phương pháp vá lỗ curvature-aware thay fairing vị trí thường bằng nội suy normal dựa PDE. Pipeline 7 giai đoạn:

1. **Trích boundary:** phát hiện loop lỗ qua half-edge boundary traversal.
2. **Tam giác hóa ban đầu:** lấp mỗi loop bằng ear clipping với tiêu chí **best-ear** (max-min interior angle) tránh artifact fan-from-pivot.
3. **Delaunay edge flip:** cải thiện chất lượng patch bằng Lawson flip hướng tới điều kiện Delaunay chiếu.
4. **Refinement:** chia nhỏ tam giác lớn để khớp mật độ mesh (target edge length từ trung bình boundary).
5. **Normal field diffusion:** coi ba thành phần normal là trường vô hướng trên patch; giải heat equation $\partial \mathbf{n}/\partial t = \Delta \mathbf{n}$ với Dirichlet boundary (normal biên cố định) bằng **implicit Euler** với cotangent Laplacian và SimplicialLDLT factorization.
6. **Spherical-cap displacement:** đẩy vertex nội bộ dọc trường normal khuếch tán với biên độ suy từ intrinsic geometry (mean radius r, mean angle θ); profile weight $w(t) = \sqrt{2t - t^2}$ đảm bảo C^1 tại biên.
7. **Taubin smoothing:** bộ lọc low-pass không co rút để khử noise cao tần từ nội suy.

Đánh giá định lượng bằng symmetric Hausdorff distance đạt 0.12% bounding-box diagonal trên Stanford Bunny.

Câu 23. Khác nhau giữa hole filling và mesh repair?

Hole filling là một subtask cụ thể trong phạm trù rộng hơn **mesh repair**. Mesh repair bao gồm mọi thao tác chuyển mesh khiếm khuyết thành biểu diễn sạch, watertight, 2-manifold. Khiếm khuyết gồm: **lỗ** (open boundary), **non-manifold edge** (kề ≥ 3 face), **non-manifold vertex** (vi phạm cấu trúc umbrella), **self-intersection** (face cắt nhau hình học), **inverted normal** (orientation không nhất quán), **duplicate vertex** (crack), **degenerate triangle** (area = 0), và **floating component** (mảnh rời rạc). Pipeline mesh repair điển hình: **merge duplicate vertex** trước, **xóa degenerate face**, **tách non-manifold edge**, **giải self-intersection**, **sửa orientation**, rồi **vá lỗ** bước cuối. Vì vậy hole filling giả định mesh đã sạch topology — áp dụng trên mesh non-manifold cho hành vi không xác định. Công cụ như MeshLab, Meshmixer, Netfabb cung cấp filter riêng cho từng bước. Nhầm lẫn nảy sinh vì một số tool thương mại quảng bá "mesh repair" nhưng thực ra chỉ làm hole filling.

Câu 24. Xử lý topology lỗ phức tạp (nhiều thành phần, island)?

Topology lỗ phức tạp nảy sinh khi scan vật lõm hoặc thủng. Ba sub-case: (i) **Nhiều lỗ rời:** đơn giản — trích từng boundary loop độc lập và vá riêng. Half-edge traversal tự nhiên cho một loop mỗi thành phần. (ii) **Lỗ có island**

naturally produces one loop per component. (ii) **Holes with islands** (a hole containing a separate boundary loop, like a donut hole inside a larger hole): the outer loop and each inner loop must be filled *jointly*, otherwise topology is wrong. Standard Liepa cannot handle this; one must use **multi-loop triangulation**, e.g., constrained Delaunay on the projected planar layout with all loops as constraints, or a **tube/bridge** approach connecting loops first then filling. (iii) **Genus-changing repairs:** sometimes topology should change (close a handle as a hole, or open a closed region). This requires user input — automatic methods cannot guess intent. Practical workflow: detect all loops; classify by enclosure tests; build a hierarchy; for each **outer-with-islands** hole, use planar CDT or volumetric approaches like Poisson restricted to the hole's bounding region. Robust libraries: CGAL polygon_mesh_processing::triangulate_hole accepts island input.

Q25. Quality criteria for a hole-filled patch?

Patch quality is evaluated along five axes:

1. **Topology:** the patch must close the loop and yield a 2-manifold (no non-manifold edges, no self-intersections within or with the surrounding mesh).
2. C^0 **continuity (positional):** patch vertices on the boundary coincide exactly with the original loop. Trivially satisfied by construction.
3. C^1 **continuity (tangent):** normals across the patch-host boundary match — no visible crease. Measured by dihedral angle deviation between adjacent triangles across the seam, ideally $< 5°$.
4. C^2 **continuity (curvature):** principal curvatures vary smoothly across the seam — no curvature discontinuity. Hard to enforce; achieved approximately by NFD or thin-plate fairing.
5. **Triangle quality:** aspect ratios > 0.3, minimum angles $> 20°$, edge length matching local mesh density.

Quantitative metrics: **symmetric Hausdorff distance** from patch to ground-truth surface (when known); **mean curvature deviation** along the boundary; **visual inspection** via reflection-line shading which exposes C^1 defects strongly. The NFD project reports Hausdorff 0.12% on bunny, illustrating quantitative reporting standard.

Q26. How is hole filling used in 3D scanning workflows?

3D scanning produces incomplete meshes due to **self-occlusion**, sensor dropout, and surface properties (specular, transparent, dark). A typical scanning pipeline is **capture → register → fuse → repair → deliver**. Hole filling is essential at the repair stage: structured-light scans (Artec, EinScan) leave holes wherever the projected pattern was occluded; LiDAR misses surfaces normal-aligned with the beam; photogrammetry (RealityCapture, Metashape) leaves holes in textureless regions. Operators use hole filling to: (i) produce a watertight mesh required by 3D-printing slicers; (ii) generate clean models for VR/AR display; (iii) enable volumetric measurements (mass, area) which require closed surfaces; (iv) prepare reference geometry for inspection. Best practice: distinguish **intentional** openings (e.g., the inside of a vase) from **scan defects** — do not auto-fill openings smaller than scanner accuracy thresholds. Tools like Geomagic Wrap, RevoScan, and MeshLab provide "small hole" auto-fill plus manual selection for large holes. Curvature-aware methods (NFD, Poisson) are critical for cultural heritage and medical scans where shape fidelity matters.

Q27. How is triangulation used in GIS / terrain modeling?

In Geographic Information Systems, terrains are commonly represented as a **Triangulated Irregular Network** (TIN), a triangulated 2.5D surface where elevation z is a function of (x, y). TINs are constructed by 2D Delaunay triangulation of survey points (LiDAR, photogrammetric, or manual). Compared to regular grids (DEMs), TINs adaptively concentrate vertices in high-relief areas and use few vertices in flat areas, achieving better compression for the same accuracy. **Constrained Delaunay** is essential because GIS data has natural **breaklines** — rivers, ridges, road centerlines, building footprints — that must appear as edges in the triangulation. Applications include: (i) hydrological flow modeling (D8/D-Infinity algorithms compute flow direction per triangle); (ii) viewshed analysis (visibility from a tower); (iii) cut-and-fill calculations for civil engineering; (iv) contour generation by intersecting triangles with horizontal planes; (v) 3D city models (CityGML LOD1-2). Robust triangulation matters because GIS coordinates have huge dynamic range (UTM meters vs centimeter accuracy); exact predicates are mandatory.

Q28. How is geometric modeling used in cultural heritage preservation?

Digital cultural heritage preservation creates 3D archives of monuments, artifacts, and historic sites that may be lost to weathering, conflict, or accident — exemplified by the Notre-Dame of Paris fire (2019) where pre-fire scan data informed reconstruction. Pipelines combine **photogrammetry** (Agisoft Metashape, RealityCapture), **laser scanning** (FARO, Leica), and **structured light** (Artec). Triangulation reconstructs surface meshes at sub-millimeter accuracy. Hole filling is critical because: artifacts have hollow interiors (vases, statues with cavities), bronze surfaces are specular, ancient stone has cracks. Curvature-aware methods (NFD, Poisson) are preferred because cultural items have rich detail that must blend naturally — flat patches look obviously synthetic. Standards include the **CIDOC CRM** ontology for metadata and **IIIF 3D** for delivery. Notable projects: CyArk's monument database, the Smithsonian's open 3D archive, ScanPyramids for the Great Pyramid. The course's My Son temple practical falls in this category — Vietnam's UNESCO heritage with significant hole-filling needs due to weathering of the Cham brick towers.

Q29. Why is robust triangulation important for medical 3D printing?

Medical 3D printing — patient-specific anatomical models, surgical guides, prosthetics, implants — requires **watertight, manifold meshes** or the slicer fails. Source data comes from **CT/MRI segmentation**: voxel volumes are converted to surfaces via Marching Cubes, producing meshes with stair-step artifacts, isolated components, non-manifold edges where thin tissue meets, and small holes from low-contrast regions. Robust triangulation matters because: (i) printer slicers (Cura, PrusaSlicer, GE Healthcare) reject non-manifold input or silently produce wrong toolpaths; (ii) FDA-cleared workflows for surgical guides require validated geometry; (iii) implants must satisfy dimensional tolerances within ± 0.1 mm; (iv) finite-element simulation for stress analysis demands quality tetrahedral meshes derived from clean surface meshes. Hole filling is sensitive: oversmoothing in cardiac models can erase important features (chordae tendineae); under-filling leaves printable but inaccurate models. Curvature-aware methods (NFD, MAT-based) are preferred. Vendors like Materialise Mimics integrate sophisticated repair pipelines

(lỗ chứa boundary loop khác bên trong, như lỗ donut trong lỗ lớn): outer loop và mỗi inner loop phải vá *đồng thời*, nếu không topology sai. Liepa chuẩn không xử lý được; phải dùng **multi-loop triangulation**, vd: constrained Delaunay trên layout phẳng chiếu với mọi loop là constraint, hoặc cách **tube/bridge** nối loop trước rồi vá. (iii) **Sửa đổi genus:** đôi khi topology cần đổi (đóng handle như lỗ, hoặc mở vùng đóng). Cần user input — phương pháp tự động không đoán được ý định. Workflow thực tế: phát hiện mọi loop; phân loại theo enclosure test; xây cây phân cấp; với mỗi lỗ **outer-with-islands**, dùng CDT phẳng hoặc Poisson hạn chế trong bounding region của lỗ. Thư viện robust: CGAL polygon_mesh_processing::triangulate_hole nhận island input.

Câu 25. Tiêu chí chất lượng patch vá?

Chất lượng patch đánh giá theo năm trục:

1. **Topology:** patch phải đóng loop và sinh 2-manifold (không non-manifold edge, không self-intersection trong patch hay với mesh xung quanh).
2. **Liên tục** C^0 **(vị trí):** vertex patch trên biên trùng chính xác với loop gốc. Tự thỏa do cách xây.
3. **Liên tục** C^1 **(tangent):** normal qua biên patch-host khớp — không gấp khúc thị giác. Đo bằng độ lệch dihedral angle giữa tam giác kề qua đường khâu, lý tưởng $< 5°$.
4. **Liên tục** C^2 **(curvature):** principal curvature biến thiên mượt qua đường khâu — không gián đoạn curvature. Khó ép buộc; xấp xỉ qua NFD hoặc thin-plate fairing.
5. **Chất lượng tam giác:** aspect ratio > 0.3, góc nhỏ nhất $> 20°$, edge length khớp mật độ mesh cục bộ.

Metric định lượng: **symmetric Hausdorff distance** từ patch tới ground-truth (nếu biết); **độ lệch curvature trung bình** dọc biên; **kiểm tra thị giác** bằng reflection-line shading — bộc lộ defect C^1 mạnh. Project NFD báo cáo Hausdorff 0.12% trên bunny, minh họa chuẩn báo cáo định lượng.

Câu 26. Hole filling dùng thế nào trong workflow scan 3D?

Scan 3D sinh mesh khuyết do **self-occlusion**, sensor dropout, và đặc tính bề mặt (specular, trong suốt, đen). Pipeline scan điển hình: **capture → register → fuse → repair → deliver**. Vá lỗ thiết yếu ở pha repair: structured-light scan (Artec, EinScan) để lại lỗ ở vùng pattern bị che; LiDAR sót bề mặt vuông góc tia; photogrammetry (RealityCapture, Metashape) để lại lỗ vùng không texture. Operator dùng vá lỗ để: (i) sinh mesh watertight cho slicer in 3D; (ii) sinh model sạch cho VR/AR; (iii) cho đo lường thể tích (khối lượng, diện tích) cần bề mặt đóng; (iv) chuẩn bị reference geometry cho inspection. Best practice: phân biệt mở **cố ý** (ví dụ trong lòng bình) với **scan defect** — không auto-fill mở nhỏ hơn ngưỡng độ chính xác scanner. Công cụ như Geomagic Wrap, RevoScan, MeshLab cung cấp auto-fill "lỗ nhỏ" cộng manual cho lỗ lớn. Phương pháp curvature-aware (NFD, Poisson) thiết yếu cho di sản văn hóa và scan y tế nơi shape fidelity quan trọng.

Câu 27. Tam giác hóa dùng thế nào trong GIS / mô hình địa hình?

Trong GIS, địa hình thường biểu diễn bằng **Triangulated Irregular Network** (TIN) — bề mặt 2.5D tam giác hóa, độ cao z là hàm của (x, y). TIN xây bằng Delaunay 2D trên survey point (LiDAR, photogrammetry, hoặc đo thủ công). So với grid đều (DEM), TIN tập trung vertex thích nghi ở vùng địa hình mạnh và dùng ít vertex ở vùng phẳng, nén tốt hơn cùng độ chính xác. **Constrained Delaunay** thiết yếu vì dữ liệu GIS có **breakline** tự nhiên — sông, sống núi, centerline đường, footprint nhà — phải xuất hiện là cạnh trong triangulation. Ứng dụng: (i) mô hình thủy văn (thuật toán D8/D-Infinity tính hướng dòng chảy mỗi tam giác); (ii) viewshed analysis (visibility từ tháp); (iii) tính cut-and-fill xây dựng dân dụng; (iv) sinh contour bằng giao tam giác với mặt phẳng ngang; (v) mô hình thành phố 3D (CityGML LOD1-2). Triangulation robust quan trọng vì tọa độ GIS có dynamic range lớn (UTM mét vs độ chính xác cm); exact predicate bắt buộc.

Câu 28. Mô hình hình học dùng thế nào trong bảo tồn di sản văn hóa?

Bảo tồn **di sản văn hóa số** tạo lưu trữ 3D di tích, hiện vật, địa điểm có thể mất do thời tiết, xung đột, tai nạn — ví dụ vụ cháy Notre-Dame Paris (2019) nơi dữ liệu scan trước cháy phục vụ tái tạo. Pipeline kết hợp **photogrammetry** (Agisoft Metashape, RealityCapture), **laser scanning** (FARO, Leica), và **structured light** (Artec). Tam giác hóa tái tạo mesh bề mặt ở độ chính xác sub-millimeter. Vá lỗ thiết yếu vì: hiện vật có lòng rỗng (bình, tượng có khoang), bề mặt đồng phản chiếu, đá cổ có nứt. Phương pháp curvature-aware (NFD, Poisson) ưu tiên vì hiện vật văn hóa có chi tiết phong phú phải hòa tự nhiên — patch phẳng nhìn rõ là nhân tạo. Chuẩn gồm ontology **CIDOC CRM** cho metadata và **IIIF 3D** cho delivery. Project nổi bật: CyArk monument database, Smithsonian open 3D archive, ScanPyramids. Bài thực hành My Sơn của môn thuộc loại này — di sản UNESCO Việt Nam với nhu cầu vá lỗ đáng kể do phong hóa các tháp Chăm.

Câu 29. Tại sao tam giác hóa robust quan trọng cho in 3D y tế?

In 3D y tế — model giải phẫu patient-specific, surgical guide, prosthetic, implant — đòi hỏi **mesh watertight, manifold** nếu không slicer hỏng. Dữ liệu nguồn từ **phân đoạn CT/MRI**: volume voxel chuyển thành bề mặt qua Marching Cubes, sinh mesh có stair-step artifact, isolated component, non-manifold edge nơi mô mỏng tiếp xúc, và lỗ nhỏ từ vùng low-contrast. Triangulation robust quan trọng vì: (i) slicer máy in (Cura, PrusaSlicer, GE Healthcare) từ chối input non-manifold hoặc lặng lẽ sinh toolpath sai; (ii) workflow FDA-cleared cho surgical guide đòi hỏi geometry validated; (iii) implant phải thỏa dung sai ± 0.1 mm; (iv) FEM simulation phân tích ứng suất cần tetrahedral mesh chất lượng từ surface mesh sạch. Vá lỗ nhạy: oversmoothing trong model tim có thể xóa feature quan trọng (chordae tendineae); under-fill cho model in được nhưng không chính xác. Phương pháp curvature-aware (NFD, MAT-based) ưu tiên. Hãng như Materialise Mimics tích hợp pipeline repair tinh vi tinh chỉnh cho workflow y tế.

specifically tuned for medical workflows.

Q30. How does triangulation affect downstream FEM simulation quality?
Finite Element Method (FEM) discretizes continuous PDEs (elasticity, heat, fluid flow) on a mesh. Triangulation quality directly governs solution accuracy and solver performance through three mechanisms: (1) **Discretization error** — convergence rates assume non-degenerate elements. Standard linear elements have error $O(h)$ where h is element diameter, but the constant blows up as $1/\sin\theta_{\min}$ for slivers, so a single bad element can dominate global error. (2) **Conditioning** — the FEM stiffness matrix has condition number $\kappa \sim h^{-2}/\sin^2\theta_{\min}$. Iterative solvers (CG, GMRES) require iteration count proportional to $\sqrt{\kappa}$, so slivers cause dramatic slowdowns. (3) **Locking** — isotropic deformation modes are spuriously stiff in poor meshes, requiring higher-order elements or special formulations. Best practices: use **Delaunay** or **centroidal Voronoi** meshing; enforce **minimum angle** $> 20°$ (2D) or **dihedral** $> 15°$ (3D); apply **adaptive refinement** guided by error estimators; **anisotropic meshing** for boundary layers. Tools like `TetGen`, `Gmsh`, `Netgen` provide quality-guaranteed mesh generation.

Câu 30. Tam giác hóa ảnh hưởng chất lượng simulation FEM thế nào?
Finite Element Method (FEM) rời rạc hóa PDE liên tục (đàn hồi, nhiệt, lưu chất) trên mesh. Chất lượng tam giác hóa quyết định trực tiếp độ chính xác nghiệm và hiệu năng solver qua ba cơ chế: (1) **Sai số rời rạc** — tốc độ hội tụ giả định element không suy biến. Element tuyến tính chuẩn có sai số $O(h)$ với h là đường kính element, nhưng hằng số bùng nổ như $1/\sin\theta_{\min}$ cho sliver, nên một element xấu có thể chi phối sai số toàn cục. (2) **Điều kiện số** — ma trận stiffness FEM có condition number $\kappa \sim h^{-2}/\sin^2\theta_{\min}$. Iterative solver (CG, GMRES) cần số lần lặp tỉ lệ $\sqrt{\kappa}$, nên sliver gây chậm dramatic. (3) **Locking** — mode biến dạng đẳng hướng cứng giả tạo trong mesh kém, đòi element bậc cao hoặc formulation đặc biệt. Best practice: dùng meshing **Delaunay** hoặc **centroidal Voronoi**; ép **góc nhỏ nhất** $> 20°$ (2D) hoặc **dihedral** $> 15°$ (3D); áp dụng **refinement thích nghi** hướng dẫn bởi error estimator; **anisotropic meshing** cho boundary layer. Công cụ như `TetGen`, `Gmsh`, `Netgen` cung cấp mesh generation đảm bảo chất lượng.

QA2b. Theoretical Q&A — Simplification & Denoising

Q1. What is mesh simplification? Why is it needed?
Mesh simplification is the process of reducing the number of vertices, edges, and faces in a triangle mesh while preserving its visual and geometric appearance as much as possible. Modern scanners and CAD tools easily produce meshes with millions of triangles, which are too heavy for real-time rendering, transmission, or physics simulation. Simplification creates lightweight models suitable for **LOD (level of detail)**, mobile/VR devices, and web streaming. It also removes redundant micro-detail that does not contribute to perception. Algorithms range from **vertex clustering**, **vertex decimation** (Schroeder), to **QEM edge collapse** (Garland & Heckbert). The goal is to minimize geometric error subject to a target triangle budget.

Q2. Explain Quadric Error Metric (QEM) edge collapse — write the math.
For each vertex v, define the **quadric** $Q = \sum_{p\in\text{planes}(v)} K_p$, where each plane $p: ax+by+cz+d=0$ contributes $K_p = \mathbf{p}\mathbf{p}^T$ with $\mathbf{p} = (a,b,c,d)^T$. The error of placing $v = (x,y,z,1)^T$ is the sum of squared distances to those planes: $\Delta(v) = v^T Q v$. For an **edge collapse** $(v_i, v_j) \to \bar{v}$, the merged quadric is $\bar{Q} = Q_i + Q_j$ and the error is $\Delta(\bar{v}) = \bar{v}^T \bar{Q} \bar{v}$. Edges are inserted into a heap keyed by this minimum error and collapsed in order. After each collapse, neighboring quadrics and heap keys are updated. QEM is fast, produces high-quality results, and is the standard for offline LOD generation.

Q3. How do you compute the quadric matrix Q for a vertex?
Steps: 1) For each face f adjacent to vertex v, compute its plane equation $ax+by+cz+d=0$ from the unit normal and one vertex. 2) Form the **fundamental quadric** $K_f = \mathbf{p}\mathbf{p}^T$, a symmetric 4×4 matrix. 3) Sum across the one-ring: $Q_v = \sum_{f\in N(v)} K_f$. The diagonal/upper triangle has 10 unique entries, so storage is compact. Optionally weight K_f by face **area** to give larger triangles more influence. For boundary edges, add a **virtual perpendicular plane** with high weight to prevent boundary shrinkage. The resulting Q encodes the squared-distance field to the local tangent planes.

Q4. How do you choose the optimal position v* after collapsing edge (v_i, v_j)?
Minimize $\Delta(v) = v^T \bar{Q} v$ where $\bar{Q} = Q_i + Q_j$. Setting $\partial\Delta/\partial v = 0$ gives the linear system: solve

$$\begin{pmatrix} q_{11} & q_{12} & q_{13} & q_{14} \\ q_{12} & q_{22} & q_{23} & q_{24} \\ q_{13} & q_{23} & q_{33} & q_{34} \\ 0 & 0 & 0 & 1 \end{pmatrix} v^* = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}$$

i.e. invert the upper-left 3×3 block. If the matrix is **singular** (degenerate planes), fall back to: (a) midpoint $(v_i+v_j)/2$, (b) endpoint with smaller Δ, or (c) line search along the edge. The optimal v^* generally lies off the edge, allowing better fit to curved surfaces.

Q5. What is progressive mesh? How does it enable streaming and geomorphing?
A **progressive mesh** (Hoppe 1996) stores a base coarse mesh M_0 plus an ordered sequence of **vertex splits** (the inverse of edge collapses): $M_0 \to M_1 \to \ldots \to M_n$. Each split adds one vertex and two faces. **Streaming:** the client renders M_0 immediately while remaining splits arrive over the network, refining detail progressively. **Geomorphing:** during LOD switching, vertex positions are linearly interpolated between M_k and M_{k+1} over a few frames, eliminating the visual "popping" artifact. Progressive meshes also support **selective refinement**, refining only regions near the camera. They are widely used in games and 3D web (glTF Draco extensions adopt similar ideas).

Q6. Compare vertex clustering vs vertex decimation vs QEM edge collapse.
Vertex clustering (Rossignac-Borrel): overlay a 3D grid, collapse all vertices in each cell to one representative. Very fast, out-of-core friendly, but loses topology and quality. **Vertex decimation** (Schroeder): iteratively delete a vertex if its local neighborhood is "flat" (distance-to-plane threshold) and re-triangulate the hole. Preserves topology, decent quality, but greedy and not error-bounded. **QEM edge collapse** (Garland-Heckbert): each collapse minimizes $v^T Q v$, giving provably good error. Slower than clustering, faster than decimation per quality level, and is the de-facto standard. Quality: QEM > Decimation > Clustering. Speed: Clustering > QEM > Decimation.

Q7. What metrics measure simplification quality?
Hausdorff distance $d_H(A,B) = \max\{\sup_{a\in A}\inf_{b\in B}\|a-b\|, \sup_{b\in B}\inf_{a\in A}\|a-b\|\}$ — worst-case symmetric distance, very sensitive to outliers. **RMS error** $\sqrt{\frac{1}{|S|}\sum_{p\in S} d(p,M')^2}$ over sampled points S on the original mesh — average-case, smoother metric. Tools like **Metro** and **MeshLab** compute both. Other measures: **normal deviation** (visual smoothness), **volume preservation**, and perceptual metrics (**MSDM2**). For texture-mapped models, also track UV/texture distortion. Report errors as a percentage of the bounding-box diagonal so they are

QA2b. Câu hỏi lý thuyết — Đơn giản hóa & Khử nhiễu

Câu 1. Đơn giản hóa lưới là gì? Vì sao cần?
Đơn giản hóa lưới (mesh simplification) là quá trình giảm số đỉnh, cạnh, mặt của lưới tam giác mà vẫn giữ hình dáng trực quan và hình học gần nguyên gốc nhất có thể. Máy quét và CAD hiện đại tạo ra lưới vài triệu tam giác — quá nặng để render real-time, truyền mạng, hay mô phỏng vật lý. Đơn giản hóa tạo mô hình nhẹ phù hợp với **LOD (mức độ chi tiết)**, thiết bị mobile/VR, và 3D web. Nó cũng loại bỏ chi tiết vi mô không đóng góp vào nhận thức. Thuật toán phổ biến: **vertex clustering**, **vertex decimation** (Schroeder), và **QEM edge collapse** (Garland-Heckbert). Mục tiêu: tối thiểu sai số hình học với ngân sách tam giác cho trước.

Câu 2. Quadric Error Metric (QEM) edge collapse — viết công thức.
Với mỗi đỉnh v, **quadric** là $Q = \sum_p K_p$ trong đó mỗi mặt phẳng $p: ax+by+cz+d=0$ đóng góp $K_p = \mathbf{p}\mathbf{p}^T$ với $\mathbf{p} = (a,b,c,d)^T$. Sai số khi đặt $v = (x,y,z,1)^T$ là tổng bình phương khoảng cách đến các mặt phẳng đó: $\Delta(v) = v^T Q v$. Khi **gộp cạnh** $(v_i, v_j) \to \bar{v}$: quadric mới $\bar{Q} = Q_i + Q_j$, sai số $\Delta(\bar{v}) = \bar{v}^T \bar{Q} \bar{v}$. Đẩy các cạnh vào heap khóa bằng sai số tối thiểu này và gộp theo thứ tự. Sau mỗi lần gộp, cập nhật quadric và khóa heap của hàng xóm. QEM nhanh, chất lượng cao, là chuẩn de-facto cho LOD offline.

Câu 3. Cách tính ma trận quadric Q cho một đỉnh?
Các bước: 1) Với mỗi mặt f kề đỉnh v, tính phương trình mặt phẳng $ax+by+cz+d=0$ từ pháp tuyến đơn vị và một đỉnh. 2) Lập **quadric cơ bản** $K_f = \mathbf{p}\mathbf{p}^T$, ma trận đối xứng 4×4. 3) Cộng dồn quanh **one-ring**: $Q_v = \sum_{f\in N(v)} K_f$. Vì đối xứng nên chỉ cần lưu 10 phần tử riêng biệt. Tùy chọn: nhân thêm **diện tích** mặt f để tam giác lớn ảnh hưởng nhiều hơn. Với cạnh biên, thêm **mặt phẳng vuông góc ảo** có trọng số lớn để chống co biên. Kết quả Q mã hóa trường khoảng cách bình phương đến các mặt phẳng tiếp tuyến cục bộ.

Câu 4. Chọn vị trí tối ưu v^* sau khi gộp cạnh (v_i, v_j) thế nào?
Tối thiểu hóa $\Delta(v) = v^T \bar{Q} v$ với $\bar{Q} = Q_i + Q_j$. Đạo hàm bằng 0 dẫn đến hệ tuyến tính:

$$\begin{pmatrix} q_{11} & q_{12} & q_{13} & q_{14} \\ q_{12} & q_{22} & q_{23} & q_{24} \\ q_{13} & q_{23} & q_{33} & q_{34} \\ 0 & 0 & 0 & 1 \end{pmatrix} v^* = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}$$

tức là nghịch đảo khối 3×3 trên-trái. Nếu ma trận **suy biến** (mặt phẳng thoái hóa), dự phòng bằng: (a) trung điểm $(v_i+v_j)/2$, (b) đỉnh có Δ nhỏ hơn, hoặc (c) line search dọc cạnh. v^* tối ưu thường nằm ngoài cạnh, cho phép khớp tốt hơn với mặt cong.

Câu 5. Progressive mesh là gì? Nó hỗ trợ streaming và geomorphing ra sao?
Progressive mesh (Hoppe 1996) lưu lưới thô gốc M_0 kèm chuỗi **vertex split** có thứ tự (đảo của edge collapse): $M_0 \to M_1 \to \ldots \to M_n$. Mỗi split thêm 1 đỉnh và 2 mặt. **Streaming:** client render M_0 ngay, các split còn lại đến qua mạng để tinh chỉnh dần. **Geomorphing:** khi chuyển LOD, nội suy tuyến tính vị trí đỉnh giữa M_k và M_{k+1} trong vài frame, tránh hiện tượng "popping". Progressive mesh còn hỗ trợ **selective refinement** — chỉ tinh chỉnh vùng gần camera. Được dùng rộng rãi trong games và 3D web (glTF Draco có ý tưởng tương tự).

Câu 6. So sánh vertex clustering, vertex decimation, QEM edge collapse.
Vertex clustering (Rossignac-Borrel): phủ lưới 3D, gộp tất cả đỉnh trong mỗi ô về một đại diện. Rất nhanh, chạy out-of-core được, nhưng mất topology và chất lượng kém. **Vertex decimation** (Schroeder): lặp lại xóa đỉnh nếu vùng quanh nó "phẳng" (ngưỡng khoảng cách-mặt phẳng) rồi tam giác hóa lại lỗ. Giữ topology, chất lượng khá, nhưng tham lam và không ràng buộc sai số. **QEM edge collapse** (Garland-Heckbert): mỗi lần gộp tối thiểu hóa $v^T Q v$, chứng minh được tốt. Chậm hơn clustering, nhanh hơn decimation cùng mức chất lượng. Chất lượng: QEM > Decimation > Clustering. Tốc độ: Clustering > QEM > Decimation.

Câu 7. Các metric đo chất lượng đơn giản hóa?
Hausdorff distance $d_H(A,B) = \max\{\sup_a \inf_b \|a-b\|, \sup_b \inf_a \|a-b\|\}$ — khoảng cách đối xứng tệ-nhất, nhạy với outlier. **RMS error** $\sqrt{\frac{1}{|S|}\sum d(p,M')^2}$ trên các điểm sampled S trên lưới gốc — trung bình, mượt hơn. Công cụ **Metro** và **MeshLab** tính cả hai. Các đại lượng khác: **độ lệch pháp tuyến** (mượt thị giác), **bảo toàn thể tích**, và metric tri giác (**MSDM2**). Với mô hình có texture, thêm **distortion UV/texture**. Báo cáo sai số dưới dạng phần trăm đường chéo bounding-box để bất biến tỷ lệ.

Câu 8. LOD là gì? Liên hệ với simplification?
LOD (level of detail) render các độ phân giải khác nhau tùy khoảng cách, kích thước trên màn hình, hoặc mức quan trọng. **LOD rời rạc**: lưu vài lưới đã tính trước (100%, 50%, 25%, 10%) và chuyển theo ngưỡng khoảng cách. **LOD liên tục** (CLOD): dùng progressive mesh, tinh chỉnh từng đỉnh. **HLOD** gộp nhiều object thành proxy đơn giản cho cụm xa (Unreal Engine 4/5). Thuật toán simplification (chủ yếu QEM) sinh chuỗi LOD offline. LOD giảm

scale-invariant.

Q8. What is LOD (level of detail)? How does it relate to simplification?

LOD renders different mesh resolutions depending on distance, screen size, or importance. **Discrete LOD** stores a few precomputed meshes (e.g., 100%, 50%, 25%, 10%) and switches based on distance thresholds. **Continuous LOD** (CLOD) uses a progressive mesh and refines vertex-by-vertex. **Hierarchical LOD** (HLOD) groups multiple objects into one simplified proxy for distant clusters (Unreal Engine 4/5). Simplification algorithms (mainly QEM) generate the LOD chain offline. LOD reduces **vertex shader** load, draw-call cost, and memory bandwidth. Modern engines (Unreal Nanite) virtualize geometry into a meshlet hierarchy, blending LOD techniques with cluster culling.

Q9. How is QEM used to create LOD models for VR/games?

Pipeline: 1) Author or scan the high-poly hero asset (e.g., 1M triangles). 2) Run QEM offline to generate LOD0 (full), LOD1 (50%), LOD2 (20%), LOD3 (5%), LOD4 (1%). 3) **Bake** normal maps from LOD0 onto LOD1+ to recover surface detail visually. 4) Constrain QEM to **preserve UV seams, sharp edges, and material boundaries** via boundary-plane penalties. 5) Export with progressive mesh metadata if streaming. 6) The engine selects LOD by projected screen size each frame. For **VR** (90+ Hz, two eyes), lower LODs are crucial, and **foveated** LOD pushes higher detail only at gaze. QEM's quality-per-budget makes it ideal for tight VR frame times.

Q10. What is feature-preserving simplification? Why is it important?

Feature-preserving simplification explicitly retains **sharp edges, corners, silhouettes, and textures** that carry semantic meaning. Without it, mechanical parts lose chamfers, characters lose facial creases, and CAD models lose dimensions. Techniques: 1) Detect features via **dihedral angle** threshold ($> 60° =$ sharp). 2) Add high-weight **constraint planes** along feature edges to the QEM matrix. 3) Forbid collapses that would destroy a feature edge. 4) Preserve **boundary** loops similarly. 5) Use **appearance-based** QEM that incorporates color/UV gradients (Garland-Heckbert 1998). Important because human perception is highly sensitive to silhouettes and creases; even small errors there are immediately visible.

Q11. What is noise in 3D data and where does it come from?

Noise is unwanted random/systematic deviation of measured points from the true surface. Sources: 1) **Sensor noise** — laser speckle, photon shot noise in ToF/structured-light cameras (Kinect, RealSense). 2) **Calibration error** — between cameras and laser. 3) **Registration error** — when fusing multiple scans (ICP residuals). 4) **Surface properties** — specular, transparent, dark surfaces give bad returns. 5) **Environmental** — ambient light, vibration, motion blur. 6) **Discretization** — voxel grid quantization. Noise is usually modeled as **Gaussian** along the surface normal, but real LiDAR also has **outliers** (multi-path, mixed pixels) and structured artifacts (banding).

Q12. Given a noisy point cloud, how do you make it more regular? List algorithms.

Pipeline: 1) **Outlier removal**: Statistical Outlier Removal (SOR), Radius Outlier Removal. 2) **Downsample**: Voxel-grid filter or Poisson-disk sampling for uniform density. 3) **Smooth in place**: **MLS** (Moving Least Squares) projects each point to a local polynomial fit. 4) **Resample/regularize**: **WLOP** (Weighted Locally Optimal Projection) redistributes points uniformly even on noisy input. 5) **Bilateral point-cloud filter** preserves features while smoothing. 6) **Normal estimation** via PCA, then orient consistently (minimum spanning tree). 7) Optionally **reconstruct mesh** (Poisson, BPA) and apply mesh denoising. 8) Modern: deep models like **PointCleanNet**, **Score-Based** denoising.

Q13. Explain Laplacian smoothing — formula and why it causes shrinkage.

For each vertex v_i, the **umbrella operator**/Laplacian is

$$L(v_i) = \frac{1}{|N(i)|} \sum_{j \in N(i)} (v_j - v_i)$$

The update is $v_i \leftarrow v_i + \lambda L(v_i)$ with $0 < \lambda < 1$. It diffuses geometry like the heat equation $\partial v/\partial t = \Delta v$. **Shrinkage:** L acts as a low-pass filter that attenuates all non-zero spatial frequencies, so after many iterations a closed surface converges toward its centroid (a sphere shrinks to a point). It also **erodes sharp features** because creases are high-frequency content. Use **Taubin** or pin boundary vertices to mitigate.

Q14. Compare uniform Laplacian vs cotangent Laplacian — which is better?

Uniform Laplacian weights each neighbor equally: $L_u(v_i) = \frac{1}{|N(i)|} \sum (v_j - v_i)$. Simple, fast, but depends on triangulation: irregular meshes drift tangentially (not just normally). **Cotangent Laplacian**: $L_c(v_i) = \frac{1}{2A_i} \sum_j (\cot \alpha_{ij} + \cot \beta_{ij})(v_j - v_i)$, where α, β are the angles opposite edge (i, j) and A_i the Voronoi area. This is the proper discretization of the **Laplace-Beltrami** operator and approximates **mean curvature normal**: $\Delta v = -2H\mathbf{n}$. Cotangent is better for smoothing, parameterization, and physics, because it is intrinsic and triangulation-independent (when angles stay non-obtuse). Drawback: cotangent weights can be **negative** in obtuse triangles, requiring clamping or remeshing.

Q15. Explain Taubin $\lambda|\mu$ smoothing — why two passes prevent shrinkage.

Taubin (1995): alternate two updates per iteration:

$$v_i^{(t+1/2)} = v_i^{(t)} + \lambda L(v_i^{(t)}), \quad v_i^{(t+1)} = v_i^{(t+1/2)} + \mu L(v_i^{(t+1/2)})$$

with $0 < \lambda < -\mu$, e.g. $\lambda = 0.33, \mu = -0.34$. The first (positive λ) shrinks; the second (negative μ) inflates. The combined transfer function is $f(k) = (1 - \lambda k)(1 - \mu k)$ in the spectral domain of the Laplacian. The **pass-band frequency** $k_{PB} = \frac{1}{\lambda} + \frac{1}{\mu}$ satisfies $f(k_{PB}) = 1$ — frequencies below k_{PB} are preserved, higher ones attenuated. Choosing $\lambda + \mu < 0$ gives a true low-pass filter without DC shrinkage. No matrix solve needed; it's still a local averaging.

Q16. Explain bilateral mesh denoising. How does it preserve features?

Bilateral mesh denoising (Fleishman 2003, Jones 2003) extends 2D bilateral filtering to meshes. For vertex v_i with normal n_i, predict its new position by displacing along n_i:

$$v_i' = v_i + n_i \cdot \frac{\sum_{j \in N(i)} w_c(\|v_j - v_i\|)\, w_s(\langle n_i, v_j - v_i \rangle)\, \langle n_i, v_j - v_i \rangle}{\sum w_c w_s}$$

tải **vertex shader**, draw call, băng thông bộ nhớ. Engine hiện đại (Unreal Nanite) ảo hóa hình học thành phân cấp meshlet, kết hợp LOD với cluster culling.

Câu 9. QEM tạo LOD cho VR/games như thế nào?

Pipeline: 1) Tạo asset hero high-poly (vd 1M tam giác). 2) Chạy QEM offline sinh LOD0 (đầy đủ), LOD1 (50%), LOD2 (20%), LOD3 (5%), LOD4 (1%). 3) **Bake** normal map từ LOD0 lên LOD1+ để giữ chi tiết bề mặt. 4) Ràng buộc QEM **giữ UV seam, cạnh sắc, biên material** bằng penalty mặt biên. 5) Export kèm metadata progressive nếu streaming. 6) Engine chọn LOD theo kích thước chiếu trên màn hình mỗi frame. Với **VR** (90+ Hz, 2 mắt), LOD thấp là then chốt; **foveated LOD** chỉ giữ chi tiết cao tại điểm nhìn. Tỷ lệ chất lượng/ngân sách của QEM rất phù hợp với frame time chặt của VR.

Câu 10. Simplification giữ đặc trưng (feature-preserving) là gì? Tại sao quan trọng?

Simplification giữ đặc trưng cố tình giữ lại **cạnh sắc, góc, đường bao silhouette, texture** — những thứ mang ý nghĩa ngữ nghĩa. Không có nó, chi tiết cơ khí mất chamfer, nhân vật mất nếp mặt, mô hình CAD mất kích thước. Kỹ thuật: 1) Phát hiện feature qua ngưỡng **góc nhị diện** ($> 60° =$ sắc). 2) Thêm **mặt phẳng ràng buộc** trọng số cao dọc cạnh feature vào ma trận QEM. 3) Cấm collapse phá hủy cạnh feature. 4) Bảo toàn vòng **biên** tương tự. 5) Dùng QEM **appearance-based** tích hợp gradient màu/UV (Garland-Heckbert 1998). Quan trọng vì thị giác con người rất nhạy với silhouette và crease — sai số nhỏ ở đó cũng dễ thấy.

Câu 11. Nhiễu trong dữ liệu 3D là gì, đến từ đâu?

Nhiễu là độ lệch ngẫu nhiên/hệ thống của điểm đo so với bề mặt thật. Nguồn: 1) **Sensor noise** — speckle laser, shot noise photon trong camera ToF/structured-light (Kinect, RealSense). 2) **Calibration** — sai lệch giữa camera và laser. 3) **Đăng ký** — residual khi ICP gộp nhiều scan. 4) **Tính chất bề mặt** — bóng, trong suốt, tối cho tín hiệu kém. 5) **Môi trường** — ánh sáng, rung, motion blur. 6) **Lượng tử hóa** voxel grid. Nhiễu thường mô hình hóa **Gauss** dọc pháp tuyến, nhưng LiDAR thực còn có **outlier** (multi-path, mixed pixel) và artifact có cấu trúc (banding).

Câu 12. Có một point cloud nhiễu, làm sao chuẩn hóa? Liệt kê thuật toán.

Pipeline: 1) **Loại outlier**: Statistical Outlier Removal (SOR), Radius Outlier Removal. 2) **Downsample**: Voxel-grid hoặc Poisson-disk để mật độ đều. 3) **Làm mượt tại chỗ**: **MLS** chiếu mỗi điểm lên đa thức cục bộ. 4) **Resample**: **WLOP** phân bố lại điểm đều ngay cả trên dữ liệu nhiễu. 5) **Bilateral** cho point cloud, giữ feature khi khử nhiễu. 6) **Ước lượng pháp tuyến** bằng PCA, sau đó định hướng nhất quán (cây bao trùm tối thiểu). 7) Tùy chọn **tái dựng lưới** (Poisson, BPA) rồi áp dụng denoising lưới. 8) Hiện đại: deep learning như **PointCleanNet**, **Score-Based** denoising.

Câu 13. Laplacian smoothing — công thức và vì sao gây co?

Với mỗi đỉnh v_i, toán tử **umbrella**/Laplacian là

$$L(v_i) = \frac{1}{|N(i)|} \sum_{j \in N(i)} (v_j - v_i)$$

Cập nhật: $v_i \leftarrow v_i + \lambda L(v_i)$ với $0 < \lambda < 1$. Nó khuếch tán hình học như phương trình nhiệt $\partial v/\partial t = \Delta v$. **Co (shrinkage):** L là bộ lọc thông thấp triệt tiêu mọi tần số không gian khác 0, nên sau nhiều lần lặp, mặt kín hội tụ về tâm khối lượng (hình cầu co thành điểm). Nó cũng **xói mòn feature** vì crease là tần số cao. Khắc phục: dùng **Taubin** hoặc ghim đỉnh biên.

Câu 14. Uniform Laplacian vs cotangent Laplacian — cái nào tốt hơn?

Uniform: trọng số bằng nhau $L_u(v_i) = \frac{1}{|N(i)|} \sum (v_j - v_i)$. Đơn giản, nhanh, nhưng phụ thuộc tam giác hóa: lưới không đều bị trôi tiếp tuyến (không chỉ pháp tuyến). **Cotangent**: $L_c(v_i) = \frac{1}{2A_i} \sum_j (\cot \alpha_{ij} + \cot \beta_{ij})(v_j - v_i)$, α, β là góc đối cạnh (i, j), A_i là diện tích Voronoi. Đây là rời rạc hóa đúng của **Laplace-Beltrami**, xấp xỉ **vector pháp tuyến nhân độ cong trung bình**: $\Delta v = -2H\mathbf{n}$. Cotangent tốt hơn cho làm mượt, parameterization, vật lý — vì nội tại, không phụ thuộc tam giác hóa (khi góc không tù). Nhược: trọng số cotangent có thể **âm** ở tam giác tù, cần clamp hoặc remesh.

Câu 15. Taubin $\lambda|\mu$ — vì sao 2 pass chống co? Điều kiện pass-band.

Taubin (1995): mỗi vòng lặp xen kẽ 2 cập nhật:

$$v^{(t+1/2)} = v^{(t)} + \lambda L(v^{(t)}), \quad v^{(t+1)} = v^{(t+1/2)} + \mu L(v^{(t+1/2)})$$

với $0 < \lambda < -\mu$, vd $\lambda = 0.33, \mu = -0.34$. Pass đầu ($\lambda$ dương) co; pass sau (μ âm) phồng. Hàm truyền tổng hợp trong miền phổ Laplacian là $f(k) = (1 - \lambda k)(1 - \mu k)$. **Tần số pass-band** $k_{PB} = \frac{1}{\lambda} + \frac{1}{\mu}$ thỏa $f(k_{PB}) = 1$ — tần số nhỏ hơn k_{PB} được giữ, lớn hơn bị suy giảm. Chọn $\lambda + \mu < 0$ cho bộ lọc thông thấp thực sự, không co DC. Không cần giải hệ ma trận; vẫn chỉ là trung bình cục bộ.

Câu 16. Bilateral mesh denoising. Tại sao giữ được feature?

Bilateral mesh denoising (Fleishman 2003, Jones 2003) mở rộng bilateral 2D sang lưới. Với đỉnh v_i pháp tuyến n_i, dự đoán vị trí mới bằng dịch dọc n_i:

$$v_i' = v_i + n_i \cdot \frac{\sum_j w_c(\|v_j - v_i\|)\, w_s(\langle n_i, v_j - v_i \rangle)\, \langle n_i, v_j - v_i \rangle}{\sum w_c w_s}$$

$w_c(x) = \exp(-x^2/2\sigma_c^2)$ là trọng số **khoảng cách**, $w_s(y) = \exp(-y^2/2\sigma_s^2)$ là trọng số **độ lệch dọc pháp tuyến**. **Giữ feature:** qua cạnh sắc, hàng xóm cách xa mặt phẳng tiếp tuyến của v_i, w_s bé, không kéo v_i qua bên kia — feature giữ nguyên còn vùng phẳng vẫn được làm mượt. Tinh chỉnh σ_c theo kích thước neighborhood, σ_s theo biên độ nhiễu.

where $w_c(x) = \exp(-x^2/2\sigma_c^2)$ is the **spatial** (closeness) weight and $w_s(y) = \exp(-y^2/2\sigma_s^2)$ is the **range** (similarity along normal) weight. **Feature preservation:** across a sharp edge, neighbors lie far from v_i's tangent plane, so w_s becomes small and they don't pull v_i across — features stay sharp while flat regions still smooth. Tune σ_c to neighborhood size and σ_s to noise amplitude.

Q17. What is mean curvature flow smoothing?

Mean curvature flow (MCF) evolves the surface along its normal at a speed proportional to mean curvature: $\partial v/\partial t = -H\mathbf{n}$. Discretely, this equals applying the **cotangent Laplacian**: $\Delta v = -2H\mathbf{n}$, so MCF is $v_{t+1} = v_t + \tau \cdot \frac{1}{2} L_c(v_t)$. It smooths surfaces optimally in a geometric sense (curvature-shortening). **Implicit MCF** (Desbrun 1999) solves $(I - \tau L_c)v_{t+1} = v_t$ for unconditional stability with large τ. Mean curvature flow is intrinsic — purely geometric, not topology-dependent. Drawback: still shrinks closed surfaces (sphere collapses to point); use **volume-preserving** or **conformalized** MCF (Kazhdan 2012) to keep size.

Q18. Compare Laplacian vs Taubin vs bilateral mesh denoising.

Laplacian (1 pass): simplest, fast, *shrinks* the model and *blurs features*. Use only on small noise. **Taubin $\lambda|\mu$:** no shrinkage (low-pass with proper pass-band), still feature-blurring because it is linear. Fast, no matrix solve. **Bilateral:** non-linear, weights neighbors by both spatial and normal-difference Gaussians — *feature preserving* (sharp creases survive). Slower and needs tuning σ_c, σ_s. Modern: **normal-based bilateral** (Zheng 2011) filters face normals first then updates vertex positions; even better. Rule of thumb: smooth/organic data → Taubin; CAD/feature-rich → bilateral or ℓ_0 minimization.

Q19. Explain Statistical Outlier Removal (SOR) for point clouds.

SOR (PCL implementation) removes points whose mean distance to their k nearest neighbors deviates too far from the global distribution. **Steps:** 1) For each point p_i, find its k nearest neighbors and compute the mean distance d_i. 2) Compute the global mean μ and standard deviation σ of $\{d_i\}$. 3) A point is an **outlier** if $d_i > \mu + \alpha\sigma$ (typically $\alpha = 1$ to 2). 4) Remove all outliers. SOR assumes the overall density is roughly Gaussian and works well for sparse outliers from sensor noise. Parameters: k controls locality (e.g., 30), α controls aggressiveness. It does *not* preserve fine detail equally — dense/sparse boundaries can be misclassified.

Q20. What is Radius Outlier Removal? When to use vs SOR?

Radius Outlier Removal (ROR) keeps only points that have at least $N_{\min}$ neighbors within radius r. **Steps:** 1) For each point, range-search neighbors within r. 2) If count $< N_{\min}$, remove. Simpler than SOR, no statistics needed. **When to use:** 1) When you know the expected sensor density (e.g., LiDAR at 10 m gives ∼1 cm spacing). 2) For very sparse outliers (single floaters). 3) Real-time pipelines that can't afford SOR's two-pass statistics. **SOR vs ROR:** SOR adapts to local density, ROR uses fixed thresholds. Use SOR when density varies (close vs far objects); use ROR when density is uniform and speed matters.

Q21. What is voxel grid downsampling?

Voxel grid downsampling overlays a regular 3D grid of cubes (voxels) of size L on the point cloud and replaces all points in each voxel with their **centroid** (or one representative). **Steps:** 1) Compute bounding box, divide by leaf size. 2) Hash each point into its voxel. 3) Average points (and normals/colors) per voxel. Output: roughly uniform-density cloud, fewer points. Benefits: fast, cache-friendly, reduces memory for downstream ICP/registration. Drawbacks: fixed grid causes **aliasing** and shifts at voxel boundaries. Variants: **octree-based** adaptive voxels, or **Poisson-disk** sampling for blue-noise distribution. Used in PCL (`pcl::VoxelGrid`), Open3D, and ROS perception stacks.

Q22. What is MLS (Moving Least Squares)? How is it used for denoising?

Moving Least Squares (Levin 1998, Alexa 2003) projects each input point to a smooth surface defined locally. **Steps:** 1) For point p, gather neighbors within radius h. 2) Fit a **local reference plane** by weighted PCA with Gaussian weights $w(\|q-p\|) = \exp(-\|q-p\|^2/h^2)$. 3) On that plane, fit a low-degree **polynomial** (often quadratic) approximating the height field of neighbors. 4) Project p onto this polynomial to get p'. The MLS surface is smooth (infinitely differentiable in h) and noise is averaged out at scale h. Used in PCL (`pcl::MovingLeastSquares`), Meshlab. Drawbacks: bandwidth h trade-off — too small keeps noise, too large blurs features.

Q23. What is WLOP? How does it differ from MLS?

WLOP (Weighted Locally Optimal Projection, Huang 2009) generates a uniformly distributed, denoised set of **output** points $\{x_i\}$ from noisy input $\{p_j\}$, even *without* normal estimation. It minimizes:

$$E(X) = \sum_i \sum_j \theta(\|x_i - p_j\|)\|x_i - p_j\| - \mu \sum_i \sum_{i' \neq i} \theta(\|x_i - x_{i'}\|)\eta(\|x_i - x_{i'}\|)$$

The first **attraction** term pulls x_i toward input data. The second **repulsion** term spreads output points apart for uniform density. Local density weights θ down-weight crowded regions. **Differences from MLS:** (a) MLS projects each input point individually and may inherit non-uniformity; WLOP redistributes; (b) WLOP needs no normals; (c) MLS needs a smooth surface assumption, WLOP handles outlier-rich data better. Use WLOP for raw scanner output that is highly non-uniform.

Q24. How do you estimate normals for a point cloud (PCA-based)?

Steps: 1) For point p_i, find k-NN or radius-r neighbors $\{q_j\}$. 2) Compute the centroid $\bar{p}$ and **covariance matrix** $C = \frac{1}{k}\sum(q_j - \bar{p})(q_j - \bar{p})^T$. 3) Eigen-decompose: $C = V\Lambda V^T$ with $\lambda_0 \leq \lambda_1 \leq \lambda_2$. The eigenvector for the smallest λ_0 is the normal n_i (perpendicular to the local fitted plane). 4) The ratio $\lambda_0/(\lambda_0 + \lambda_1 + \lambda_2)$ is the **surface variation**/curvature proxy. 5) **Orient consistently**: compute a Riemannian graph of nearest neighbors, build a minimum spanning tree, and propagate orientation by flipping if $n_i \cdot n_j < 0$ (Hoppe 1992). Robust variants weight neighbors by distance or use RANSAC plane fitting.

Q25. Compare mesh smoothing vs point cloud smoothing — what's different?

Mesh smoothing relies on explicit **connectivity**: each vertex has a fixed one-ring of neighbors, so Laplacian/Taubin/cotangent operators are well-defined and feature-preserving filters can use dihedral angles. **Point cloud smoothing** has no connectivity — you must reconstruct neighborhoods on-the-fly via k-NN or radius search

Câu 17. Mean curvature flow smoothing là gì?

Mean curvature flow (MCF) tiến hóa bề mặt dọc pháp tuyến với tốc độ tỷ lệ độ cong trung bình: $\partial v/\partial t = -H\mathbf{n}$. Rời rạc, đây chính là áp dụng **cotangent Laplacian**: $\Delta v = -2H\mathbf{n}$, nên MCF là $v_{t+1} = v_t + \tau \cdot \frac{1}{2} L_c(v_t)$. Làm mượt tối ưu theo nghĩa hình học (curvature-shortening). **Implicit MCF** (Desbrun 1999) giải $(I - \tau L_c)v_{t+1} = v_t$ để ổn định vô điều kiện với τ lớn. MCF là nội tại — thuần hình học, không phụ thuộc topology. Nhược: vẫn co mặt kín (cầu sụp về điểm); dùng MCF **bảo toàn thể tích** hoặc **conformalized** (Kazhdan 2012) để giữ kích thước.

Câu 18. So sánh Laplacian vs Taubin vs bilateral mesh denoising.

Laplacian (1 pass): đơn giản, nhanh, *co* mô hình và *làm mờ feature*. Chỉ dùng khi nhiễu nhỏ. **Taubin $\lambda|\mu$:** không co (thông thấp với pass-band đúng), vẫn làm mờ feature vì tuyến tính. Nhanh, không cần giải ma trận. **Bilateral:** phi tuyến, có trọng số cả khoảng cách lẫn độ lệch pháp tuyến — *giữ feature* (cạnh sắc tồn tại). Chậm hơn, cần tinh chỉnh σ_c, σ_s. Hiện đại: **normal-based bilateral** (Zheng 2011) lọc pháp tuyến mặt trước rồi cập nhật vị trí; còn tốt hơn. Nguyên tắc: dữ liệu hữu cơ/mượt → Taubin; CAD/nhiều feature → bilateral hoặc tối thiểu hóa ℓ_0.

Câu 19. Statistical Outlier Removal (SOR) cho point cloud?

SOR (PCL) loại bỏ điểm có khoảng cách trung bình đến k hàng xóm gần nhất lệch quá xa phân phối toàn cục. **Các bước:** 1) Với mỗi điểm p_i, tìm k-NN và tính khoảng cách trung bình d_i. 2) Tính trung bình toàn cục μ và độ lệch chuẩn σ của $\{d_i\}$. 3) Điểm là **outlier** nếu $d_i > \mu + \alpha\sigma$ (thường $\alpha = 1$ đến 2). 4) Xóa outlier. SOR giả định mật độ tổng thể xấp xỉ Gauss và tốt với outlier thưa do nhiễu sensor. Tham số: k kiểm soát locality (vd 30), α kiểm soát độ aggressive. SOR *không* giữ chi tiết nhỏ tốt như nhau — biên dày/thưa có thể bị phân loại sai.

Câu 20. Radius Outlier Removal là gì? Khi nào dùng so với SOR?

Radius Outlier Removal (ROR) chỉ giữ điểm có ít nhất $N_{\min}$ hàng xóm trong bán kính r. **Các bước:** 1) Range-search neighbors trong r cho mỗi điểm. 2) Nếu count $< N_{\min}$, xóa. Đơn giản hơn SOR, không cần thống kê. **Khi nào dùng:** 1) Khi biết mật độ sensor kỳ vọng (vd LiDAR ở 10m cho khoảng cách ∼1cm). 2) Cho outlier rất thưa (điểm trôi đơn lẻ). 3) Pipeline real-time không đủ thời gian SOR 2-pass. **SOR vs ROR:** SOR thích nghi với mật độ cục bộ, ROR dùng ngưỡng cố định. Dùng SOR khi mật độ biến thiên (vật gần vs xa); dùng ROR khi mật độ đều và cần nhanh.

Câu 21. Voxel grid downsampling là gì?

Voxel grid downsampling phủ lưới khối lập phương đều (voxel) cạnh L lên đám mây điểm và thay tất cả điểm trong mỗi voxel bằng **centroid** (hoặc đại diện). **Các bước:** 1) Tính bounding box, chia theo leaf size. 2) Hash mỗi điểm vào voxel của nó. 3) Trung bình điểm (và pháp tuyến/màu) mỗi voxel. Output: cloud mật độ xấp xỉ đều, ít điểm hơn. Lợi: nhanh, cache-friendly, giảm bộ nhớ cho ICP/registration sau đó. Hại: lưới cố định gây **aliasing** và dịch ở biên voxel. Biến thể: **voxel thích nghi** dựa trên octree, hoặc **Poisson-disk** cho phân bố blue-noise. Dùng trong PCL (`pcl::VoxelGrid`), Open3D, ROS perception stacks.

Câu 22. MLS là gì? Dùng để khử nhiễu thế nào?

Moving Least Squares (Levin 1998, Alexa 2003) chiếu mỗi điểm input về một bề mặt mượt định nghĩa cục bộ. **Các bước:** 1) Với điểm p, lấy hàng xóm trong bán kính h. 2) Khớp **mặt phẳng tham chiếu** cục bộ bằng PCA có trọng số $w(\|q-p\|) = \exp(-\|q-p\|^2/h^2)$. 3) Trên mặt phẳng đó, khớp **đa thức** bậc thấp (thường bậc 2) xấp xỉ trường độ cao của hàng xóm. 4) Chiếu p lên đa thức để được p'. Mặt MLS mượt (khả vi vô hạn theo h) và nhiễu được trung bình hóa ở scale h. Dùng trong PCL (`pcl::MovingLeastSquares`), Meshlab. Hạn chế: bandwidth h — quá nhỏ giữ nhiễu, quá lớn làm mờ feature.

Câu 23. WLOP là gì? Khác MLS thế nào?

WLOP (Weighted Locally Optimal Projection, Huang 2009) sinh ra tập điểm **output** $\{x_i\}$ phân bố đều, đã khử nhiễu, từ input nhiễu $\{p_j\}$, ngay cả *không cần* pháp tuyến. Tối thiểu hóa:

$$E(X) = \sum_i \sum_j \theta(\|x_i - p_j\|)\|x_i - p_j\| - \mu \sum_i \sum_{i' \neq i} \theta(\|x_i - x_{i'}\|)\eta(\|x_i - x_{i'}\|)$$

Số hạng đầu **hấp dẫn** kéo x_i về dữ liệu input. Số hạng sau **đẩy** làm các điểm output trải đều. Trọng số θ giảm trọng vùng dày. **Khác MLS:** (a) MLS chiếu từng điểm input riêng và kế thừa không đều; WLOP phân bố lại; (b) WLOP không cần pháp tuyến; (c) MLS giả định bề mặt mượt, WLOP xử lý dữ liệu nhiều outlier tốt hơn. Dùng WLOP cho output scanner thô không đều cao.

Câu 24. Ước lượng pháp tuyến point cloud bằng PCA?

Các bước: 1) Với điểm p_i, tìm k-NN hoặc neighbors bán kính r là $\{q_j\}$. 2) Tính centroid $\bar{p}$ và **ma trận hiệp phương sai** $C = \frac{1}{k}\sum(q_j - \bar{p})(q_j - \bar{p})^T$. 3) Phân rã trị riêng: $C = V\Lambda V^T$ với $\lambda_0 \leq \lambda_1 \leq \lambda_2$. Vector riêng ứng λ_0 nhỏ nhất là pháp tuyến n_i (vuông góc mặt phẳng cục bộ). 4) Tỷ số $\lambda_0/(\lambda_0 + \lambda_1 + \lambda_2)$ là **độ biến thiên bề mặt**/proxy cho độ cong. 5) **Định hướng nhất quán**: dựng đồ thị Riemann của k-NN, xây cây bao trùm tối thiểu, lan truyền hướng bằng cách lật nếu $n_i \cdot n_j < 0$ (Hoppe 1992). Biến thể robust: trọng số theo khoảng cách hoặc fit RANSAC.

Câu 25. So sánh smoothing lưới và smoothing point cloud — khác nhau gì?

Smoothing lưới dựa vào **kết nối** tường minh: mỗi đỉnh có one-ring cố định, nên Laplacian/Taubin/cotangent định nghĩa tốt và bộ lọc giữ feature có thể dùng góc nhị diện. **Smoothing point cloud** không có kết nối — phải xây neighborhood động bằng k-NN/bán kính mỗi lần. Phương pháp: MLS, WLOP, bilateral PC, hoặc deep learning. Smoothing lưới còn quản lý **topology** và đảm bảo manifold; PC không giữ topology. Lưới có pháp tuyến tường minh từ mặt; PC cần ước lượng PCA — bản thân nhiễu. Smoothing PC còn phải xử lý **biến thiên mật độ** (gần vs xa). Pipeline phổ biến: smooth PC → tái dựng lưới → smooth lưới.

Câu 26. Mô tả pipeline geometric modeling đầy-đủ.

Các giai đoạn: 1) **Thu nhận** — laser scan, photogrammetry, depth camera, hoặc CAD. 2) **Đăng ký** — gộp nhiều

every time. Operations: MLS, WLOP, bilateral PC, or learned models. Mesh smoothing also handles **topology** and can guarantee manifoldness; PC methods don't preserve any topology. Mesh has explicit normals from face geometry; PC needs PCA estimation that itself can be noisy. PC smoothing must additionally manage **density variation** (close vs far). Often the pipeline is: smooth PC → reconstruct mesh → smooth mesh.

Q26. Describe the geometric modeling pipeline end-to-end.

Stages: 1) **Acquisition** — laser scan, photogrammetry, depth camera, or CAD authoring. 2) **Registration** — align multiple scans (ICP, global SLAM). 3) **Cleaning** — remove outliers (SOR/ROR), fill gaps. 4) **Downsample** — voxel grid, Poisson-disk. 5) **Normal estimation** — PCA + consistent orientation. 6) **Surface reconstruction** — Poisson, Ball-Pivoting, Marching Cubes (from implicit). 7) **Mesh repair** — fill holes, fix non-manifold edges. 8) **Denoising** — Taubin, bilateral, MCF. 9) **Simplification & LOD** — QEM, progressive mesh. 10) **Parameterization & UV** — for texturing. 11) **Export** — OBJ/PLY/STL/glTF. 12) **Rendering / Application** — game engine, 3D printer, FEA, VR.

Q27. Common 3D file formats — compare OBJ, PLY, STL, OFF.

OBJ (Wavefront): ASCII, supports vertices, normals, UVs, materials (`.mtl`), polygon faces (not just triangles). De-facto interchange for art assets. **PLY** (Stanford): ASCII or binary, supports arbitrary per-vertex/per-face properties (color, intensity, custom). Standard for scanner output and research. **STL**: ASCII or binary, only triangle soup with face normals — *no shared vertex indexing, no color, no UV*. The standard for 3D printing/CAD slicers. **OFF**: ASCII, very simple — header line, vertex list, face list with index. Used in academia (Princeton Shape Benchmark). Also worth knowing: **glTF 2.0** (binary, PBR, animations — modern web/games), **FBX** (proprietary, animation-rich), **USD** (Pixar, scene graph).

Q28. What are the main 3D representations?

1) **Point cloud** — unordered set of (x, y, z) points; native for scanners; no topology. 2) **Triangle/poly mesh** — vertices + edges + faces; explicit connectivity; standard for rendering. 3) **NURBS** — non-uniform rational B-spline surfaces; smooth, parametric, exact for CAD. 4) **Implicit / SDF** — $f(x, y, z) = 0$; supports easy CSG, constant-time inside test; needs Marching Cubes to mesh. 5) **Voxel grid** — 3D pixels; native for medical/volumetric data; high memory. 6) **CSG** — Constructive Solid Geometry, boolean tree of primitives; classic CAD. 7) **Subdivision surfaces** — coarse mesh + refinement rules; smooth at any LOD. 8) **Neural fields** (NeRF, occupancy networks) — MLP encodes geometry/radiance.

Q29. When would you choose mesh vs implicit vs voxel representation?

Mesh: best for *rendering* and *games* — GPU loves triangles, surface-only memory, explicit UV. Bad for boolean ops and topology change. **Implicit / SDF:** best for *modeling*, *CSG booleans*, *ray-marching* (shadertoy), *collision detection* — closed-form inside test. Bad for direct rendering without conversion. **Voxel:** best for *volumetric data* (medical CT/MRI), *simulation* (fluids, voxel-based physics), *games like Minecraft*. Bad for memory: $O(n^3)$ growth — solved by sparse octrees (**VDB**, **NanoVDB**). **Rule of thumb:** render = mesh, sculpt = SDF, simulate = voxel. Modern engines mix: Unreal Nanite uses mesh, Lumen uses SDF, fluid sims use VDB.

Q30. What is C^0, C^1, C^2 continuity? Why does it matter?

At the boundary between two curves/surfaces: C^0 = positions match (no gap). C^1 = positions and first derivatives (**tangents**) match — surfaces visually smooth, no "kink". C^2 = positions, tangents, and second derivatives (**curvature**) match — even smoother, important for highlights and reflections. G^n (geometric continuity) is the looser version: tangent/curvature *directions* match, magnitudes can differ. **Why it matters:** 1) Class-A surfaces in automotive/product design need G^2 for clean reflections. 2) Animation paths need at least C^1 to avoid jerk. 3) Subdivision schemes (Catmull-Clark) produce C^2 except at extraordinary vertices. 4) Bezier patches need careful boundary tangent matching for C^1.

Q31. How is point cloud denoising used in LiDAR / autonomous driving?

LiDAR returns are noisy and contain spurious points from **rain, fog, snow, exhaust, dust, sensor multipath**. Pipeline in AVs (Waymo, Tesla, Mobileye): 1) **Range filter** — remove very near/far returns. 2) **ROR/SOR** — remove single floating points (rain droplets). 3) **Ground plane fit** (RANSAC) — separate ground from objects. 4) **Voxel downsample** — for real-time 3D detectors (PointPillars, CenterPoint). 5) **Temporal denoising** — accumulate frames, vote on persistence (random rain points won't reappear). 6) **Weather-specific filters** — DROR (Dynamic Radius Outlier Removal) for snow. Clean point clouds are critical for downstream **3D object detection**, **tracking**, and **SLAM**. Even small false positives can trigger phantom braking.

Q32. How is mesh simplification used in real-time rendering / games?

Games can't render 100M-triangle assets at 60-120 Hz. **Uses of simplification:** 1) **Discrete LOD chains** (LOD0-4) generated by QEM offline, switched by screen size. 2) **HLOD** — distant clusters of buildings merged into single proxy meshes (Unreal). 3) **Imposters** — even simpler than meshes, just billboards from extreme LOD. 4) **Shadow LOD** — separate, more aggressive simplification for shadow casters. 5) **Collision LOD** — simplified physics meshes for runtime. 6) **Streaming** — progressive mesh / Nanite virtualized geometry. 7) **VR/mobile** — aggressive LOD because thermal/perf budget is tight. The combination saves **vertex shader cost**, draw calls, memory bandwidth, and shadow-pass cost. Without simplification, modern open-world games would be impossible.

Q33. How is denoising used in medical 3D scanning?

Medical CT/MRI/ultrasound and surface scanners (intra-oral dental, body scanners) produce noisy volumes/meshes. **Uses:** 1) **CT reconstruction** — iterative algorithms (SART, MBIR) include denoising priors (TV, BM3D). 2) **Surface extraction** — Marching Cubes followed by Taubin smoothing for organ surfaces. 3) **Dental crown design** — bilateral mesh denoising preserves cusp/groove features while removing scanner speckle. 4) **Surgical planning** — point cloud denoising on registered intra-op scans for AR overlay. 5) **Implant/prosthesis fitting** — clean meshes for 3D printing. 6) **Radiomics / AI** — denoised images give more reliable features for diagnosis. Denoising must be **feature-preserving** — it cannot blur tumor edges or vessel walls, which is why bilateral and edge-aware methods dominate.

Q34. Why is geometric modeling fundamental to digital twins?

scan (ICP, SLAM toàn cục). 3) **Làm sạch** — xóa outlier (SOR/ROR), lấp khe. 4) **Downsample** — voxel grid, Poisson-disk. 5) **Ước lượng pháp tuyến** — PCA + định hướng nhất quán. 6) **Tái dựng bề mặt** — Poisson, Ball-Pivoting, Marching Cubes (từ implicit). 7) **Sửa lưới** — lấp lỗ, sửa cạnh non-manifold. 8) **Khử nhiễu** — Taubin, bilateral, MCF. 9) **Đơn giản hóa & LOD** — QEM, progressive mesh. 10) **Parameterization & UV** — cho texturing. 11) **Export** — OBJ/PLY/STL/glTF. 12) **Render / Ứng dụng** — game engine, in 3D, FEA, VR.

Câu 27. Các định dạng 3D — so sánh OBJ, PLY, STL, OFF.

OBJ (Wavefront): ASCII, hỗ trợ vertex, normal, UV, material (`.mtl`), face đa giác (không chỉ tam giác). Chuẩn de-facto trao đổi asset. **PLY** (Stanford): ASCII hoặc binary, hỗ trợ thuộc tính tùy ý per-vertex/per-face (color, intensity, custom). Chuẩn cho output scanner và nghiên cứu. **STL**: ASCII hoặc binary, chỉ là tập tam giác với pháp tuyến mặt — *không có index dùng chung, không có color, không có UV*. Chuẩn cho in 3D / CAD slicer. **OFF**: ASCII, rất đơn giản — header, vertex list, face list theo index. Dùng trong học thuật (Princeton Shape Benchmark). Cần biết thêm: **glTF 2.0** (binary, PBR, animation — web/games hiện đại), **FBX** (proprietary, animation), **USD** (Pixar, scene graph).

Câu 28. Các biểu diễn 3D chính?

1) **Point cloud** — tập điểm (x, y, z) không thứ tự; native cho scanner; không topology. 2) **Lưới tam giác/đa giác** — đỉnh + cạnh + mặt; kết nối tường minh; chuẩn render. 3) **NURBS** — non-uniform rational B-spline; mượt, tham số, chính xác cho CAD. 4) **Implicit / SDF** — $f(x, y, z) = 0$; CSG dễ, kiểm tra inside O(1); cần Marching Cubes để mesh. 5) **Voxel grid** — pixel 3D; native cho dữ liệu y khoa/khối; tốn bộ nhớ. 6) **CSG** — Constructive Solid Geometry, cây boolean của primitive; CAD cổ điển. 7) **Subdivision** — lưới thô + quy tắc tinh chỉnh; mượt tại mọi LOD. 8) **Neural fields** (NeRF, occupancy net) — MLP mã hóa hình học/radiance.

Câu 29. Khi nào chọn lưới vs implicit vs voxel?

Lưới: tốt nhất cho *render* và *game* — GPU yêu tam giác, bộ nhớ chỉ surface, UV tường minh. Kém với boolean và thay đổi topology. **Implicit / SDF:** tốt cho *modeling*, *boolean CSG*, *ray-marching* (shadertoy), *collision* — kiểm tra inside dạng đóng. Kém khi render trực tiếp không qua chuyển đổi. **Voxel:** tốt cho *dữ liệu khối* (CT/MRI y khoa), *mô phỏng* (chất lỏng, vật lý voxel), *game kiểu Minecraft*. Kém về bộ nhớ: tăng $O(n^3)$ — giải bằng octree thưa (**VDB**, **NanoVDB**). **Nguyên tắc:** render = lưới, sculpt = SDF, simulate = voxel. Engine hiện đại trộn: Unreal Nanite dùng lưới, Lumen dùng SDF, mô phỏng chất lỏng dùng VDB.

Câu 30. C^0, C^1, C^2 continuity là gì? Tại sao quan trọng?

Tại biên giữa hai đường cong/bề mặt: C^0 = vị trí trùng (không có khe). C^1 = vị trí và đạo hàm bậc 1 (**tiếp tuyến**) trùng — bề mặt mượt thị giác, không có "gãy". C^2 = vị trí, tiếp tuyến, đạo hàm bậc 2 (**độ cong**) trùng — mượt hơn nữa, quan trọng cho highlight và phản xạ. G^n (continuity hình học) là phiên bản lỏng hơn: *hướng* tiếp tuyến/độ cong trùng, độ lớn có thể khác. **Tại sao quan trọng:** 1) Class-A surface ô tô/sản phẩm cần G^2 cho phản xạ sạch. 2) Đường animation cần ít nhất C^1 để tránh giật. 3) Subdivision (Catmull-Clark) cho C^2 trừ đỉnh đặc biệt. 4) Bezier patch cần khớp tiếp tuyến biên cẩn thận để có C^1.

Câu 31. Khử nhiễu point cloud trong LiDAR / xe tự lái?

LiDAR có nhiễu và điểm giả từ **mưa, sương, tuyết, khói thải, bụi, multipath sensor**. Pipeline trong AV (Waymo, Tesla, Mobileye): 1) **Lọc range** — xóa return rất gần/rất xa. 2) **ROR/SOR** — xóa điểm trôi đơn (giọt mưa). 3) **Khớp mặt đất** (RANSAC) — tách mặt đất khỏi vật. 4) **Voxel downsample** — cho 3D detector real-time (PointPillars, CenterPoint). 5) **Khử nhiễu thời gian** — tích lũy frame, vote bền vững (điểm mưa ngẫu nhiên không lặp lại). 6) **Lọc theo thời tiết** — DROR (Dynamic ROR) cho tuyết. Cloud sạch là then chốt cho **phát hiện vật 3D**, **tracking**, **SLAM**. False positive nhỏ cũng có thể gây phanh ảo.

Câu 32. Đơn giản hóa lưới trong render real-time / game?

Game không thể render asset 100M tam giác ở 60-120 Hz. **Ứng dụng:** 1) **Chuỗi LOD rời rạc** (LOD0-4) sinh bởi QEM offline, chuyển theo kích thước màn hình. 2) **HLOD** — cụm tòa nhà xa gộp thành proxy đơn (Unreal). 3) **Imposter** — đơn giản hơn cả lưới, chỉ là billboard từ LOD cực thấp. 4) **Shadow LOD** — đơn giản hóa aggressive hơn riêng cho shadow caster. 5) **Collision LOD** — lưới vật lý đơn giản runtime. 6) **Streaming** — progressive mesh / Nanite virtualized geometry. 7) **VR/mobile** — LOD aggressive vì ngân sách nhiệt/perf chặt. Tổ hợp tiết kiệm **vertex shader cost**, draw call, băng thông, shadow pass. Không có simplification, game open-world hiện đại không khả thi.

Câu 33. Khử nhiễu trong scan 3D y khoa?

CT/MRI/siêu âm và scanner bề mặt (intra-oral nha khoa, body scanner) cho khối/lưới nhiễu. **Ứng dụng:** 1) **Tái dựng CT** — thuật toán lặp (SART, MBIR) gồm prior khử nhiễu (TV, BM3D). 2) **Trích bề mặt** — Marching Cubes rồi Taubin smoothing cho bề mặt nội tạng. 3) **Thiết kế mão răng** — bilateral mesh denoising giữ feature đỉnh/rãnh trong khi xóa speckle scanner. 4) **Lập kế hoạch phẫu thuật** — khử nhiễu point cloud trên scan intra-op đã đăng ký để overlay AR. 5) **Khớp implant/prosthesis** — lưới sạch để in 3D. 6) **Radiomics / AI** — ảnh đã khử nhiễu cho feature đáng tin hơn. Khử nhiễu phải **giữ feature** — không được làm mờ biên khối u hoặc thành mạch, vì vậy bilateral và edge-aware chiếm ưu thế.

Câu 34. Tại sao geometric modeling là nền tảng của digital twin?

Digital twin là bản sao 3D ảo đồng bộ của tài sản vật lý (nhà máy, tòa nhà, thành phố, cơ thể). Geometric modeling cung cấp: 1) **Thu nhận** — laser scan / photogrammetry để chụp thực tế as-built. 2) **Làm sạch + khử nhiễu** — mô hình dùng được từ scan thô. 3) **Đơn giản hóa + LOD** — twin thành phố có hàng tỷ tam giác; LOD mới dùng tương tác được. 4) **Lưới + parameterization** — đặt sensor, mô phỏng (CFD, FEM), phân tích nhiệt. 5) **Cập nhật liên tục** — re-scan, đăng ký, diff với lưới trước để phát hiện thay đổi (vd biến dạng kết cấu). 6) **Tương tác liên thông** — định dạng chuẩn (USD, IFC, glTF) chia sẻ giữa công cụ. Không có geometric modeling, digital twin chỉ là CAD drawing — thiếu tính trung thực cao, sensor-driven, tiến hóa thời gian làm nên "twin".

Câu 35. QEM kết hợp Marching Cubes trong y khoa thế nào?

Pipeline: 1) Thu khối CT/MRI. 2) Phân đoạn nội tạng bằng ngưỡng/AI để có trường nhị phân hoặc vô hướng. 3) Chạy **Marching Cubes** (Lorensen 1987) trích iso-surface — thường *vài triệu tam giác nhỏ*, có aliasing răng cưa. 4) Áp dụng **Taubin smoothing** để xóa artifact răng cưa voxel. 5) Chạy **QEM simplification** để giảm còn vài

A **digital twin** is a synchronized 3D virtual replica of a physical asset (factory, building, city, body). Geometric modeling provides: 1) **Acquisition** — laser scan / photogrammetry to capture as-built reality. 2) **Cleaning + denoising** — usable model from raw scans. 3) **Simplification + LOD** — twins of cities have billions of triangles; LOD enables interactive use. 4) **Mesh + parameterization** — for sensor placement, simulations (CFD, FEM), thermal analysis. 5) **Continuous update** — re-scan, register, diff with previous mesh to detect change (e.g. structural deformation). 6) **Interoperability** — standard formats (USD, IFC, glTF) for sharing across tools. Without geometric modeling, digital twins would just be CAD drawings — they'd lack the high-fidelity, sensor-driven, time-evolving nature that makes them "twins".

Q35. How is QEM combined with Marching Cubes in medical visualization?
Pipeline: 1) Acquire CT/MRI volume. 2) Segment the organ via thresholding/AI to get binary or scalar field. 3) Run **Marching Cubes** (Lorensen 1987) to extract iso-surface mesh — typically *millions of small triangles*, often staircase-aliased. 4) Apply **Taubin smoothing** to remove voxel staircase artifacts. 5) Run **QEM simplification** to reduce to tens of thousands of triangles, preserving anatomical features (vessel curvature, tumor shape). 6) Compute **normals + UV** for shading. 7) Render with PBR for surgical planning, VR/AR overlay, or 3D print. The combo is essential because Marching Cubes outputs are too dense for interactive VR but contain the correct topology — QEM + smoothing make them practical while QEM with feature-preservation keeps clinically critical detail (e.g., vessel branching, tumor margins).

chục nghìn tam giác, giữ feature giải phẫu (cong mạch, hình khối u). 6) Tính **pháp tuyến + UV** cho shading. 7) Render PBR cho lập kế hoạch phẫu thuật, overlay VR/AR, hoặc in 3D. Tổ hợp này thiết yếu vì output Marching Cubes quá dày cho VR tương tác nhưng có topology đúng — QEM + smoothing biến chúng thành thực dụng, còn QEM giữ feature giữ chi tiết quan trọng lâm sàng (vd nhánh mạch, biên khối u).

QA3a. Theoretical Q&A — Modalities & Reconstruction

Q1. What types of input data exist in medical imaging? List and briefly describe each.
Medical imaging produces several data types depending on modality. **2D projection images** (X-ray radiographs) are single shadow images formed by attenuating X-rays. **Tomographic slice stacks** (CT, MRI) are 2D cross-sections that combine into a 3D **volumetric scalar field** $f(x, y, z)$ stored as voxels with intensity values. **Sinograms** are raw projection data $p(\rho, \theta)$ from CT before reconstruction. **k-space data** are raw frequency-domain samples acquired by MRI before inverse Fourier transform. **Time-series volumes** (fMRI, 4D-CT) add a temporal axis $f(x, y, z, t)$. **Functional/molecular data** (PET, SPECT) encode tracer concentration. Finally, **point clouds and surface meshes** arise after segmentation for surgical planning. All are typically packaged in the **DICOM** format with metadata.

Q2. Explain how MRI works (physics → image): magnetic field, RF pulse, T1/T2 relaxation.
MRI exploits the magnetic properties of **hydrogen protons** in water. (1) A strong static field B_0 (1.5–3T) aligns proton spins along its axis, producing net magnetization M_z. (2) The protons precess at the **Larmor frequency** $\omega = \gamma B_0$, where γ is the gyromagnetic ratio (42.58 MHz/T for ^{1}H). (3) An **RF pulse** at ω tips spins into the transverse plane, creating M_{xy} that induces a signal in receiver coils. (4) After the pulse, two relaxations occur: **T1** (spin-lattice, M_z recovery) and **T2** (spin-spin, M_{xy} decay). (5) **Gradient coils** encode spatial position via frequency and phase, filling **k-space**. (6) An **inverse 2D Fourier transform** of k-space yields the image. Different tissues have different T1/T2, producing contrast.

Q3. What is the difference between T1, T2, FLAIR, and DWI MRI?
T1-weighted: short TR/TE; fat appears bright, water dark; great for anatomy and post-contrast (gadolinium) imaging. **T2-weighted**: long TR/TE; water/edema appears bright, ideal for detecting pathology, inflammation, tumors. **FLAIR** (Fluid-Attenuated Inversion Recovery) is a T2 variant that nulls free CSF signal so periventricular lesions (e.g., MS plaques) stand out. **DWI** (Diffusion-Weighted Imaging) measures Brownian motion of water molecules using bipolar gradients; restricted diffusion (acute stroke, dense tumors) appears bright. Each sequence highlights different tissue properties from the same hardware by varying **TR, TE, TI, and gradient schemes**.

Q4. Explain how CT scanner works (X-ray, sinogram, projections).
A CT scanner has an **X-ray tube** and a ring of detectors that rotate around the patient. (1) X-rays pass through the body; tissues attenuate them following Beer's law $I = I_0 e^{-\int \mu(s)\, ds}$. (2) Each detector measures the line integral $p_\theta(\rho) = \int \mu(x, y)\, d\ell$ along that ray. (3) Rotating from $\theta = 0$ to π collects many projections forming a **sinogram** (a point in space traces a sinusoid). (4) Reconstruction (FBP or iterative) inverts the Radon transform to recover the 2D map $\mu(x, y)$. (5) Values are normalized to **Hounsfield Units**. Modern **helical multi-detector CT** acquires whole volumes in seconds.

Q5. What are Hounsfield Units? Give typical values for air, water, soft tissue, bone.
Hounsfield Units (HU) normalize CT attenuation relative to water:

$$\mathrm{HU} = 1000 \cdot \frac{\mu - \mu_w}{\mu_w}$$

By definition, **water = 0 HU** and **air =** -1000 **HU**. Typical values: **fat** -100 to -50, **soft tissue** $+30$ to $+80$, **blood** $+30$ to $+45$, **muscle** $+10$ to $+40$, **cancellous bone** $+300$ to $+400$, **cortical bone** $+700$ to $+3000$, **metal/contrast** $> +1000$. HU windowing (window/level) maps a narrow HU range to grayscale: e.g., bone window $W = 2000, L = 400$; lung window $W = 1500, L = -600$. This standardization lets clinicians compare scans across machines.

Q6. Compare MRI vs CT — strengths, weaknesses.
CT strengths: fast (seconds), excellent for **bone, lung, acute hemorrhage, trauma**, widely available, cheap. **CT weaknesses**: **ionizing radiation**, poor soft-tissue contrast, beam-hardening artifacts near metal. **MRI strengths**: **superb soft-tissue contrast** (brain, spine, joints, pelvis), no ionizing radiation, multi-parametric (T1/T2/DWI/fMRI), arbitrary slice planes. **MRI weaknesses**: slow (minutes), expensive, contraindicated for pacemakers/ferromagnetic implants, claustrophobia, sensitive to motion. **Use CT** for emergency, bone, lung, and quick screening; **use MRI** for neurological, musculoskeletal, oncologic, and cardiac soft-tissue evaluation.

Q7. How does ultrasound (US) work? When is it preferred?
A piezoelectric transducer emits high-frequency **sound pulses (2–15 MHz)** into tissue and receives reflected echoes.

QA3a. Câu hỏi lý thuyết — Phương thức tạo ảnh & Tái tạo

Câu 1. Có những loại dữ liệu đầu vào nào trong y học hình ảnh?
Y học hình ảnh sinh ra nhiều loại dữ liệu tùy theo modality. **Ảnh chiếu 2D** (X-quang) là một ảnh bóng do tia X bị suy giảm. **Chồng lát cắt tomographic** (CT, MRI) là các mặt cắt 2D ghép thành **trường vô hướng 3D** $f(x, y, z)$ lưu dưới dạng voxel. **Sinogram** là dữ liệu chiếu thô $p(\rho, \theta)$ từ CT trước khi tái tạo. **k-space** là mẫu thô miền tần số do MRI thu thập trước khi biến đổi Fourier ngược. **Khối thời gian** (fMRI, 4D-CT) thêm trục thời gian $f(x, y, z, t)$. **Dữ liệu chức năng/phân tử** (PET, SPECT) mã hóa nồng độ chất đánh dấu. Cuối cùng, **point cloud và mesh bề mặt** xuất hiện sau segmentation phục vụ phẫu thuật. Tất cả đóng gói trong định dạng **DICOM** kèm metadata.

Câu 2. MRI hoạt động thế nào (vật lý → ảnh)?
MRI khai thác từ tính của **proton hydrogen** trong nước. (1) Từ trường tĩnh mạnh B_0 (1.5–3T) căn chỉnh spin proton dọc trục, tạo từ hóa M_z. (2) Proton tiến động (precess) ở **tần số Larmor** $\omega = \gamma B_0$ (42.58 MHz/T với ^{1}H). (3) Một **xung RF** ở ω lật spin sang mặt phẳng ngang, tạo M_{xy} cảm ứng tín hiệu trong cuộn thu. (4) Sau xung, hai quá trình hồi phục: **T1** (spin-mạng, hồi M_z) và **T2** (spin-spin, suy giảm M_{xy}). (5) **Cuộn gradient** mã hóa vị trí qua tần số và pha, lấp đầy **k-space**. (6) Biến đổi **Fourier ngược 2D** của k-space cho ảnh. Mô khác nhau có T1/T2 khác nhau, tạo độ tương phản.

Câu 3. Khác biệt T1, T2, FLAIR, DWI?
T1-weighted: TR/TE ngắn; mỡ sáng, nước tối; tốt cho giải phẫu và sau tiêm gadolinium. **T2-weighted**: TR/TE dài; nước/phù sáng, lý tưởng để phát hiện bệnh lý, viêm, u. **FLAIR** là biến thể T2 triệt tín hiệu CSF tự do để tổn thương cạnh não thất (ví dụ MS) nổi bật. **DWI** đo chuyển động Brown của phân tử nước qua gradient lưỡng cực; khuếch tán hạn chế (đột quỵ cấp, u đặc) sáng. Mỗi sequence làm nổi bật tính chất mô khác nhau từ cùng phần cứng bằng cách thay đổi **TR, TE, TI, và gradient**.

Câu 4. CT scanner hoạt động thế nào?
Máy CT có **ống tia X** và vành detector quay quanh bệnh nhân. (1) Tia X xuyên cơ thể; mô làm suy giảm theo định luật Beer $I = I_0 e^{-\int \mu(s)\, ds}$. (2) Mỗi detector đo tích phân đường $p_\theta(\rho) = \int \mu(x, y)\, d\ell$. (3) Quay từ $\theta = 0$ đến π thu nhiều projection tạo **sinogram** (một điểm vẽ thành đường sin). (4) Tái tạo (FBP hoặc iterative) đảo ngược biến đổi Radon để khôi phục bản đồ 2D $\mu(x, y)$. (5) Giá trị chuẩn hóa thành **Hounsfield Units**. CT đa dãy xoắn ốc hiện đại thu cả khối trong vài giây.

Câu 5. Hounsfield Units là gì? Giá trị điển hình?
Hounsfield Units (HU) chuẩn hóa độ suy giảm CT theo nước:

$$\mathrm{HU} = 1000 \cdot \frac{\mu - \mu_w}{\mu_w}$$

Theo định nghĩa, **nước = 0 HU** và **khí =** -1000 **HU**. Giá trị điển hình: **mỡ** -100 đến -50, **mô mềm** $+30$ đến $+80$, **máu** $+30$ đến $+45$, **cơ** $+10$ đến $+40$, **xương xốp** $+300$ đến $+400$, **xương vỏ** $+700$ đến $+3000$, **kim loại/cản quang** $> +1000$. Windowing HU (window/level) ánh xạ dải HU hẹp sang thang xám: ví dụ window xương $W = 2000, L = 400$; window phổi $W = 1500, L = -600$. Chuẩn hóa này cho phép so sánh giữa các máy.

Câu 6. So sánh MRI vs CT.
Ưu điểm CT: nhanh (vài giây), tuyệt vời cho **xương, phổi, xuất huyết cấp, chấn thương**, phổ biến, rẻ. **Nhược điểm CT**: **tia ion hóa**, độ tương phản mô mềm kém, nhiễu beam-hardening gần kim loại. **Ưu điểm MRI**: **độ tương phản mô mềm xuất sắc** (não, cột sống, khớp, vùng chậu), không tia ion hóa, đa tham số (T1/T2/DWI/fMRI), mặt cắt tùy ý. **Nhược điểm MRI**: chậm (vài phút), đắt, chống chỉ định với máy tạo nhịp/cấy ghép sắt từ, gây sợ hãi không gian kín, nhạy với chuyển động. **Dùng CT** cho cấp cứu, xương, phổi, sàng lọc nhanh; **dùng MRI** cho thần kinh, cơ-xương-khớp, ung bướu mô mềm, tim.

Câu 7. Siêu âm hoạt động thế nào? Khi nào dùng?
Đầu dò áp điện phát **xung âm tần cao (2–15 MHz)** vào mô và nhận sóng dội. Thời gian bay cho độ sâu ($d = ct/2$, $c \approx 1540$ m/s), biên độ dội ánh xạ thành độ sáng **B-mode**. **Doppler** đo vận tốc dòng máu qua dịch tần số. Siêu âm được ưu tiên cho **sản khoa** (an toàn cho thai), **siêu âm tim**, **mạch máu**, **tạng bụng**, và **cấp cứu tại giường**. Ưu: thời gian thực, di động, rẻ, không tia. Nhược: kém qua xương/khí, phụ thuộc kỹ thuật viên, trường nhìn hẹp.

Câu 8. PET hoạt động thế nào? FDG đo gì?

The time-of-flight gives depth ($d = ct/2$, $c \approx 1540$ m/s), and echo amplitude maps to brightness in **B-mode**. **Doppler mode** measures blood-flow velocity from frequency shifts. Ultrasound is preferred for **obstetrics** (no radiation safe for fetus), **cardiac echo**, **vascular flow**, **abdominal organs**, and **point-of-care** bedside imaging. Strengths: real-time, portable, cheap, no radiation. Weaknesses: poor through bone/air, operator-dependent, limited field of view.

Q8. How does PET imaging work? What does FDG measure?

Positron Emission Tomography injects a radioactive tracer that emits **positrons**; each positron annihilates with an electron producing two **511 keV gamma photons** traveling 180° apart. A ring of detectors records **coincidence events**, and reconstruction (OSEM iterative) yields a 3D map of tracer concentration. **FDG (^{18}F-fluorodeoxyglucose)** is a glucose analog trapped in metabolically active cells, revealing tumors, inflammation, and brain activity. PET shows **function/metabolism**, not anatomy. Other tracers (PSMA, amyloid, ^{68}Ga-DOTATATE) target specific receptors. Spatial resolution is modest (∼4–6 mm) but molecular sensitivity is unmatched.

Q9. PET-CT vs PET-MRI?

PET-CT fuses functional PET with anatomic CT acquired sequentially in the same gantry; CT also provides **attenuation correction**. It is fast, robust, and standard for oncology staging, but adds CT radiation dose. **PET-MRI** replaces CT with MRI, giving **superior soft-tissue contrast** and no extra radiation, ideal for pediatric, neurological, and pelvic cancers. PET-MRI is more expensive, slower, and attenuation correction is harder (MRI doesn't directly measure electron density — requires Dixon-based or atlas methods). Choice depends on indication, dose concern, and availability.

Q10. What is DICOM? Why is it the standard?

DICOM (Digital Imaging and Communications in Medicine) is the universal standard for storing and transmitting medical images. Each **DICOM file** contains pixel data plus a header of **tagged metadata**: patient ID, study/series UIDs, modality, acquisition parameters (TR, TE, kVp, mAs), pixel spacing, slice thickness, orientation matrix. It defines **network services** (C-STORE, C-FIND, C-MOVE) for PACS communication. Standardization ensures **interoperability** across vendors, enables longitudinal comparison, supports AI pipelines, and provides legal/audit trails. Without DICOM, every scanner would speak its own dialect.

Q11. What is fMRI and what does it measure?

Functional MRI measures brain activity via the **BOLD signal** (Blood-Oxygen-Level-Dependent). When neurons fire, local blood flow increases beyond oxygen consumption, raising the ratio of oxyhemoglobin (diamagnetic) to deoxyhemoglobin (paramagnetic) and lengthening T2*. A **T2*-weighted EPI sequence** captures this small (∼1–5%) signal change every 1–2 seconds. Subjects perform tasks (block or event-related design) and statistical models (GLM) localize activated regions. fMRI provides **spatial resolution** of millimeters but **temporal resolution** of seconds (vs EEG's ms). Used for presurgical planning, cognitive neuroscience, and resting-state connectivity.

Q12. What is SPECT? How does it differ from PET?

SPECT (Single-Photon Emission CT) uses tracers emitting a **single gamma photon** (e.g., ^{99m}Tc, ^{123}I) detected by a rotating gamma camera with a **collimator**. Reconstruction is similar to CT (FBP/OSEM) on projection data. Differences vs PET: SPECT photons are single (not coincidence pairs), so **collimators** are needed, reducing sensitivity by ∼100×. Resolution is lower (∼10 mm vs 4 mm), but tracers are cheaper and longer-lived (no cyclotron needed). SPECT is widely used for **cardiac perfusion, bone scans, thyroid, brain (DAT-scan)**.

Q13. Reconstruct a 2D CT slice via Filtered Back Projection (FBP).

FBP inverts the Radon transform. Steps: (1) **Acquire** projections $p_\theta(\rho)$ for $\theta \in [0, \pi]$. (2) **1D Fourier transform** each projection: $P_\theta(\nu) = \mathcal{F}\{p_\theta\}$. (3) **Apply ramp filter** $|\nu|$ (often windowed by Ram-Lak, Shepp-Logan, or Hann to reduce noise): $\tilde{P}_\theta(\nu) = |\nu| P_\theta(\nu)$. (4) **Inverse 1D FT** to get filtered projections $\tilde{p}_\theta(\rho)$. (5) **Back-project** by smearing each filtered projection along its ray and integrating over angles:

$$f(x, y) = \int_0^\pi \tilde{p}_\theta(x \cos\theta + y \sin\theta)\, d\theta$$

The ramp filter compensates for the $1/|\nu|$ blurring inherent in plain back-projection. FBP is fast, analytic, but noisy at low dose.

Q14. What is the Radon transform / sinogram?

The **Radon transform** maps a 2D function $f(x, y)$ to its line integrals:

$$p_\theta(\rho) = \int_{-\infty}^{\infty} f(\rho \cos\theta - s \sin\theta,\ \rho \sin\theta + s \cos\theta)\, ds$$

where ρ is the perpendicular distance and θ the angle of the line. Stacking $p_\theta(\rho)$ as a 2D image with ρ on the horizontal and θ on the vertical axis gives the **sinogram**. A point object traces a sinusoid (hence the name). CT reconstruction = inverting this transform. The Radon transform underlies all tomography and is invertible only if projections cover at least 180° (parallel beam).

Q15. What is the Central Slice Theorem?

The **Central Slice Theorem** (Fourier Slice Theorem) states: the **1D Fourier transform of a projection at angle** θ equals a **1D radial slice through the 2D Fourier transform of the object** at the same angle:

$$\mathcal{F}_1\{p_\theta(\rho)\}(\nu) = F(\nu \cos\theta,\ \nu \sin\theta)$$

This is the theoretical foundation of CT reconstruction: collecting projections at many angles fills the 2D Fourier plane radially, and an inverse 2D FT recovers f. Because radial sampling is denser near the origin, a $|\nu|$ **ramp filter** compensates for this density before back-projection — which is exactly FBP.

Q16. How is MRI reconstructed from k-space?

k-space is the 2D (or 3D) frequency domain of the image. Each MRI readout fills one trajectory in k-space (line for Cartesian, spiral, radial). Steps: (1) **Acquire** all required k-space samples by sweeping phase-encoding gradients and frequency-encoding readouts. (2) For Cartesian sampling, apply **inverse 2D FFT**: $I(x, y) = \mathcal{F}^{-1}\{S(k_x, k_y)\}$.

PET (Positron Emission Tomography) tiêm chất đánh dấu phóng xạ phát **positron**; mỗi positron hủy với electron sinh hai **photon gamma 511 keV** bay ngược chiều 180°. Vành detector ghi **sự kiện trùng phùng** (coincidence), và tái tạo (OSEM iterative) cho bản đồ 3D nồng độ chất đánh dấu. **FDG (^{18}F-fluorodeoxyglucose)** là chất tương tự glucose bị giữ trong tế bào chuyển hóa cao, chỉ điểm u, viêm, hoạt động não. PET cho thấy **chức năng/chuyển hóa**, không phải giải phẫu. Các tracer khác (PSMA, amyloid, ^{68}Ga-DOTATATE) nhắm receptor cụ thể. Độ phân giải không gian khiêm tốn (∼4–6 mm) nhưng độ nhạy phân tử vô địch.

Câu 9. PET-CT vs PET-MRI?

PET-CT kết hợp PET chức năng với CT giải phẫu thu liên tiếp trong cùng gantry; CT cũng cung cấp **hiệu chỉnh suy giảm**. Nhanh, mạnh mẽ, chuẩn cho phân giai đoạn ung thư, nhưng thêm liều xạ CT. **PET-MRI** thay CT bằng MRI, cho **tương phản mô mềm vượt trội** và không thêm tia, lý tưởng cho ung thư nhi, thần kinh, vùng chậu. PET-MRI đắt hơn, chậm hơn, hiệu chỉnh suy giảm khó hơn (MRI không đo trực tiếp mật độ điện tử — cần phương pháp Dixon hoặc atlas). Lựa chọn tùy chỉ định, lo ngại liều, và sẵn có.

Câu 10. DICOM là gì? Tại sao là chuẩn?

DICOM (Digital Imaging and Communications in Medicine) là chuẩn toàn cầu để lưu và truyền ảnh y tế. Mỗi **file DICOM** chứa dữ liệu pixel cộng header **metadata gắn tag**: ID bệnh nhân, UID study/series, modality, tham số thu (TR, TE, kVp, mAs), pixel spacing, slice thickness, ma trận hướng. Định nghĩa **dịch vụ mạng** (C-STORE, C-FIND, C-MOVE) cho giao tiếp PACS. Chuẩn hóa đảm bảo **liên thông** giữa các hãng, cho phép so sánh dài hạn, hỗ trợ pipeline AI, và tạo dấu vết pháp lý/kiểm toán. Không có DICOM, mỗi máy sẽ nói tiếng riêng.

Câu 11. fMRI là gì và đo gì?

fMRI đo hoạt động não qua **tín hiệu BOLD** (Blood-Oxygen-Level-Dependent). Khi neuron phóng điện, lưu lượng máu cục bộ tăng vượt mức tiêu thụ oxy, làm tăng tỉ lệ oxyhemoglobin (nghịch từ) so với deoxyhemoglobin (thuận từ) và kéo dài T2*. **Sequence EPI T2*-weighted** bắt thay đổi tín hiệu nhỏ (∼1–5%) mỗi 1–2 giây. Đối tượng thực hiện task (block hoặc event-related) và mô hình thống kê (GLM) định vị vùng kích hoạt. fMRI cho **độ phân giải không gian** milimét nhưng **độ phân giải thời gian** chỉ vài giây (so với EEG mili giây). Dùng cho lập kế hoạch tiền phẫu, khoa học thần kinh nhận thức, và kết nối resting-state.

Câu 12. SPECT là gì? Khác PET ra sao?

SPECT (Single-Photon Emission CT) dùng chất đánh dấu phát **một photon gamma đơn** (ví dụ ^{99m}Tc, ^{123}I) được phát hiện bởi gamma camera xoay với **collimator**. Tái tạo tương tự CT (FBP/OSEM) trên dữ liệu projection. Khác biệt với PET: photon SPECT là đơn (không phải cặp trùng phùng), nên cần **collimator**, giảm độ nhạy ∼100 lần. Độ phân giải thấp hơn (∼10 mm vs 4 mm), nhưng tracer rẻ hơn và sống lâu hơn (không cần cyclotron). SPECT dùng nhiều cho **tưới máu cơ tim, xạ hình xương, tuyến giáp, não (DAT-scan)**.

Câu 13. Tái tạo lát cắt CT 2D bằng FBP.

FBP đảo ngược biến đổi Radon. Các bước: (1) **Thu** projection $p_\theta(\rho)$ với $\theta \in [0, \pi]$. (2) **Fourier 1D** từng projection: $P_\theta(\nu) = \mathcal{F}\{p_\theta\}$. (3) **Áp ramp filter** $|\nu|$ (thường có cửa sổ Ram-Lak, Shepp-Logan, hoặc Hann để giảm nhiễu): $\tilde{P}_\theta(\nu) = |\nu| P_\theta(\nu)$. (4) **FT 1D ngược** để được projection đã lọc $\tilde{p}_\theta(\rho)$. (5) **Back-project** bằng cách trải mỗi projection đã lọc dọc tia và tích phân theo góc:

$$f(x, y) = \int_0^\pi \tilde{p}_\theta(x \cos\theta + y \sin\theta)\, d\theta$$

Ramp filter bù cho làm mờ $1/|\nu|$ vốn có trong back-projection thuần. FBP nhanh, giải tích, nhưng nhiễu khi liều thấp.

Câu 14. Biến đổi Radon / sinogram là gì?

Biến đổi Radon ánh xạ hàm 2D $f(x, y)$ thành các tích phân đường:

$$p_\theta(\rho) = \int_{-\infty}^{\infty} f(\rho \cos\theta - s \sin\theta,\ \rho \sin\theta + s \cos\theta)\, ds$$

trong đó ρ là khoảng cách vuông góc và θ là góc đường thẳng. Xếp $p_\theta(\rho)$ thành ảnh 2D với ρ ngang và θ dọc cho **sinogram**. Một điểm vẽ thành đường sin (vì thế tên gọi). Tái tạo CT = đảo ngược biến đổi này. Biến đổi Radon là nền tảng mọi tomography và chỉ khả nghịch nếu projection phủ tối thiểu 180° (chùm song song).

Câu 15. Định lý lát cắt trung tâm là gì?

Định lý lát cắt trung tâm (Fourier Slice Theorem) nói: **biến đổi Fourier 1D của projection ở góc** θ bằng **lát cắt xuyên tâm 1D qua biến đổi Fourier 2D của vật** ở cùng góc:

$$\mathcal{F}_1\{p_\theta(\rho)\}(\nu) = F(\nu \cos\theta,\ \nu \sin\theta)$$

Đây là nền tảng lý thuyết của tái tạo CT: thu projection ở nhiều góc lấp đầy mặt phẳng Fourier 2D theo tia, và FT 2D ngược khôi phục f. Vì lấy mẫu xuyên tâm dày hơn gần gốc, **ramp filter** $|\nu|$ bù mật độ này trước back-projection — chính là FBP.

Câu 16. Tái tạo MRI từ k-space.

k-space là miền tần số 2D (hoặc 3D) của ảnh. Mỗi readout MRI lấp một quỹ đạo trong k-space (dòng cho Cartesian, xoắn ốc, xuyên tâm). Các bước: (1) **Thu** đủ mẫu k-space bằng quét gradient phase-encoding và readout frequency-encoding. (2) Với mẫu Cartesian, áp **FFT 2D ngược**: $I(x, y) = \mathcal{F}^{-1}\{S(k_x, k_y)\}$. (3) Với phi Cartesian (xuyên tâm, xoắn ốc), **regrid** về Cartesian rồi FFT, hoặc dùng **NUFFT**. (4) Kết hợp dữ liệu nhiều cuộn dùng **sum-of-squares**

(3) For non-Cartesian (radial, spiral), **regrid** to Cartesian then FFT, or use **NUFFT**. (4) Combine multi-coil data using **sum-of-squares** or sensitivity-weighted combination. (5) Apply intensity correction. Center of k-space encodes contrast/low frequencies; periphery encodes edges/detail. Undersampling enables acceleration (SENSE, GRAPPA, compressed sensing).

Q17. Build a 3D volume from a stack of 2D MRI slices.
(1) **Read DICOM** stack, sort by `InstanceNumber` or `SliceLocation`. (2) Use header metadata (`PixelSpacing`, `SliceThickness`, `ImagePositionPatient`, `ImageOrientationPatient`) to compute world-space voxel positions. (3) **Resample** to isotropic spacing if needed (trilinear). (4) Store as a 3D array $V[i,j,k]$. (5) **Visualize** via: (a) **Multi-planar reformat (MPR)** — axial/coronal/sagittal slices; (b) **Volume rendering** with a transfer function; (c) **Marching Cubes** surface extraction at an iso-value; (d) **MIP** (Maximum Intensity Projection) for vascular display. Tools: VTK, ITK-SNAP, 3D Slicer.

Q18. Marching Cubes: 256 cases, 15 unique.
Marching Cubes (Lorensen & Cline 1987) extracts an iso-surface $f(x,y,z)=c$ from a scalar volume. Steps: (1) Slide a logical cube over each set of 8 neighboring voxels. (2) For each corner, mark inside ($f \geq c$) or outside ($f < c$) — 8 bits. (3) The 8-bit code ($2^8 = 256$ **cases**) indexes a precomputed **triangulation lookup table**. (4) For each edge crossed by the surface, place a vertex by **linear interpolation**:

$$\mathbf{v} = \mathbf{p}_1 + \frac{c - f_1}{f_2 - f_1}(\mathbf{p}_2 - \mathbf{p}_1)$$

(5) Connect vertices into triangles per the lookup table; advance to next cube. By symmetries (rotations + complement), 256 cases reduce to **15 unique topologies**. Output: a triangle mesh approximating the iso-surface.

Q19. Volume rendering (DVR) vs Surface rendering (Marching Cubes).
Direct Volume Rendering ray-casts through the volume, accumulating color/opacity via a **transfer function**: $C_{out} = \sum_i C_i \alpha_i \prod_{j<i}(1-\alpha_j)$. It shows **all internal structure semi-transparently**, preserves all data, ideal for soft tissue and exploration, but slow and harder to measure. **Surface rendering (Marching Cubes)** extracts a polygon mesh at one iso-value; fast to render afterwards on GPU, easy to measure/3D-print/cut, but **commits to one threshold** and discards data. **Use DVR** for diagnostic exploration, multi-tissue display; **use surface** for surgical planning, 3D printing, simulation, mesh-based analysis.

Q20. What is a transfer function in volume rendering?
A **transfer function** maps scalar voxel values (and optionally gradient magnitude) to **color and opacity**: $\mathrm{TF}(v) \to (R,G,B,\alpha)$. It controls which tissues are visible: e.g., assign $\alpha = 0$ to soft tissue and $\alpha = 0.8$ red to bone in a CT to show only the skeleton. **1D TFs** use intensity only; **2D TFs** add gradient magnitude to highlight boundaries. Designing a good TF is the **hardest part of DVR** — often interactive with histogram widgets. Preset TFs for CT (bone, lung, vessel) and MRI ship with viewers. The TF is essentially a visual segmentation without explicit labels.

Q21. Iterative reconstruction (MLEM, OSEM) vs FBP.
FBP is fast, analytic, exact for noiseless full data but **amplifies noise** (ramp filter) and produces streaks at low dose / sparse views. **MLEM** (Maximum Likelihood Expectation Maximization) models photon statistics (Poisson) and iteratively updates: $f^{(n+1)} = f^{(n)} \cdot A^T(p/Af^{(n)})/A^T\mathbf{1}$. **OSEM** (Ordered Subset EM) accelerates MLEM by partitioning projections into subsets, $\sim$10× speedup. Iterative methods incorporate **system models** (geometry, scatter, attenuation) and **regularization** (TV, edge-preserving), giving better SNR at low dose, but are **slower** and can over-smooth. Modern CT/PET use iterative as default; deep-learning reconstruction is emerging.

Q22. SENSE and GRAPPA parallel imaging for MRI.
Parallel imaging accelerates MRI by undersampling k-space and using **multi-coil arrays** (8–64 coils), each with a known spatial sensitivity. **SENSE** (SENSitivity Encoding) works in image space: undersampled k-space gives an aliased image per coil; SENSE solves a linear system with the coil sensitivities C_i to unfold the aliases at each pixel. **GRAPPA** (GeneRalized Autocalibrating Partial Parallel Acquisition) works in k-space: it uses a fully sampled **auto-calibration** region to learn linear weights that synthesize missing k-space lines from neighboring acquired lines across coils. Acceleration factors $R = 2$–4 are routine, reducing scan time at the cost of $\sqrt{R}$ SNR.

Q23. What is compressed sensing in MRI?
Compressed sensing (CS) reconstructs MR images from **far fewer k-space samples** than Nyquist requires, exploiting two facts: (1) MR images are **sparse** in some transform domain (wavelets, total variation, finite differences); (2) random/incoherent sampling spreads aliasing as noise-like artifacts. CS solves:

$$\min_x \|\Psi x\|_1 \quad \text{s.t.} \quad \|F_u x - y\|_2 \leq \epsilon$$

where Ψ is the sparsifying transform, F_u undersampled Fourier, y measured k-space. Nonlinear iterative solvers (FISTA, ADMM) recover x. CS enables 4–10× acceleration, especially for dynamic MRI (cardiac, contrast). Combined with parallel imaging and deep learning (variational networks), it underlies modern fast MRI.

Q24. VTK pipeline for medical visualization.
VTK (Visualization Toolkit) uses a data-flow pipeline: **Source → Filter → Mapper → Actor → Renderer → RenderWindow**. For CT: (1) `vtkDICOMImageReader` loads volume. (2) Filters preprocess: `vtkImageThreshold`, `vtkImageGaussianSmooth`. (3) `vtkMarchingCubes` or `vtkContourFilter` extracts iso-surface; *or* `vtkSmartVolumeMapper` does GPU ray-cast DVR. (4) Optionally `vtkDecimatePro`, `vtkSmoothPolyDataFilter`. (5) `vtkPolyDataMapper` converts to graphics primitives. (6) `vtkActor` adds material/color; `vtkVolume` for DVR with `vtkColorTransferFunction` and `vtkPiecewiseFunction`. (7) `vtkRenderer` composites; `vtkRenderWindow` displays; `vtkRenderWindowInteractor` handles input. 3D Slicer and ParaView are built on VTK.

Q25. From CT scan to 3D-printable bone mesh — pipeline.
(1) **Acquire** CT DICOM stack. (2) **Load** into 3D Slicer / ITK-SNAP. (3) **Window/level** to bone (HU > +200). (4) **Segment** bone by thresholding (HU > 200–300) plus **region-growing** or AI tools to remove vessels with contrast and table. (5) **Morphological cleanup**: closing, fill holes, island removal. (6) **Marching Cubes** to generate triangle mesh at iso-value. (7) **Smooth** (Laplacian or Taubin) and **decimate** to reasonable triangle count. (8) hoặc trọng số sensitivity. (5) Hiệu chỉnh cường độ. Tâm k-space mã hóa tương phản/tần thấp; rìa mã hóa cạnh/chi tiết. Lấy mẫu thưa cho phép tăng tốc (SENSE, GRAPPA, compressed sensing).

Câu 17. Xây khối 3D từ chồng lát cắt MRI 2D.
(1) **Đọc DICOM** stack, sắp xếp theo `InstanceNumber` hoặc `SliceLocation`. (2) Dùng metadata header (`PixelSpacing`, `SliceThickness`, `ImagePositionPatient`, `ImageOrientationPatient`) để tính vị trí voxel trong không gian thế giới. (3) **Resample** sang spacing đẳng hướng nếu cần (trilinear). (4) Lưu thành mảng 3D $V[i,j,k]$. (5) **Hiển thị** qua: (a) **Multi-planar reformat (MPR)** — các lát axial/coronal/sagittal; (b) **Volume rendering** với transfer function; (c) **Marching Cubes** trích bề mặt iso-value; (d) **MIP** (Maximum Intensity Projection) hiển thị mạch máu. Công cụ: VTK, ITK-SNAP, 3D Slicer.

Câu 18. Marching Cubes: 256 trường hợp, 15 duy nhất.
Marching Cubes (Lorensen & Cline 1987) trích iso-surface $f(x,y,z)=c$ từ khối vô hướng. Bước: (1) Trượt khối logic qua mỗi nhóm 8 voxel kề. (2) Mỗi đỉnh đánh dấu trong ($f \geq c$) hoặc ngoài ($f < c$) — 8 bit. (3) Mã 8-bit ($2^8 = 256$ **trường hợp**) tra bảng **lookup tam giác hóa** đã tính trước. (4) Với mỗi cạnh bị bề mặt cắt, đặt đỉnh bằng **nội suy tuyến tính**:

$$\mathbf{v} = \mathbf{p}_1 + \frac{c - f_1}{f_2 - f_1}(\mathbf{p}_2 - \mathbf{p}_1)$$

(5) Nối đỉnh thành tam giác theo bảng tra; chuyển sang khối kế. Bằng đối xứng (xoay + phần bù), 256 trường hợp giảm xuống **15 topology duy nhất**. Đầu ra: mesh tam giác xấp xỉ iso-surface.

Câu 19. Volume rendering (DVR) vs Surface rendering.
Direct Volume Rendering ray-cast xuyên khối, tích lũy màu/độ mờ qua **transfer function**: $C_{out} = \sum_i C_i \alpha_i \prod_{j<i}(1-\alpha_j)$. Hiển thị **toàn bộ cấu trúc bên trong bán trong suốt**, giữ tất cả dữ liệu, lý tưởng cho mô mềm và khám phá, nhưng chậm và khó đo lường. **Surface rendering (Marching Cubes)** trích mesh polygon ở một iso-value; render nhanh trên GPU, dễ đo/in 3D/cắt, nhưng **cố định một ngưỡng** và bỏ phần dữ liệu khác. **Dùng DVR** cho khám phá chẩn đoán, hiển thị đa mô; **dùng surface** cho lập kế hoạch phẫu thuật, in 3D, mô phỏng, phân tích trên mesh.

Câu 20. Transfer function trong volume rendering là gì?
Transfer function ánh xạ giá trị voxel vô hướng (và tùy chọn độ lớn gradient) sang **màu và độ mờ**: $\mathrm{TF}(v) \to (R,G,B,\alpha)$. Nó kiểm soát mô nào hiện ra: ví dụ gán $\alpha = 0$ cho mô mềm và $\alpha = 0.8$ đỏ cho xương trên CT để chỉ thấy bộ xương. **TF 1D** chỉ dùng cường độ; **TF 2D** thêm độ lớn gradient để làm nổi biên. Thiết kế TF tốt là **phần khó nhất của DVR** — thường tương tác với widget histogram. Preset TF cho CT (xương, phổi, mạch) và MRI có sẵn trong viewer. TF về cơ bản là segmentation thị giác không cần nhãn rõ ràng.

Câu 21. Iterative (MLEM, OSEM) vs FBP.
FBP nhanh, giải tích, chính xác cho dữ liệu đầy đủ không nhiễu nhưng **khuếch đại nhiễu** (do ramp filter) và sinh streak khi liều thấp / ít góc. **MLEM** (Maximum Likelihood Expectation Maximization) mô hình thống kê photon (Poisson) và cập nhật lặp: $f^{(n+1)} = f^{(n)} \cdot A^T(p/Af^{(n)})/A^T\mathbf{1}$. **OSEM** (Ordered Subset EM) tăng tốc MLEM bằng phân nhóm projection, tăng tốc $\sim$10 lần. Phương pháp lặp tích hợp **mô hình hệ thống** (hình học, tán xạ, suy giảm) và **regularization** (TV, bảo cạnh), cho SNR tốt hơn ở liều thấp, nhưng **chậm hơn** và có thể quá mịn. CT/PET hiện đại dùng iterative làm mặc định; tái tạo deep-learning đang nổi.

Câu 22. SENSE và GRAPPA cho MRI song song.
Parallel imaging tăng tốc MRI bằng lấy mẫu thưa k-space và dùng **mảng đa cuộn** (8–64 cuộn), mỗi cuộn có sensitivity không gian biết trước. **SENSE** (SENSitivity Encoding) hoạt động trong miền ảnh: k-space thưa cho ảnh aliased mỗi cuộn; SENSE giải hệ tuyến tính với sensitivity C_i để tách aliasing tại mỗi pixel. **GRAPPA** (GeneRalized Autocalibrating Partial Parallel Acquisition) hoạt động trong k-space: dùng vùng **auto-calibration** đầy đủ để học trọng số tuyến tính tổng hợp các dòng k-space thiếu từ dòng thu kề trên các cuộn. Tăng tốc $R = 2$–4 là thường quy, giảm thời gian quét đổi lấy SNR giảm $\sqrt{R}$.

Câu 23. Compressed sensing trong MRI là gì?
Compressed sensing (CS) tái tạo ảnh MR từ **ít mẫu k-space hơn nhiều** so với Nyquist yêu cầu, khai thác hai sự thật: (1) ảnh MR **thưa** trong miền biến đổi nào đó (wavelet, total variation, sai phân hữu hạn); (2) lấy mẫu ngẫu nhiên/không tương quan trải aliasing thành nhiễu. CS giải:

$$\min_x \|\Psi x\|_1 \quad \text{s.t.} \quad \|F_u x - y\|_2 \leq \epsilon$$

với Ψ là biến đổi sparsifying, F_u Fourier thưa, y k-space đo. Solver lặp phi tuyến (FISTA, ADMM) khôi phục x. CS cho phép tăng tốc 4–10 lần, đặc biệt cho MRI động (tim, cản quang). Kết hợp với parallel imaging và deep learning (variational network), nó nền tảng cho MRI nhanh hiện đại.

Câu 24. Pipeline VTK cho hiển thị y học.
VTK (Visualization Toolkit) dùng pipeline luồng dữ liệu: **Source → Filter → Mapper → Actor → Renderer → RenderWindow**. Với CT: (1) `vtkDICOMImageReader` nạp khối. (2) Filter tiền xử lý: `vtkImageThreshold`, `vtkImageGaussianSmooth`. (3) `vtkMarchingCubes` hoặc `vtkContourFilter` trích iso-surface; *hoặc* `vtkSmartVolumeMapper` ray-cast DVR trên GPU. (4) Tùy chọn `vtkDecimatePro`, `vtkSmoothPolyDataFilter`. (5) `vtkPolyDataMapper` chuyển sang nguyên thủy đồ họa. (6) `vtkActor` thêm vật liệu/màu; `vtkVolume` cho DVR với `vtkColorTransferFunction` và `vtkPiecewiseFunction`. (7) `vtkRenderer` tổng hợp; `vtkRenderWindow` hiển thị; `vtkRenderWindowInteractor` xử lý input. 3D Slicer và ParaView xây trên VTK.

Câu 25. Từ CT sang mesh xương in 3D — pipeline.
(1) **Thu** stack CT DICOM. (2) **Nạp** vào 3D Slicer / ITK-SNAP. (3) **Window/level** sang xương (HU > +200). (4) **Segment** xương bằng ngưỡng (HU > 200–300) cộng **region-growing** hoặc công cụ AI để loại mạch cản quang và bàn. (5) **Dọn hình thái**: closing, lấp lỗ, loại đảo. (6) **Marching Cubes** sinh mesh tam giác ở iso-value. (7) **Smooth** (Laplacian hoặc Taubin) và **decimate** về số tam giác hợp lý. (8) **Sửa**: đảm bảo manifold, watertight, không tự cắt (MeshLab, Meshmixer). (9) **Xuất STL/3MF**. (10) **Slice** cho in (Cura/PrusaSlicer) và in trên FDM/SLA.

Repair: ensure manifold, watertight, no self-intersections (MeshLab, Meshmixer). (9) **Export STL/3MF**. (10) **Slice** for printing (Cura/PrusaSlicer) and print on FDM/SLA. Used for orthopedic implant planning, education, surgical guides.

Q26. How does Marching Cubes help surgical planning?
After CT/MRI segmentation, **Marching Cubes** converts each labeled organ (tumor, vessel, bone) into a **closed triangle mesh**. Surgeons can: (1) view the 3D mesh from any angle, rotating in real time on GPU; (2) measure distances, volumes, angles for resection planning; (3) print physical models for tactile rehearsal; (4) feed meshes into **VR/AR systems** to overlay anatomy on the patient; (5) plan tool trajectories avoiding critical structures. Compared with raw volume display, surface meshes are **lightweight, easy to manipulate, and 3D-print ready**, making MC the workhorse of patient-specific surgical planning.

Q27. How is volume rendering used clinically?
Direct Volume Rendering is used for: (1) **CT angiography** — visualize vessels and stenoses in 3D without segmentation; (2) **Virtual colonoscopy/bronchoscopy** — fly-through of hollow organs; (3) **Maxillofacial / orthopedic planning** — skull and bone overview; (4) **Radiation therapy planning** — target and OAR display; (5) **Patient communication** — intuitive 3D images for consent. Modern GPU ray-casting renders 512^3 volumes at >30 fps. Cinematic rendering (path tracing, Monte Carlo) further improves realism. DVR complements MPR slices by giving spatial context without requiring perfect segmentation.

Q28. Why is real-time reconstruction important in interventional procedures?
Interventional radiology (catheter navigation, biopsy, ablation, surgery under fluoroscopy or MR) needs imaging **updated within milliseconds** so the clinician can react. Specifically: (1) **tool tracking** requires latency < 100 ms; (2) **respiratory/cardiac motion** demands frame rates > 5 Hz; (3) **closed-loop control** (MR-guided focused ultrasound, robotic surgery) needs deterministic sub-frame latency. Slow reconstruction means stale guidance, prolonged radiation/anesthesia, and increased complication risk. Hence **GPU-accelerated FBP, parallel imaging with rapid reconstruction, real-time compressed sensing, and AI-based reconstruction** are critical research areas.

Q29. How is image processing used to detect tumor signals in MRI?
A typical **tumor detection pipeline**: (1) **Multi-sequence acquisition** (T1, T1+Gd, T2, FLAIR, DWI/ADC) gives complementary contrasts. (2) **Preprocessing**: bias-field correction (N4), skull stripping, registration of sequences, intensity normalization. (3) **Feature extraction**: voxel intensities across sequences, texture (GLCM, LBP), gradients, location atlas. (4) **Segmentation** using thresholding, region growing, level sets, or modern **CNN/U-Net** models (BraTS challenge). (5) **Post-processing**: connected components, volume computation, contour overlay. (6) **Radiomics**: extract shape and texture features for prognosis. Output: tumor mask, volume, and similarity to template lesions, helping radiologists detect, classify, and follow tumors over time.

Dùng cho lập kế hoạch implant chỉnh hình, giảng dạy, surgical guide.

Câu 26. Marching Cubes hỗ trợ lập kế hoạch phẫu thuật.
Sau segmentation CT/MRI, **Marching Cubes** chuyển mỗi cơ quan có nhãn (u, mạch, xương) thành **mesh tam giác kín**. Bác sĩ có thể: (1) xem mesh 3D từ mọi góc, xoay thời gian thực trên GPU; (2) đo khoảng cách, thể tích, góc cho lập kế hoạch cắt bỏ; (3) in mô hình vật lý để diễn tập xúc giác; (4) đưa mesh vào **hệ VR/AR** chồng giải phẫu lên bệnh nhân; (5) lập quỹ đạo dụng cụ tránh cấu trúc quan trọng. So với hiển thị khối thô, mesh bề mặt **nhẹ, dễ thao tác, sẵn sàng in 3D**, biến MC thành công cụ chủ lực của lập kế hoạch phẫu thuật cá nhân hóa.

Câu 27. Volume rendering trong lâm sàng.
Direct Volume Rendering dùng cho: (1) **CT angiography** — hiển thị mạch và hẹp 3D không cần segmentation; (2) **Nội soi đại tràng/phế quản ảo** — bay xuyên tạng rỗng; (3) **Lập kế hoạch hàm mặt / chỉnh hình** — tổng quan sọ và xương; (4) **Lập kế hoạch xạ trị** — hiển thị target và OAR; (5) **Giao tiếp với bệnh nhân** — ảnh 3D trực quan để đồng thuận. Ray-cast GPU hiện đại render khối 512^3 ở >30 fps. Cinematic rendering (path tracing, Monte Carlo) tăng độ chân thực. DVR bổ sung cho lát MPR bằng cách cho ngữ cảnh không gian mà không cần segmentation hoàn hảo.

Câu 28. Tại sao tái tạo thời gian thực quan trọng trong can thiệp?
Can thiệp X-quang (đưa ống thông, sinh thiết, đốt, phẫu thuật dưới fluoroscopy hoặc MR) cần ảnh **cập nhật trong mili giây** để bác sĩ phản ứng. Cụ thể: (1) **theo dõi dụng cụ** cần độ trễ < 100 ms; (2) **chuyển động hô hấp/tim** đòi tốc độ khung > 5 Hz; (3) **điều khiển vòng kín** (siêu âm hội tụ dẫn MR, phẫu thuật robot) cần độ trễ dưới khung xác định. Tái tạo chậm nghĩa là dẫn đường lỗi thời, kéo dài thời gian xạ/gây mê, và tăng nguy cơ biến chứng. Vì thế **FBP tăng tốc GPU, parallel imaging tái tạo nhanh, compressed sensing thời gian thực, và tái tạo dựa AI** là hướng nghiên cứu quan trọng.

Câu 29. Xử lý ảnh phát hiện u trên MRI.
Pipeline phát hiện u điển hình: (1) **Thu đa sequence** (T1, T1+Gd, T2, FLAIR, DWI/ADC) cho tương phản bổ sung. (2) **Tiền xử lý**: hiệu chỉnh bias-field (N4), loại sọ, đăng ký các sequence, chuẩn hóa cường độ. (3) **Trích đặc trưng**: cường độ voxel qua các sequence, texture (GLCM, LBP), gradient, atlas vị trí. (4) **Segmentation** bằng ngưỡng, region growing, level set, hoặc mô hình **CNN/U-Net** hiện đại (cuộc thi BraTS). (5) **Hậu xử lý**: thành phần liên thông, tính thể tích, vẽ contour. (6) **Radiomics**: trích đặc trưng hình dạng và texture cho tiên lượng. Đầu ra: mặt nạ u, thể tích, và độ tương đồng với tổn thương mẫu, giúp bác sĩ X-quang phát hiện, phân loại, và theo dõi u theo thời gian.

QA3b. Theoretical Q&A — Segmentation, BraTS & Papers

Q1. End-to-end pipeline from patient to clinical image.
The full chain has **seven stages**: (1) **Acquisition** — patient scanned by CT/MRI/US/PET; raw **k-space** or **sinogram** data captured. (2) **Reconstruction** — inverse Fourier (MRI) or filtered back-projection (CT) builds a volume. (3) **Preprocessing** — **N4 bias correction**, denoising, **intensity normalization** (Nyul), **skull-stripping**. (4) **Registration** — align modalities or atlas to patient space. (5) **Segmentation** — extract organs/lesions (U-Net, nnU-Net). (6) **Quantification** — volume, shape, radiomics features. (7) **Reporting / CDSS** — visualization, DICOM-SR, decision support to clinician. Each step propagates error, so QC at every stage is critical.

Q2. Bias field correction (N4) — why for MRI?
Bias field is a smooth, low-frequency **multiplicative** intensity inhomogeneity caused by **B1 RF coil non-uniformity** in MRI; the same tissue (e.g., white matter) appears brighter on one side of the brain and darker on the other. **N4ITK** (Tustison 2010) is an improved **N3** algorithm that models the log-bias as a **B-spline** surface and iteratively sharpens the histogram via **deconvolution**. Without N4, intensity-based segmentation (Otsu, k-means, even CNNs) fails because the same tissue class has different intensities. N4 is mandatory in BraTS preprocessing. CT does not need it (Hounsfield units are calibrated).

Q3. Image registration — rigid vs affine vs B-spline vs Demons vs LDDMM.
Registration aligns image I to reference J by finding transform T minimizing dissimilarity. **Rigid** (6 DoF: 3 rot + 3 trans) — same patient, same modality, different time. **Affine** (12 DoF) — adds scaling + shear; useful for inter-subject coarse alignment. **B-spline FFD** — non-rigid, control-grid deformation; smooth, invertibility not guaranteed. **Demons** (Thirion) — optical-flow style, fast, no explicit regularizer beyond Gaussian smoothing. **LDDMM** (Large Deformation Diffeomorphic Metric Mapping) — geodesic in space of diffeomorphisms; guarantees **topology preservation** (no folds), used in computational anatomy.

Q4. Similarity metrics in registration.
Choice depends on modality match. **SSD** (sum-of-squared-differences) $\sum(I-J)^2$ — only for same modality, same intensity scale. **NCC** (normalized cross-correlation) — robust to linear intensity changes, used in mono-modal MRI. **Mutual Information** $\mathrm{MI}(I,J) = \sum p(i,j)\log\frac{p(i,j)}{p(i)p(j)}$ — gold standard for **multi-modal** (MRI↔CT, T1↔T2) since it only assumes statistical dependence, not intensity equality. **NMI** (normalized MI) reduces overlap bias. For deep-learning registration (VoxelMorph) we use **NCC + smoothness loss** on the displacement field.

Q5. Common denoising methods for medical images.
Gaussian filter — simple, blurs edges. **Median filter** — removes salt-and-pepper, preserves edges, used on US speckle. **Bilateral filter** — edge-preserving via spatial+intensity kernel. **Anisotropic diffusion** (Perona-Malik) — PDE that diffuses along but not across edges. **Non-Local Means (NLM)** — averages similar patches across the image; gold standard before deep learning. **BM3D** — block-matching + 3D collaborative filtering, near-optimal for

QA3b. Câu hỏi lý thuyết — Phân vùng, BraTS & Bài báo

Câu 1. Pipeline đầy đủ từ bệnh nhân đến ảnh lâm sàng.
Chuỗi gồm **7 bước**: (1) **Acquisition** — bệnh nhân được chụp CT/MRI/US/PET, thu dữ liệu thô **k-space** hoặc **sinogram**. (2) **Reconstruction** — Fourier nghịch (MRI) hay filtered back-projection (CT) dựng volume. (3) **Tiền xử lý** — **N4 bias correction**, khử nhiễu, **intensity normalization** (Nyul), **skull-stripping**. (4) **Registration** — căn chỉnh các modality hoặc atlas về không gian bệnh nhân. (5) **Segmentation** — tách cơ quan / tổn thương (U-Net, nnU-Net). (6) **Định lượng** — thể tích, hình dáng, radiomics. (7) **Báo cáo / CDSS** — visualization, DICOM-SR, hỗ trợ quyết định cho bác sĩ. Mọi bước đều propagate lỗi nên QC ở mỗi tầng là bắt buộc.

Câu 2. N4 bias correction — vì sao MRI cần?
Bias field là độ không đồng nhất cường độ **nhân tính (multiplicative)** tần số thấp do **B1 RF coil** không đồng đều trên MRI; cùng một mô (ví dụ chất trắng) lại sáng bên này, tối bên kia. **N4ITK** (Tustison 2010) cải tiến từ **N3**, mô hình hóa log-bias bằng mặt **B-spline** và lặp deconvolution để histogram sắc nét hơn. Không có N4, segmentation theo cường độ (Otsu, k-means, kể cả CNN) sẽ thất bại vì cùng class lại có cường độ khác nhau. Trong BraTS, N4 là bước bắt buộc. CT không cần (đơn vị Hounsfield đã được hiệu chuẩn).

Câu 3. Registration — rigid, affine, B-spline, Demons, LDDMM.
Registration căn ảnh I về ảnh tham chiếu J bằng cách tìm transform T tối thiểu hóa khoảng cách. **Rigid** (6 DoF: 3 quay + 3 tịnh tiến) — cùng bệnh nhân, cùng modality, theo dõi theo thời gian. **Affine** (12 DoF) — thêm scaling + shear, dùng cho căn chỉnh thô liên-bệnh nhân. **B-spline FFD** — non-rigid, biến dạng theo lưới control point, mượt nhưng không đảm bảo invertible. **Demons** (Thirion) — kiểu optical-flow, nhanh, regularize bằng làm mượt Gaussian. **LDDMM** — geodesic trên không gian diffeomorphism, đảm bảo **bảo toàn topology** (không bị gấp nếp), dùng trong computational anatomy.

Câu 4. Metric tương đồng trong registration.
Tùy modality. **SSD** $\sum(I-J)^2$ — chỉ dùng được cùng modality, cùng thang. **NCC** (normalized cross-correlation) — bền với biến đổi tuyến tính, dùng MRI cùng modality. **Mutual Information** $\mathrm{MI}(I,J) = \sum p(i,j)\log\frac{p(i,j)}{p(i)p(j)}$ — chuẩn vàng cho **multi-modal** (MRI↔CT, T1↔T2) vì chỉ giả định phụ thuộc thống kê, không cần intensity bằng nhau. **NMI** (chuẩn hóa MI) giảm bias do overlap. Với deep registration (VoxelMorph) thường dùng **NCC + smoothness loss** trên trường dịch chuyển.

Câu 5. Phương pháp khử nhiễu phổ biến.
Gaussian — đơn giản, làm mờ cạnh. **Median** — trị nhiễu muối-tiêu, giữ cạnh, dùng cho nhiễu speckle siêu âm. **Bilateral** — giữ cạnh nhờ kernel kết hợp không gian + cường độ. **Anisotropic diffusion** (Perona-Malik) — PDE khuếch tán dọc theo cạnh, không qua cạnh. **Non-Local Means (NLM)** — trung bình các patch tương tự khắp ảnh, là chuẩn vàng tiền-DL. **BM3D** — block-matching + 3D collaborative filter, gần tối ưu với nhiễu Gaussian. **DnCNN / Noise2Noise** — deep learning, học residual nhiễu. MRI cần mô hình **Rician noise**.

Gaussian noise. **DnCNN / Noise2Noise** — deep learning, learns residual noise. For MRI **Rician noise** model is needed.

Q6. Intensity normalization (Nyul) — why across scanners?
MRI intensities are **not calibrated** (unlike CT Hounsfield units): the same brain on a Siemens 3T and a Philips 1.5T can have wildly different intensity ranges. This destroys generalization. **Nyul & Udupa (2000)** normalization picks a set of **histogram landmarks** (e.g., percentiles 1, 10, 20, ..., 99) on a training corpus, computes their mean positions, then **piecewise-linearly maps** every new scan's landmarks to those targets. Alternatives: **z-score** per-volume, **white-stripe** (normalize to white-matter peak), **histogram matching**. Without normalization, a model trained on hospital A fails on hospital B — a major **domain-shift** problem.

Q7. What is segmentation? Why most important step?
Segmentation assigns a label to every voxel: $S : \Omega \to \{0, 1, \ldots, K\}$, partitioning the image into anatomically/pathologically meaningful regions. It is the bridge from pixels to **quantitative biomarkers**: tumor **volume**, growth rate (RECIST), shape descriptors, radiomics. Diagnosis ("is there a tumor?"), treatment planning (radiotherapy contours), surgical guidance, and outcome prediction all require segmentation. Classification only says "yes/no"; segmentation says "where + how big". This is why BraTS, LiTS, KiTS challenges all center on segmentation.

Q8. Otsu thresholding — formula.
Otsu (1979) finds the threshold t^* that **maximizes between-class variance** (equivalent to minimizing within-class variance) over the gray-level histogram. With class probabilities $\omega_0(t), \omega_1(t)$ and means $\mu_0(t), \mu_1(t)$:

$$t^* = \arg\max_t \ \sigma_B^2(t) = \omega_0(t)\,\omega_1(t)\,[\mu_0(t) - \mu_1(t)]^2.$$

It assumes a **bimodal histogram** (foreground + background). Pros: parameter-free, fast $O(L)$. Cons: fails on multi-modal histograms, sensitive to noise and uneven illumination (use **adaptive/local Otsu**). Multi-class extension exists (**multi-Otsu**).

Q9. Region growing.
Region growing starts from one or more **seed pixels** and iteratively absorbs neighbors satisfying a **homogeneity criterion** (e.g., $|I(p) - \mu_R| < \tau$ or gradient-based). It is **semi-automatic** (user picks seed) and intuitive, used in liver/lung segmentation in older PACS tools. Weaknesses: highly seed-sensitive, suffers from **leakage** when boundaries are weak, can't separate touching structures, and threshold τ is hard to tune. Variants: **symmetric region growing**, **statistical region merging**, **confidence connected** (ITK).

Q10. Watershed — over-segmentation & fix.
Watershed treats the gradient image as a topographic surface; "flooding" from local minima creates basins separated by **watershed lines** along ridges. Each basin = one region. Problem: **every local minimum becomes a region** → severe **over-segmentation** on noisy gradients (hundreds of basins). Fixes: (1) **Marker-controlled watershed** — flood only from user/algorithm-supplied markers; (2) **H-minima transform** — suppress shallow minima; (3) **Pre-smoothing** the gradient (Gaussian/bilateral); (4) **Region merging** as post-process based on similarity. Marker-controlled watershed is the standard variant used in cell segmentation.

Q11. Active contours / snakes — energy formula.
A **snake** is a parametric curve $C(s) = (x(s), y(s))$, $s \in [0, 1]$ that evolves to minimize:

$$E_{\text{snake}} = \int_0^1 \Big(\underbrace{\alpha|C'(s)|^2 + \beta|C''(s)|^2}_{E_{\text{int}}} + \underbrace{E_{\text{ext}}(C(s))}_{-|\nabla I|^2} \Big)\, ds.$$

Internal energy: α controls elasticity (resists stretching), β controls rigidity (resists bending). **External energy** pulls the curve toward edges (high $|\nabla I|$). Solved by gradient descent with finite-difference Euler-Lagrange. Weaknesses: needs good initialization, can't change topology, fails in concavities. **GVF snake** (Gradient Vector Flow) extends capture range.

Q12. Level set methods — evolution PDE.
Level set (Osher-Sethian 1988) embeds the contour as the zero level set of a higher-dimensional function $\phi(\mathbf{x}, t)$: $C(t) = \{\mathbf{x} : \phi(\mathbf{x}, t) = 0\}$. Evolution PDE:

$$\frac{\partial \phi}{\partial t} + F\,|\nabla \phi| = 0,$$

where speed F depends on curvature $\kappa = \nabla\cdot(\nabla\phi/|\nabla\phi|)$ and image features. **Chan-Vese** model uses region statistics (mean inside/outside) instead of edges. Big advantage over snakes: **automatic topology change** (split/merge). Cost: solve PDE on whole grid (use **narrow band** or **fast marching** for speedup). Re-initialize ϕ as signed distance periodically.

Q13. U-Net — why standard for medical seg?
U-Net (Ronneberger 2015) is a fully-convolutional **encoder-decoder** with **skip connections** between symmetric levels. Encoder = contracting path (conv-conv-pool, channels double); decoder = expanding path (up-conv, concat skip, conv-conv). Skips inject **high-resolution detail** into deep features → sharp boundaries. Reasons it dominates medical: (1) works with **small datasets** (data augmentation + skips); (2) end-to-end pixel labels; (3) easily extended to 3D (3D U-Net, V-Net); (4) modular — backbone can be swapped (ResNet, EfficientNet, ViT). Variants: Attention U-Net, U-Net++, TransU-Net. The original BraTS 2015 paper already showed U-Net winning.

Q14. V-Net vs U-Net.
V-Net (Milletari 2016) is the **3D** counterpart of U-Net for volumetric data (MRI, CT). Differences: (1) **3D convolutions** ($k \times k \times k$) instead of 2D; (2) **residual connections** inside each block (V-Net came before ResU-Net was popular); (3) introduced the **Dice loss** as primary objective:

$$\mathcal{L}_{\text{Dice}} = 1 - \frac{2\sum p_i g_i}{\sum p_i^2 + \sum g_i^2},$$

Câu 6. Nyul normalization — vì sao cần xuyên scanner?
Cường độ MRI **không được hiệu chuẩn** (khác CT có Hounsfield): cùng não nhưng máy Siemens 3T và Philips 1.5T cho dải cường độ rất khác. Điều này phá vỡ generalization. **Nyul & Udupa (2000)** chọn một tập **landmark** histogram (percentile 1, 10, 20, ..., 99) trên corpus train, tính vị trí trung bình, rồi **ánh xạ piecewise-linear** mỗi scan mới về các landmark đó. Lựa chọn khác: **z-score** từng volume, **white-stripe** (chuẩn theo đỉnh chất trắng), **histogram matching**. Không có normalization, model train ở bệnh viện A sẽ thất bại ở bệnh viện B — bài toán **domain shift** kinh điển.

Câu 7. Segmentation là gì? Vì sao quan trọng nhất?
Segmentation gán nhãn cho từng voxel: $S : \Omega \to \{0, 1, \ldots, K\}$, chia ảnh thành các vùng có ý nghĩa giải phẫu/bệnh lý. Đây là cầu nối từ pixel sang **biomarker định lượng**: **thể tích** u, tốc độ phát triển (RECIST), hình dáng, radiomics. Chẩn đoán ("có u không?"), lập kế hoạch xạ trị (vẽ contour), dẫn hướng phẫu thuật, dự báo tiên lượng — đều cần segmentation. Classification chỉ trả lời "có/không"; segmentation trả lời "ở đâu + lớn cỡ nào". Vì vậy BraTS, LiTS, KiTS đều xoay quanh segmentation.

Câu 8. Otsu — công thức.
Otsu (1979) chọn ngưỡng t^* **tối đa hóa phương sai giữa-lớp** (tương đương tối thiểu phương sai trong-lớp) trên histogram. Với xác suất $\omega_0(t), \omega_1(t)$ và trung bình $\mu_0(t), \mu_1(t)$:

$$t^* = \arg\max_t \ \sigma_B^2(t) = \omega_0(t)\,\omega_1(t)\,[\mu_0(t) - \mu_1(t)]^2.$$

Giả định histogram **lưỡng phương thức** (foreground + background). Ưu: không tham số, $O(L)$. Nhược: thất bại nếu nhiều mode, nhạy với nhiễu và chiếu sáng không đều (dùng **Otsu cục bộ / adaptive**). Có biến thể **multi-Otsu** cho nhiều class.

Câu 9. Region growing.
Region growing bắt đầu từ một (hoặc vài) **pixel mầm** và liên tục hợp nhất các neighbor thoả tiêu chí **đồng nhất** (ví dụ $|I(p) - \mu_R| < \tau$ hay theo gradient). Đây là phương pháp **bán tự động** (người dùng chọn seed), trực quan, từng được dùng trong segment gan/phổi trên các tool PACS đời cũ. Nhược: rất nhạy seed, hay bị **rò rỉ (leakage)** khi biên yếu, không tách được hai cấu trúc dính nhau, ngưỡng τ khó tinh chỉnh. Biến thể: **symmetric region growing**, **statistical region merging**, **confidence connected** (ITK).

Câu 10. Watershed — vì sao over-segment, cách khắc phục.
Watershed coi ảnh gradient là bề mặt địa hình; "ngập nước" từ các cực tiểu cục bộ tạo các bồn chứa, ngăn cách bởi **watershed line**. Mỗi bồn = một vùng. Vấn đề: **mỗi cực tiểu cục bộ thành một vùng → over-segmentation** nghiêm trọng trên gradient nhiễu (hàng trăm vùng). Khắc phục: (1) **Marker-controlled watershed** — chỉ ngập từ marker do người/thuật toán cung cấp; (2) **H-minima transform** — bỏ các cực tiểu nông; (3) **Smooth** gradient trước (Gaussian/bilateral); (4) **Region merging** hậu xử lý. Marker-controlled là biến thể chuẩn trong segmentation tế bào.

Câu 11. Snakes — công thức năng lượng.
Snake là đường tham số $C(s) = (x(s), y(s))$, $s \in [0, 1]$ tiến hoá để tối thiểu:

$$E_{\text{snake}} = \int_0^1 \Big(\underbrace{\alpha|C'(s)|^2 + \beta|C''(s)|^2}_{E_{\text{int}}} + \underbrace{E_{\text{ext}}(C(s))}_{-|\nabla I|^2} \Big)\, ds.$$

Năng lượng nội: α điều khiển **độ đàn hồi**, β điều khiển **độ cứng (chống cong)**. **Năng lượng ngoài** kéo đường về phía cạnh ($|\nabla I|$ lớn). Giải bằng gradient descent qua Euler-Lagrange sai phân hữu hạn. Nhược: cần khởi tạo tốt, không đổi topology, kẹt ở vùng lõm. **GVF snake** mở rộng vùng hấp dẫn.

Câu 12. Level set — PDE tiến hoá.
Level set (Osher-Sethian 1988) nhúng đường viền vào mức 0 của hàm cao chiều $\phi(\mathbf{x}, t)$: $C(t) = \{\mathbf{x} : \phi(\mathbf{x}, t) = 0\}$. PDE tiến hoá:

$$\frac{\partial \phi}{\partial t} + F\,|\nabla \phi| = 0,$$

với tốc độ F phụ thuộc curvature $\kappa = \nabla\cdot(\nabla\phi/|\nabla\phi|)$ và đặc trưng ảnh. **Chan-Vese** dùng thống kê vùng (mean trong/ngoài) thay cạnh. Lợi thế lớn so với snake: **tự động đổi topology** (tách/gộp). Chi phí: giải PDE trên cả lưới (dùng **narrow band** hoặc **fast marching** để tăng tốc). Cần re-init ϕ thành signed distance định kỳ.

Câu 13. U-Net — vì sao chuẩn cho y tế?
U-Net (Ronneberger 2015) là kiến trúc fully-convolutional dạng **encoder-decoder** có **skip connection** đối xứng. Encoder = đường thu hẹp (conv-conv-pool, channel gấp đôi); decoder = đường mở rộng (up-conv, concat skip, conv-conv). Skip "tiêm" **chi tiết độ phân giải cao** vào feature sâu → biên sắc nét. Lý do thống trị y tế: (1) hiệu quả với **dataset nhỏ** (augmentation + skip); (2) end-to-end, ra label per-pixel; (3) dễ mở rộng 3D (3D U-Net, V-Net); (4) backbone modular (ResNet, EfficientNet, ViT). Biến thể: Attention U-Net, U-Net++, TransU-Net. BraTS 2015 đã thấy U-Net thắng.

Câu 14. V-Net khác U-Net.
V-Net (Milletari 2016) là phiên bản **3D** của U-Net cho dữ liệu volumetric (MRI, CT). Khác biệt: (1) **conv 3D** ($k \times k \times k$) thay vì 2D; (2) **residual connection** bên trong mỗi block (V-Net ra trước ResU-Net); (3) đề xuất **Dice loss** làm objective chính:

$$\mathcal{L}_{\text{Dice}} = 1 - \frac{2\sum p_i g_i}{\sum p_i^2 + \sum g_i^2},$$

xử lý trực tiếp **class imbalance** (foreground rất nhỏ); (4) downsampling bằng **strided conv** thay max-pool. V-Net được kiểm chứng đầu tiên trên **MRI tuyến tiền liệt**. Ngày nay nnU-Net thống nhất 2D/3D U-Net/V-Net.

Câu 15. nnU-Net — "no-new-Net".
nnU-Net (Isensee, Nature Methods 2021) là **framework tự cấu hình**, không phải kiến trúc mới. Luận điểm: U-Net cấu hình đúng cách thắng các kiến trúc cầu kỳ trên hầu hết tác vụ y tế. Cho một dataset, nnU-Net tự

which directly handles **class imbalance** (small foreground); (4) downsampling via **strided conv** not max-pool. V-Net was originally validated on **prostate MRI**. Today nnU-Net unifies 2D/3D U-Net/V-Net.

Q15. nnU-Net — "no-new-Net".

nnU-Net (Isensee, 2021, Nature Methods) is a **self-configuring framework**, not a new architecture. The thesis: a properly configured plain U-Net beats fancy custom networks on most medical tasks. Given a dataset, nnU-Net automatically infers **patch size, batch size, spacing, normalization, network depth, augmentation, training schedule** from a "data fingerprint" (median shape, spacing, modality). It produces 2D / 3D-full / 3D-cascade variants and **ensembles** the best. Won 19+ segmentation challenges (BraTS, KiTS, AMOS). Lesson: **good engineering > fancy architecture**. It is the modern baseline you must beat to publish.

Q16. Classical vs deep learning segmentation.

Classical (Otsu, region growing, watershed, snakes, level sets, atlas-based, Random Forest+features): interpretable, no training data, fast, parameter-tunable; but brittle, requires hand-crafted features, fails on heterogeneous tumors. **Deep learning** (U-Net, V-Net, nnU-Net, SAM-Med): end-to-end, state-of-the-art Dice, generalizes within distribution; but needs annotated data (expensive in medicine), black-box, vulnerable to **domain shift**. In practice: classical for QC / coarse ROI / preprocessing (skull strip), DL for the main segmentation. Hybrid pipelines (DL-init + level-set refine) combine both.

Q17. SAM and SAM-Med.

SAM (Segment Anything Model, Meta 2023) is a **foundation model** for segmentation trained on 1B masks (SA-1B). Architecture: ViT-H image encoder + prompt encoder (point/box/mask) + lightweight mask decoder. Promptable, zero-shot on natural images. **Limitation on medical**: trained on natural images → poor on MRI/CT/path. **SAM-Med2D / SAM-Med3D / MedSAM** fine-tune SAM on medical corpora (≈1.5M medical masks). They enable **interactive segmentation** (click a tumor, get a mask), promising for clinical workflows where fully-automatic is risky. Still struggles on small lesions and cross-modality generalization.

Q18. Dice and Jaccard / IoU.

Dice (Sørensen-Dice): $D = \dfrac{2|A \cap B|}{|A| + |B|} = \dfrac{2\,TP}{2\,TP + FP + FN}$. **Jaccard / IoU**: $J = \dfrac{|A \cap B|}{|A \cup B|} = \dfrac{TP}{TP + FP + FN}$. Both range $[0, 1]$, 1=perfect. Relation: $D = \dfrac{2J}{1+J}$ and $J = \dfrac{D}{2-D}$; Dice is always $\geq$ Jaccard. Dice is more forgiving (penalizes errors less). Both are **volume-overlap** metrics — they ignore boundary smoothness. In medicine Dice is the de-facto reporting metric (BraTS, MSD).

Q19. Hausdorff95.

Hausdorff distance $H(A, B) = \max\{\sup_{a \in A} \inf_{b \in B} \|a - b\|,\ \sup_{b \in B} \inf_{a \in A} \|a - b\|\}$ measures the **worst-case boundary error**. It is extremely sensitive to outliers (one stray voxel ruins it). **HD95** = 95th percentile of the same distances → robust to outliers while still capturing boundary quality. Preferred over Dice when **boundary accuracy matters**: radiotherapy contours, surgical planning, cortex/vessel segmentation. Reported in millimeters (so spacing matters). BraTS uses HD95 alongside Dice.

Q20. ASSD.

Average Symmetric Surface Distance:

$$\mathrm{ASSD}(A, B) = \frac{1}{|\partial A| + |\partial B|} \Big(\sum_{a \in \partial A} d(a, \partial B) + \sum_{b \in \partial B} d(b, \partial A) \Big),$$

the mean distance between surface points of prediction and ground truth, in mm. Symmetric (averaged both ways), unlike one-sided versions. More stable than HD95 because it averages over the whole surface rather than taking a high quantile. Used together with Dice and HD95 in BraTS, ACDC, MSD. Lower = better; sub-millimeter ASSD is excellent for organs, 1–3 mm typical for tumors.

Q21. High Dice but poor HD95/ASSD — why?

The metrics measure different things. **Dice** is a global **volume overlap**; if 95% of the voxels are correct Dice is high. **HD95/ASSD** are **boundary** metrics. Failure modes: (1) **Spurious far false positives** — a small predicted blob far from the lesion barely changes Dice but spikes HD95. (2) **Holes inside** the predicted mask — Dice drops slightly, surface distances can grow if topology is wrong. (3) **Jagged boundaries** — Dice insensitive, ASSD increases. Paper 1 (Multi-scale 3D-GAN) shows this paradox on **ET**: Dice up, HD95/ASSD worse → the GAN improves volume but creates boundary noise.

Q22. The BraTS challenge.

BraTS (Brain Tumor Segmentation) is a yearly **MICCAI challenge** (since 2012) on multi-institutional pre-operative MRI of glioma patients, run by U. Penn (CBICA). It distributes hundreds to thousands of pre-processed (co-registered, skull-stripped, 1 mm isotropic, $240 \times 240 \times 155$) multi-modal scans with expert tumor labels. Tasks expanded: segmentation (core), survival prediction, uncertainty, missing-modality (BraTS-2023 has Africa, pediatric, meningioma sub-challenges). It is the **de facto benchmark** for medical 3D segmentation; nnU-Net, Vox2Vox, Swin-UNetR all compete here.

Q23. Four MRI modalities in BraTS.

(1) **T1** — anatomy, fat bright; (2) **T1-Gd / T1ce** — post-gadolinium contrast, **enhancing tumor (ET)** and blood-brain-barrier breakdown light up; (3) **T2** — water/edema bright; (4) **FLAIR** — T2 with CSF suppression, **whole tumor (WT) edema** clearly visible. Multi-modality is essential because each tumor compartment is best seen in a different sequence: **ET**→T1ce, **edema/WT**→FLAIR, **necrosis**→T1, **tumor core**→T1ce+T2. Networks fuse them as 4-channel input.

Q24. WT, TC, ET sub-regions.

BraTS labels are nested: **label 1** necrotic core, **label 2** edema, **label 4** enhancing. Evaluation regions: **WT (Whole Tumor)** = labels $\{1, 2, 4\}$ — everything abnormal, primarily on FLAIR. **TC (Tumor Core)** = $\{1, 4\}$ — solid + necrotic, on T1ce. **ET (Enhancing Tumor)** = $\{4\}$ — actively growing rim on T1ce. ET is the hardest

suy ra **patch size, batch size, spacing, normalization, depth, augmentation, training schedule** từ "data fingerprint" (median shape, spacing, modality). Nó tạo các biến thể 2D / 3D-full / 3D-cascade rồi **ensemble** cái tốt nhất. Thắng 19+ challenge (BraTS, KiTS, AMOS). Bài học: **kỹ thuật tốt > kiến trúc fancy**. Đây là baseline bạn phải đánh bại để công bố.

Câu 16. Cổ điển vs deep learning.

Cổ điển (Otsu, region growing, watershed, snake, level set, atlas, Random Forest+feature): dễ giải thích, không cần data, nhanh, có thể tinh chỉnh; nhưng dễ vỡ, cần feature thủ công, thất bại với u không đồng nhất. **Deep learning** (U-Net, V-Net, nnU-Net, SAM-Med): end-to-end, Dice cao nhất, generalize tốt trong phân phối; nhưng cần dữ liệu nhãn (rất đắt trong y tế), black-box, dễ tổn thương **domain shift**. Thực tế: cổ điển dùng cho QC / khoanh ROI thô / preprocessing (skull strip), DL cho segment chính. Pipeline lai (DL khởi tạo + level-set tinh chỉnh) kết hợp cả hai.

Câu 17. SAM và SAM-Med.

SAM (Segment Anything, Meta 2023) là **foundation model** cho segmentation, train trên 1B mask (SA-1B). Kiến trúc: ViT-H image encoder + prompt encoder (point/box/mask) + mask decoder nhẹ. Promptable, zero-shot trên ảnh tự nhiên. **Hạn chế y tế**: train trên ảnh tự nhiên → kém trên MRI/CT/giải phẫu bệnh. **SAM-Med2D / SAM-Med3D / MedSAM** fine-tune SAM trên corpus y tế (≈1.5M mask). Cho phép **segmentation tương tác** (click vào u → ra mask), hứa hẹn cho workflow lâm sàng nơi tự động hoàn toàn còn rủi ro. Vẫn yếu với tổn thương nhỏ và generalize liên modality.

Câu 18. Dice và Jaccard / IoU.

Dice: $D = \dfrac{2|A \cap B|}{|A| + |B|} = \dfrac{2\,TP}{2\,TP + FP + FN}$. **Jaccard / IoU**: $J = \dfrac{|A \cap B|}{|A \cup B|} = \dfrac{TP}{TP + FP + FN}$. Cả hai $\in [0, 1]$, 1=hoàn hảo. Quan hệ: $D = \dfrac{2J}{1+J}$, $J = \dfrac{D}{2-D}$; Dice luôn $\geq$ Jaccard. Dice "dễ tính" hơn (phạt lỗi nhẹ hơn). Cả hai là **overlap thể tích** — không quan tâm độ mượt biên. Trong y tế Dice là metric báo cáo chuẩn (BraTS, MSD).

Câu 19. Hausdorff95.

Khoảng cách Hausdorff $H(A, B) = \max\{\sup_{a \in A} \inf_{b \in B} \|a - b\|,\ \sup_{b \in B} \inf_{a \in A} \|a - b\|\}$ đo **lỗi biên tệ nhất**. Cực kỳ nhạy với outlier (một voxel lạc cũng làm hỏng). **HD95** = phân vị 95% của cùng các khoảng cách → bền với outlier mà vẫn phản ánh chất lượng biên. Ưu tiên hơn Dice khi **độ chính xác biên** quan trọng: vẽ contour xạ trị, lập kế hoạch phẫu thuật, segment vỏ não/mạch máu. Đơn vị mm (nên spacing quan trọng). BraTS dùng HD95 cùng Dice.

Câu 20. ASSD.

Average Symmetric Surface Distance:

$$\mathrm{ASSD}(A, B) = \frac{1}{|\partial A| + |\partial B|} \Big(\sum_{a \in \partial A} d(a, \partial B) + \sum_{b \in \partial B} d(b, \partial A) \Big),$$

khoảng cách trung bình giữa điểm bề mặt dự đoán và ground truth, đơn vị mm. Đối xứng (trung bình hai chiều). Ổn định hơn HD95 vì lấy trung bình toàn bề mặt, không phải phân vị cao. Dùng cùng Dice và HD95 trong BraTS, ACDC, MSD. Nhỏ = tốt; ASSD dưới 1mm là xuất sắc với cơ quan, 1–3mm là điển hình với u.

Câu 21. Dice cao nhưng HD95/ASSD tệ — vì sao?

Hai metric đo hai thứ khác nhau. **Dice** là **overlap thể tích** toàn cục; 95% voxel đúng là Dice cao. **HD95/ASSD** là metric **biên**. Các kiểu lỗi: (1) **FP nhỏ ở xa** — không ảnh hưởng Dice nhưng đẩy HD95 lên. (2) **Lỗ rỗng bên trong** mask — Dice giảm nhẹ, surface distance tăng nếu topology sai. (3) **Biên răng cưa** — Dice không nhạy, ASSD tăng. Bài 1 (Multi-scale 3D-GAN) cho thấy nghịch lý này trên **ET**: Dice tăng nhưng HD95/ASSD tệ hơn → GAN cải thiện thể tích nhưng tạo nhiễu biên.

Câu 22. BraTS challenge.

BraTS (Brain Tumor Segmentation) là **thử thách MICCAI** hằng năm (từ 2012) trên ảnh MRI tiền phẫu đa-trung-tâm của bệnh nhân glioma, do U. Penn (CBICA) tổ chức. Phát hành hàng trăm đến hàng ngàn ca đã được tiền xử lý (đồng đăng ký, skull-strip, isotropic 1 mm, $240 \times 240 \times 155$) đa modality kèm nhãn chuyên gia. Mở rộng dần: segmentation (lõi), dự báo sống, uncertainty, missing-modality (BraTS-2023 có sub-challenge châu Phi, nhi khoa, meningioma). Đây là **benchmark de facto** cho segmentation 3D y tế; nnU-Net, Vox2Vox, Swin-UNetR đều cạnh tranh ở đây.

Câu 23. Bốn modality MRI trong BraTS.

(1) **T1** — giải phẫu, mỡ sáng; (2) **T1-Gd / T1ce** — sau tiêm gadolinium, **enhancing tumor (ET)** và phá vỡ hàng rào máu não bất thuốc; (3) **T2** — nước/phù sáng; (4) **FLAIR** — T2 ức chế CSF, **whole tumor (WT) / phù** hiện rõ. Đa modality cần thiết vì mỗi vùng u rõ nhất ở một sequence: **ET**→T1ce, **phù/WT**→FLAIR, **hoại tử**→T1, **lõi u**→T1ce+T2. Mạng feed 4 channel.

Câu 24. WT, TC, ET.

Nhãn BraTS lồng nhau: **label 1** hoại tử, **label 2** phù, **label 4** enhancing. Vùng đánh giá: **WT (Whole Tumor)** = $\{1, 2, 4\}$ — toàn bộ bất thường, chủ yếu trên FLAIR. **TC (Tumor Core)** = $\{1, 4\}$ — phần đặc + hoại tử, trên T1ce. **ET (Enhancing Tumor)** = $\{4\}$ — viền đang phát triển trên T1ce. ET khó nhất (nhỏ, không đều), WT dễ nhất. Báo cáo Dice/HD95 từng vùng rồi trung bình. Phẫu thuật và xạ trị dựa trên các vùng này.

Câu 25. Pipeline chuẩn segment u não.

(1) **Chụp**: 4 modality (T1, T1ce, T2, FLAIR). (2) **Đồng đăng ký** về T1 (rigid). (3) **Resample** về isotropic 1 mm. (4) **N4 bias correction** từng modality. (5) **Skull-strip** (HD-BET / ROBEX). (6) **Chuẩn hóa cường độ** (z-score trong mask não). (7) Stack 4 channel, đẩy vào **3D U-Net / nnU-Net**. (8) **Hậu xử lý**: connected-component, bỏ các đảo nhỏ, threshold ET nếu quá ít voxel (mẹo BraTS: zero ET nếu nhỏ). (9) Tính Dice/HD95 cho WT/TC/ET. (10) Visualize overlay cho bác sĩ X quang QC.

Câu 26. Skull stripping.

(smallest, irregular), WT the easiest. Reported metrics are Dice/HD95 per region, then averaged. Surgical and radiotherapy planning rely on these sub-regions.

Q25. Standard pipeline for brain tumor segmentation.
(1) **Acquisition**: 4 modalities (T1, T1ce, T2, FLAIR). (2) **Co-register** to T1 reference (rigid). (3) **Resample** to 1 mm isotropic. (4) **N4 bias correction** on each modality. (5) **Skull-strip** (HD-BET / ROBEX). (6) **Intensity normalize** (z-score within brain mask). (7) Stack 4 channels, feed into **3D U-Net / nnU-Net**. (8) **Post-process**: connected-component, remove small islands, threshold ET if too few voxels (BraTS trick: zero out ET if <small count). (9) Compute Dice/HD95 per WT/TC/ET. (10) Visualize overlays for radiologist QC.

Q26. Skull stripping.
Skull stripping (brain extraction) removes non-brain tissue (skull, scalp, eyes, neck) keeping only brain parenchyma. Needed because (a) non-brain has high T1 fat that confuses normalization, (b) reduces ROI for the network. Classical: **BET** (FSL, deformable surface), **BSE** (BrainSuite, edge+morphology), **ROBEX** (random forest + active shape), **3dSkullStrip** (AFNI). Deep: **HD-BET** (DKFZ, U-Net trained on multi-modal MRI) — current SOTA, robust to pathologies. BraTS scans come pre-stripped using HD-BET.

Q27. End-to-end tumor detection in MRI.
(1) Patient gets multi-modal MRI (T1, T1ce, T2, FLAIR). (2) DICOM → NIfTI conversion (`dcm2niix`). (3) Preprocessing: **co-register** all modalities, **N4 bias correction**, **skull-strip** (HD-BET), **intensity normalize** (z-score). (4) Run **3D nnU-Net** (or Multi-scale 3D-GAN) trained on BraTS, output 4-class voxel-wise probability. (5) Argmax + connected-component cleanup. (6) Compute volumes for WT/TC/ET, generate **3D mesh** (marching cubes), overlay on MRI in viewer (3D Slicer). (7) Radiologist reviews; **XAI heatmap** (Grad-CAM) shows where the network looked. (8) Report exported as DICOM-SR back to PACS for clinical use.

Q28. Signals detectable in MRI.
MRI distinguishes tissues by **T1, T2, proton density, diffusion, perfusion**. Detectable abnormalities: (1) **Tumor** — mass effect, contrast enhancement on T1ce; (2) **Edema** — bright on T2/FLAIR around lesions; (3) **Hemorrhage** — variable depending on age (acute = T2 dark, chronic = T1 bright then T2* bloom); (4) **Infarct (stroke)** — bright on **DWI**, dark on ADC, acutely; (5) **Demyelination** (MS) — periventricular FLAIR plaques; (6) **Abscess** — ring-enhancing with restricted diffusion inside; (7) **Microbleeds** — **SWI** blooming; (8) **Functional activity** — BOLD fMRI; (9) **Tracts** — DTI tractography.

Q29. Paper 1 summary (Multi-scale 3D-GAN, BraTS 2021).
Problem: brain-tumor segmentation on BraTS 2021. **Architecture**: Generator = **3D Residual U-Net** + a novel **Bridge module** that aggregates multi-scale features before the Discriminator; Discriminator = standard 3D CNN. **Algorithm 1**: dynamically adjusts the number of **D-epochs** per G-epoch using the **linear-regression slope** of adversarial loss → keeps G/D balanced. **Loss**: **Dice + MSE adversarial**. **Results**: Dice ET 82.3, WT 88.6, TC 83.1, beating U-Net, V-Net, VoxelGAN, Vox2Vox baselines. **Contribution**: GAN with multi-scale Bridge + adaptive D-training.

Q30. Bridge module — why useful?
The **Bridge** sits between Generator's multi-scale feature pyramid and the Discriminator. It **aggregates features from multiple resolutions** (deep semantic + shallow texture) before D evaluates real-vs-fake. Benefit: D no longer judges only the final mask but also **intermediate scales**, forcing G to produce masks consistent across scales — helpful for tumors of varying size. It is conceptually similar to **PatchGAN with FPN**. Weakness in the paper: the design lacks a dedicated figure and there is **no ablation** isolating its contribution from the GAN baseline.

Q31. Algorithm 1 — dynamic D-epoch adjustment.
Classical GAN training is unstable: if D becomes too strong, gradients to G vanish; too weak, G diverges. Algorithm 1 monitors the **adversarial loss** of D over a sliding window of l recent epochs and computes the **linear-regression slope** k. If the slope is too negative (D rapidly winning, loss dropping fast) → reduce D-epochs. If slope too positive or near zero (D struggling) → increase D-epochs. A threshold β separates cases. Net effect: automatic G–D balance without manual scheduling. Paper provides no theoretical justification for the linear-regression choice and no sensitivity study on k, l, β.

Q32. Loss — Dice + MSE, why combine?
The total loss is $\mathcal{L} = \mathcal{L}_{\text{Dice}}(G(x), y) + \lambda\, \mathcal{L}_{\text{MSE-adv}}(D(G(x)), 1)$. **Dice loss** directly optimizes overlap and handles **class imbalance** (tumor $\ll$ background). **MSE adversarial** (LSGAN-style) replaces the original cross-entropy GAN loss and gives **smoother gradients**, fewer vanishing-gradient problems, more stable training. Combining: Dice ensures voxel correctness, adversarial term forces **globally realistic shapes** (label distribution similar to real masks). Trade-off: GAN may improve Dice but degrade boundary metrics (the ET paradox).

Q33. Weaknesses of Paper 1 & improvements.
(1) Evaluation only on a **validation split** of training data, no submission to the BraTS **online server** → not directly comparable to leaderboard. (2) **No cross-validation** (no mean±std). (3) **No ablation** for Bridge or Algorithm 1 individually. (4) Unexplained **HD95/ASSD paradox** on ET. (5) Hyperparameters k, l, β not investigated. (6) No model size, FLOPs, training/inference time. (7) No statistical significance test. (8) No code release. **Fixes**: 5-fold CV with std, ablation grid, BraTS-server submission, hyperparameter sensitivity, code release on GitHub, parameter / FLOPs table, paired Wilcoxon test.

Q34. HD95/ASSD paradox on ET — why an issue?
Multi-scale 3D-GAN reports **higher Dice** than the ResU-Net (no-GAN) variant on ET, yet **worse HD95** (9.6mm vs 6.4mm) and **worse ASSD** (3.37 vs 2.35). This means the GAN **adds correctly-labeled voxels** (raising overlap) but also produces **boundary noise / spurious far blobs** (worsening surface distance). For **enhancing tumor**, where boundaries directly inform **neurosurgical resection margins** and **radiotherapy GTV**, boundary errors are clinically dangerous. The paper should have analyzed this — perhaps the MSE adversarial term over-fits to global mass and hallucinates rim voxels. Mitigation: post-process with connected-component, add boundary-aware

Skull stripping (brain extraction) loại bỏ mô không phải não (sọ, da đầu, mắt, cổ), chỉ giữ nhu mô não. Cần thiết vì (a) mỡ ngoài não T1 cao gây nhiễu normalization, (b) thu hẹp ROI cho mạng. Cổ điển: **BET** (FSL, mặt biến dạng), **BSE** (BrainSuite, cạnh + morphology), **ROBEX** (random forest + active shape), **3dSkullStrip** (AFNI). Deep: **HD-BET** (DKFZ, U-Net trên đa modality MRI) — SOTA hiện nay, bền với bệnh lý. Dữ liệu BraTS đã được skull-strip sẵn bằng HD-BET.

Câu 27. Quy trình end-to-end phát hiện u não.
(1) Bệnh nhân chụp MRI đa modality (T1, T1ce, T2, FLAIR). (2) DICOM → NIfTI (`dcm2niix`). (3) Tiền xử lý: **đồng đăng ký** các modality, **N4**, **skull-strip** (HD-BET), **chuẩn hóa cường độ** (z-score). (4) Chạy **3D nnU-Net** (hoặc Multi-scale 3D-GAN) đã train BraTS, ra xác suất 4 lớp per-voxel. (5) Argmax + clean connected-component. (6) Tính thể tích WT/TC/ET, dựng **lưới 3D** bằng marching cubes, overlay lên MRI ở viewer (3D Slicer). (7) Bác sĩ duyệt; **XAI heatmap** (Grad-CAM) cho thấy mạng đã "nhìn" đâu. (8) Báo cáo xuất DICOM-SR trở lại PACS để dùng lâm sàng.

Câu 28. Tín hiệu MRI có thể phát hiện.
MRI phân biệt mô bằng **T1, T2, mật độ proton, khuếch tán, tưới máu**. Các bất thường: (1) **U** — khối choáng chỗ, bắt thuốc T1ce; (2) **Phù** — sáng T2/FLAIR quanh tổn thương; (3) **Xuất huyết** — biến đổi theo tuổi (cấp = T2 tối, mãn = T1 sáng rồi T2* "bloom"); (4) **Nhồi máu (đột quỵ)** — sáng **DWI**, tối ADC, ở giai đoạn cấp; (5) **Mất myelin** (MS) — mảng FLAIR cạnh não thất; (6) **Áp xe** — viền bắt thuốc với DWI hạn chế bên trong; (7) **Vi xuất huyết** — **SWI** bloom; (8) **Hoạt động chức năng** — fMRI BOLD; (9) **Sợi trục** — DTI tractography.

Câu 29. Tóm tắt Bài 1 (Multi-scale 3D-GAN, BraTS 2021).
Bài toán: segment u não trên BraTS 2021. **Kiến trúc**: Generator = **3D Residual U-Net** + module **Bridge** mới gộp đặc trưng đa-tỷ-lệ trước khi đưa vào Discriminator; D là CNN 3D chuẩn. **Algorithm 1**: tự điều chỉnh số **epoch của D** theo **độ dốc hồi quy tuyến tính** của adversarial loss → giữ G/D cân bằng. **Loss**: **Dice + MSE adversarial**. **Kết quả**: Dice ET 82.3, WT 88.6, TC 83.1, vượt U-Net, V-Net, VoxelGAN, Vox2Vox. **Đóng góp**: GAN có Bridge đa tỷ lệ + huấn luyện D thích ứng.

Câu 30. Module Bridge — vì sao hữu ích?
Bridge đặt giữa kim tự tháp đặc trưng đa tỷ lệ của G và Discriminator. Nó **gộp đặc trưng nhiều độ phân giải** (ngữ nghĩa sâu + texture nông) trước khi D đánh giá real-vs-fake. Lợi: D không chỉ đánh giá mask cuối mà cả **tỷ lệ trung gian**, ép G tạo mask nhất quán xuyên tỷ lệ — hữu ích cho u kích thước đa dạng. Về ý tưởng giống **PatchGAN với FPN**. Yếu điểm bài viết: thiếu hình minh họa riêng và **không có ablation** tách đóng góp Bridge khỏi GAN baseline.

Câu 31. Algorithm 1 — điều chỉnh epoch D động.
GAN train không ổn: D quá mạnh → gradient đến G tiêu biến; D quá yếu → G phân kỳ. Algorithm 1 theo dõi **adversarial loss** của D trên cửa sổ trượt l epoch gần nhất, tính **độ dốc hồi quy tuyến tính** k. Nếu k âm sâu (D thắng nhanh, loss giảm mạnh) → giảm số epoch D. Nếu k dương hoặc gần 0 (D đang đuối) → tăng epoch D. Ngưỡng β phân biệt. Hệ quả: tự động cân bằng G–D không cần lịch tay. Bài thiếu **biện minh lý thuyết** cho việc dùng hồi quy tuyến tính và không khảo sát độ nhạy k, l, β.

Câu 32. Loss Dice + MSE — vì sao kết hợp?
Tổng loss $\mathcal{L} = \mathcal{L}_{\text{Dice}}(G(x), y) + \lambda\, \mathcal{L}_{\text{MSE-adv}}(D(G(x)), 1)$. **Dice loss** tối ưu trực tiếp overlap và xử lý **class imbalance** (u $\ll$ background). **MSE adversarial** (kiểu LSGAN) thay cross-entropy GAN gốc, cho **gradient mượt hơn**, ít vanishing-gradient, train ổn định. Kết hợp: Dice đảm bảo voxel đúng, term adversarial ép **hình dạng tổng thể realistic** (phân phối nhãn giống mask thật). Đánh đổi: GAN có thể tăng Dice nhưng làm xấu metric biên (nghịch lý ET).

Câu 33. Yếu điểm Bài 1 & cải thiện.
(1) Đánh giá chỉ trên **validation split** của tập train, không submit **server BraTS chính thức** → không so sánh trực tiếp leaderboard. (2) **Không cross-validation** (không có mean±std). (3) **Không ablation** riêng Bridge và Algorithm 1. (4) **Nghịch lý HD95/ASSD** trên ET không được giải thích. (5) Hyperparameter k, l, β chưa khảo sát. (6) Không báo cáo size model, FLOPs, thời gian train/inference. (7) Không kiểm định ý nghĩa thống kê. (8) Không công bố code. **Sửa**: 5-fold CV với std, lưới ablation, submit server BraTS, khảo sát độ nhạy hyperparameter, code GitHub, bảng tham số / FLOPs, kiểm định Wilcoxon ghép cặp.

Câu 34. Nghịch lý HD95/ASSD trên ET — vì sao quan trọng?
Multi-scale 3D-GAN báo **Dice cao hơn** ResU-Net (không GAN) trên ET, nhưng **HD95 tệ hơn** (9.6mm vs 6.4mm) và **ASSD tệ hơn** (3.37 vs 2.35). Nghĩa là GAN **thêm đúng voxel** (tăng overlap) nhưng cũng tạo **nhiễu biên / đốm xa giả**. Với **enhancing tumor**, biên trực tiếp xác định **ranh giới cắt phẫu thuật** và **GTV xạ trị** → lỗi biên nguy hiểm về lâm sàng. Bài đáng lẽ phải phân tích — có thể MSE adversarial overfit khối tổng thể và "ảo giác" voxel viền. Giảm thiểu: hậu xử lý connected-component, thêm boundary-aware loss (Boundary loss, HD loss).

Câu 35. Tóm tắt Bài 2 (review ANN→DCNN, 2024).
Luận điểm: "paradigm shift" từ **ANN cổ điển (feature thủ công)** sang **deep CNN (học feature end-to-end)** trong xử lý ảnh y tế. Lập luận chính: (i) pipeline cũ = trích đặc trưng thủ công (texture, hình dạng, GLCM) + ANN classifier; (ii) pipeline mới = ảnh thô → DCNN **tự học** feature; (iii) Hai bảng tổng hợp so sánh nghiên cứu ANN và DCNN; cột "feature extraction" **biến mất** ở DCNN. (iv) Thừa nhận hạn chế DCNN: cần dữ liệu lớn, dễ overfit, black-box. Bài chủ yếu mang tính **nhập môn / sư phạm**.

Câu 36. Điểm mạnh ví dụ Sobel của Bài 2.
Bài đi qua một phép **tích chập số cụ thể** giữa patch ảnh nhỏ và kernel **Sobel**, áp lên ảnh X-quang bàn tay. Tính từng bước trên giá trị thật, sau đó hiển thị kết quả là bản đồ cạnh. Đây là **rất tốt cho dạy học**: người mới thấy chính xác **convolution làm gì**, vì sao lớp đầu CNN học các filter giống cạnh, và feature extraction "tự nhiên" xuất hiện ra sao. Cầu nối từ trừu tượng ("CNN học đặc trưng") sang cụ thể (toán ma trận) — khoảng trống thường gặp ở các paper review.

Câu 37. Yếu điểm Bài 2.

loss (Boundary loss, HD loss).

Q35. Paper 2 summary (ANN→DCNN review, 2024).

Thesis: a "paradigm shift" from **classic ANNs (with hand-crafted features)** to **deep CNNs (end-to-end feature learning)** in medical image processing. Key arguments: (i) old pipeline = manual feature extraction (texture, shape, GLCM) + ANN classifier; (ii) new pipeline = raw images → DCNN that **learns** features. (iii) Two summary tables compare ANN vs DCNN studies; the "feature extraction" column disappears for DCNN. (iv) Acknowledges DCNN limitations: data-hungry, overfitting, black-box. The paper is mostly **introductory / pedagogical**.

Q36. Strength of Paper 2's Sobel kernel example.

The paper walks through a **concrete numerical convolution** between a small image patch and a **Sobel** edge kernel, applied to a hand X-ray. It is computed step by step on actual values, then visualized as the produced edge map. This is excellent **didactically**: a beginner sees exactly **what convolution does**, why a CNN's first layers learn edge-like filters, and how feature extraction emerges automatically. It bridges the abstract ("CNN learns features") with the concrete (matrix math), a common gap in survey papers.

Q37. Weaknesses of Paper 2.

(1) Published 2024 but **ignores Vision Transformers, SAM, Med-PaLM, LLaVA-Med** — the actual current paradigm shift. (2) Focuses on **classification**, almost no **segmentation** (no U-Net / nnU-Net) and no key datasets (BraTS, LIDC, CheXpert). (3) English/Western-only, **no low-resource languages**, no Vietnamese-medical-NLP discussion, no medical knowledge graph. (4) No **XAI** or **uncertainty quantification** (Grad-CAM, conformal prediction, Dempster-Shafer) despite EU AI Act high-risk classification. (5) No **PRISMA** review methodology, no inclusion/exclusion criteria, no positioning vs Litjens 2017 / Esteva 2019. (6) Long historical 1980s–90s section, short on current CNN mechanisms; AI/ML/DL definition boxes too basic for a Q1 journal.

Q38. Why ignoring ViT and SAM in 2024 is fatal.

By 2024 the field had clearly moved beyond plain CNNs. **Vision Transformers** (ViT, Swin, MaxViT) and hybrid CNN-Transformers (TransU-Net, Swin-UNetR) dominate medical leaderboards. **Foundation models** (SAM, MedSAM, Med-PaLM-2, LLaVA-Med) are reshaping how we think about training: pretrain-once, prompt-many. Calling ANN→DCNN the "paradigm shift" of 2024 is anachronistic — the actual shift is **DCNN → Transformers / foundation models**. A review missing this is **out of date on arrival** and gives readers a wrong mental model of where to invest research effort.

Q39. Missing: low-resource languages & XAI.

The paper gives **zero discussion** of medical AI in non-English settings — no Vietnamese, no Bahasa, no Hindi, no Arabic. For Vietnamese CDSS this is a critical gap: lack of labeled corpora, code-mixed Vietnamese–English–Latin terminology, no Vietnamese medical KG, and ICD-10 terms often loaned. Equally absent: **XAI** — Grad-CAM, integrated gradients, SHAP for radiomics, conformal prediction for trustworthy intervals. The **EU AI Act** classifies CDSS as **high-risk**, mandating explainability and uncertainty. A 2024 medical-AI review without these is incomplete from both technical and regulatory angles.

Q40. Overall takeaway from both papers.

Paper 1 (3D-GAN) shows the **technical-research** pattern: novel idea (Bridge + adaptive D-epochs), competitive Dice, but **rigor gaps** (no CV, no ablation, no server-validation, paradox unexplained). Paper 2 (ANN→DCNN) shows the **survey** pitfall: pedagogically good intro, but **misses the current frontier** (ViT, foundation models, XAI, low-resource). For my own thesis on **Vietnamese CDSS**, lessons are: (i) always benchmark on official servers with CV, ablations, and code release; (ii) cover modern paradigms (Transformers, SAM-Med); (iii) explicitly address **low-resource Vietnamese** medical NLP and **XAI/uncertainty** for high-risk classification under EU AI Act / FDA.

Q41. How image processing supports diagnosis.

Image processing turns raw scans into **quantitative, reproducible biomarkers** that augment human reading. (1) **Detection** — CADe systems flag suspicious lesions (lung nodules, microcalcifications). (2) **Segmentation** — gives volume, shape, growth-rate (RECIST). (3) **Classification** — benign vs malignant, grading. (4) **Registration / longitudinal** — same patient over time, quantify change. (5) **Radiomics** — hundreds of texture/shape features feed survival models. (6) **Decision support** — fuse imaging with EHR, labs, genomics for personalized treatment. The radiologist **stays in the loop**; AI is a second reader, not a replacement.

Q42. End-to-end CDSS pipeline.

(1) **Data ingestion**: DICOM imaging from PACS, EHR (HL7/FHIR), labs, prescriptions, pathology, genomics. (2) **Preprocessing**: registration, normalization, NLP on free-text Vietnamese notes. (3) **Feature extraction**: imaging biomarkers (segmentation → volumetrics, radiomics) + structured features. (4) **Model inference**: multi-modal model (e.g., imaging-CNN + tabular MLP + text BERT) outputs risk scores or differentials. (5) **XAI overlay**: Grad-CAM heatmap, SHAP for tabular, citations for text. (6) **Uncertainty**: conformal prediction set, calibrated probability. (7) **Recommendation**: ranked diagnoses + suggested next test, CPG-aligned. (8) **Clinician UI**: integrated into EMR, one-click accept/override, audit logged. (9) **Feedback loop**: outcomes captured, used to retrain.

Q43. XAI in medical AI — methods.

XAI (eXplainable AI) makes black-box models interpretable; mandatory in medical AI for trust, debugging, regulation. Key methods: (1) **Grad-CAM** — gradient-weighted class activation maps; visual heatmaps over CT/MRI of "where the model looked". (2) **Integrated Gradients / SmoothGrad** — pixel attributions. (3) **SHAP / LIME** — model-agnostic feature importance, common for tabular EHR. (4) **Attention rollouts** for ViT. (5) **Conformal prediction** — distribution-free, gives **prediction sets with calibrated coverage** (e.g., 90% set "this contains the true diagnosis"). (6) **Dempster–Shafer** — combine evidence from multi-source. Required by **EU AI Act** for high-risk CDSS.

(1) Xuất bản 2024 nhưng **bỏ qua Vision Transformer, SAM, Med-PaLM, LLaVA-Med** — đúng paradigm shift hiện nay. (2) Chủ yếu **classification**, gần như không có **segmentation** (không U-Net / nnU-Net) và không nhắc dataset chính (BraTS, LIDC, CheXpert). (3) Chỉ nói tiếng Anh / phương Tây, **không đề cập low-resource language**, không nói NLP y tế tiếng Việt, không có medical knowledge graph. (4) Không **XAI** hay **uncertainty** (Grad-CAM, conformal prediction, Dempster-Shafer) dù EU AI Act xếp CDSS là high-risk. (5) Không **PRISMA**, không tiêu chí lựa chọn, không định vị so với Litjens 2017 / Esteva 2019. (6) Phần lịch sử 1980s–90s dài, phần cơ chế CNN ngắn; box định nghĩa AI/ML/DL quá cơ bản với tạp chí Q1.

Câu 38. Vì sao bỏ qua ViT, SAM năm 2024 là sai lầm chí mạng.

Đến 2024 ngành đã rõ ràng vượt khỏi CNN thuần. **Vision Transformer** (ViT, Swin, MaxViT) và CNN-Transformer lai (TransU-Net, Swin-UNetR) thống trị bảng xếp hạng y tế. **Foundation model** (SAM, MedSAM, Med-PaLM-2, LLaVA-Med) đang định hình lại cách train: pretrain một lần, prompt nhiều lần. Gọi ANN→DCNN là "paradigm shift" của 2024 là lạc hậu — dịch chuyển thực sự đang là **DCNN → Transformer / foundation model**. Một review thiếu phần này là **lỗi thời ngay khi xuất bản** và đưa ra mô hình tư duy sai cho người đọc về nơi nên đầu tư nghiên cứu.

Câu 39. Thiếu sót: low-resource language & XAI.

Bài viết **không hề bàn** về AI y tế ngoài bối cảnh tiếng Anh — không Việt, không Bahasa, không Hindi, không Ả-rập. Với CDSS tiếng Việt đây là khoảng trống chí tử: thiếu corpus có nhãn, thuật ngữ pha trộn Việt-Anh-Latin, không có KG y tế tiếng Việt, ICD-10 thường vay mượn. Tương tự thiếu: **XAI** — Grad-CAM, integrated gradients, SHAP cho radiomics, conformal prediction cho khoảng tin cậy. **EU AI Act** xếp CDSS là **high-risk**, bắt buộc explainability và uncertainty. Một review y-tế-AI 2024 thiếu các phần này là không đầy đủ cả về kỹ thuật lẫn pháp lý.

Câu 40. Bài học chung từ hai bài.

Bài 1 (3D-GAN) cho thấy mẫu của **nghiên cứu kỹ thuật**: ý tưởng mới (Bridge + epoch D thích ứng), Dice cạnh tranh, nhưng **thiếu chặt chẽ** (không CV, không ablation, không server-validate, không giải thích nghịch lý). Bài 2 (ANN→DCNN) cho thấy cạm bẫy của **survey**: nhập môn tốt, nhưng **lỡ biên giới hiện tại** (ViT, foundation model, XAI, low-resource). Với luận văn của tôi về **CDSS tiếng Việt**, bài học: (i) luôn benchmark trên server chính thức kèm CV, ablation, code release; (ii) bao phủ paradigm hiện đại (Transformer, SAM-Med); (iii) bàn rõ **NLP y tế tiếng Việt low-resource** và **XAI/uncertainty** cho high-risk theo EU AI Act / FDA.

Câu 41. Image processing hỗ trợ chẩn đoán thế nào?

Image processing biến scan thô thành **biomarker định lượng, có thể tái lập**, bổ sung cho mắt người. (1) **Phát hiện** — CADe đánh dấu tổn thương (nốt phổi, vi vôi hóa). (2) **Segmentation** — cho thể tích, hình dáng, tốc độ phát triển (RECIST). (3) **Classification** — lành/ác, độ ác. (4) **Registration / dọc thời gian** — cùng bệnh nhân nhiều thời điểm, định lượng thay đổi. (5) **Radiomics** — hàng trăm đặc trưng texture/hình dáng vào model sống còn. (6) **Hỗ trợ quyết định** — fuse imaging với EHR, lab, gen cho điều trị cá nhân hoá. Bác sĩ **vẫn ở trong vòng**; AI là người đọc thứ hai, không thay thế.

Câu 42. Pipeline CDSS end-to-end.

(1) **Thu nhận dữ liệu**: ảnh DICOM từ PACS, EHR (HL7/FHIR), lab, đơn thuốc, giải phẫu bệnh, gen. (2) **Tiền xử lý**: registration, normalization, NLP trên văn bản tiếng Việt. (3) **Trích đặc trưng**: biomarker ảnh (segment → thể tích, radiomics) + đặc trưng cấu trúc. (4) **Inference**: model đa modality (imaging-CNN + tabular MLP + text BERT) ra điểm rủi ro hoặc chẩn đoán phân biệt. (5) **Lớp XAI**: Grad-CAM heatmap, SHAP cho tabular, trích dẫn cho text. (6) **Uncertainty**: tập conformal, xác suất hiệu chuẩn. (7) **Khuyến nghị**: chẩn đoán xếp hạng + xét nghiệm tiếp theo, theo CPG. (8) **Giao diện bác sĩ**: tích hợp EMR, một-cú-click chấp nhận / phản hồi, có log audit. (9) **Vòng phản hồi**: kết quả thực được ghi nhận, dùng retrain.

Câu 43. XAI trong AI y tế.

XAI (eXplainable AI) làm cho model black-box dễ hiểu; bắt buộc trong AI y tế để tin cậy, debug, tuân thủ. Phương pháp chính: (1) **Grad-CAM** — bản đồ kích hoạt theo gradient; heatmap "vùng mạng nhìn" trên CT/MRI. (2) **Integrated Gradients / SmoothGrad** — quy gán pixel. (3) **SHAP / LIME** — model-agnostic, phổ biến cho EHR tabular. (4) **Attention rollout** cho ViT. (5) **Conformal prediction** — không cần phân phối, cho **tập dự đoán có coverage hiệu chuẩn** (ví dụ: tập 90% "chứa chẩn đoán đúng"). (6) **Dempster–Shafer** — kết hợp bằng chứng đa nguồn. **EU AI Act** bắt buộc cho CDSS high-risk.

Câu 44. Khung pháp lý cho AI y tế.

Mỹ: **FDA 510(k)** (tương đương predicate device), **De Novo**, **PMA** (rủi ro cao nhất), guidance **Software as a Medical Device (SaMD)**, và mới có **Predetermined Change Control Plan (PCCP)** cho cập nhật AI. **Châu Âu**: **CE marking theo MDR 2017/745**, phân lớp I/IIa/IIb/III. **EU AI Act** (2024) xếp đa số CDSS là **high-risk**, bắt buộc quản lý rủi ro, quản trị dữ liệu, minh bạch, giám sát con người, độ chính xác, tính bền, an ninh mạng, giám sát hậu thị trường. **ISO 13485** (QMS), **IEC 62304** (vòng đời phần mềm), **ISO 14971** (rủi ro). Việt Nam dựa Thông tư 30/2015 BYT và đang dần áp dụng các ISO.

Câu 45. Thách thức triển khai AI y tế ở Việt Nam.

(1) **Khan hiếm dữ liệu**: ít corpus y tế tiếng Việt có nhãn công khai; PACS bị khoá theo bệnh viện. (2) **Ngôn ngữ**: bệnh án trộn **Việt-Anh-Latin** (ví dụ "U não thể glioblastoma multiforme grade IV"); chưa có KG y tế tiếng Việt phổ biến; ánh xạ ICD-10 không nhất quán. (3) **Hạ tầng**: PACS cũ, EMR phân mảnh, GPU thiếu ở bệnh viện tỉnh. (4) **Domain shift**: model train ở nước ngoài (BraTS, ImageNet) kém trên máy quét VN và dân số VN. (5) **Pháp lý**: chưa có hành lang AI y tế rõ ràng; dựa Thông tư 30/2015 & ISO. (6) **Riêng tư**: PDPL 2023 nhưng tiêu chuẩn ẩn danh hóa còn yếu. (7) **Niềm tin**: bác sĩ cần XAI tiếng Việt. (8) **Nhân lực**: thiếu chuyên gia tin học y tế. Giải pháp: federated learning xuyên bệnh viện, KG y tế tiếng Việt, fine-tune foundation model bản địa (PhoBERT-Med, ViMedSAM).

Q44. Regulatory frameworks for medical AI.
USA: **FDA 510(k)** (substantial equivalence to predicate device), **De Novo**, **PMA** (premarket approval, highest risk), **Software as a Medical Device (SaMD)** guidance, and the new **Predetermined Change Control Plan (PCCP)** for AI updates. **Europe**: **CE marking under MDR 2017/745** (Medical Device Regulation), with classification I/IIa/IIb/III. **EU AI Act** (2024) classifies most CDSS as **high-risk**, demanding risk management, data governance, transparency, human oversight, accuracy, robustness, cybersecurity, post-market monitoring. **ISO 13485** (QMS), **IEC 62304** (software lifecycle), **ISO 14971** (risk). Vietnam relies on Ministry of Health Circular 30/2015 and is gradually adopting ISO frameworks.

Q45. Challenges of medical AI in Vietnamese healthcare.
(1) **Data scarcity**: few public Vietnamese-labeled medical corpora; PACS data is hospital-locked. (2) **Language**: clinical notes mix **Vietnamese, English, Latin** terminology (e.g., "U não thể glioblastoma multiforme grade IV"); no widely-adopted Vietnamese medical knowledge graph; ICD-10 mappings inconsistent. (3) **Infrastructure**: legacy PACS, fragmented EMRs, low GPU availability in provincial hospitals. (4) **Domain shift**: foreign-trained models (BraTS, ImageNet) underperform on Vietnamese scanners and demographics. (5) **Regulation**: no clear local AI-medical pathway yet; relying on Circular 30/2015 & ISO. (6) **Privacy**: PDPL 2023, but anonymization standards weak. (7) **Trust**: clinicians need XAI in Vietnamese. (8) **Workforce**: shortage of medical-informatics specialists. Solutions: federated learning across hospitals, Vietnamese medical KG, local foundation-model fine-tunes (PhoBERT-Med, ViMedSAM).

QA4. Theoretical Q&A — Bezier & De Casteljau

Q1. What is a Bezier curve? Write the formula.
A **Bezier curve** of degree n is a parametric polynomial curve defined by $n+1$ **control points** $P_0, P_1, \ldots, P_n$ and the **Bernstein basis**. The curve is

$$B(t) = \sum_{i=0}^{n} \binom{n}{i} (1-t)^{n-i} t^i P_i, \quad t \in [0,1].$$

It **interpolates** P_0 and P_n, only **approximates** the inner points, and lies inside the **convex hull** of its control polygon. Cubic ($n = 3$) Beziers are the workhorse in fonts, SVG, and animation curves because they are smooth, intuitive, and cheap to evaluate.

Q2. What are control points and the control polygon?
Control points $P_0, \ldots, P_n$ are the user-editable handles that shape a Bezier curve. The polyline connecting them in order, $P_0 P_1 \cdots P_n$, is the **control polygon**. Only the endpoints P_0 and P_n are touched by the curve; intermediate points pull the curve towards themselves but never lie on it (in general). The control polygon gives a quick visual approximation of the curve and bounds it inside its convex hull. Designers manipulate control points instead of sampling the curve directly.

Q3. What is the Bernstein polynomial? Write the formula.
The **Bernstein polynomial** of degree n, index i, is

$$B_{i,n}(t) = \binom{n}{i} (1-t)^{n-i} t^i, \quad i = 0, 1, \ldots, n.$$

For example, the cubic Bernsteins are $B_{0,3} = (1-t)^3$, $B_{1,3} = 3(1-t)^2 t$, $B_{2,3} = 3(1-t)t^2$, $B_{3,3} = t^3$. They are non-negative on $[0,1]$, sum to 1, are symmetric ($B_{i,n}(t) = B_{n-i,n}(1-t)$), and have a single peak at $t = i/n$. They serve as **blending functions** that mix the control points.

Q4. What is the Bernstein basis?
The set $\{B_{0,n}(t), B_{1,n}(t), \ldots, B_{n,n}(t)\}$ is a **basis** of the polynomial space $\mathbb{P}_n$ (polynomials of degree $\leq n$). Any polynomial of degree $\leq n$ can be written uniquely as $\sum c_i B_{i,n}(t)$. Compared with the **monomial basis** $\{1, t, t^2, \ldots\}$, the Bernstein basis is much better conditioned numerically on $[0,1]$ and the coefficients c_i have a clear geometric meaning — they are control points. This is the algebraic reason Bezier curves are well-behaved.

Q5. What is the parameter t in a Bezier curve?
The **parameter** $t \in [0,1]$ is the abstract "time" or "progress" along the curve: $t = 0$ gives P_0, $t = 1$ gives P_n, and intermediate t values give points along the curve. It is **not** arc length — equal steps in t do **not** produce equally spaced points on the curve, so re-parametrization (arc-length parametrization) is needed for uniform-speed animation. We can also extend t outside $[0,1]$ to extrapolate, but the convex hull guarantee is then lost.

Q6. What is a Bezier surface? Tensor product formula.
A **Bezier surface** of degree (m,n) uses an $(m+1) \times (n+1)$ grid of control points $P_{i,j}$, called the **control net**. It is the **tensor product** of two univariate Beziers:

$$S(u,v) = \sum_{i=0}^{m} \sum_{j=0}^{n} B_{i,m}(u) B_{j,n}(v) P_{i,j}, \quad (u,v) \in [0,1]^2.$$

For each fixed v, $S(\cdot, v)$ is a Bezier curve in u, and vice versa. The **bicubic patch** ($m = n = 3$, 16 control points) is the standard CAD primitive.

Q7. What is a control net?
A **control net** is the 2D analogue of a control polygon: a topologically rectangular grid $\{P_{i,j}\}_{0 \leq i \leq m,\, 0 \leq j \leq n}$ of control points that defines a Bezier surface. The four **corners** of the net lie on the surface; the four **boundary curves** are themselves Bezier curves of the boundary rows/columns; interior points only pull the surface. The net's convex hull contains the surface — a useful bound for collision detection and rendering culling.

Q8. List the key properties of Bezier curves.
(1) **Endpoint interpolation**: $B(0) = P_0$, $B(1) = P_n$. (2) **Convex hull**: curve $\subseteq$ convex hull of control polygon. (3) **Affine invariance**: applying an affine map to control points = applying it to the curve. (4) **Variation diminishing**:

QA4. Câu hỏi lý thuyết — Bezier & De Casteljau

Câu 1. Đường cong Bezier là gì? Viết công thức.
Đường cong Bezier bậc n là đường cong tham số dạng đa thức được định nghĩa bởi $n+1$ **control point** $P_0, P_1, \ldots, P_n$ và **Bernstein basis**. Công thức:

$$B(t) = \sum_{i=0}^{n} \binom{n}{i} (1-t)^{n-i} t^i P_i, \quad t \in [0,1].$$

Đường cong **nội suy** (đi qua) P_0 và P_n, chỉ **xấp xỉ** các điểm giữa, và luôn nằm trong **convex hull** của control polygon. Bezier bậc 3 ($n = 3$) phổ biến nhất trong font, SVG, animation vì mượt, trực quan và rẻ để tính.

Câu 2. Control point và control polygon là gì?
Control point $P_0, \ldots, P_n$ là các điểm điều khiển mà người dùng kéo thả để tạo hình đường Bezier. Đường gấp khúc nối chúng theo thứ tự $P_0 P_1 \cdots P_n$ gọi là **control polygon**. Chỉ P_0 và P_n nằm trên curve; các điểm giữa chỉ **kéo** curve về phía chúng. Control polygon cho ta hình dung nhanh dáng curve và giới hạn curve trong convex hull của nó. Designer chỉnh control point thay vì lấy mẫu trực tiếp trên curve.

Câu 3. Bernstein polynomial là gì? Viết công thức.
Bernstein polynomial bậc n, chỉ số i:

$$B_{i,n}(t) = \binom{n}{i} (1-t)^{n-i} t^i, \quad i = 0, 1, \ldots, n.$$

Ví dụ, Bernstein bậc 3: $B_{0,3} = (1-t)^3$, $B_{1,3} = 3(1-t)^2 t$, $B_{2,3} = 3(1-t)t^2$, $B_{3,3} = t^3$. Chúng **không âm** trên $[0,1]$, **tổng bằng 1**, **đối xứng** ($B_{i,n}(t) = B_{n-i,n}(1-t)$), đạt cực đại tại $t = i/n$. Đóng vai trò **blending function** trộn các control point.

Câu 4. Bernstein basis là gì?
Tập $\{B_{0,n}(t), \ldots, B_{n,n}(t)\}$ là một **cơ sở** của không gian đa thức $\mathbb{P}_n$ (đa thức bậc $\leq n$). Mọi đa thức bậc $\leq n$ viết duy nhất dạng $\sum c_i B_{i,n}(t)$. So với **monomial basis** $\{1, t, t^2, \ldots\}$, Bernstein basis **ổn định số** hơn trên $[0,1]$ và hệ số c_i có ý nghĩa hình học rõ — chính là control point. Đây là lý do đại số khiến Bezier "ngoan".

Câu 5. Tham số t trong Bezier là gì?
Tham số $t \in [0,1]$ là "thời gian" trừu tượng dọc theo curve: $t = 0$ cho P_0, $t = 1$ cho P_n, các giá trị giữa cho các điểm dọc curve. **Không phải** arc length — chia đều t **không** cho điểm cách đều trên curve, nên cần re-parametrize theo arc length nếu muốn animation chạy đều. Có thể extend t ngoài $[0,1]$ để ngoại suy nhưng mất tính chất convex hull.

Câu 6. Bezier surface là gì? Công thức tensor product.
Bezier surface bậc (m,n) dùng lưới $(m+1) \times (n+1)$ control point $P_{i,j}$ gọi là **control net**. Là **tensor product** của hai Bezier 1 chiều:

$$S(u,v) = \sum_{i=0}^{m} \sum_{j=0}^{n} B_{i,m}(u) B_{j,n}(v) P_{i,j}, \quad (u,v) \in [0,1]^2.$$

Cố định v thì $S(\cdot, v)$ là Bezier curve theo u, và ngược lại. **Bicubic patch** ($m = n = 3$, 16 control point) là primitive chuẩn trong CAD.

Câu 7. Control net là gì?
Control net là phiên bản 2D của control polygon: lưới chữ nhật $\{P_{i,j}\}$ định nghĩa Bezier surface. Bốn **góc** của net nằm trên surface; bốn **boundary curve** cũng là Bezier curve của các hàng/cột biên; điểm trong chỉ **kéo** surface. Convex hull của net bao toàn surface — hữu ích cho collision detection và culling khi render.

Câu 8. Liệt kê các tính chất chính của Bezier curve.
(1) **Nội suy đầu mút**: $B(0) = P_0$, $B(1) = P_n$. (2) **Convex hull**: curve nằm trong convex hull của control polygon. (3) **Affine invariance**: phép affine lên control point = lên curve. (4) **Variation diminishing**: curve không "lượn"

the curve is no more wiggly than its control polygon. (5) **Symmetry**: reversing control points reverses the curve. (6) **Tangent at endpoints**: $B'(0) = n(P_1 - P_0)$, $B'(1) = n(P_n - P_{n-1})$. (7) **Partition of unity**: $\sum_i B_{i,n}(t) = 1$. (8) **Polynomial precision**: any polynomial of degree $\leq n$ is reproducible.

Q9. Endpoint interpolation property?
At $t = 0$, all Bernstein values vanish except $B_{0,n}(0) = 1$, so $B(0) = P_0$. At $t = 1$, all vanish except $B_{n,n}(1) = 1$, giving $B(1) = P_n$. Hence the curve always passes **exactly** through the first and last control points. This makes Bezier ideal for **stitching** curves end-to-end (e.g. glyph contours, animation paths) because matching neighbours is just placing P_n of one curve at P_0 of the next.

Q10. Explain the convex hull property.
For every $t \in [0, 1]$, the Bernstein values $B_{i,n}(t)$ are non-negative and sum to 1, so $B(t) = \sum B_{i,n}(t)P_i$ is a **convex combination** of the control points. Thus $B(t)$ lies in the **convex hull** of $\{P_0, \ldots, P_n\}$. Practical consequences: (i) bounding box for rendering and collision culling; (ii) guarantees the curve does not "shoot off" wildly even with bad control points; (iii) used in subdivision-based intersection algorithms.

Q11. Why does the curve always stay inside the control polygon?
Because partition of unity ($\sum B_{i,n}(t) = 1$) plus non-negativity ($B_{i,n}(t) \geq 0$) means each $B(t)$ is a **convex combination** of the control points. A convex combination always lies inside the **convex hull**, which is contained in (or equals) the bounding region of the control polygon. Geometric intuition: the Bernsteins "blend" control points like weights summing to 1, so the result cannot escape the cluster of points.

Q12. Partition of unity — why does it matter?
Partition of unity states $\sum_{i=0}^{n} B_{i,n}(t) = 1$ for all t. This is what guarantees **affine invariance** (translating all control points by v translates the curve by v) and the **convex hull** property (when combined with non-negativity). Without it, simply moving the world would change the shape — clearly nonsensical for a geometric primitive. Algebraically it follows from the binomial theorem: $\sum \binom{n}{i}(1-t)^{n-i}t^i = ((1-t)+t)^n = 1$.

Q13. What is affine invariance? Why important?
Affine invariance means: for any affine map T (translation, rotation, scale, shear), applying T to all control points and then evaluating the curve gives the same result as evaluating first then applying T, i.e. $T(B(t)) = \sum B_{i,n}(t)\,T(P_i)$. Important because the entire 3D pipeline (modelview matrix) is affine — we can transform **just** the control points (cheap, $O(n)$) instead of every sampled point on the curve. This is why graphics APIs store curves as control points, not tessellated meshes.

Q14. What is the variation diminishing property?
Variation diminishing: any straight line L intersects the Bezier curve at most as many times as it intersects the control polygon. So the curve cannot "wiggle" more than its polygon — it is a **smoothed** version of the polygon. Practical meaning: if the polygon is convex, so is the curve; if the polygon is monotone in x, so is the curve. This is why drag-editing Bezier control points feels predictable to designers.

Q15. Symmetry property?
The Bernstein basis satisfies $B_{i,n}(t) = B_{n-i,n}(1-t)$. Consequently, if we **reverse** the control points to $P_n, P_{n-1}, \ldots, P_0$, the resulting curve traces the **same geometric shape** but with parameter direction reversed: $B_{\text{rev}}(t) = B(1-t)$. There is no algorithmic preference for the "front" or "back" of a Bezier curve — symmetry is built in. This simplifies many derivations (e.g. analyses at $t = 1$ are mirror images of analyses at $t = 0$).

Q16. Tangents at the endpoints of a Bezier curve?
Differentiating, the derivative of a Bezier curve of degree n is itself a Bezier of degree $n-1$ with control points $n(P_{i+1} - P_i)$:

$$B'(t) = n \sum_{i=0}^{n-1} B_{i,n-1}(t)\,(P_{i+1} - P_i).$$

At endpoints: $B'(0) = n(P_1 - P_0)$ and $B'(1) = n(P_n - P_{n-1})$. So the curve leaves P_0 tangent to the segment $\overline{P_0P_1}$ and arrives at P_n tangent to $\overline{P_{n-1}P_n}$, with magnitudes scaled by n. This is exactly why dragging the second control point rotates the start tangent.

Q17. What is the De Casteljau algorithm? Why useful?
De Casteljau evaluates a Bezier curve at parameter t by **repeated linear interpolation** of control points. Given $\{P_i\}$, set $P_i^{(0)} = P_i$ and recurse $P_i^{(k)} = (1-t)P_i^{(k-1)} + t\,P_{i+1}^{(k-1)}$ until a single point $P_0^{(n)} = B(t)$ remains. It is **numerically stable** (only convex combinations), works in any dimension, and as a side effect produces **subdivision** control points for free. It costs $O(n^2)$ multiplications — fine for typical $n = 3$.

Q18. Write the De Casteljau recursion.
For $k = 1, \ldots, n$ and $i = 0, \ldots, n-k$:

$$P_i^{(k)}(t) = (1-t)\,P_i^{(k-1)}(t) + t\,P_{i+1}^{(k-1)}(t), \quad P_i^{(0)} = P_i.$$

The Bezier value is $B(t) = P_0^{(n)}(t)$. Geometrically each step replaces the current control polygon by the polygon of midpoint-weighted interpolations, shrinking the polygon one vertex at a time. After n steps, the polygon collapses to a single point — the curve point at t.

Q19. De Casteljau vs direct Bernstein evaluation?
Direct Bernstein: compute each $B_{i,n}(t)$ via the explicit formula and dot with P_i. Cost: $O(n)$ multiplications per evaluation if Bernsteins are precomputed, but it suffers **cancellation** when $(1-t)^k$ and t^k are large/small. **De Casteljau**: $O(n^2)$ but uses only convex combinations — **no cancellation**, monotone error growth, plus you get subdivision data for free. Use direct evaluation when n is large and precision is fine; use De Casteljau for robustness, subdivision, splitting, and intersection.

Q20. Why is De Casteljau numerically stable?
Each step is a **convex combination** $(1-t)A + tB$ with $t \in [0, 1]$, so the result lies between A and B. No subtraction

nhiều hơn control polygon. (5) **Đối xứng**: đảo ngược control point thì đảo ngược curve. (6) **Tiếp tuyến đầu mút**: $B'(0) = n(P_1 - P_0)$, $B'(1) = n(P_n - P_{n-1})$. (7) **Partition of unity**: $\sum_i B_{i,n}(t) = 1$. (8) **Polynomial precision**: tái tạo được mọi đa thức bậc $\leq n$.

Câu 9. Tính chất nội suy đầu mút?
Tại $t = 0$, mọi Bernstein bằng 0 trừ $B_{0,n}(0) = 1$, nên $B(0) = P_0$. Tại $t = 1$, mọi Bernstein bằng 0 trừ $B_{n,n}(1) = 1$, nên $B(1) = P_n$. Do đó curve luôn đi **chính xác** qua điểm đầu và cuối. Điều này làm Bezier lý tưởng để **ghép nối** curve liên tiếp (đường viền glyph, animation path) vì chỉ cần đặt P_n của curve này trùng P_0 của curve sau.

Câu 10. Giải thích tính chất convex hull.
Với mọi $t \in [0, 1]$, các Bernstein $B_{i,n}(t) \geq 0$ và tổng bằng 1, nên $B(t) = \sum B_{i,n}(t)P_i$ là **tổ hợp lồi** của control point. Do đó $B(t)$ nằm trong **convex hull** của $\{P_0, \ldots, P_n\}$. Ứng dụng: (i) bounding box cho render và collision culling; (ii) đảm bảo curve không "bay" lung tung dù control point xấu; (iii) dùng trong thuật toán intersection bằng subdivision.

Câu 11. Vì sao curve luôn nằm trong control polygon?
Vì partition of unity ($\sum B_{i,n}(t) = 1$) cộng với không âm ($B_{i,n}(t) \geq 0$) nghĩa là mỗi $B(t)$ là **tổ hợp lồi** của các control point. Tổ hợp lồi luôn nằm trong **convex hull**, mà convex hull bị bao bởi (hoặc trùng) miền của control polygon. Trực quan: Bernsteins "trộn" các control point như các trọng số tổng bằng 1 nên kết quả không thể "bay" ra khỏi cụm điểm.

Câu 12. Partition of unity — vì sao quan trọng?
Partition of unity nghĩa là $\sum_{i=0}^{n} B_{i,n}(t) = 1$ với mọi t. Đây là điều đảm bảo **affine invariance** (dịch tất cả control point đi v thì curve cũng dịch v) và **convex hull** (khi kết hợp với không âm). Không có nó, việc dịch chuyển thế giới sẽ làm biến dạng curve — vô lý cho một primitive hình học. Đại số: từ nhị thức Newton $\sum \binom{n}{i}(1-t)^{n-i}t^i = ((1-t)+t)^n = 1$.

Câu 13. Affine invariance là gì? Quan trọng vì sao?
Affine invariance: với mọi phép affine T (translation, rotation, scale, shear), áp T lên control point rồi tính curve bằng tính curve rồi áp T, tức $T(B(t)) = \sum B_{i,n}(t)\,T(P_i)$. Quan trọng vì toàn bộ pipeline 3D (modelview matrix) đều affine — ta chỉ cần transform **control point** (rẻ, $O(n)$) thay vì transform mọi điểm sampled. Vì vậy graphics API lưu curve dạng control point, không phải mesh đã tessellate.

Câu 14. Variation diminishing là gì?
Variation diminishing: mọi đường thẳng L cắt Bezier curve không nhiều hơn số lần cắt control polygon. Curve không thể "lượn" nhiều hơn polygon — là phiên bản **smoothed** của polygon. Ý nghĩa: nếu polygon lồi thì curve lồi; polygon đơn điệu theo x thì curve cũng vậy. Đó là lý do drag control point trong design tool rất "predictable".

Câu 15. Tính đối xứng?
Bernstein basis có $B_{i,n}(t) = B_{n-i,n}(1-t)$. Hệ quả: nếu **đảo ngược** control point thành $P_n, P_{n-1}, \ldots, P_0$, curve vẽ **cùng hình** nhưng hướng tham số ngược: $B_{\text{rev}}(t) = B(1-t)$. Không có "đầu" hay "đuôi" ưu tiên — đối xứng là built-in. Đơn giản hoá nhiều suy luận (phân tích tại $t = 1$ chỉ là ảnh gương của tại $t = 0$).

Câu 16. Tiếp tuyến tại đầu mút Bezier?
Lấy đạo hàm, derivative của Bezier bậc n là Bezier bậc $n-1$ với control point $n(P_{i+1} - P_i)$:

$$B'(t) = n \sum_{i=0}^{n-1} B_{i,n-1}(t)\,(P_{i+1} - P_i).$$

Tại đầu mút: $B'(0) = n(P_1 - P_0)$ và $B'(1) = n(P_n - P_{n-1})$. Curve rời P_0 tiếp xúc đoạn $\overline{P_0P_1}$, đến P_n tiếp xúc $\overline{P_{n-1}P_n}$, độ lớn nhân n. Vì vậy kéo control point thứ hai sẽ xoay tangent đầu.

Câu 17. De Casteljau là gì? Vì sao hữu ích?
De Casteljau tính Bezier tại t bằng cách **nội suy tuyến tính lặp** các control point. Cho $\{P_i\}$, đặt $P_i^{(0)} = P_i$ và đệ quy $P_i^{(k)} = (1-t)P_i^{(k-1)} + t\,P_{i+1}^{(k-1)}$ cho đến khi còn một điểm $P_0^{(n)} = B(t)$. **Ổn định số** (chỉ tổ hợp lồi), làm việc ở mọi chiều, và "miễn phí" cho ra control point của **subdivision**. Chi phí $O(n^2)$ phép nhân — quá đủ với $n = 3$.

Câu 18. Viết đệ quy De Casteljau.
Với $k = 1, \ldots, n$ và $i = 0, \ldots, n-k$:

$$P_i^{(k)}(t) = (1-t)\,P_i^{(k-1)}(t) + t\,P_{i+1}^{(k-1)}(t), \quad P_i^{(0)} = P_i.$$

Giá trị curve là $B(t) = P_0^{(n)}(t)$. Hình học: mỗi bước thay polygon hiện tại bằng polygon các điểm nội suy trung gian, mỗi lần "co" đi một đỉnh. Sau n bước, polygon co về một điểm — chính là điểm trên curve tại t.

Câu 19. So sánh De Casteljau với Bernstein trực tiếp?
Bernstein trực tiếp: tính từng $B_{i,n}(t)$ rồi dot với P_i. Chi phí $O(n)$ phép nhân nếu Bernsteins precomputed, nhưng **cancellation** khi $(1-t)^k$ và t^k chênh lệch lớn. **De Casteljau**: $O(n^2)$ nhưng chỉ dùng tổ hợp lồi — **không cancellation**, sai số tăng đều, và miễn phí subdivision. Dùng trực tiếp khi n lớn và precision đủ; dùng De Casteljau khi cần robustness, subdivision, splitting, intersection.

Câu 20. Vì sao De Casteljau ổn định số?
Mỗi bước là **tổ hợp lồi** $(1-t)A + tB$ với $t \in [0, 1]$, kết quả nằm giữa A và B. Không có phép trừ giữa hai số gần

of nearly-equal numbers, no large t^k or $(1-t)^k$ cancellations. The intermediate points stay within the convex hull of the original control points, so floating-point error is **bounded by** the spread of those points and grows linearly. By contrast, Bernstein direct evaluation expands $(1-t)^k t^{n-k}$ which can over/underflow and cancel for t near 0 or 1.

Q21. How does De Casteljau give subdivision for free?
After running De Casteljau at parameter t^*, the **left edge** of the triangle $(P_0^{(0)}, P_0^{(1)}, \ldots, P_0^{(n)})$ gives the control points of the curve restricted to $[0, t^*]$, and the **right edge** $(P_0^{(n)}, P_1^{(n-1)}, \ldots, P_n^{(0)})$ gives the control points for $[t^*, 1]$. Both halves are themselves Bezier curves of the same degree. This split is the basis of **adaptive rendering**, **intersection**, and **trimming** algorithms — recurse until each piece is "flat enough", then draw line segments.

Q22. Geometric interpretation of De Casteljau?
Imagine the control polygon as a sequence of segments. At parameter t, place a point at fraction t along each segment $\overline{P_i P_{i+1}}$ — these are the level-1 points. Connect them to form a new polygon with one fewer vertex, and repeat. Each recursion is a **shrinking polygon**; after n recursions only one point remains, the curve point at t. The construction is purely **linear interpolation** — pencil-and-ruler-friendly, no algebra needed.

Q23. Worked: $P_0 = (0,0), P_1 = (1,3), P_2 = (3,3), P_3 = (4,0)$, compute $B(0.5)$ via De Casteljau.
With $t = 0.5$, $1-t = 0.5$. **Level 1**: $P_0^{(1)} = 0.5(0,0) + 0.5(1,3) = (0.5, 1.5)$; $P_1^{(1)} = 0.5(1,3) + 0.5(3,3) = (2,3)$; $P_2^{(1)} = 0.5(3,3) + 0.5(4,0) = (3.5, 1.5)$. **Level 2**: $P_0^{(2)} = 0.5(0.5,1.5) + 0.5(2,3) = (1.25, 2.25)$; $P_1^{(2)} = 0.5(2,3) + 0.5(3.5,1.5) = (2.75, 2.25)$. **Level 3**: $B(0.5) = P_0^{(3)} = 0.5(1.25,2.25) + 0.5(2.75,2.25) = (2, 2.25)$.

Q24. Same setup, compute $B(0.75)$ via De Casteljau.
With $t = 0.75$, $1-t = 0.25$. **Level 1**: $P_0^{(1)} = 0.25(0,0) + 0.75(1,3) = (0.75, 2.25)$; $P_1^{(1)} = 0.25(1,3) + 0.75(3,3) = (2.5, 3)$; $P_2^{(1)} = 0.25(3,3) + 0.75(4,0) = (3.75, 0.75)$. **Level 2**: $P_0^{(2)} = 0.25(0.75,2.25) + 0.75(2.5,3) = (2.0625, 2.8125)$; $P_1^{(2)} = 0.25(2.5,3) + 0.75(3.75,0.75) = (3.4375, 1.3125)$. **Level 3**: $B(0.75) = 0.25(2.0625,2.8125) + 0.75(3.4375,1.3125) = (3.0625, 1.6875)$.

Q25. Same setup, compute $B(0.5)$ via direct Bernstein. Verify.
Cubic Bernsteins at $t = 0.5$: $B_{0,3} = 0.125$, $B_{1,3} = 3(0.25)(0.5) = 0.375$, $B_{2,3} = 3(0.5)(0.25) = 0.375$, $B_{3,3} = 0.125$. $B(0.5) = 0.125(0,0) + 0.375(1,3) + 0.375(3,3) + 0.125(4,0)$. $x = 0 + 0.375 + 1.125 + 0.5 = 2$. $y = 0 + 1.125 + 1.125 + 0 = 2.25$. So $B(0.5) = (2, 2.25)$ — **matches** the De Casteljau result of Q23.

Q26. Tangents at $t = 0$ and $t = 1$ for the same curve.
Using $B'(0) = n(P_1 - P_0)$ and $B'(1) = n(P_n - P_{n-1})$ with $n = 3$: $B'(0) = 3((1,3) - (0,0)) = (3,9)$. Direction: $(1,3)$, slope 3. $B'(1) = 3((4,0) - (3,3)) = (3,-9)$. Direction: $(1,-3)$, slope -3. So the curve leaves $(0,0)$ steeply upward along direction $(1,3)$ and arrives at $(4,0)$ steeply downward along $(1,-3)$ — symmetric about the vertical $x = 2$, as expected from the symmetric control polygon.

Q27. Subdivide the cubic at $t = 0.5$ — left and right control polygons.
From the De Casteljau triangle at $t = 0.5$ (Q23): **Left polygon** (curve on $[0, 0.5]$): $P_0^{(0)} = (0,0)$, $P_0^{(1)} = (0.5, 1.5)$, $P_0^{(2)} = (1.25, 2.25)$, $P_0^{(3)} = (2, 2.25)$. **Right polygon** (curve on $[0.5, 1]$): $P_0^{(3)} = (2, 2.25)$, $P_1^{(2)} = (2.75, 2.25)$, $P_2^{(1)} = (3.5, 1.5)$, $P_3^{(0)} = (4, 0)$. The two halves share the midpoint $B(0.5) = (2, 2.25)$ as a common endpoint and form a C^1 join (the original curve is smooth, after all).

Q28. Why is a Bezier surface called tensor product?
The basis of the surface space is the **outer product** of two univariate Bernstein bases:

$$\{B_{i,m}(u) B_{j,n}(v)\}_{i,j}.$$

Algebraically this is the **tensor product** of the two basis sets. Geometrically, fixing v leaves a Bezier curve in u, and fixing u leaves a Bezier curve in v — the surface is "woven" from two families of orthogonal Bezier curves. Tensor-product structure is what allows **separable evaluation** and clean mathematical analysis.

Q29. Row-first vs column-first surface evaluation?
Row-first: for each row i, evaluate the Bezier curve in v with control points $\{P_{i,j}\}_{j=0..n}$ to get $Q_i(v)$. Then evaluate a Bezier curve in u on $\{Q_0, \ldots, Q_m\}$ to get $S(u,v)$. **Column-first**: swap roles of u and v. Both give the same point — tensor products commute. Choice depends on caching / SIMD layout. For a **bicubic**, total cost is $4 + 1 = 5$ De Casteljau passes either way: 4 row passes (cost $O(n^2)$ each) plus 1 final pass.

Q30. De Casteljau on a Bezier surface — step by step.
Goal: evaluate $S(u^*, v^*)$ on a 4×4 bicubic patch. **Step 1**: For each row $i = 0,1,2,3$ run De Casteljau with $t = v^*$ on the 4 control points $\{P_{i,0}, P_{i,1}, P_{i,2}, P_{i,3}\}$ to obtain Q_i = row i Bezier curve at v^*. **Step 2**: Run De Casteljau with $t = u^*$ on the 4 derived points $\{Q_0, Q_1, Q_2, Q_3\}$. **Step 3**: The final point is $S(u^*, v^*)$. Total: 5 univariate De Casteljau evaluations of degree 3.

Q31. What is a bicubic Bezier patch? How many control points?
A **bicubic Bezier patch** has degree $(3,3)$. It uses $(3+1) \times (3+1) = \mathbf{16}$ control points $\{P_{i,j}\}_{0 \le i,j \le 3}$. The four corners $P_{0,0}, P_{0,3}, P_{3,0}, P_{3,3}$ lie on the surface; the four boundary curves are cubic Beziers. Bicubic is the **minimum** degree allowing C^1 joins between patches with shape control on both the corner positions and the corner tangents — that is why CAD systems standardise on bicubic patches.

Q32. What are isoparametric curves?
An **isoparametric curve** (iso-curve) is a curve on a Bezier surface obtained by **fixing one parameter**: $u = u_0$ (a v-iso) or $v = v_0$ (a u-iso). Each iso-curve is itself a Bezier curve of degree m or n, with control points obtained by running univariate De Casteljau on the rows or columns of the control net at the fixed parameter. Iso-curves are used for **wireframe rendering**, **contour previews**, and as **trim curves**.

Q33. Bezier vs B-spline vs NURBS — when to use which?
Bezier: simple, single-segment polynomial; degree tied to number of control points; no local control; great for fonts, simple animation. **B-spline**: piecewise polynomial with a knot vector; **local control** (move one control point, only

bằng, không cancellation lớn của t^k hay $(1-t)^k$. Điểm trung gian luôn ở trong convex hull của control point gốc, sai số floating-point **bị chặn** bởi đường kính cụm điểm và tăng tuyến tính. Ngược lại, Bernstein trực tiếp khai triển $(1-t)^k t^{n-k}$ dễ over/underflow và cancellation khi t gần 0 hoặc 1.

Câu 21. Subdivision miễn phí từ De Casteljau?
Sau khi chạy De Casteljau tại t^*, **cạnh trái** của tam giác $(P_0^{(0)}, P_0^{(1)}, \ldots, P_0^{(n)})$ là control point của curve trên $[0, t^*]$, và **cạnh phải** $(P_0^{(n)}, P_1^{(n-1)}, \ldots, P_n^{(0)})$ là control point trên $[t^*, 1]$. Cả hai nửa đều là Bezier cùng bậc. Đây là cơ sở của **adaptive rendering**, **intersection**, **trimming** — đệ quy cho đến khi đủ "phẳng" thì vẽ line segment.

Câu 22. Diễn giải hình học De Casteljau?
Hình dung control polygon là chuỗi đoạn thẳng. Tại tham số t, đặt một điểm ở vị trí tỉ lệ t trên mỗi đoạn $\overline{P_i P_{i+1}}$ — đó là level-1. Nối chúng tạo polygon mới ít hơn một đỉnh, lặp lại. Mỗi đệ quy là **polygon co lại**; sau n đệ quy còn một điểm — chính là điểm trên curve tại t. Toàn bộ chỉ là **nội suy tuyến tính** — vẽ tay bằng compass thước được, không cần đại số.

Câu 23. Bài tập: $P_0 = (0,0), P_1 = (1,3), P_2 = (3,3), P_3 = (4,0)$, tính $B(0.5)$ bằng De Casteljau.
Với $t = 0.5$, $1-t = 0.5$. **Level 1**: $P_0^{(1)} = 0.5(0,0) + 0.5(1,3) = (0.5, 1.5)$; $P_1^{(1)} = 0.5(1,3) + 0.5(3,3) = (2,3)$; $P_2^{(1)} = 0.5(3,3) + 0.5(4,0) = (3.5, 1.5)$. **Level 2**: $P_0^{(2)} = 0.5(0.5,1.5) + 0.5(2,3) = (1.25, 2.25)$; $P_1^{(2)} = 0.5(2,3) + 0.5(3.5,1.5) = (2.75, 2.25)$. **Level 3**: $B(0.5) = P_0^{(3)} = 0.5(1.25,2.25) + 0.5(2.75,2.25) = (2, 2.25)$.

Câu 24. Cùng setup, tính $B(0.75)$ bằng De Casteljau.
Với $t = 0.75$, $1-t = 0.25$. **Level 1**: $P_0^{(1)} = 0.25(0,0) + 0.75(1,3) = (0.75, 2.25)$; $P_1^{(1)} = 0.25(1,3) + 0.75(3,3) = (2.5, 3)$; $P_2^{(1)} = 0.25(3,3) + 0.75(4,0) = (3.75, 0.75)$. **Level 2**: $P_0^{(2)} = 0.25(0.75,2.25) + 0.75(2.5,3) = (2.0625, 2.8125)$; $P_1^{(2)} = 0.25(2.5,3) + 0.75(3.75,0.75) = (3.4375, 1.3125)$. **Level 3**: $B(0.75) = 0.25(2.0625,2.8125) + 0.75(3.4375,1.3125) = (3.0625, 1.6875)$.

Câu 25. Cùng setup, tính $B(0.5)$ trực tiếp bằng Bernstein. Kiểm tra.
Bernstein bậc 3 tại $t = 0.5$: $B_{0,3} = 0.125$, $B_{1,3} = 3(0.25)(0.5) = 0.375$, $B_{2,3} = 3(0.5)(0.25) = 0.375$, $B_{3,3} = 0.125$. $B(0.5) = 0.125(0,0) + 0.375(1,3) + 0.375(3,3) + 0.125(4,0)$. $x = 0 + 0.375 + 1.125 + 0.5 = 2$. $y = 0 + 1.125 + 1.125 + 0 = 2.25$. Vậy $B(0.5) = (2, 2.25)$ — **khớp** với De Casteljau ở Câu 23.

Câu 26. Tiếp tuyến tại $t = 0$ và $t = 1$ của cùng curve.
Dùng $B'(0) = n(P_1 - P_0)$ và $B'(1) = n(P_n - P_{n-1})$ với $n = 3$: $B'(0) = 3((1,3) - (0,0)) = (3,9)$. Hướng $(1,3)$, slope 3. $B'(1) = 3((4,0) - (3,3)) = (3,-9)$. Hướng $(1,-3)$, slope -3. Curve rời $(0,0)$ đi lên dốc theo hướng $(1,3)$, đến $(4,0)$ đi xuống theo $(1,-3)$ — đối xứng qua đường $x = 2$, đúng như control polygon đối xứng.

Câu 27. Chia đôi cubic tại $t = 0.5$ — control polygon trái và phải.
Từ tam giác De Casteljau tại $t = 0.5$ (Câu 23): **Polygon trái** (curve trên $[0, 0.5]$): $P_0^{(0)} = (0,0)$, $P_0^{(1)} = (0.5, 1.5)$, $P_0^{(2)} = (1.25, 2.25)$, $P_0^{(3)} = (2, 2.25)$. **Polygon phải** (curve trên $[0.5, 1]$): $P_0^{(3)} = (2, 2.25)$, $P_1^{(2)} = (2.75, 2.25)$, $P_2^{(1)} = (3.5, 1.5)$, $P_3^{(0)} = (4, 0)$. Hai nửa chia sẻ midpoint $B(0.5) = (2, 2.25)$ làm endpoint chung, ghép C^1 (curve gốc vốn smooth).

Câu 28. Vì sao Bezier surface gọi là tensor product?
Cơ sở không gian surface là **outer product** của hai Bernstein basis 1 chiều:

$$\{B_{i,m}(u) B_{j,n}(v)\}_{i,j}.$$

Đại số đây chính là **tensor product** hai tập cơ sở. Hình học, cố định v còn lại Bezier curve theo u, cố định u còn lại Bezier curve theo v — surface "dệt" từ hai họ Bezier curve trực giao. Cấu trúc tensor product cho phép **separable evaluation** và phân tích toán học sạch sẽ.

Câu 29. Đánh giá surface theo hàng trước hay cột trước?
Row-first: với mỗi hàng i, tính Bezier curve theo v với control point $\{P_{i,j}\}_j$ để có $Q_i(v)$. Rồi tính Bezier curve theo u trên $\{Q_0, \ldots, Q_m\}$ ra $S(u,v)$. **Column-first**: đổi vai trò u, v. Cùng kết quả — tensor product giao hoán. Chọn tuỳ cache / SIMD layout. Với **bicubic**, tổng cộng $4 + 1 = 5$ pass De Casteljau: 4 row pass (mỗi pass $O(n^2)$) + 1 pass cuối.

Câu 30. De Casteljau cho Bezier surface — từng bước.
Mục tiêu: tính $S(u^*, v^*)$ trên bicubic 4×4. **Bước 1**: Với mỗi hàng $i = 0,1,2,3$ chạy De Casteljau với $t = v^*$ trên 4 control point $\{P_{i,0}, \ldots, P_{i,3}\}$ ra Q_i = Bezier curve hàng i tại v^*. **Bước 2**: Chạy De Casteljau với $t = u^*$ trên 4 điểm $\{Q_0, Q_1, Q_2, Q_3\}$. **Bước 3**: Kết quả cuối là $S(u^*, v^*)$. Tổng: 5 lần De Casteljau 1 chiều bậc 3.

Câu 31. Bicubic Bezier patch là gì? Bao nhiêu control point?
Bicubic Bezier patch có bậc $(3,3)$. Dùng $(3+1) \times (3+1) = \mathbf{16}$ control point $\{P_{i,j}\}_{0 \le i,j \le 3}$. Bốn góc $P_{0,0}, P_{0,3}, P_{3,0}, P_{3,3}$ nằm trên surface; bốn boundary curve là Bezier bậc 3. Bicubic là bậc **tối thiểu** cho phép ghép C^1 giữa các patch và kiểm soát cả vị trí lẫn tangent ở góc — vì vậy CAD chuẩn hoá bicubic.

Câu 32. Isoparametric curve là gì?
Isoparametric curve (iso-curve) là curve trên Bezier surface khi **cố định một tham số**: $u = u_0$ (v-iso) hoặc $v = v_0$ (u-iso). Mỗi iso-curve là Bezier curve bậc m hoặc n, với control point lấy từ De Casteljau 1 chiều trên hàng/cột của control net tại tham số cố định. Dùng cho **wireframe render**, **contour preview**, và **trim curve**.

Câu 33. Bezier vs B-spline vs NURBS — khi nào dùng?
Bezier: 1 segment đa thức đơn giản; bậc gắn liền số control point; không local control; tốt cho font, animation đơn giản. **B-spline**: piecewise polynomial với knot vector; **local control** (kéo 1 control point chỉ đổi vài segment

nearby segments change); decoupled degree and segment count; best for editable smooth curves. **NURBS** (Non-Uniform Rational B-Spline): B-spline + per-control-point weights, written in homogeneous coords; can represent **conics exactly** (circles, ellipses, paraboloids); industry standard for CAD/CAM. Use Bezier for art assets, B-spline for editable shapes, NURBS for engineering.

Q34. Hermite curve — compare with Bezier.
A **cubic Hermite curve** is defined by two endpoints P_0, P_1 and two endpoint tangents T_0, T_1:

$$H(t) = h_{00}P_0 + h_{10}T_0 + h_{01}P_1 + h_{11}T_1,$$

with the standard Hermite blending functions. A cubic Hermite is **equivalent** to a cubic Bezier with control points $P_0, P_0 + \frac{1}{3}T_0, P_1 - \frac{1}{3}T_1, P_1$. Hermite is more natural when the user supplies tangents directly (e.g. animation keyframes); Bezier is more natural when the user supplies handles. They are two parameterisations of the same cubic family.

Q35. Catmull-Rom spline — when preferred?
A **Catmull-Rom** spline is a C^1 cubic spline that **interpolates** every given knot point and computes tangents from neighbours: $T_i = \frac{1}{2}(P_{i+1} - P_{i-1})$. Preferred when you have a sequence of **must-pass-through** points (camera paths, animation keyframes, road centrelines) and don't want to fiddle with handles. Downside: not local in shape if you change one point — it perturbs four segments — and lacks the convex hull property, so it can overshoot.

Q36. NURBS surface — why CAD industry standard?
NURBS = Non-Uniform Rational B-Spline. Each control point has a weight $w_{i,j}$, and the surface is

$$S(u,v) = \frac{\sum_{i,j} N_{i,p}(u)N_{j,q}(v)w_{i,j}P_{i,j}}{\sum_{i,j} N_{i,p}(u)N_{j,q}(v)w_{i,j}}.$$

Why standard: (i) exact representation of **conics, quadrics, tori** — needed for mechanical parts; (ii) local control via knot vectors; (iii) affine + projective invariance; (iv) supported by every major CAD kernel (Parasolid, ACIS, OpenCASCADE) and exchange format (STEP, IGES).

Q37. Degree elevation — formula and uses.
Degree elevation: convert a Bezier of degree n with controls $\{P_i\}$ into the **same curve** represented as a Bezier of degree $n+1$ with controls $\{P_i^*\}$:

$$P_i^* = \tfrac{i}{n+1}P_{i-1} + \left(1 - \tfrac{i}{n+1}\right)P_i, \quad i = 0, \ldots, n+1,$$

with P_{-1}, P_{n+1} ignored. Used to: (i) merge curves of different degrees into a common degree before joining/blending; (ii) give designers more control points without changing the shape; (iii) prepare data for algorithms (e.g. Bezier-Clip intersection) that need matched degrees.

Q38. Derivative of a Bezier curve?
The derivative of a degree-n Bezier is a degree-$(n-1)$ Bezier with control points $D_i = n(P_{i+1} - P_i)$:

$$B'(t) = \sum_{i=0}^{n-1} B_{i,n-1}(t)\, n(P_{i+1} - P_i).$$

This is called the **hodograph**. Used for: (i) tangent vectors at any t (just evaluate the hodograph at t); (ii) curvature analysis $\kappa = \frac{|B' \times B''|}{|B'|^3}$; (iii) checking inflection points; (iv) arc-length parametrisation via $\int |B'(t)|\, dt$.

Q39. C^0, C^1, C^2 continuity — how to enforce?
Two Bezier segments B_1 (controls $P_0, \ldots, P_n$) and B_2 (controls $Q_0, \ldots, Q_m$) joined at $B_1(1) = B_2(0)$: C^0: positions match $\Rightarrow Q_0 = P_n$. C^1: tangents match $\Rightarrow$ for equal degrees $n = m$, $Q_1 - Q_0 = P_n - P_{n-1}$, i.e. $P_{n-1}, P_n = Q_0, Q_1$ are collinear and equally spaced. C^2: in addition, the second derivatives match — adds a constraint on $P_{n-2}, P_{n-1}, Q_0, Q_1, Q_2$ involving combinations of consecutive differences.

Q40. Joining two cubic Beziers with G^1 continuity?
G^1 **(geometric)** requires only that tangent **directions** match at the join, not magnitudes. For cubics: $Q_0 = P_3$ and $Q_1 - Q_0 = \alpha(P_3 - P_2)$ for some scalar $\alpha > 0$. So $P_2, P_3 = Q_0, Q_1$ are collinear and on the same side, but the spacing can differ. This is a **weaker** but more flexible condition than C^1; commonly used in font design where shape continuity matters but the parameter speed is irrelevant.

Q41. Bezier curves in font design / typography?
Modern fonts (PostScript Type 1, TrueType, OpenType, SVG) store glyph outlines as sequences of **Bezier segments**. PostScript and OpenType-CFF use **cubic** Beziers; TrueType uses **quadratic** Beziers (cheaper to evaluate on early rasterisers). Each glyph is a closed contour stitched from many short Bezier pieces with C^0 or G^1 continuity at junctions, plus a fill rule (even-odd or non-zero). Resolution-independence and editability via control points are why Beziers won the typography war.

Q42. Bezier curves in vector graphics (SVG, Illustrator)?
SVG path commands `C` (cubic) and `Q` (quadratic) directly draw Bezier segments; the `T/S` variants give shorthand smooth joins. Adobe Illustrator's pen tool exposes the four control points of each cubic Bezier as the anchor point and two handles. Operations like **stroking**, **offset curves**, and **boolean ops** are implemented by adaptively subdividing Beziers (often with De Casteljau) until each piece is "flat enough" to treat as a line segment — then standard polygon algorithms apply.

Q43. Bezier surfaces in CAD?
Early CAD systems (Bezier at Renault, Coons at MIT) used Bezier patches to model car body panels and aircraft skins — areas where smooth, free-form, mathematically tractable surfaces were essential. Modern CAD has migrated to **NURBS** (which is a Bezier-superset), but Bezier patches remain the building block: every NURBS surface can be **converted** into a network of Bezier patches via knot insertion, and downstream algorithms (rendering, intersection, meshing) often work on the Bezier representation.

Q44. Bezier in animation (keyframe interpolation)?

quanh); bậc và số segment độc lập; tốt cho curve smooth chỉnh sửa được. **NURBS**: B-spline + weight cho từng control point, viết homogeneous; biểu diễn **conic chính xác** (đường tròn, ellipse, paraboloid); chuẩn công nghiệp CAD/CAM. Dùng Bezier cho asset nghệ thuật, B-spline cho hình chỉnh sửa, NURBS cho kỹ thuật.

Câu 34. Hermite curve — so sánh Bezier.
Hermite curve bậc 3 định nghĩa bởi 2 điểm P_0, P_1 và 2 tangent T_0, T_1:

$$H(t) = h_{00}P_0 + h_{10}T_0 + h_{01}P_1 + h_{11}T_1,$$

với các Hermite blending function chuẩn. Cubic Hermite **tương đương** cubic Bezier với control point $P_0, P_0 + \frac{1}{3}T_0, P_1 - \frac{1}{3}T_1, P_1$. Hermite tự nhiên khi người dùng cho tangent trực tiếp (animation keyframe); Bezier tự nhiên khi cho handle. Hai cách parameterise cùng một họ cubic.

Câu 35. Catmull-Rom — khi nào dùng?
Catmull-Rom là cubic spline C^1 **nội suy** mọi knot point và tính tangent từ hàng xóm: $T_i = \frac{1}{2}(P_{i+1} - P_{i-1})$. Dùng khi có chuỗi điểm **bắt buộc phải đi qua** (camera path, animation keyframe, tim đường) và không muốn loay hoay với handle. Nhược: đổi một điểm sẽ ảnh hưởng 4 segment, không có convex hull nên có thể overshoot.

Câu 36. NURBS surface — vì sao chuẩn CAD?
NURBS = Non-Uniform Rational B-Spline. Mỗi control point có weight $w_{i,j}$:

$$S(u,v) = \frac{\sum_{i,j} N_{i,p}(u)N_{j,q}(v)w_{i,j}P_{i,j}}{\sum_{i,j} N_{i,p}(u)N_{j,q}(v)w_{i,j}}.$$

Lý do: (i) biểu diễn **conic, quadric, torus** chính xác — cần cho cơ khí; (ii) local control qua knot vector; (iii) bất biến affine + projective; (iv) mọi CAD kernel lớn (Parasolid, ACIS, OpenCASCADE) và format trao đổi (STEP, IGES) hỗ trợ.

Câu 37. Degree elevation — công thức và mục đích.
Degree elevation: chuyển Bezier bậc n với control $\{P_i\}$ thành **cùng curve** ở bậc $n+1$ với control $\{P_i^*\}$:

$$P_i^* = \tfrac{i}{n+1}P_{i-1} + \left(1 - \tfrac{i}{n+1}\right)P_i, \quad i = 0, \ldots, n+1,$$

với P_{-1}, P_{n+1} bỏ qua. Dùng để: (i) gộp curve khác bậc về cùng bậc trước khi nối/blend; (ii) cho designer thêm control point mà không đổi hình; (iii) chuẩn bị dữ liệu cho thuật toán (ví dụ Bezier-Clip) cần cùng bậc.

Câu 38. Đạo hàm của Bezier curve?
Đạo hàm Bezier bậc n là Bezier bậc $n-1$ với control point $D_i = n(P_{i+1} - P_i)$:

$$B'(t) = \sum_{i=0}^{n-1} B_{i,n-1}(t)\, n(P_{i+1} - P_i).$$

Gọi là **hodograph**. Dùng cho: (i) tangent tại mọi t (chỉ cần đánh giá hodograph tại t); (ii) curvature $\kappa = \frac{|B' \times B''|}{|B'|^3}$; (iii) kiểm tra điểm uốn; (iv) arc-length parametrisation qua $\int |B'(t)|\, dt$.

Câu 39. Liên tục C^0, C^1, C^2 — cách enforce?
Hai Bezier B_1 (control $P_0, \ldots, P_n$) và B_2 (control $Q_0, \ldots, Q_m$) nối tại $B_1(1) = B_2(0)$: C^0: vị trí $\Rightarrow Q_0 = P_n$. C^1: tangent $\Rightarrow$ với cùng bậc $n = m$, $Q_1 - Q_0 = P_n - P_{n-1}$, tức $P_{n-1}, P_n = Q_0, Q_1$ thẳng hàng và cách đều. C^2: thêm đạo hàm bậc 2 khớp — ràng buộc thêm trên $P_{n-2}, P_{n-1}, Q_0, Q_1, Q_2$ qua tổ hợp các sai phân liên tiếp.

Câu 40. Nối hai cubic Bezier với G^1?
G^1 **(geometric)** chỉ yêu cầu **hướng** tangent khớp, không cần độ lớn. Với cubic: $Q_0 = P_3$ và $Q_1 - Q_0 = \alpha(P_3 - P_2)$ với $\alpha > 0$. Tức $P_2, P_3 = Q_0, Q_1$ thẳng hàng, cùng phía, nhưng khoảng cách có thể khác. Là điều kiện **yếu** hơn nhưng linh hoạt hơn C^1; phổ biến trong thiết kế font, nơi cần hình mượt nhưng tốc độ tham số không quan trọng.

Câu 41. Bezier trong thiết kế font / typography?
Font hiện đại (PostScript Type 1, TrueType, OpenType, SVG) lưu đường viền glyph dạng chuỗi **Bezier segment**. PostScript và OpenType-CFF dùng **cubic** Bezier; TrueType dùng **quadratic** Bezier (rasterise rẻ hơn trên máy đời cũ). Mỗi glyph là contour kín ghép từ nhiều Bezier ngắn với C^0 hoặc G^1 tại nối, kèm fill rule (even-odd hoặc non-zero). Resolution-independence và editability qua control point khiến Bezier thắng trận typography.

Câu 42. Bezier trong vector graphics (SVG, Illustrator)?
Lệnh path SVG `C` (cubic) và `Q` (quadratic) vẽ trực tiếp Bezier; `T/S` là shorthand smooth. Pen tool của Illustrator hiển thị 4 control point cubic Bezier dưới dạng anchor + 2 handle. Các phép **stroke**, **offset curve**, **boolean op** được hiện thực bằng adaptive subdivision (thường De Casteljau) cho đến khi mỗi mảnh đủ "phẳng" để coi như line — rồi áp thuật toán polygon chuẩn.

Câu 43. Bezier surface trong CAD?
CAD đời đầu (Bezier ở Renault, Coons ở MIT) dùng Bezier patch để mô hình thân xe, vỏ máy bay — những vùng cần surface free-form mượt, dễ xử lý toán học. CAD hiện đại đã chuyển sang **NURBS** (siêu tập Bezier), nhưng Bezier patch vẫn là khối xây: mọi NURBS surface có thể **convert** thành lưới Bezier patch qua knot insertion, và thuật toán downstream (render, intersection, meshing) thường làm việc trên dạng Bezier.

Câu 44. Bezier trong animation (keyframe)?

Animation timing curves (the "f-curves" in Maya, Blender, After Effects) are typically cubic Beziers from one keyframe value to the next. The two endpoint control points are the keyframe values; the two interior handles control **ease-in / ease-out**. The CSS3 `cubic-bezier(x1,y1,x2,y2)` function is exactly this: a cubic Bezier with $P_0 = (0,0)$, $P_1 = (x_1, y_1)$, $P_2 = (x_2, y_2)$, $P_3 = (1,1)$ that maps animation time $\to$ progress.

Q45. Bezier curves in robotics path planning?
Robot arms, drones, and autonomous vehicles use Bezier curves to generate **smooth, C^2-continuous** trajectories that respect velocity / acceleration limits. Compared with piecewise-linear paths, Bezier paths reduce jerk (the third derivative), which lowers actuator wear and battery drain. The convex hull property gives **guaranteed safety bounds** for collision checking — if the convex hull is obstacle-free, so is the trajectory. Quintic Beziers ($n = 5$) are common because they give independent control of position, velocity, and acceleration at both ends.

Q46. Fastest way to compute $B(t)$ at one t for 4 control points?
For a single t on a cubic ($n = 3$): **De Casteljau** costs 3+2+1 = 6 linear interpolations, each one multiplication-add per coordinate, so ~ 12 FLOPs in 2D. Alternative: Horner's rule on the polynomial form $B(t) = ((c_3t+c_2)t+c_1)t+c_0$ where c_i are precomputed Bernstein-coefficient combinations — slightly cheaper if only one t value is needed and stability isn't critical. For **many** t samples, precompute the Bernsteins or use Horner. For **robust** subdivision and intersection, always De Casteljau.

Q47. Compute $S(0.5, 0.5)$ on a 4×4 control net — step by step.
Let the net be $P_{i,j}$, $i, j \in \{0, 1, 2, 3\}$. **Step 1**: For each row i, run cubic De Casteljau in v at 0.5 on $\{P_{i,0}, P_{i,1}, P_{i,2}, P_{i,3}\}$, producing Q_i. **Step 2**: Run cubic De Casteljau in u at 0.5 on $\{Q_0, Q_1, Q_2, Q_3\}$. **Step 3**: Output is $S(0.5, 0.5)$. Total: 4 row reductions of cost 6 lerps + 1 final reduction of 6 lerps = 30 linear interpolations.

Q48. Bezier curve at $t = 0.25$ — same control points as Q23?
With $t = 0.25$, $1 - t = 0.75$. **Level 1**: $P_0^{(1)} = 0.75(0,0) + 0.25(1,3) = (0.25, 0.75)$; $P_1^{(1)} = 0.75(1,3) + 0.25(3,3) = (1.5, 3)$; $P_2^{(1)} = 0.75(3,3) + 0.25(4,0) = (3.25, 2.25)$. **Level 2**: $P_0^{(2)} = 0.75(0.25, 0.75) + 0.25(1.5, 3) = (0.5625, 1.3125)$; $P_1^{(2)} = 0.75(1.5, 3) + 0.25(3.25, 2.25) = (1.9375, 2.8125)$. **Level 3**: $B(0.25) = 0.75(0.5625, 1.3125) + 0.25(1.9375, 2.8125) = (0.9375, 1.6875)$. Note: by symmetry of the polygon about $x = 2$, $B(0.25)$ and $B(0.75)$ should mirror — indeed $0.9375 + 3.0625 = 4 = 2 \cdot 2$ and the y-values match 1.6875. ✓

Q49. Why might floating-point errors accumulate in Bernstein direct evaluation but not in De Casteljau?
Direct evaluation expands $B_{i,n}(t) = \binom{n}{i}(1-t)^{n-i}t^i$. For t near 0 or 1, one of t^i and $(1-t)^{n-i}$ is tiny, the other near 1, and the binomial coefficient can be large — multiplying tiny by large amplifies relative error, and summing terms with mixed signs (after substitution into the polynomial) causes **cancellation**. De Casteljau uses only **convex combinations** of points, which never amplify error: the absolute error of each lerp is bounded by the spread of inputs, and the scheme is forward-stable. This is a classic **Wilkinson stability** result.

Q50. Cost of De Casteljau for degree-n curve?
At level k ($k = 1, \ldots, n$), there are $n - k + 1$ linear interpolations, each costing 2 multiplications and 1 addition per coordinate (or one fused multiply-add: $(1-t)A + tB$). Total scalar lerps: $\sum_{k=1}^{n}(n-k+1) = \frac{n(n+1)}{2}$. So $O(n^2)$ **operations**, and for d-dimensional points, $\sim d \cdot n(n+1)/2$ multiplies. For $n = 3, d = 3$: ~ 18 multiplies. Compared with direct Bernstein at $O(n)$ if Bernsteins are precomputed, De Casteljau is constant-factor slower but stable; for the $n = 3$ case used 99% of the time, the difference is negligible.

Đường timing animation (f-curve trong Maya, Blender, After Effects) thường là cubic Bezier giữa 2 keyframe. Hai control point đầu là giá trị keyframe; hai handle giữa điều khiển **ease-in / ease-out**. Hàm CSS3 `cubic-bezier(x1,y1,x2,y2)` chính là cubic Bezier với $P_0 = (0,0)$, $P_1 = (x_1, y_1)$, $P_2 = (x_2, y_2)$, $P_3 = (1,1)$ map animation time $\to$ progress.

Câu 45. Bezier trong path planning robotics?
Cánh tay robot, drone, xe tự hành dùng Bezier curve sinh quỹ đạo **mượt,** C^2 tôn trọng giới hạn vận tốc / gia tốc. So với path piecewise-linear, Bezier giảm jerk (đạo hàm bậc 3), giảm hao actuator và pin. Tính chất convex hull cho **biên an toàn** cho collision check — convex hull free thì trajectory cũng free. Quintic Bezier ($n = 5$) phổ biến vì kiểm soát độc lập vị trí, vận tốc, gia tốc tại cả hai đầu.

Câu 46. Cách nhanh nhất tính $B(t)$ với 4 control point?
Với một t trên cubic ($n = 3$): **De Casteljau** tốn 3+2+1 = 6 phép nội suy tuyến tính, mỗi phép 1 multiply-add / coord, tổng ~ 12 FLOPs ở 2D. Cách khác: Horner trên dạng đa thức $B(t) = ((c_3t + c_2)t + c_1)t + c_0$ với c_i là tổ hợp Bernstein precomputed — rẻ hơn chút nếu chỉ 1 t và không cần stability. Với **nhiều** mẫu t, precompute Bernstein hoặc Horner. Với **robust** subdivision/intersection, luôn De Casteljau.

Câu 47. Tính $S(0.5, 0.5)$ trên control net 4×4 — từng bước.
Cho net $P_{i,j}$, $i, j \in \{0, 1, 2, 3\}$. **Bước 1**: Với mỗi hàng i, chạy cubic De Casteljau theo v tại 0.5 trên $\{P_{i,0}, P_{i,1}, P_{i,2}, P_{i,3}\}$, ra Q_i. **Bước 2**: Chạy cubic De Casteljau theo u tại 0.5 trên $\{Q_0, \ldots, Q_3\}$. **Bước 3**: Kết quả là $S(0.5, 0.5)$. Tổng: 4 row reduction $\times$ 6 lerp + 1 final reduction $\times$ 6 lerp = 30 lerp.

Câu 48. Tính $B(0.25)$ — cùng control point Câu 23?
Với $t = 0.25$, $1 - t = 0.75$. **Level 1**: $P_0^{(1)} = 0.75(0,0) + 0.25(1,3) = (0.25, 0.75)$; $P_1^{(1)} = 0.75(1,3) + 0.25(3,3) = (1.5, 3)$; $P_2^{(1)} = 0.75(3,3) + 0.25(4,0) = (3.25, 2.25)$. **Level 2**: $P_0^{(2)} = 0.75(0.25, 0.75) + 0.25(1.5, 3) = (0.5625, 1.3125)$; $P_1^{(2)} = 0.75(1.5, 3) + 0.25(3.25, 2.25) = (1.9375, 2.8125)$. **Level 3**: $B(0.25) = 0.75(0.5625, 1.3125) + 0.25(1.9375, 2.8125) = (0.9375, 1.6875)$. Lưu ý: do polygon đối xứng qua $x = 2$, $B(0.25)$ và $B(0.75)$ phải đối xứng — đúng vì $0.9375 + 3.0625 = 4 = 2 \cdot 2$ và y cùng 1.6875. ✓

Câu 49. Vì sao Bernstein trực tiếp tích lũy sai số floating-point còn De Casteljau thì không?
Bernstein trực tiếp khai triển $B_{i,n}(t) = \binom{n}{i}(1-t)^{n-i}t^i$. Khi t gần 0 hoặc 1, một trong t^i hoặc $(1-t)^{n-i}$ rất nhỏ, cái kia gần 1, và binomial có thể lớn — nhân nhỏ với lớn khuếch đại sai số tương đối, và cộng các số hạng khác dấu (sau khi mở rộng) gây **cancellation**. De Casteljau chỉ **tổ hợp lồi**, không khuếch đại sai số: sai số tuyệt đối mỗi lerp bị chặn bởi đường kính cụm input, scheme là forward-stable. Đây là kết quả **Wilkinson stability** kinh điển.

Câu 50. Chi phí De Casteljau cho curve bậc n?
Tại level k ($k = 1, \ldots, n$) có $n - k + 1$ phép nội suy, mỗi phép 2 multiply + 1 add / coord (hoặc 1 fused multiply-add: $(1-t)A + tB$). Tổng số lerp: $\sum_{k=1}^{n}(n-k+1) = \frac{n(n+1)}{2}$. Vậy $O(n^2)$, với điểm d chiều thì $\sim d \cdot n(n+1)/2$ multiply. Với $n = 3, d = 3$: ~ 18 multiply. So với Bernstein trực tiếp $O(n)$ khi Bernstein đã precomputed, De Casteljau chậm hơn hằng số nhưng stable; với $n = 3$ dùng 99% trường hợp, chênh lệch không đáng kể.

5. Glossary EN ↔ VI / Từ vựng song ngữ

General CG / Chung
computer graphics – đồ họa máy tính
rendering – kết xuất / dựng hình
real-time rendering – kết xuất thời gian thực
offline rendering – kết xuất ngoại tuyến
ray tracing – dò tia
ray casting – phóng tia
path tracing – dò đường truyền tia
rasterization – quét mành / điểm hóa
pipeline – ống dẫn / quy trình xử lý
graphics pipeline – ống dẫn đồ họa
vertex – đỉnh
edge – cạnh
face – mặt
polygon – đa giác
primitive – nguyên thủy / khối cơ bản
frame buffer – bộ đệm khung hình
depth buffer / Z-buffer – bộ đệm độ sâu
stencil buffer – bộ đệm khuôn
shader – bộ đổ bóng / shader
vertex shader – shader đỉnh
fragment shader / pixel shader – shader điểm ảnh
geometry shader – shader hình học
compute shader – shader tính toán
fragment – mảnh / phân đoạn
pixel – điểm ảnh
texel – điểm họa tiết
voxel – điểm thể tích
viewport – khung nhìn / cổng nhìn
clipping – cắt / xén
culling – loại bỏ
back-face culling – loại bỏ mặt sau
frustum culling – loại bỏ ngoài hình chóp cụt
occlusion culling – loại bỏ bị che khuất
projection – phép chiếu
perspective projection – chiếu phối cảnh
orthographic projection – chiếu trực giao / song song
transformation – phép biến đổi
rotation – phép quay
translation – phép tịnh tiến
scaling – phép co giãn / tỉ lệ
shearing – phép trượt
reflection – phép phản chiếu
homogeneous coordinate – tọa độ thuần nhất
world coordinate – tọa độ thế giới
camera coordinate – tọa độ máy ảnh
screen coordinate – tọa độ màn hình
model-view matrix – ma trận mô hình-quan sát
OpenGL – thư viện đồ họa OpenGL
DirectX – thư viện DirectX
Vulkan – API Vulkan
GPU – bộ xử lý đồ họa
CPU – bộ xử lý trung tâm
anti-aliasing (AA) – khử răng cưa
MSAA – khử răng cưa đa mẫu
level of detail (LOD) – mức độ chi tiết
texture – họa tiết / vân bề mặt
texture mapping – ánh xạ họa tiết
UV mapping – ánh xạ tọa độ UV
mipmap – mipmap / đa mức họa tiết
bump mapping – ánh xạ độ gồ ghề
normal mapping – ánh xạ pháp tuyến
displacement mapping – ánh xạ dịch chuyển
normal vector – vec-tơ pháp tuyến
tangent – tiếp tuyến
binormal / bitangent – vec-tơ phụ tuyến
lighting – chiếu sáng
ambient light – ánh sáng môi trường
diffuse light – ánh sáng khuếch tán
specular light – ánh sáng phản chiếu
point light – nguồn sáng điểm
directional light – nguồn sáng định hướng
spot light – nguồn sáng pha
shading – tạo bóng / đổ bóng
flat shading – đổ bóng phẳng
Gouraud shading – đổ bóng Gouraud
Phong shading – đổ bóng Phong
PBR – kết xuất dựa trên vật lý
material – chất liệu / vật liệu
albedo – phản chiếu khuếch tán cơ bản
roughness – độ nhám
metallic – độ kim loại
shadow mapping – ánh xạ bóng đổ
global illumination – chiếu sáng toàn cục
ambient occlusion (AO) – che khuất môi trường
HDR – dải tương phản cao
tone mapping – ánh xạ tông màu
gamma correction – hiệu chỉnh gamma
double buffering – đệm kép
vsync – đồng bộ dọc
frame rate (FPS) – tốc độ khung hình
Geometric Modeling
geometric modeling – mô hình hóa hình học
point cloud – đám mây điểm
triangle mesh – lưới tam giác
polygon mesh – lưới đa giác
quad mesh – lưới tứ giác
wireframe – khung dây
manifold – đa tạp
non-manifold – không đa tạp
boundary – biên
boundary edge – cạnh biên
hole – lỗ thủng
hole filling – vá lỗ / lấp lỗ
watertight – kín nước / liền khối
closed surface – mặt kín
open surface – mặt hở
orientable – có hướng
genus – chi (số lỗ)
Euler characteristic – đặc trưng Euler
topology – tô-pô / hình học liên kết
geometry – hình học (tọa độ)
half-edge – nửa cạnh
winged-edge – cạnh có cánh
adjacency – kề / lân cận
normal – pháp tuyến
vertex normal – pháp tuyến đỉnh
face normal – pháp tuyến mặt
surface – bề mặt
curve – đường cong
simplification – đơn giản hóa
decimation – giảm lưới / giảm mật độ
mesh simplification – đơn giản hóa lưới
remeshing – tạo lại lưới
triangulation – tam giác hóa
Delaunay triangulation – tam giác hóa Delaunay
constrained Delaunay – Delaunay có ràng buộc
Voronoi diagram – biểu đồ Voronoi
circumcircle – đường tròn ngoại tiếp
in-circle predicate – vị từ trong đường tròn
empty circle property – tính chất đường tròn rỗng
convex hull – bao lồi
flip operation / edge flip – phép lật cạnh
fairing – làm mượt / chỉnh trơn
smoothing – làm trơn
denoising – khử nhiễu
noise removal – loại bỏ nhiễu
outlier – điểm ngoại lai
sliver triangle – tam giác mỏng / tam giác kim
regular sampling – lấy mẫu đều
irregular sampling – lấy mẫu không đều
alpha shape – hình dạng alpha
ball pivoting (BPA) – xoay bóng
Poisson reconstruction – tái tạo Poisson
Marching Cubes – thuật toán Marching Cubes
isosurface – mặt đẳng trị
isovalue – giá trị đẳng trị
scalar field – trường vô hướng
signed distance function (SDF) – hàm khoảng cách có dấu
implicit surface – mặt ẩn
explicit surface – mặt tường minh
parametric surface – mặt tham số
NURBS – B-spline hữu tỷ không đều
B-spline – đường spline cơ sở B
Bezier – đường Bezier
control point – điểm điều khiển
control polygon – đa giác điều khiển
control net – lưới điều khiển
knot – nút
knot vector – vec-tơ nút
basis function – hàm cơ sở
tensor product – tích tensor
subdivision surface – mặt chia nhỏ
subdivision – chia nhỏ / mịn hóa
Catmull-Clark – Catmull-Clark
Loop subdivision – chia nhỏ Loop
refinement – mịn hóa / tinh chỉnh
edge collapse – gộp cạnh
vertex split – tách đỉnh
quadric error metric (QEM) – độ đo sai số bậc hai
Laplacian – toán tử Laplace
Laplacian smoothing – làm trơn Laplace
cotangent weight – trọng số cotang
Taubin smoothing – làm trơn Taubin
bilateral filter – bộ lọc song phương
Gaussian filter – bộ lọc Gauss
mean curvature – độ cong trung bình
Gaussian curvature – độ cong Gauss
principal curvature – độ cong chính
moving least squares (MLS) – bình phương tối thiểu di động
WLOP – vận hành định vị địa phương có trọng số
PCA – phân tích thành phần chính
kNN – k láng giềng gần nhất
octree – cây bát phân
KD-tree – cây KD
BVH – phân cấp bao đóng
bounding box – hộp bao
AABB – hộp bao trục căn chỉnh
OBB – hộp bao có hướng
registration – căn chỉnh / khớp lưới
ICP (Iterative Closest Point) – thuật toán điểm gần nhất lặp
Hausdorff distance – khoảng cách Hausdorff
advancing front – mặt tiến
Greedy projection – chiếu tham lam
Medical Imaging
medical image – ảnh y tế
MRI – chụp cộng hưởng từ
CT scan – chụp cắt lớp vi tính
ultrasound (US) – siêu âm
PET – chụp cắt lớp phát xạ positron
X-ray – tia X / chụp X-quang
SPECT – chụp cắt lớp đơn photon
OCT – chụp cắt lớp quang học
fMRI – cộng hưởng từ chức năng
DICOM – chuẩn ảnh y tế DICOM
NIfTI – định dạng NIfTI
modality – phương thức / phương pháp ảnh
slice – lát cắt
volume – khối thể tích
volume rendering – kết xuất khối
multi-planar reformatting (MPR) – tái dựng đa mặt phẳng
k-space – không gian k
sinogram – ảnh sin / sinogram
Radon transform – biến đổi Radon
Filtered Back Projection (FBP) – chiếu ngược có lọc
Hounsfield Unit (HU) – đơn vị Hounsfield
T1-weighted – ảnh trọng số T1
T2-weighted – ảnh trọng số T2
FLAIR – ảnh FLAIR
DWI – ảnh khuếch tán
ADC – hệ số khuếch tán biểu kiến
contrast agent – thuốc cản quang / chất tương phản
gadolinium – gadolini
iodine contrast – cản quang iốt
segmentation – phân đoạn
semantic segmentation – phân đoạn ngữ nghĩa
instance segmentation – phân đoạn đối tượng
registration – khớp ảnh / căn chỉnh ảnh
rigid registration – khớp ảnh cứng
non-rigid registration – khớp ảnh phi cứng
classification – phân loại
detection – phát hiện
brain tumor – u não
glioma – u thần kinh đệm
meningioma – u màng não
BraTS – thử thách phân đoạn u não BraTS
lesion – tổn thương
edema – phù nề
hemorrhage – xuất huyết
necrosis – hoại tử
enhancing tumor – u ngấm thuốc
whole tumor – toàn bộ u
tumor core – lõi u
skull stripping – loại bỏ hộp sọ
bias field – nhiễu trường thiên lệch
N4 bias correction – hiệu chỉnh thiên lệch N4
intensity normalization – chuẩn hóa cường độ

histogram matching – khớp biểu đồ
Otsu thresholding – ngưỡng Otsu
thresholding – phân ngưỡng
watershed – phân vùng đường phân thủy
region growing – phát triển vùng
snake / active contour – đường viền chủ động
level set – tập mức
morphological operation – phép toán hình thái
erosion – co
dilation – giãn
opening – mở
closing – đóng
U-Net – mạng U-Net
V-Net – mạng V-Net
nnU-Net – mạng nnU-Net (tự cấu hình)
encoder-decoder – bộ mã hóa-giải mã
skip connection – kết nối tắt
Dice coefficient – hệ số Dice
Jaccard / IoU – chỉ số Jaccard / giao trên hợp
Hausdorff distance – khoảng cách Hausdorff
ASSD – khoảng cách bề mặt đối xứng trung bình
sensitivity / recall – độ nhạy
specificity – độ đặc hiệu
precision – độ chính xác (precision)
accuracy – độ chính xác tổng
F1 score – điểm F1
ROC curve – đường ROC
AUC – diện tích dưới đường cong
ground truth – nhãn thật / nhãn chuẩn
label – nhãn
annotation – chú thích
augmentation – tăng cường dữ liệu
transfer learning – học chuyển giao
fine-tuning – tinh chỉnh
pretrained model – mô hình đã huấn luyện sẵn
GAN – mạng đối kháng sinh
generator – bộ sinh
discriminator – bộ phân biệt
cGAN – GAN có điều kiện
diagnosis – chẩn đoán
prognosis – tiên lượng
CDSS – hệ thống hỗ trợ quyết định lâm sàng
XAI – AI giải thích được
Grad-CAM – bản đồ kích hoạt theo gradient
saliency map – bản đồ nổi bật
attention map – bản đồ chú ý
VR / AR / XR
virtual reality (VR) – thực tế ảo
augmented reality (AR) – thực tế tăng cường
mixed reality (MR) – thực tế hỗn hợp
extended reality (XR) – thực tế mở rộng
HMD / head-mounted display – kính/mũ đeo đầu
immersion – sự đắm chìm / nhập vai
presence – cảm giác hiện diện
telepresence – hiện diện từ xa
stereoscopic – lập thể
stereo rendering – kết xuất lập thể
parallax – thị sai
depth perception – nhận thức độ sâu
binocular disparity – chênh lệch hai mắt
accommodation – điều tiết
vergence – hội tụ mắt
vergence-accommodation conflict – mâu thuẫn hội tụ-điều tiết
FOV (field of view) – góc nhìn
IPD (interpupillary distance) – khoảng cách đồng tử
refresh rate – tốc độ làm mới
latency – độ trễ
motion-to-photon latency – độ trễ chuyển động đến photon

tracking – theo dõi / bám
6DoF – 6 bậc tự do
3DoF – 3 bậc tự do
inside-out tracking – theo dõi từ trong ra
outside-in tracking – theo dõi từ ngoài vào
lighthouse – trạm phát Lighthouse
IMU – đơn vị đo quán tính
accelerometer – gia tốc kế
gyroscope – con quay hồi chuyển
magnetometer – từ kế
SLAM – định vị và lập bản đồ đồng thời
visual SLAM – SLAM thị giác
marker-based AR – AR dựa trên dấu mốc
markerless AR – AR không dấu mốc
fiducial marker – dấu mốc nhận dạng
ArUco / QR marker – dấu ArUco / QR
anchor – điểm neo
plane detection – phát hiện mặt phẳng
feature point – điểm đặc trưng
occlusion – che khuất
pass-through – nhìn xuyên (camera)
optical see-through – nhìn xuyên quang học
video see-through – nhìn xuyên video
waveguide – ống dẫn sóng
OLED – màn hình OLED
micro-OLED – micro-OLED
LCD – màn hình LCD
foveated rendering – kết xuất theo điểm nhìn
eye tracking – theo dõi mắt
lens distortion – méo thấu kính
barrel distortion – méo dạng thùng
pincushion distortion – méo dạng gối
chromatic aberration – quang sai sắc
motion sickness – say tàu xe ảo
cybersickness – say không gian mạng
vection – ảo giác chuyển động
controller – tay điều khiển
haptic feedback – phản hồi xúc giác
force feedback – phản hồi lực
ray casting (VR) – phóng tia chọn vật
teleportation – dịch chuyển tức thời
snap turn – xoay nhanh
smooth locomotion – di chuyển trơn
room scale – quy mô phòng
guardian / chaperone – vùng an toàn
Unity – engine Unity
Unreal Engine – engine Unreal
ARKit – ARKit (Apple)
ARCore – ARCore (Google)
Vuforia – Vuforia
OpenXR – chuẩn OpenXR
MRTK – bộ công cụ MR (Microsoft)
WebXR – WebXR (XR trên web)
eMuseum – bảo tàng điện tử
eTourism – du lịch điện tử
digital heritage – di sản số
digital twin – bản sao số
avatar – nhân vật đại diện
metaverse – vũ trụ ảo
Bezier & Splines
Bezier curve – đường cong Bezier
Bezier surface – mặt Bezier
Bernstein polynomial – đa thức Bernstein
Bernstein basis – cơ sở Bernstein
control point – điểm điều khiển
control polygon – đa giác điều khiển
control net – lưới điều khiển
parameter t – tham số t
parameterization – tham số hóa
interpolation – nội suy

approximation – xấp xỉ
De Casteljau algorithm – thuật toán De Casteljau
recursion – đệ quy
subdivision – chia đôi / chia nhỏ
degree – bậc
degree elevation – nâng bậc
degree reduction – hạ bậc
derivative – đạo hàm
hodograph – đồ thị đạo hàm / hodograph
tangent vector – vec-tơ tiếp tuyến
normal vector – vec-tơ pháp tuyến
curvature – độ cong
torsion – độ xoắn
convex hull property – tính chất bao lồi
partition of unity – phân hoạch đơn vị
affine invariance – bất biến affine
variation diminishing – giảm biến thiên
symmetry – đối xứng
endpoint interpolation – nội suy điểm đầu cuối
isoparametric curve – đường đẳng tham số
tensor product surface – mặt tích tensor
bicubic – bậc ba kép / song bậc ba
biquadratic – bậc hai kép
rational Bezier – Bezier hữu tỷ
B-spline – B-spline
NURBS – NURBS
knot vector – vec-tơ nút
uniform knot – nút đều
non-uniform knot – nút không đều
Hermite curve – đường Hermite
Catmull-Rom spline – spline Catmull-Rom
cubic spline – spline bậc ba
spline – spline / đường ghép trơn
continuity – tính liên tục
C0 continuity – liên tục C0 (cùng vị trí)
C1 continuity – liên tục đạo hàm bậc 1
C2 continuity – liên tục đạo hàm bậc 2
G1 continuity – liên tục hình học G1
G2 continuity – liên tục hình học G2
smoothness – độ trơn
blending function – hàm pha trộn
weight – trọng số
rational basis – cơ sở hữu tỷ
Coons patch – mảnh Coons
Gregory patch – mảnh Gregory
control mesh – lưới điều khiển
patch – mảnh
Math / Toán
matrix – ma trận
vector – vec-tơ
scalar – vô hướng
determinant – định thức
eigenvalue – giá trị riêng
eigenvector – vec-tơ riêng
inverse matrix – ma trận nghịch đảo
transpose – chuyển vị
identity matrix – ma trận đơn vị
orthogonal matrix – ma trận trực giao
symmetric matrix – ma trận đối xứng
positive definite – xác định dương
rank – hạng
trace – vết
dot product / inner product – tích vô hướng
cross product – tích có hướng
outer product – tích ngoài
norm – chuẩn
L_2 **norm** – chuẩn L_2 / chuẩn Euclid
gradient – gradient / vec-tơ độ dốc
divergence – tán xạ / div
Laplacian – toán tử Laplace

Hessian – ma trận Hess
Jacobian – ma trận Jacobi
partial derivative – đạo hàm riêng
integral – tích phân
Fourier transform – biến đổi Fourier
FFT – biến đổi Fourier nhanh
convolution – tích chập
kernel – nhân (toán)
basis – cơ sở
span – bao tuyến tính
linear combination – tổ hợp tuyến tính
projection – phép chiếu
orthogonal – trực giao
orthonormal – trực chuẩn
plane – mặt phẳng
line – đường thẳng
ray – tia
intersection – giao
distance – khoảng cách
angle – góc
area – diện tích
volume – thể tích
polynomial – đa thức
recursion – đệ quy
iteration – lặp
convergence – hội tụ
optimization – tối ưu hóa
minimization – cực tiểu hóa
maximization – cực đại hóa
gradient descent – giảm theo gradient
stochastic gradient descent – giảm gradient ngẫu nhiên
regression – hồi quy
regularization – chính quy hóa
least squares – bình phương tối thiểu
SVD – phân tích giá trị suy biến
quaternion – số quaternion
Euler angle – góc Euler
barycentric coordinate – tọa độ trọng tâm
Algorithms / Performance
algorithm – thuật toán
complexity – độ phức tạp
time complexity – độ phức tạp thời gian
space complexity – độ phức tạp không gian
big-O notation – ký hiệu O lớn
recursion – đệ quy
dynamic programming – quy hoạch động
greedy algorithm – thuật toán tham lam
divide and conquer – chia để trị
backtracking – quay lui
heap – đống / heap
priority queue – hàng đợi ưu tiên
stack – ngăn xếp
queue – hàng đợi
hash table – bảng băm
BFS – duyệt theo chiều rộng
DFS – duyệt theo chiều sâu
Dijkstra – thuật toán Dijkstra
A* search – tìm kiếm A sao
sort – sắp xếp
search – tìm kiếm
binary search – tìm kiếm nhị phân
graph – đồ thị
tree – cây
octree – cây bát phân
quadtree – cây tứ phân
KD-tree – cây KD
R-tree – cây R
BVH – phân cấp khối bao
spatial index – chỉ mục không gian

nearest neighbor – láng giềng gần nhất
parallel computing – tính toán song song
concurrency – đồng thời
thread – luồng
GPU computing – tính toán trên GPU
CUDA – nền tảng CUDA
OpenCL – nền tảng OpenCL
SIMD – một lệnh nhiều dữ liệu
cache – bộ nhớ đệm
memory bandwidth – băng thông bộ nhớ
benchmark – đánh giá hiệu năng
profiling – đo lường hiệu năng

optimization – tối ưu hóa
lookup table (LUT) – bảng tra cứu
precomputation – tính trước
lazy evaluation – ước lượng trễ
streaming – truyền dòng
out-of-core – ngoài bộ nhớ chính
Deep Learning (bonus)
neural network – mạng nơ-ron
deep learning – học sâu
CNN – mạng nơ-ron tích chập
RNN – mạng nơ-ron hồi quy
transformer – mạng transformer

attention – cơ chế chú ý
self-attention – tự chú ý
layer – tầng / lớp
convolution layer – tầng tích chập
pooling – gộp / lấy mẫu xuống
max pooling – gộp lớn nhất
batch normalization – chuẩn hóa lô
dropout – bỏ ngẫu nhiên
activation function – hàm kích hoạt
ReLU – hàm ReLU
sigmoid – hàm sigmoid
softmax – hàm softmax

loss function – hàm mất mát
cross-entropy – entropy chéo
Dice loss – mất mát Dice
backpropagation – lan truyền ngược
epoch – chu kỳ huấn luyện
batch – lô
learning rate – tốc độ học
overfitting – quá khớp
underfitting – thiếu khớp
validation set – tập kiểm định
NeRF – trường bức xạ thần kinh
diffusion model – mô hình khuếch tán